

BEDROCK ARCHITECTURE

Programmer's Reference Manual

Bounded Byte-Addressed CISC Architecture

CONTENTS

I	Architectural Foundations	1
1	Overview	2
1.1	Contract Position	2
1.2	Design Goals	2
1.3	Architectural Profile	2
2	Terminology and Compatibility	4
2.1	Address Values	4
2.2	Segmentation and Paging	4
2.3	Control Flow	5
2.4	Tracing Terms	5
2.5	Architectural State	5
2.6	Compatibility	5
2.7	Compatibility Contract	6
2.8	Reserved Fields	6
2.9	Reserved Encodings	6
2.10	CPUID Compatibility	7
3	Conformance	8
3.1	Conformance Claim	8
3.2	Normative Material and Specificity	8
3.3	Implementation-Defined Disclosures	8
4	Programming Model	10
4.1	Register Model	10
4.2	FLAGS and STATUS Registers	10
4.3	FFLAGS and FSTATUS Registers	11
4.4	Floating-Point Register Model	12
4.5	Segment Registers	12
4.6	Segment Register Operand Class	12
4.7	Control Registers	13
5	Data Formats	17

5.1	Integer Data Sizes	17
5.2	Little-Endian Memory Organization	17
5.3	Floating-Point Data Formats	18
II	Encoding, Addressing, and Execution	19
6	Instruction Encoding	20
6.1	Instruction Stream and Record Boundary	20
6.2	Instruction Header Formats	20
6.3	Instruction Payload Ordering	23
6.4	Representative Encodings	24
7	Effective Addressing Modes	25
7.1	Compact EA Addressing Modes	26
7.2	Extended EA Descriptor	32
7.3	Extended EA Addressing Modes	34
8	Memory Address Translation	42
8.1	Multi-Page Accesses	43
8.2	Paging Stage	43
8.3	LA48 Four-Level Paging	45
8.4	LA57 Five-Level Paging	46
8.5	Page-Table Entry Format	47
8.6	Translation-Cache Identity and Invalidation	48
8.7	Leaf Address Types and Access Ranges	49
9	Instruction Execution Model	51
9.1	Architectural Result Priority	51
9.2	Implicit Stack Operations	51
9.3	Operand Evaluation and EA Defaults	52
9.4	Shared Side-Effect Rules	52
10	Flags and Condition Codes	54
10.1	Condition Encoding	54
10.2	Flag Meanings	55
10.3	Conditional Consumers	55
10.4	Floating-Point Interactions	55

11 Memory Model	56
11.1 Baseline Ordering	56
11.2 Ordinary Load Values and Preserved Order	56
11.3 PTE Cache Policies	56
11.4 Slot-Addressed Transactions	57
11.5 Access Atomicity and Alignment	57
11.6 Cache-Maintenance Blocks	58
11.7 Atomic Memory-Order Selectors	58
11.8 Fence Instructions	59
11.9 Global Sequentially Consistent Order	59
12 Streaming Execution Model	60
12.1 Architectural Scalar Semantics	60
12.2 Repeat Syntax and Body Eligibility	60
12.3 Counter, Condition, and Control Rules	60
12.4 Fault and Restart Semantics	61
III System Programming	62
13 Privileged Execution Model	63
13.1 Privilege and Transition State	63
13.2 Supervisor Call Entry	63
13.3 User Return and Context Ownership	63
13.4 Architectural Event Boundary	64
14 Architectural Event Processing Model	65
14.1 Event Code and Sources	65
14.2 Event Priority and Debug Trace	67
14.3 Entry Admission and Event Depth	67
14.4 Supervisor Event Stacks and Nesting	68
14.5 Event Entry State	69
14.6 Architectural Event Frame	70
14.7 Event Payloads	71
14.7.1 Page-Fault Error Code	72
14.7.2 Other Exception Error Codes	73
14.7.3 Auxiliary Payloads	74

14.8	Machine-Check and Bus-Error Continuation	76
14.9	Repeat Continuation	77
14.10	Delivery Failure and Shutdown	77
14.11	Event Reset and Context State	78
15	CPUID Feature Discovery	80
15.1	Overview	80
15.2	Query Model	80
15.3	Discovery Classes	81
15.4	Leaf Directory	81
15.5	Base Identity	82
15.6	Optional-Extension Directory	83
15.7	FPTRANSA Accuracy Discovery	83
15.8	Implementation-Class Directory	85
15.9	Cache-Topology Discovery	86
15.10	Performance-Counter Discovery	87
15.11	SAVE-Area Layout Discovery	87
15.12	Bit Fields	89
16	Processor-State Save and Restore	90
IV	Instruction Set Reference	92
	Reading an Instruction Description	93
17	General Instructions	94
17.1	Summary	94
18	Floating-Point Instructions	347
18.1	Common Floating-Point Semantics	347
18.1.1	Input, NaN, Rounding, and Result Order	347
18.1.2	Exception Causes and Commit	348
18.1.3	Comparison, Minimum, and Maximum	348
18.1.4	Conversions	349
18.2	Summary	349
19	Approximate Floating-Point Transcendental Instructions	440

19.1	FPTRANSA ULP Error Model	440
19.2	Summary	440
A	Reference Indexes and Revision History	479
A.1	Canonical Field Names	479
A.2	Architectural State Index	479
A.3	Architectural Event Index	480
A.4	CPUID Feature Index	481
A.5	Architecture Revision History	483

LIST OF TABLES

2-1	Reserved Field Defaults	6
3-1	Implementation-Defined Disclosure Register	8
4-1	Floating-Point State Fields	11
4-2	Segment Register Operand Encoding	12
4-3	Control-Register Selectors	13
4-4	WRCR Selector Rules	13
4-5	PTCR Fields	14
4-6	ASCR Fields	15
4-7	ECR Fields	15
4-8	URCTL Fields	15
4-9	PMC Fields	16
5-1	Scalar Data Sizes	17
6-1	Instruction Length Truth Table	22
6-2	Encoding Class Summary	23
6-3	Immediate Operand Interpretation	23
7-1	Compact EA Encoding	31
7-2	EA Payload Names	33
8-1	Segment Pre-Translation Modes	43
8-2	Page-Table Entry Fields	47
8-3	Translation-Cache Entry Identity	48
8-4	Local Translation-Cache Transitions	48
8-5	Remote Translation Shutdown Protocol	48
9-1	Ordinary Implicit Stack Operations	52
10-1	Condition Code Encoding	54
10-2	Integer Condition-Code Bits	55
11-1	Normal-Memory Cache Policies	56
11-2	Atomic Memory-Order Selectors	58
11-3	Fence Ordered-Before Pairs	59
14-1	Architectural Event Classes	65
14-2	Assigned Base Exception IDs	65
14-3	Architectural Event Boundaries and Priority	66
14-4	Architectural Event Producers and Delivery State	66
14-5	Supervisor Stack Roles	68
14-6	Architectural Event Frame Fields	70

14-7	FRAME_CONTROL Fields	70
14-8	Architectural Event Frame Types and Sizes	71
14-9	Event-to-Frame-Type Assignment	71
14-10	PAGE_FAULT ERROR_CODE Fields	72
14-11	PAGE_FAULT Causes	72
14-12	ILLEGAL_INSTRUCTION Causes	73
14-13	INVALID_CONTROL_STATE Causes	73
14-14	DOUBLE_FAULT Delivery Stages	74
14-15	MACHINE_CHECK ERROR_CODE Fields	75
14-16	MACHINE_CHECK Valid Continuation Combinations	75
14-17	BUS_ERROR ERROR_CODE Fields	76
14-18	FRAME_EXT1 Repeat-Continuation Fields	77
14-19	Warm RESET Contract	78
14-20	Architectural Reset State	78
15-1	CPUID Query Selector	80
15-2	CPUID Common Leaf Header	81
15-3	CPUID Class and Leaf Directory	81
15-4	Base Identity Indexes	82
15-5	Optional-Extension Feature Bits	83
15-6	FPTRANSA Accuracy Result	84
15-7	FPTRANSA Accuracy Contracts	84
15-8	Cache-Maintenance Properties	86
15-9	Cache-Topology Descriptor	86
15-10	SAVE-AREA-LAYOUT Indexes	87
15-11	SAVE Component Descriptor A	88
15-12	SAVE Component Descriptor B	88
15-13	Floating-Point SAVE Component	88
16-1	SAVE/RESTORE Fixed Base Block	91
17-1	General Instructions Summary	94
18-1	Floating-Point Instructions Summary	349
18-2	FMOVCR Constant IDs (IEEE-754 binary64)	405
19-1	Approximate Floating-Point Transcendental Instructions Summary	440
A-1	Canonical Field Names	479
A-2	Architectural State Index	479
A-3	Architectural Event Index	480

A-4	CPUID Feature Index	481
A-5	Architecture Revision History	483

LIST OF FIGURES

4-1	User Programming Model	10
4-2	FLAGS Register Format	11
4-3	STATUS Register Format	11
4-4	FFLAGS Register Format	11
4-5	FSTATUS Register Format	11
4-6	Segment Register Format	12
5-1	Integer Register Operand Subfields	17
5-2	Little-Endian Byte Order for a 64-Bit Value	18
5-3	Instruction Start Bytes	18
5-4	Floating-Point Register Encodings	18
6-1	Instruction Header Byte Formats	21
6-2	Instruction Record Components	21
6-3	Opcode Payload Namespaces	22
6-4	SETF and CLRF Flag Bitmap	24
6-5	Short Encoding Layout for MOV.X(z:L/Q) Rn(s), Rn(d)	24
6-6	Medium Encoding Layout for MOV.X(z:B/W) Rn(s), <ea>(e)	24
6-7	Long Encoding Layout for ADC.X(z:B/W/L/Q) Rn(s), <ea>(e)	24
7-1	Compact Effective-Address Field	32
7-2	EXT0 One-Byte Descriptor Forms	32
7-3	EXT0 Two-Byte Indexed Descriptor Byte Layouts	33
8-1	Address Translation Pipeline	42
8-2	Linear-Address Paging Fields	44
8-3	LA48 Four-Level Page Walk	45
8-4	LA57 Five-Level Page Walk	46
8-5	Page-Table Entry Formats	47
14-1	Architectural Event Code	65
14-2	Architectural Event Frame	70
14-3	FRAME_CONTROL Format	70
14-4	PAGE_FAULT ERROR_CODE Format	72
14-5	Simple Exception ERROR_CODE Formats	74
14-6	DOUBLE_FAULT Auxiliary Formats	74
14-7	MACHINE_CHECK ERROR_CODE Format	75
14-8	BUS_ERROR ERROR_CODE Format	76
14-9	FRAME_EXT1 Repeat Continuation Format	77

15-1	CPUID Query Selector Format	80
15-2	CPUID Common Leaf Header Formats	81
15-3	CPUID Base Identity Result Formats	82
15-4	Optional-Extension Directory Result	83
15-5	FPTRANSA Accuracy Result Format	84
15-6	Cache-Maintenance Properties Result	86
15-7	Cache-Topology Descriptor Format	86
15-8	Performance-Counter Presence Result	87
15-9	SAVE-Area Layout CPUID Result Formats	88
16-1	SAVE/RESTORE Base Header	91

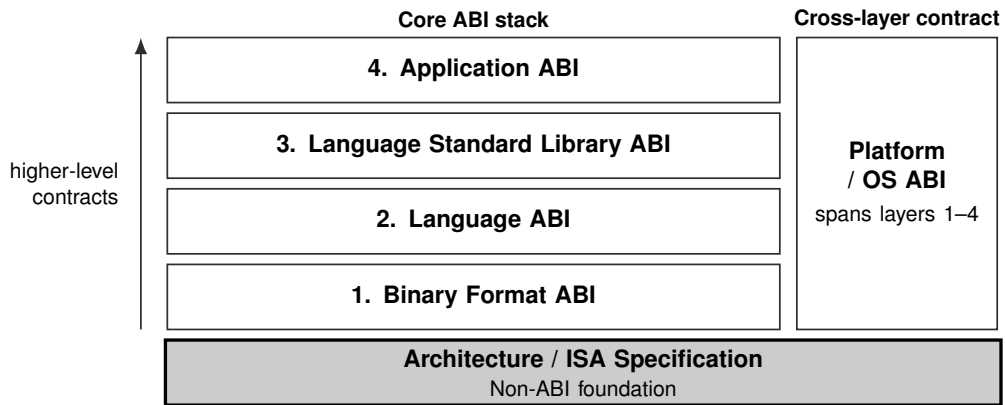
PART I

BEDROCK ARCHITECTURE

Architectural Foundations

1 OVERVIEW

1.1 Contract Position



Current document: Bedrock Programmer's Reference Manual. The shaded block identifies its primary contract position. The Platform / OS ABI spans and constrains the four core ABI layers; the ISA specification is their non-ABI architectural foundation.

1.2 Design Goals

Bedrock is a bounded, byte-addressed CISC ISA with explicit instruction lengths, compact register forms, and selected register-memory and memory-memory forms.

The manual presents programmer-visible behavior first: architectural state, operand encoding, execution semantics, and instruction descriptions. Encoding details appear where they are needed to decode or use an instruction form.

1.3 Architectural Profile

Bedrock defines sixteen 64-bit integer registers, R0 through R15. SP and PC are special architectural registers outside the Rn namespace, with SP-relative addressing tied to SS and PC-relative addressing tied to CS.

The reference is organized around programmer-visible state, operand formation, execution semantics, and instruction behavior. Byte-level instruction framing is specified separately in the Instruction Encoding chapter.

- The PC names a byte in the instruction stream; sequential execution advances it by the decoded instruction length.
- Instruction length is explicit and bounded; the base profile defines lengths from 1 to 18 bytes.
- Four integer operation sizes: B, W, L, and Q.
- The effective-address model covers register, memory, immediate, absolute, segment-qualified, indexed, and auto-update operands.

- Segment pre-translation precedes optional page-table translation.
- User and supervisor modes share the instruction set, with privileged instructions and checked user-access moves defining supervisor-only behavior.

2 TERMINOLOGY AND COMPATIBILITY

2.1 Address Values

An *effective address (EA)* is the address value or operand designator produced by an addressing mode before segment pre-translation. A memory EA produces an address, while register and immediate EAs name non-memory operands.

A *pre-segment virtual address* is a virtual-address value before segment pre-translation. Segment pre-translation either converts it to a linear address or reports the architecturally defined fault.

A *linear address* is the result of segment pre-translation. The page-table walker consumes it when paging is enabled; otherwise the memory system uses it directly.

A *physical address* is the byte address produced by page-table translation for ordinary byte-addressed memory. With paging disabled, the linear address is used directly instead.

A *memory-system address* is the value presented to the memory subsystem after every active architectural translation stage. Depending on the final access attributes, the subsystem may interpret it as an ordinary byte address or an externally acknowledged slot address.

Byte-addressed normal memory is the coherent memory-location model selected by a final translation with $AT=0$. The PTE CP field selects one of its four normal-memory cache policies. *Cacheable* names only policy $CP=0$; the other three CP values remain byte-addressed normal memory with the allocation and propagation behavior defined by the Memory Model.

A *slot-addressed transaction* is the externally acknowledged access selected by a final translation with $AT=1$. Its memory-system address is a slot address rather than a normal-memory byte address. The Memory Address Translation and Memory Model sections define its transfer, completion, and ordering rules.

Address generation is the instruction-local calculation that combines registers, displacement, index, scale, and immediate fields into an effective address. It does not by itself read memory.

2.2 Segmentation and Paging

Segment pre-translation is the stage between effective-address calculation and paging. A disabled segment passes the EA through. A translated window checks an offset and adds its base, while a bounds-only window checks an already-linear address without adding the base.

A *segment register image* is the 64-bit architectural value containing the segment base page, size exponent, size mantissa, and bounds-only enable bit.

A *disabled segment* has a zero size mantissa. It performs no segment-window check or base addition and passes the effective address through as the linear address.

A *translated segment* is enabled with bounds-only mode clear. It checks the effective address as an offset within the segment span and then adds the segment base.

A *bounds-only segment* is enabled with bounds-only mode set. It treats the effective address as already linear and checks only that it lies inside the segment window.

Page-table translation is the optional $PTCR.PE$ -controlled stage that maps a canonical linear address to a physical frame and applies PTE permissions, cache policy, and addressing type.

A *PFN* is a physical frame number. A leaf PTE combines its PFN with linear-address bits 11..0 to form a byte address when *AT*=0 or a slot address when *AT*=1.

2.3 Control Flow

A *near control transfer* is a *CALL*, *RET*, *JMP*, or conditional branch that changes only PC within the current code-segment context.

A *segment-changing control transfer* updates CS and PC in one architectural commit. Long jumps, long calls, and long returns belong to this category.

2.4 Tracing Terms

The *TRACE instruction* records its immediate marker and current instruction boundary in the implementation trace stream when that stream is enabled. A *trace unit* is the execution-completion unit sampled by *STATUS.TF.DEBUG_TRACE* is the architectural event delivered after successful completion of a TF-selected trace unit. These names identify the marker instruction, execution boundary, and event, respectively.

2.5 Architectural State

A *general-purpose register* is one of the sixteen 64-bit *Rn* integer registers, *R0* through *R15*. Ordinary integer register fields use four bits to select *R0* through *R15*.

An *Rn register* is a general-purpose integer register selected by a four-bit *Rn* field. It may hold integers, addresses, counters, selectors, or temporary data according to the instruction form.

The *stack pointer register* is *SP*. It lies outside the four-bit *Rn* namespace and uses *SS* as the fixed segment for *SP*-relative addressing.

A *segment register operand* is a 64-bit segment register exposed through the *S* operand class. The *SREG* selector table maps its eight encodings to *DS*, *SS*, and *GS0* through *GS5*. *RDSEG CS* and *PUSH CS* read the current *CS* image through their fixed *CS* forms.

A *state register* is named architectural state such as *FLAGS* or *STATUS*. *RDFLAGS*, *WRFLAGS*, *RDSTATUS*, and *WRSTATUS* access these registers.

A *control register* is a privileged register exposed through the *CR* operand class, such as *PTCR*, *ASCR*, *ECR*, *SPC*, *SCS*, and *SDS*.

2.6 Compatibility

Reserved means that software must not depend on the field's value or behavior. *Must be zero* requires software to write zero, and *read-as-zero* means reads return zero while the field remains reserved. *Ignored* means hardware accepts the field without using it for the current operation. *Preserved* requires software to retain the previous value when rewriting the containing object.

Software-defined state is reserved for software use; hardware interpretation is limited to behavior explicitly defined by the containing structure. *Implementation-defined* behavior is selected by the implementation and published through *CPUID*, platform documentation, or firmware. *Illegal* use of an opcode, effective-address form, selector, or operation raises the exception named by the defining rule.

2.7 Compatibility Contract

This section defines the default compatibility contract for reserved fields and optional features. A later section may narrow one of these defaults for a specific structure, but it must say so explicitly. This manual treats reserved and unallocated encodings as one architectural category: software must not depend on their value, decode result, or behavior.

2.8 Reserved Fields

Architected registers and architectural memory structures use these defaults unless their defining section gives a narrower rule.

Software-defined fields are not reserved fields. Software may store values in them, and hardware ignores them except where the containing structure explicitly says otherwise.

Table 2-1. Reserved Field Defaults

Field Class	Read Rule	Write or Use Rule	Fault
Architected register reserved bits	zero	must be zero	—
Control-register reserved bits	zero	must be zero	INVALID_CONTROL_STATE
Reserved selector values	—	invalid selector	INVALID_CONTROL_STATE
Consumed reserved PTE bits	—	invalid translation input	PAGE_FAULT
Reserved event-code classes or IDs	—	must not be generated by the base profile	—
Reserved architectural event-frame bits	—	must be zero	INVALID_CONTROL_STATE on ERET

2.9 Reserved Encodings

Reserved instruction opcodes, reserved extension opcodes, reserved effective-address forms, and optional instruction-group encodings whose CPUID feature bit is clear raise `ILLEGAL_INSTRUCTION` during decode.

Noncanonical encodings are invalid unless an instruction description explicitly defines an alias or priority rule. Assemblers emit canonical encodings by default; disassemblers prefer canonical spellings.

2.10 CPUID Compatibility

CPUID is the compatibility boundary for optional architectural behavior. Software must not use an optional instruction group, optional register state, or optional state image unless the corresponding CPUID bit or leaf reports support.

Unknown selectors return zero. Software ignores reserved CPUID result bits. CPUID is unprivileged. For a logical processor, CPUID results remain stable until that logical processor is reset.

3 CONFORMANCE

3.1 Conformance Claim

A Bedrock implementation conformance claim identifies the architecture revision reported by CPUID and the CPUID feature set for the logical processor to which the claim applies. For that revision and discovered feature set, the implementation accepts and executes every architecturally valid instruction record according to its encoding, state, event, memory, and instruction-specific rules. The same claim covers the architected discovery results, save images, event frames, and reset state defined for that processor.

3.2 Normative Material and Specificity

Architecture prose, ordered steps, generated architecture tables, instruction encoding diagrams, and the Operation and Detailed Semantics fields of instruction entries are normative unless a section explicitly identifies material as a summary, example, or validation status.

The following domain rules resolve different levels of specificity:

1. Instruction bytes, field values, lengths, and decoding validity follow the concrete instruction form, its field constraints, and the Instruction Encoding and Effective Addressing chapters.
2. Execution of a decoded instruction follows its Operation and Detailed Semantics. An instruction-specific rule narrows the common execution, operand, flag, memory, or event rule that it names.
3. Common architectural behavior follows the chapter that defines that behavior. A structure-specific rule narrows the defaults in Terminology and Compatibility.
4. Summary tables, examples, and validation-status files provide navigation or verification context and do not replace a normative rule.

Two normative statements at the same specificity are required to agree. A discrepancy at that level is a specification defect and is resolved by correcting the normative sources and their derived tables together.

3.3 Implementation-Defined Disclosures

An implementation-defined selection is stable for the implementation revision named by CPUID and is published through the channel named by its defining rule. The publication identifies the selected value or behavior and the processor revisions to which it applies.

Table 3-1. Implementation-Defined Disclosure Register

Item	Defining rule	Publication
cpuid_higher_classes	CPUID Feature Discovery, Base Identity and Leaf Directory	CPUID class and leaf directories plus implementation documentation for the class payload
performance_counter_ids	CPUID Performance-Counter Discovery and RDPMC	CPUID PERFORMANCE_COUNTERS presence results plus implementation documentation naming each counter event

Table 3-1 (continued)

Item	Defining rule	Publication
event_aux_syndrome	Machine-check EVENT_AUX payload	platform documentation or firmware interface documentation for the event source
fptransa_approximation	FPTRANSA ULP Error Model and CPUID accuracy contracts	CPUID accuracy-contract discovery plus implementation documentation for the deterministic approximation

4 PROGRAMMING MODEL

4.1 Register Model

The integer programming model exposes sixteen 64-bit Rn registers, R0 through R15. Instruction forms use these registers for integer values, addresses, counters, selectors, and temporary data.

SP and PC are special architectural registers outside the four-bit Rn namespace. SP-relative memory forms use SS by default; PC-relative memory forms use CS by default.

CS, DS, SS, and GS0 through GS5 are 64-bit architectural segment registers. Segment-qualified EA forms select them explicitly, while SP and PC forms omit the segment field because their segment association is fixed.

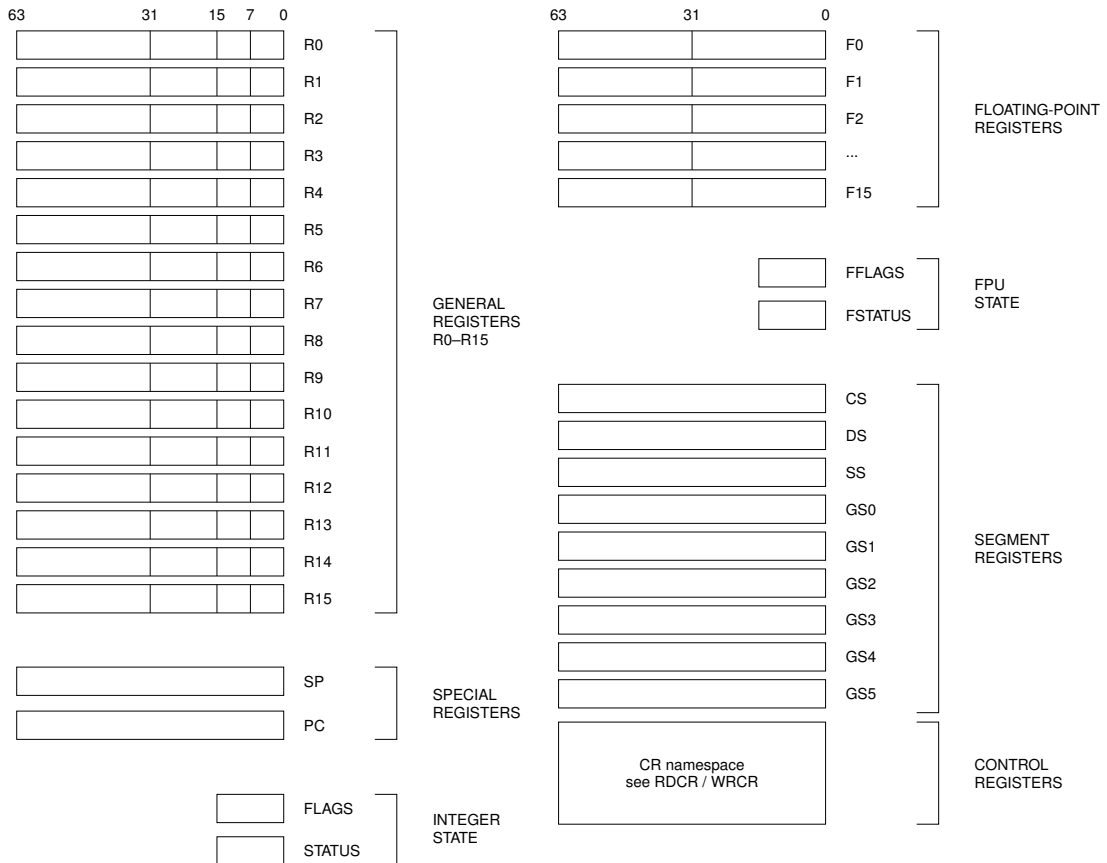
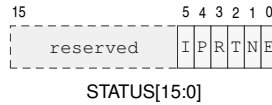


Figure 4-1. User Programming Model

4.2 FLAGS and STATUS Registers

FLAGS and STATUS are accessed through dedicated read/write instructions. Reserved bits read as zero and must remain zero.

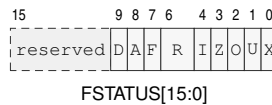
**Figure 4-2. FLAGS Register Format****Figure 4-3. STATUS Register Format**

In the diagram, I, P, R, T, N, and E abbreviate IE, PM, RF, TF, NI, and EA.

STATUS.EA is hardware-managed architectural event-active state. WRSTATUS must preserve its current value; only event entry and ERET change it. The remaining writable STATUS fields retain their instruction-defined validation rules.

4.3 FFLAGS and FSTATUS Registers

FFLAGS holds accrued IEEE-754 exception flags. FSTATUS holds the matching trap enables, rounding mode, and optional non-default floating-point modes. Both registers are present only with the floating-point extension and reset to zero.

**Figure 4-4. FFLAGS Register Format****Figure 4-5. FSTATUS Register Format**

In the diagrams, I, Z, O, U, and X denote the NV, DZ, 0F, UF, and NX flag or enable bits. In FSTATUS, D, A, F, and R denote DN, DAZ, FTZ, and RM.

Table 4-1. Floating-Point State Fields

Field	Bits	Meaning
NV . . NX	4 . . 0	accrued invalid, divide-by-zero, overflow, underflow, and inexact flags
NV_EN . . NX_EN	4 . . 0	trap enables corresponding to FFLAGS.NV through FFLAGS.NX
RM	6 . . 5	0 nearest-even; 1 toward zero; 2 toward negative; 3 toward positive
FTZ	7	flush tiny results to signed zero
DAZ	8	treat subnormal inputs as signed zero
DN	9	produce the architectural default NaN
reserved	15 . . 10	must be zero

4.4 Floating-Point Register Model

The floating-point extension exposes F0 through F15 as 64-bit registers when the floating-point instruction group is implemented.

FFLAGS holds accrued floating-point exceptions. FSTATUS holds floating-point trap enables, rounding mode, and mode controls. Their precise bit assignments are defined by the floating-point instruction set and compatibility state.

4.5 Segment Registers

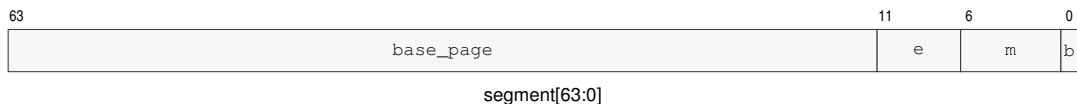


Figure 4-6. Segment Register Format

4.6 Segment Register Operand Class

Instructions that take an explicit segment-register operand use the three-bit SREG encoding below.

SP-relative EA forms do not carry this field and use SS. PC-relative EA forms do not carry this field and use CS.

SREG selects DS, SS, and GS0 through GS5. Instruction fetch and fixed PC-relative addressing select CS implicitly. RDSEG CS and PUSH CS read the current CS image. Segment-changing control transfers update CS and PC together.

Table 4-2. Segment Register Operand Encoding

Segment	Bits	Role	Use
DS	000	data	default data segment
SS	001	stack	stack segment; fixed for SP-relative forms
GS0	010	general	general segment register 0
GS1	011	general	general segment register 1
GS2	100	general	general segment register 2
GS3	101	general	general segment register 3
GS4	110	general	general segment register 4
GS5	111	general	general segment register 5

4.7 Control Registers

RDCR and WRCR use a 16-bit selector and transfer a 64-bit register image. Both instructions are supervisor-only. Selectors not listed below raise `INVALID_CONTROL_STATE`. Reserved register bits read as zero, and a write with a nonzero reserved bit raises `INVALID_CONTROL_STATE` without changing the register. The entry, stack, and user-return registers are per-logical-processor architectural state.

Table 4-3. Control-Register Selectors

Selector	Register	Use
0x0000	PTCR	page-table root and translation control
0x0001	ASCR	address-space identifier control
0x0002	ECR	architectural event-delivery control
0x0100	SPC	SYSCALL entry program counter
0x0101	SCS	SYSCALL entry code-segment image
0x0102	SDS	SYSCALL entry data-segment image
0x0108	URPC	user-return program counter
0x0109	URSP	user-return stack pointer
0x010a	URCS	user-return code-segment image
0x010b	URDS	user-return data-segment image
0x010c	URSS	user-return stack-segment image
0x010d	URCTL	user-return FLAGS, STATUS, and valid state
0x0110	EPC	common architectural event-entry PC
0x0111	ECS	architectural event-entry code-segment image
0x0112	EDS	architectural event-entry data-segment image
0x0200	SSS	supervisor-call working-stack segment image
0x0201	SSP	supervisor-call working-stack top
0x0210	ISS	maskable-interrupt stack-segment image
0x0211	ISP	maskable-interrupt stack top
0x0220	FSS	fault/exception stack-segment image
0x0221	FSP	fault/exception stack top
0x0230	DSS	double-fault stack-segment image
0x0231	DSP	double-fault stack top
0x1000	B00TPC	warm-reset target supplied by the platform at cold reset
0x1001	B00TCFG	opaque platform boot configuration preserved by warm reset
0x1100	PMC	performance-monitor enable

Table 4-4. WRCR Selector Rules

Register	Writable Image	Validation	Commit and Side Effect
PTCR	root_page, PABITS_SEL, LA57, and PE; reserved bits must be zero	PABITS_SEL is 0 or 1 and the selected root frame fits the selected physical-address width	apply the translation-cache transition selected by the changed fields
ASCR	ASID and AE; reserved bits must be zero	the complete ASCR image is validated before commit	apply the translation-cache transition selected by the changed fields

Table 4-4 (continued)

Register	Writable Image	Validation	Commit and Side Effect
ECR	MAX_EDEPTH and V; NMI_P is hardware managed and the supplied value is ignored	setting V validates EPC, ECS, EDS, and all configured event stack pairs	replace the complete writable image atomically
SPC, EPC	complete 64-bit entry address	16-byte aligned, canonical, and executable under the matching code-segment image	replace the selected register atomically
SCS, SDS, ECS, EDS, SSS, ISS, FSS, DSS	complete 64-bit segment image	apply the architectural segment-image validity rule	replace the selected register atomically
SSP, ISP, FSP, DSP	complete 64-bit configured stack top	64-byte aligned	replace the selected register atomically
URPC, URSP, URCS, URDS, URSS	complete 64-bit saved user-return value	defer complete-bank validation until URCTL.V is set or SYSRET executes	replace the selected register atomically
URCTL	V, STATUS, and FLAGS; reserved bits must be zero	setting V validates the complete user-return bank and requires saved PM and EA to be zero	replace URCTL atomically after complete-bank validation
BOOTPC	complete 64-bit warm-reset target	the value is used as the exact PC after warm RESET	replace BOOTPC atomically
BOOTCFG	complete opaque 64-bit platform boot configuration	no image transformation	replace BOOTCFG atomically
PMC	EN; reserved bits must be zero	EN is Boolean	replace PMC atomically



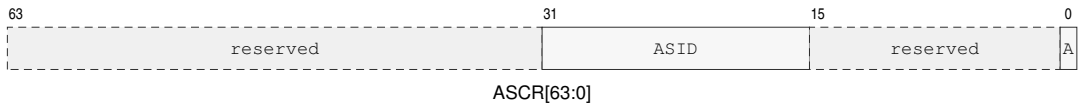
PTCR[63:0]

PTCR Format

In the diagram, PS, L, and P abbreviate PABITS_SEL, LA57, and PE.

Table 4-5. PTCR Fields

Field	Bits	Meaning
root_page	63..12	physical frame number of the page-table root
PABITS_SEL	11..8	0 selects 48 physical bits; 1 selects 56; 2..15 are invalid
LA57	7	zero selects four levels; one selects five levels
reserved	6..1	must be zero
PE	0	page-table translation enable

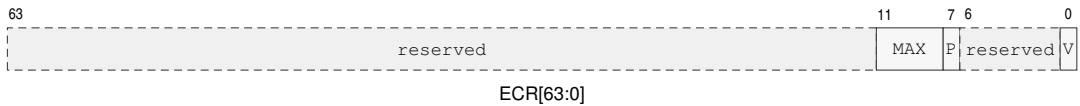


ASCR Format

A abbreviates AE in the diagram.

Table 4-6. ASCR Fields

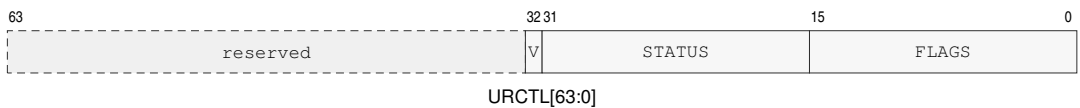
Field	Bits	Meaning
reserved	63..32	must be zero
ASID	31..16	current address-space identifier
reserved	15..1	must be zero
AE	0	address-space identifier matching enable



ECR Format

Table 4-7. ECR Fields

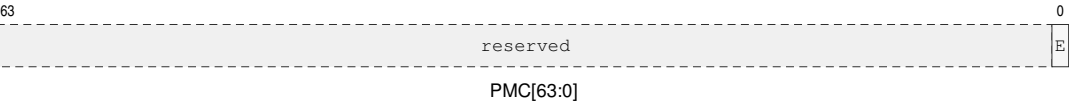
Field	Bits	Meaning
reserved	63..12	must be zero
MAX_EDEPTH	11..8	maximum depth at which a maskable interrupt may be admitted
NMI_P	7	hardware-managed coalesced pending-NMI state while entry is invalid or NI is set; WRCR ignores the supplied bit
reserved	6..1	must be zero
V	0	architectural event-entry state is valid



URCTL Format

Table 4-8. URCTL Fields

Field	Bits	Meaning
reserved	63 . . 33	must be zero
V	32	complete user-return bank contains a valid return context
STATUS	31 . . 16	saved user STATUS image
FLAGS	15 . . 0	saved user FLAGS image



PMC Format

E abbreviates EN in the diagram.

Table 4-9. PMC Fields

Field	Bits	Meaning
reserved	63 . . 1	must be zero
EN	0	enable performance-counter increments when set

SPC and EPC are exact entry addresses, must be 16-byte aligned, and must be canonical and executable under SCS and ECS respectively. SSP, ISP, FSP, and DSP are 64-byte-aligned configured stack tops and are not modified by entry or return. SCS, SDS, ECS, EDS, SSS, ISS, FSS, and DSS contain validated segment images. A write that establishes URCTL.V validates a user STATUS image with PM, EA, and reserved bits clear; SYSRET revalidates the complete bank. BOOTPC and BOOTCFG are initialized by the platform during cold reset and preserved by RESET.

5 DATA FORMATS

5.1 Integer Data Sizes

Integer operands use size suffixes to select the low-order subfield of an Rn register and the number of bytes transferred by memory operands.

Byte, word, and long operations use the low-order subfield of the selected register. Q-sized operations use the full 64-bit register.

An Rn destination write smaller than Q replaces only the selected low-order subfield; bits above the operation size are preserved. Q-sized Rn writes replace all 64 bits. Memory destination writes transfer only the selected number of bytes.

An instruction may explicitly define a full-register write, zero-extension, sign-extension, or another upper-bit rule. Such instruction-specific rules override the default subfield write rule.

Table 5-1. Scalar Data Sizes

Code	Suffix	Bits	Bytes	Name
B	.B	8	1	Byte
W	.W	16	2	Word
L	.L	32	4	Long
Q	.Q	64	8	Quad
S	.S	32	4	Single
D	.D	64	8	Double

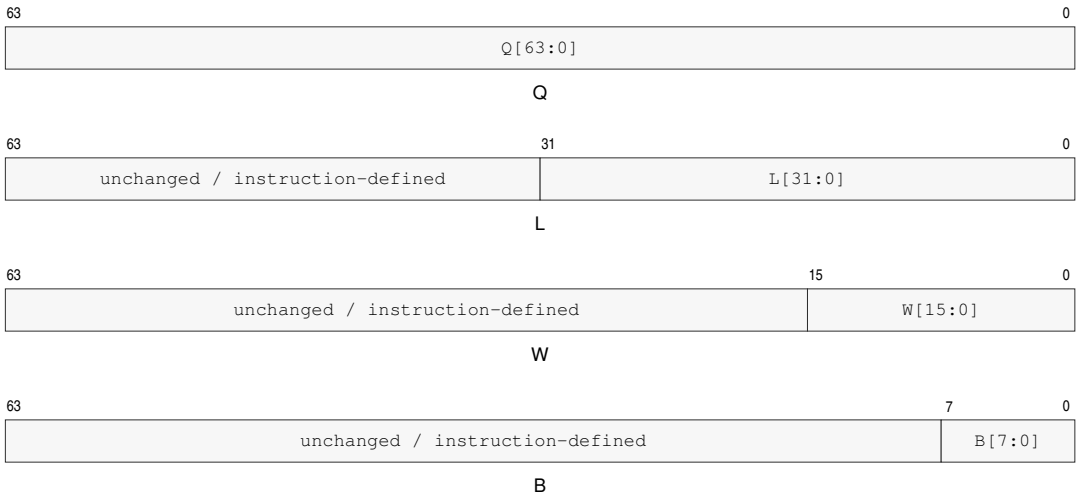


Figure 5-1. Integer Register Operand Subfields

5.2 Little-Endian Memory Organization

Integer data, immediates, displacements, and absolute address payloads are little-endian. Multi-byte numeric payloads are stored least significant byte first. The decoder consumes instruction-header fields byte by byte in instruction-stream order.

For an N-byte integer value stored at byte address A, byte A contains bits 7..0, byte A+1 contains bits 15..8, and so on.

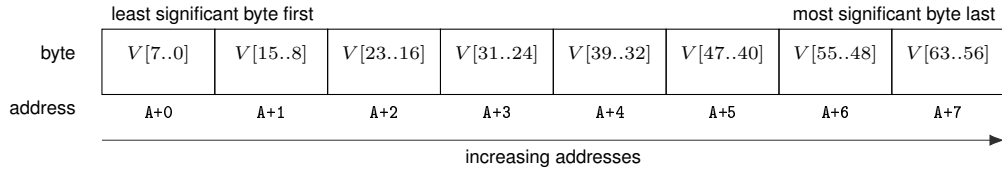


Figure 5-2. Little-Endian Byte Order for a 64-Bit Value

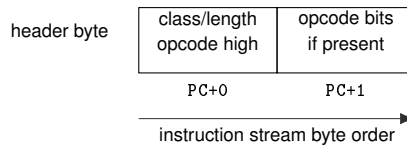


Figure 5-3. Instruction Start Bytes

5.3 Floating-Point Data Formats

Floating-point scalar formats are little-endian in memory as well. S is IEEE-754 binary32 with 24 bits of significand precision, minimum normal exponent -126 , and minimum subnormal quantum 2^{-149} . D is IEEE-754 binary64 with 53 bits of significand precision, minimum normal exponent -1022 , and minimum subnormal quantum 2^{-1074} .

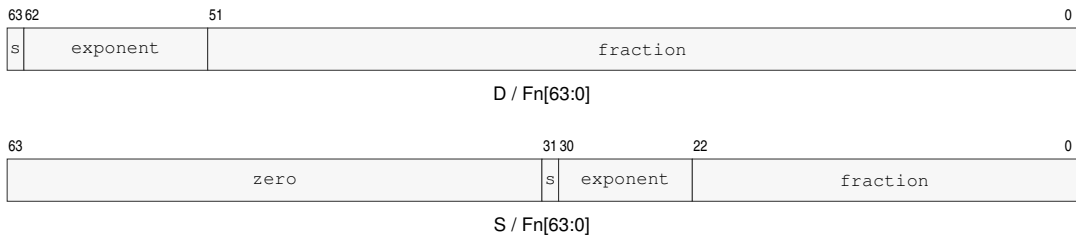


Figure 5-4. Floating-Point Register Encodings

For a NaN, the most significant fraction bit is the quiet bit: bit 22 for S and bit 51 for D. s denotes the sign bit. An S result occupies Fn bits 31..0 and writes zero to Fn bits 63..32.

The bit-level floating-point encoding, NaN policy, exception flags, and rounding behavior are defined by the floating-point instruction and state descriptions; their memory byte order follows the same least-significant-byte-first rule as integer data.

PART II

BEDROCK ARCHITECTURE

Encoding, Addressing, and Execution

6 INSTRUCTION ENCODING

6.1 Instruction Stream and Record Boundary

The byte addressed by PC is byte 0 of the current instruction. Instruction decoding determines the complete instruction length before operand evaluation begins.

Any remaining bytes selected by the instruction length belong to the opcode payload, EA descriptors, immediates, displacements, or instruction-specific payloads. The detailed length and opcode-class truth tables are in this chapter.

The instruction boundary is fixed by byte 0 for extrashort and short forms, and by the first two bytes for extended forms, before operand evaluation begins. An encoded length that cannot contain the selected form raises `ILLEGAL_INSTRUCTION` before any architectural state is changed.

The encoded length must contain every required opcode byte, descriptor, immediate, displacement, bitmap, and instruction-specific payload. For an extended instruction, every trailing byte after the selected concrete form is uninterpreted padding; every byte value is valid. Padding never extends an operand, descriptor, or opcode.

Before decode validation or operand evaluation, instruction fetch and permission checks cover the complete encoded record, including padding. An inaccessible padding byte therefore reports the applicable instruction-fetch fault. An undersized record raises `ILLEGAL_INSTRUCTION.INSUFFICIENT_LENGTH` before architectural state changes.

6.2 Instruction Header Formats

Instruction framing is byte-oriented. Byte 0 alone identifies extrashort and short instructions; extended instructions use byte 0 and byte 1 to determine both total instruction length and opcode class.

Extended byte0[5:2] is L, and the total instruction length is 3+L bytes. The opcode stream begins at byte0[1:0], continues through byte1, and then consumes later bytes as required by the opcode class.

An extended instruction is invalid if its encoded total length is shorter than the opcode class minimum: 3 bytes for medium, 4 bytes for long, and 5 bytes for extralong.

`LEN n, <instruction>` requests an extended record whose total size is n bytes, where $3 \leq n \leq 18$. The requested size must cover the concrete form. Without `LEN`, the assembler emits the shortest valid record. It fills requested trailing padding with zero. A disassembler emits `LEN n`, for an overlong record so reassembly preserves the length; padding byte values are not preserved. Extrashort and short forms retain their fixed record lengths.

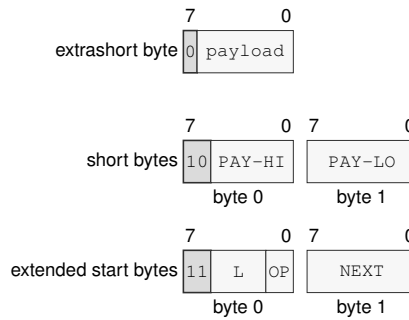
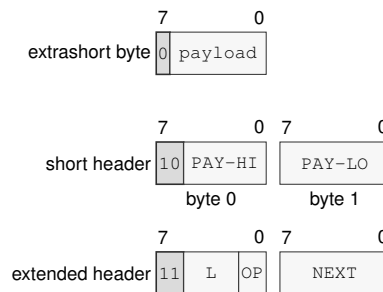
**Figure 6-1. Instruction Header Byte Formats**

Figure 6-1 annotations are read from high to low bit. In byte 0, bit 7 distinguishes extrashort framing; bits 7..6 select short (10) or extended (11) framing. For short instructions, bits 5..2 and 1..0 continue the payload. For extended instructions, bits 5..2 are *L*, bits 1..0 begin the opcode stream, and byte 1 continues that stream. Thus *L* encodes the total length as $3 + L$ bytes directly in Figure 6-1.

**Figure 6-2. Instruction Record Components**

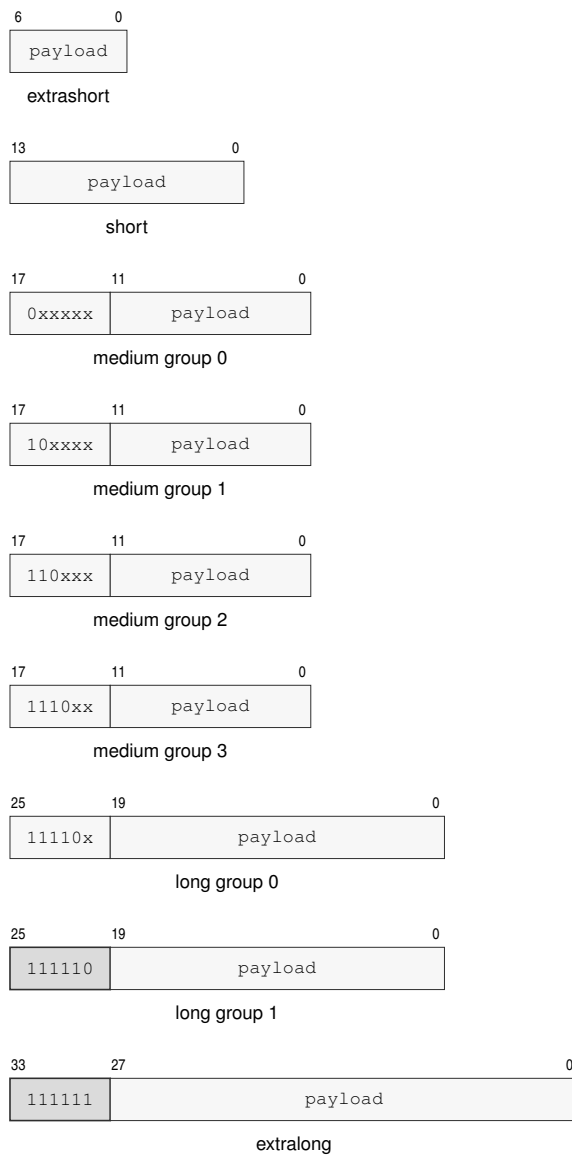


Figure 6-3. Opcode Payload Namespaces

Table 6-1. Instruction Length Truth Table

Byte 0 Pattern	Byte 1 Pattern	Bytes
0xxxxxxx	—	1
10xxxxxx	xxxxxxx	2
11000000	bbbbxxxx	3
11000100	bbbbxxxx	4
11001000	bbbbxxxx	5

Table 6-1 (continued)

Byte 0 Pattern	Byte 1 Pattern	Bytes
11001100	bbbbxxxx	6
11010000	bbbbxxxx	7
11010100	bbbbxxxx	8
11011000	bbbbxxxx	9
11011100	bbbbxxxx	10
11100000	bbbbxxxx	11
11100100	bbbbxxxx	12
11101000	bbbbxxxx	13
11101100	bbbbxxxx	14
11110000	bbbbxxxx	15
11110100	bbbbxxxx	16
11111000	bbbbxxxx	17
11111100	bbbbxxxx	18

Table 6-2. Encoding Class Summary

Class	Payload Bits	Header or Opcode Selector	Opcode Bytes	Validity
extrashort	7	0xxxxxxx	1	exactly 1 byte
short	14	10xxxxxxx	2	exactly 2 bytes
medium	18	0xxxxx, 10xxxx, 110xxx, 1110xx	3	any extended instruction length
long	26	11110x, 111110	4	instruction length at least 4 bytes
extralong	34	111111	5	instruction length at least 5 bytes

6.3 Instruction Payload Ordering

The instruction header occupies one byte for extrashort forms, two bytes for short forms, and at least two start bytes for extended forms. If a descriptor, immediate, displacement, or bitmap follows, those payload bytes occupy increasing addresses in decode order.

Signed displacement and immediate payloads use two's-complement representation at their declared width before any sign extension performed by the consuming instruction or EA form.

Table 6-3. Immediate Operand Interpretation

Operand	Bits	Value	Range	Extension	Applies When
pair_id	3	identifier	0..7	none	PUSH/POPP canonical register-pair selector
pt_level	3	enumeration	1..5	none	PTQUERY page-table level selector
flags_bitmap	4	bitmap	0..15	none	SETF/CLRF integer FLAGS masks
imm6	6	unsigned	0..63	zero-extend	integer forms with an explicit B/W/L/Q operation size
imm7	7	unsigned	0..127	zero-extend	EXTRACT register-pair forms

PUSHP and POPP interpret the 3-bit `pair_id` as a canonical register-pair index, so every encoding from 0..7 is valid. PTQUERY uses a 3-bit `pt_level`; assigned levels are 1..5, while encodings 0, 6, and 7 are reserved. SETF and CLRf interpret the 4-bit `flags_bitmap` range 0..15 as the low FLAGS nibble: bit 3 selects Z, bit 2 selects N, bit 1 selects C, and bit 0 selects V.

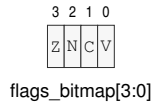


Figure 6-4. SETF and CLRf Flag Bitmap

An 6-bit `imm6` value in 0..63 is zero-extended to the selected operation size before use when that size is wider than 6 bits. EXTRACT uses the 7-bit `imm7` range 0..127 as the offset into its concatenated register pair.

6.4 Representative Encodings

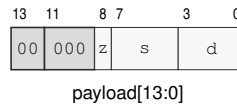


Figure 6-5. Short Encoding Layout for MOV.X(z:L/Q) Rn(s), Rn(d)

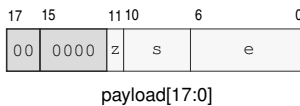


Figure 6-6. Medium Encoding Layout for MOV.X(z:B/W) Rn(s), <ea>(e)

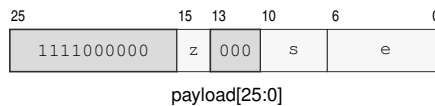


Figure 6-7. Long Encoding Layout for ADC.X(z:B/W/L/Q) Rn(s), <ea>(e)

7 EFFECTIVE ADDRESSING MODES

An effective-address field is an operand descriptor with an instruction-defined role and interpretation width. A value role reads or writes the selected register, immediate, or memory value. An address role uses a register or immediate as an address value and uses a memory-form EA as the calculated address without an initial data read. A control-target role uses a register or immediate value directly and reads an interpretation-width target from a memory form. Memory forms continue into segment pre-translation and paging.

The seven-bit compact EA field is embedded in the opcode payload. Any displacement, absolute address, immediate, or EXT0 descriptor bytes are appended as EA payload bytes. When an instruction contains multiple payload-bearing operands, payloads are consumed in instruction operand order.

Multi-byte scalar EA payloads, such as displacements, absolute addresses, and immediates, are little-endian. Payload names ending in *s* are signed and are sign-extended before use; payload names without *s* are unsigned unless the consuming instruction explicitly says otherwise. The decoder consumes EXT0 descriptor bytes in the displayed stream order, byte 0 followed by byte 1, and bit numbering restarts at 7 for each byte.

Indexed scale is the EA interpretation width declared by the consuming form. B, W, L, and Q widths select scales 1, 2, 4, and 8. A form whose width is `operation_size` uses its selected instruction size. LEA and SEGLEA use their suffix; size-less address and control-target forms use Q.

For every compact or EXT0 PC-based EA, the PC term is the architectural PC naming byte 0 of the current instruction. The PC term remains that instruction-start value throughout operand evaluation.

7.1 Compact EA Addressing Modes

Compact EA forms encode direct Rn and SP operands, common Rn/SP/PC memory operands, absolute memory operands, immediate operands, and the EXT0 escape.

Compact memory forms that do not name SP or PC use the operation default data segment. SP memory forms are fixed to SS, and PC memory forms are fixed to CS.

Register Direct

Rn(r)

SP

EA encoding: 000rrrr selects Rn(r); 1101000 selects SP.

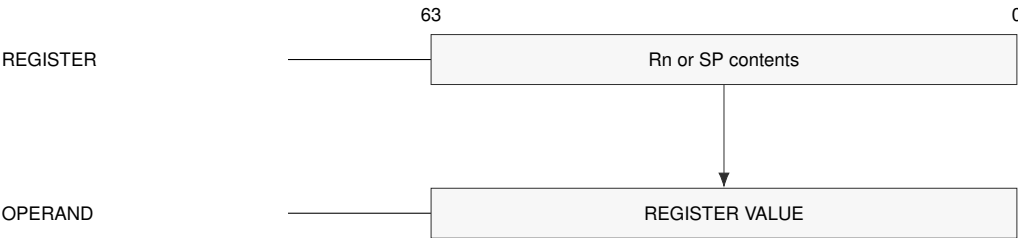
EA type: register operand, not a memory reference.

Generation: The operand value is the selected register value. No effective memory address is generated.

Segment: No segment is selected.

Payload: No appended EA payload bytes.

Update: No auto-update.



Schema — Direct register operand

Rn Memory $[Rn(r)]$ $[Rn(r) + disp8s]$ $[Rn(r) + disp16s]$ $[Rn(r) + disp32s]$ $[Rn(r) + disp64]$

EA encoding: 001rrrr has no displacement; 010rrrr, 011rrrr, 100rrrr, and 101rrrr select disp8s, disp16s, disp32s, and disp64.

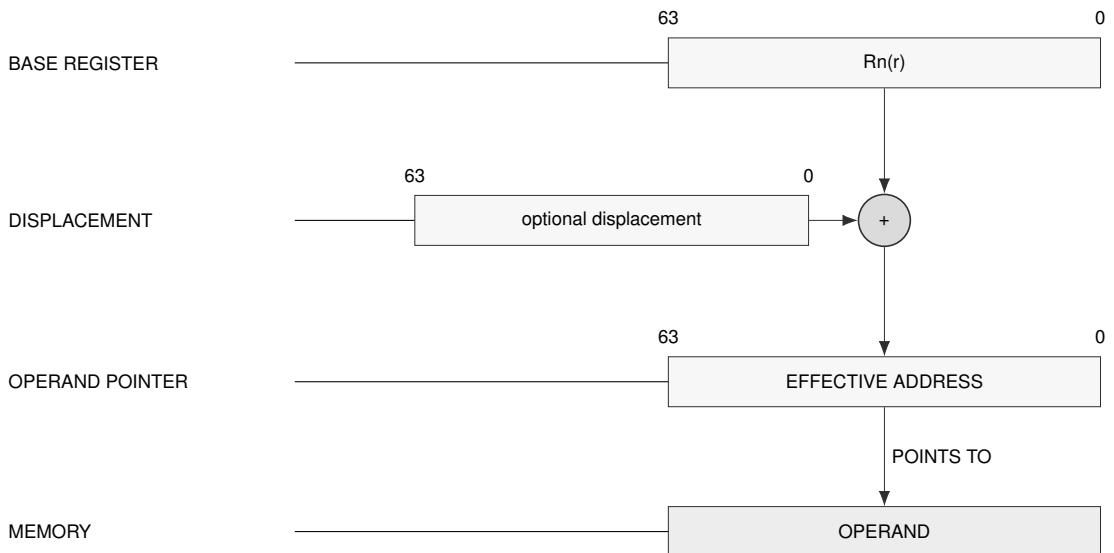
EA type: memory operand.

Generation: offset = temporary $Rn(r)$ + displacement. The no-displacement form uses displacement 0.

Segment: Uses the operation default data segment unless the instruction defines a different default.

Payload: Only displacement forms append payload bytes. Displacement payload is little-endian and has the size named by the EA form.

Update: No auto-update in compact Rn memory forms.



Schema — Rn memory address generation

SP Memory

- [SP]
- [SP + disp8s]
- [SP + disp16s]
- [SP + disp32s]
- [SP + disp64]

EA encoding: 1101001 has no displacement; 1100000, 1100001, 1100010, and 1100011 select disp8s, disp16s, disp32s, and disp64.

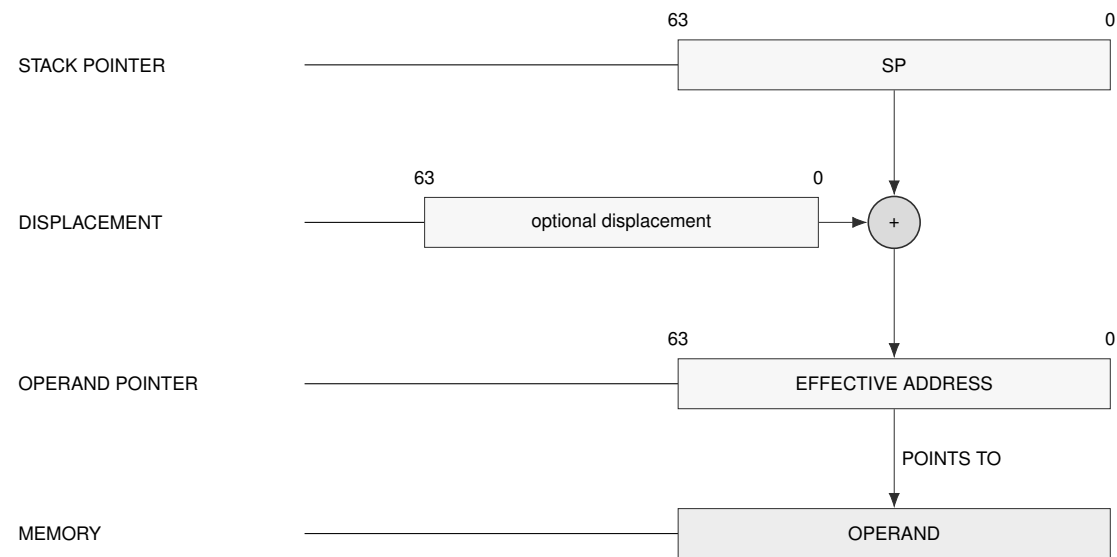
EA type: memory operand.

Generation: offset = temporary SP + displacement. The no-displacement form uses displacement 0.

Segment: Fixed to SS; no segment field is encoded.

Payload: Only displacement forms append payload bytes. Displacement payload is little-endian and has the size named by the EA form.

Update: No auto-update in compact SP memory forms.



Schema — SP memory address generation

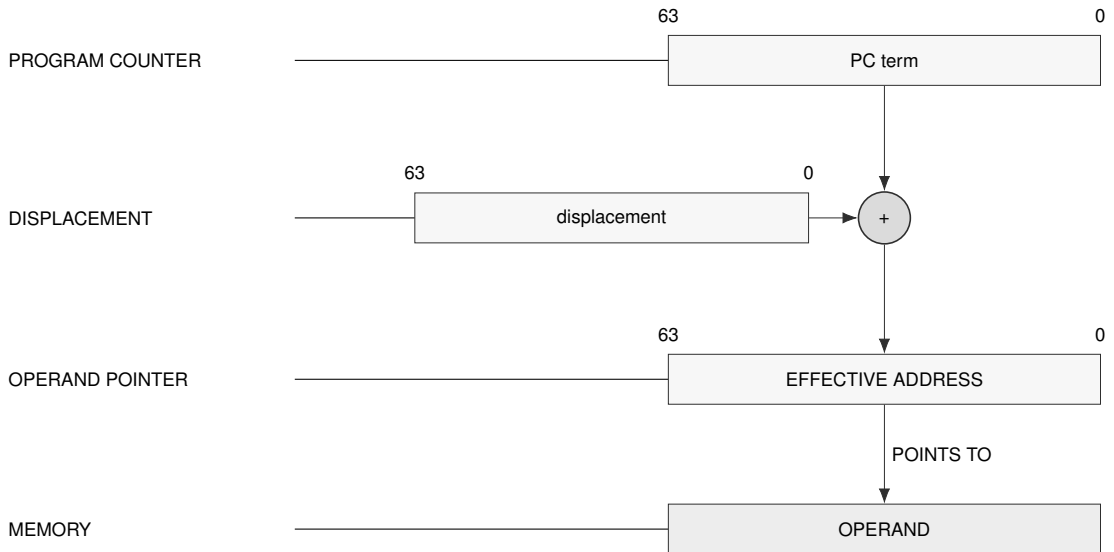
PC Memory

[PC + disp8s]

[PC + disp16s]

[PC + disp32s]

[PC + disp64]

EA encoding: 1100100, 1100101, 1100110, and 1100111 select disp8s, disp16s, disp32s, and disp64.**EA type:** memory operand.**Generation:** offset = instruction-start PC + displacement. PC names byte 0 of the current instruction.**Segment:** Fixed to CS; no segment field is encoded.**Payload:** A displacement payload is always present. Payload is little-endian and has the size named by the EA form.**Update:** No auto-update for PC forms.**Schema — PC-relative address generation**

Absolute Memory
[abs32s]
[abs64]

EA encoding: 1101010 selects abs32s; 1101011 selects abs64.

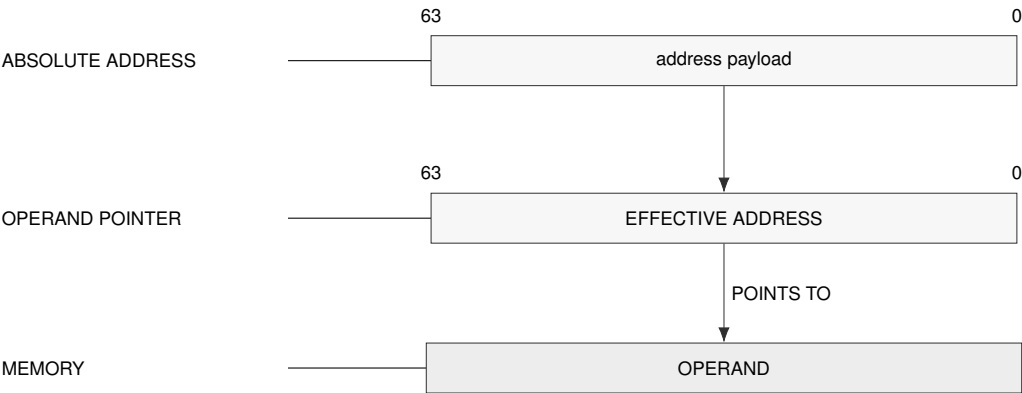
EA type: memory operand.

Generation: offset = sign-extended abs32s or unsigned abs64.

Segment: Uses the operation default data segment unless the instruction defines a different default.

Payload: The absolute address payload is little-endian and has the size named by the EA form.

Update: No auto-update.



Schema — Absolute memory address generation

Immediate
imm8s
imm16s
imm32s
imm64

EA encoding: 1101100, 1101101, 1101110, and 1101111 select imm8s, imm16s, imm32s, and imm64.

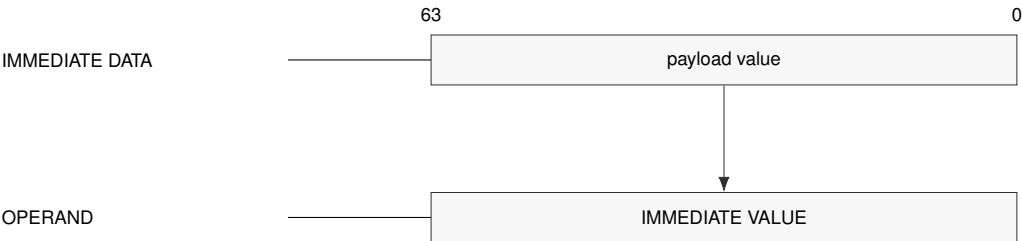
EA type: immediate operand, not a memory reference.

Generation: The operand value is decoded from the immediate payload. Signed payloads are sign-extended according to the consuming instruction's operand rule.

Segment: No segment is selected.

Payload: The immediate payload is little-endian and has the size named by the EA form.

Update: No auto-update. Immediate forms are not valid destinations unless an instruction explicitly defines such a form.



Schema — Immediate operand generation

EXT0 Escape

EXT0

EXT0/disp8s

EXT0/disp16s

EXT0/disp32s

EXT0/disp64

EA encoding: 1110100 selects no displacement; 1110000, 1110001, 1110010, and 1110011 select disp8s, disp16s, disp32s, and disp64.

EA type: memory operand described by the appended EXT0 descriptor.

Generation: The EXT0 descriptor selects segment, base, index, and auto-update behavior. The compact EA escape selects only the displacement size.

Segment: Segment-qualified forms encode SREG SEG(s); SP and PC indexed forms use SS and CS; default base-update forms use the operation default data segment.

Payload: Payload order is EXT0 descriptor byte or bytes first, then the optional displacement payload selected by the compact EA escape.

Update: Only EXT0 forms with ++ or -- update the temporary operand-evaluation image.

Table 7-1. Compact EA Encoding

Bits	Syntax	Class	Memory
000rrrr	Rn(r)	register	no
001rrrr	[Rn(r)]	memory	yes
010rrrr	[Rn(r) + disp8s]	memory	yes
011rrrr	[Rn(r) + disp16s]	memory	yes
100rrrr	[Rn(r) + disp32s]	memory	yes
101rrrr	[Rn(r) + disp64]	memory	yes
1100000	[SP + disp8s]	memory	yes
1100001	[SP + disp16s]	memory	yes
1100010	[SP + disp32s]	memory	yes
1100011	[SP + disp64]	memory	yes
1100100	[PC + disp8s]	memory	yes
1100101	[PC + disp16s]	memory	yes
1100110	[PC + disp32s]	memory	yes
1100111	[PC + disp64]	memory	yes
1101000	SP	register	no
1101001	[SP]	memory	yes
1101010	[abs32s]	memory	yes
1101011	[abs64]	memory	yes
1101100	imm8s	immediate	no
1101101	imm16s	immediate	no
1101110	imm32s	immediate	no
1101111	imm64	immediate	no
1110000	EXT0/disp8s	ext0_escape	-
1110001	EXT0/disp16s	ext0_escape	-
1110010	EXT0/disp32s	ext0_escape	-
1110011	EXT0/disp64	ext0_escape	-
1110100	EXT0	ext0_escape	-

7.2 Extended EA Descriptor

A compact EXT0 EA value selects an extended descriptor and the displacement width. The descriptor bytes then select segment, base, index, zero-base, SP/PC indexed, and auto-update behavior.

Each EXT0 descriptor selects an SREG SEG(s), fixed SS or CS through SP or PC, or the operation default data segment.

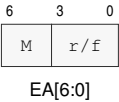


Figure 7-1. Compact Effective-Address Field

M abbreviates mode, and r/f abbreviates register/form.

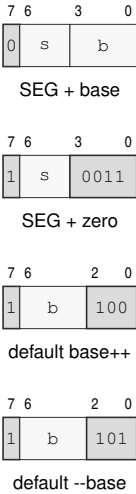
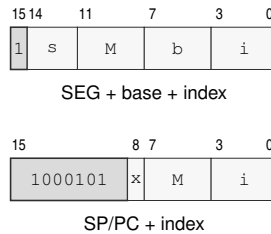


Figure 7-2. EXT0 One-Byte Descriptor Forms

**Figure 7-3. EXT0 Two-Byte Indexed Descriptor Byte Layouts**

M abbreviates mode.

Table 7-2. EA Payload Names

Payload	Bytes	Value	Use
disp8s	1	signed	displacement added to base/index expression
disp16s	2	signed	displacement added to base/index expression
disp32s	4	signed	displacement added to base/index expression
disp64	8	unsigned	displacement added to base/index expression
abs32s	4	signed	absolute address, sign-extended before use
abs64	8	unsigned	absolute address payload
imm8s	1	signed	immediate operand payload
imm16s	2	signed	immediate operand payload
imm32s	4	signed	immediate operand payload
imm64	8	unsigned	immediate operand payload
EXT0 descriptor	1 or 2	encoded	extended EA descriptor; present only for EXT0 escapes

For a compact displacement, absolute address, or immediate, the selected payload follows the EA field at that operand's payload position. EXT0 begins with the compact escape value and one or two descriptor bytes; a displaced EXT0 form places its selected displacement after those descriptor bytes. When an instruction has more than one payload-bearing EA operand, the decoder consumes each complete EA payload sequence in instruction operand order.

7.3 Extended EA Addressing Modes

EXT0 is used when compact EA cannot express the operand: explicit segment selection, indexed addressing, zero-base addressing, SP/PC indexed addressing, or auto-update addressing.

Each EXT0 form selects an encoded SREG SEG(s), fixed SS or CS through SP or PC, or the operation default data segment.

EXT0 Explicit Segment Base

[SEG(s):Rn(b) + displacement]

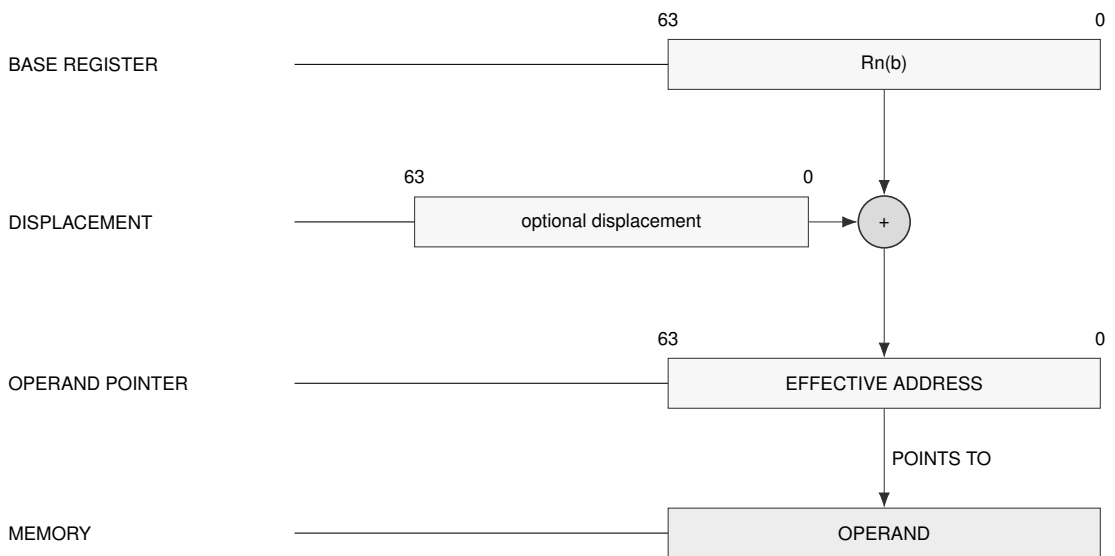
Descriptor: 0sssbbbb, one byte.

Generation: offset = temporary Rn(b) + displacement.

Segment: SEG(s) selects DS, SS, or GS0..GS5.

Payload: One descriptor byte plus the optional displacement selected by the compact EXT0 escape.

Update: No auto-update.



Schema — EXT0 explicit segment base

EXT0 Explicit Segment Indexed

$$[\text{SEG}(s):\text{Rn}(b) + \text{Rn}(i) * \text{scale} + \text{displacement}]$$

$$[\text{SEG}(s):\text{Rn}(b) + \text{Rn}(i)++ * \text{scale} + \text{displacement}]$$

$$[\text{SEG}(s):\text{Rn}(b) + --\text{Rn}(i) * \text{scale} + \text{displacement}]$$

Descriptor: Byte 0 is 1sss0010, 1sss0000, or 1sss0001 for no update, postincrement, or predecrement.

Byte 1 is bbbbiiii.

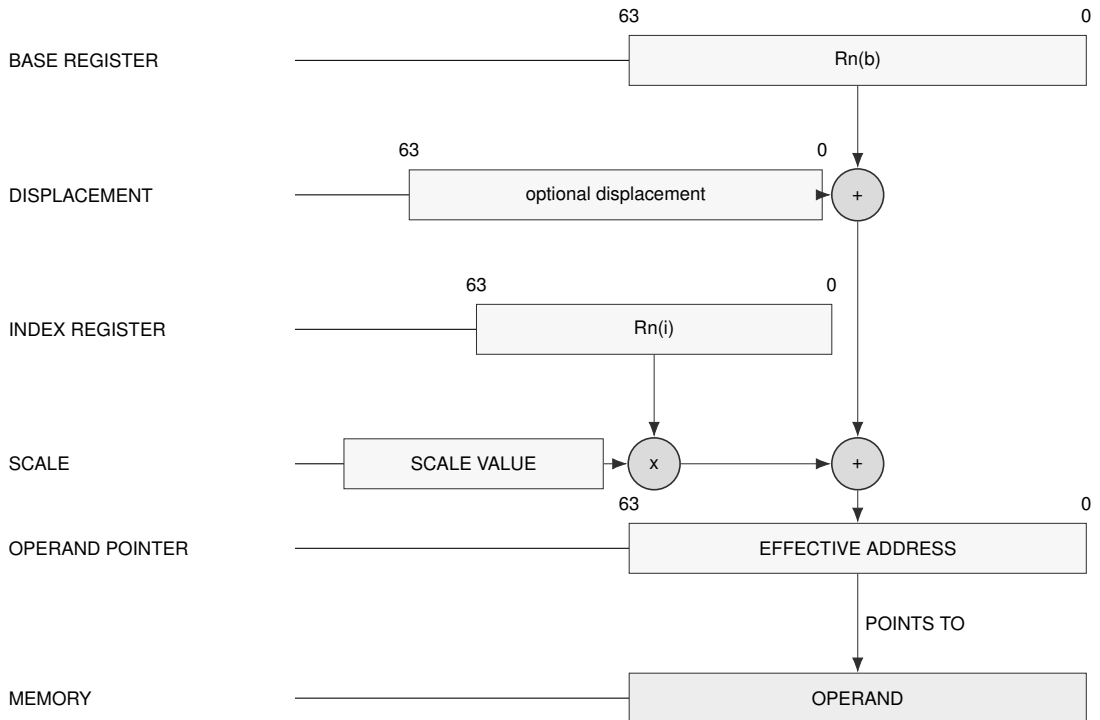
Generation: offset = temporary $\text{Rn}(b) + \text{index_term} * \text{scale} + \text{displacement}$.

Segment: $\text{SEG}(s)$ selects DS, SS, or GS0..GS5.

Scale: Scale is implicit in the consuming instruction. B/W/L/Q normally imply 1/2/4/8.

Payload: Two descriptor bytes plus the optional displacement selected by the compact EXT0 escape.

Update: Index postincrement uses the current temporary $\text{Rn}(i)$, then increments it by one element. Index predecrement decrements temporary $\text{Rn}(i)$ by one element before use.



Schema — EXT0 explicit segment indexed

EXT0 Explicit Segment Zero Base

[SEG(s):0 + displacement]

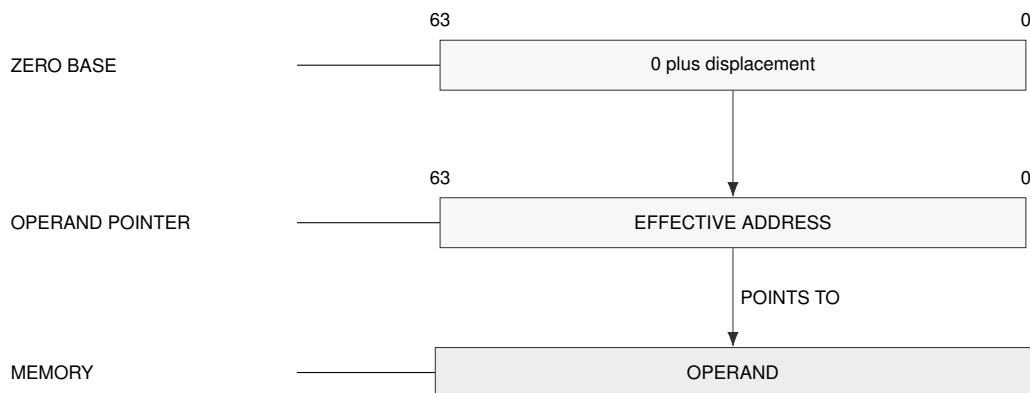
Descriptor: 1sss0011, one byte.

Generation: offset = displacement. Without a displacement payload, offset is 0.

Segment: SEG(s) selects DS, SS, or GS0..GS5.

Payload: One descriptor byte plus the optional displacement selected by the compact EXT0 escape.

Update: No auto-update.



Schema — EXT0 zero-base address generation

EXT0 Explicit Segment Base Auto-Update

[SEG(s):Rn(b)++ + displacement]

[SEG(s):--Rn(b) + displacement]

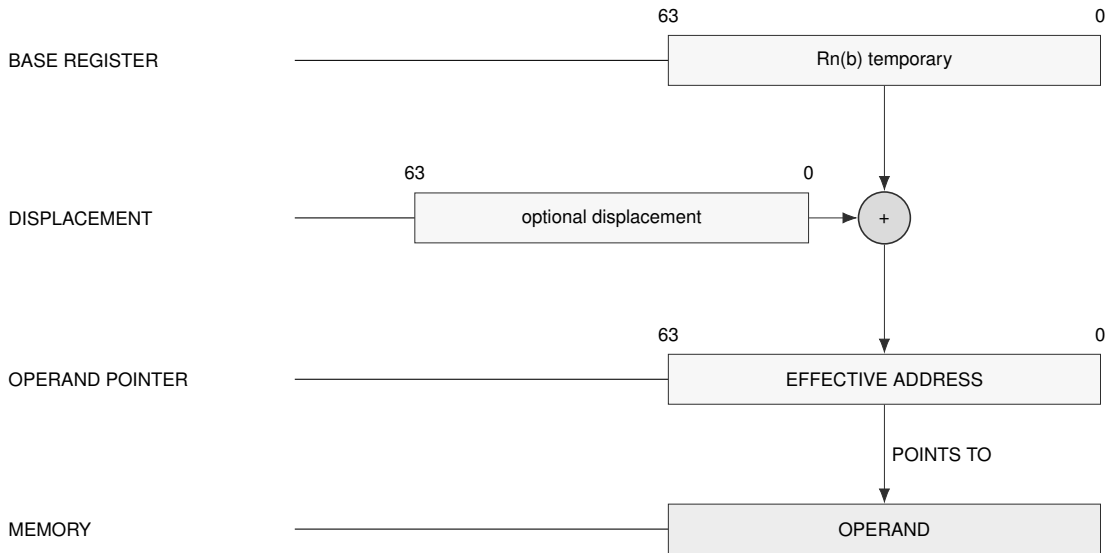
Descriptor: Byte 0 is 1sss1000. Byte 1 is bbbb0000 for base postincrement or bbbb0001 for base predecrement.

Generation: offset = base_term + displacement.

Segment: SEG(s) selects DS, SS, or GS0..GS5.

Payload: Two descriptor bytes plus the optional displacement selected by the compact EXT0 escape.

Update: Base postincrement uses the current temporary Rn(b), then increments it by the memory access size. Base predecrement decrements temporary Rn(b) by the memory access size before use.



Schema — EXT0 base auto-update

EXT0 Explicit Segment Zero-Base Indexed

$$[\text{SEG}(s):0 + \text{Rn}(i) * \text{scale} + \text{displacement}]$$

$$[\text{SEG}(s):0 + \text{Rn}(i)++ * \text{scale} + \text{displacement}]$$

$$[\text{SEG}(s):0 + --\text{Rn}(i) * \text{scale} + \text{displacement}]$$

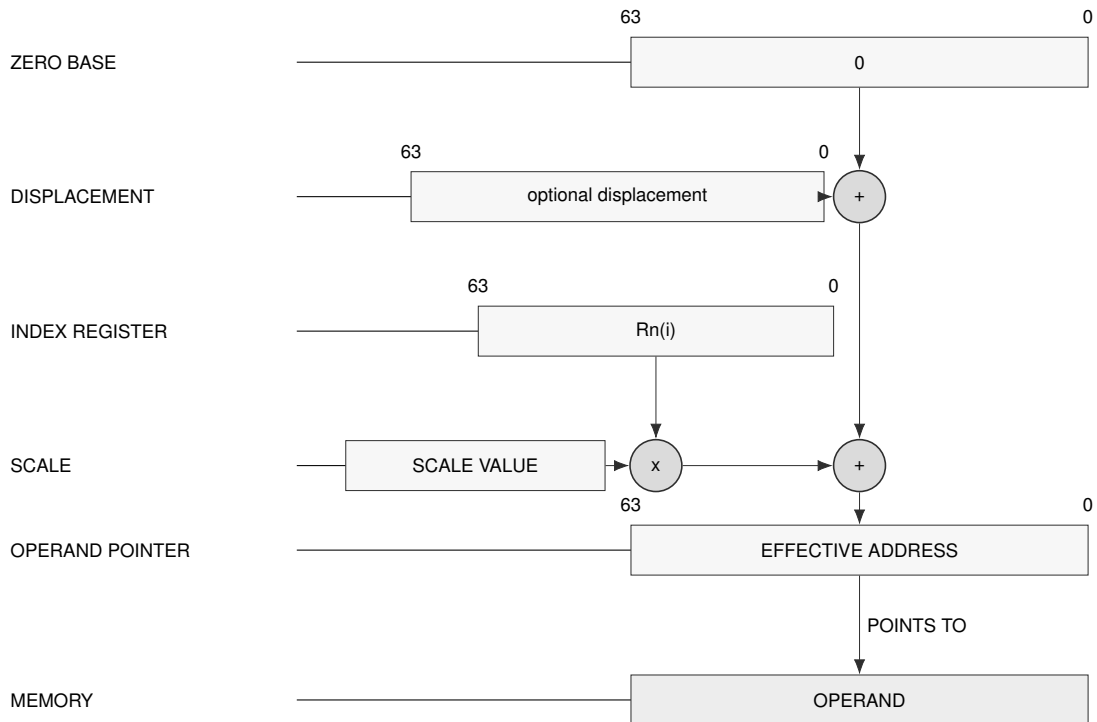
Descriptor: Byte 0 is 1sss1001. Byte 1 is 0010iiii, 0000iiii, or 0001iiii for no update, postincrement, or predecrement.

Generation: $\text{offset} = \text{index_term} * \text{scale} + \text{displacement}$.

Segment: SEG(s) selects DS, SS, or GS0..GS5.

Payload: Two descriptor bytes plus the optional displacement selected by the compact EXT0 escape.

Update: Index update follows the same one-element temporary-image rule as other indexed update forms.



Schema — EXT0 zero-base indexed

EXT0 SP/PC Indexed

$[SP + Rn(i) * scale + displacement]$
 $[SP + Rn(i)++ * scale + displacement]$
 $[SP + --Rn(i) * scale + displacement]$
 $[PC + Rn(i) * scale + displacement]$
 $[PC + Rn(i)++ * scale + displacement]$
 $[PC + --Rn(i) * scale + displacement]$

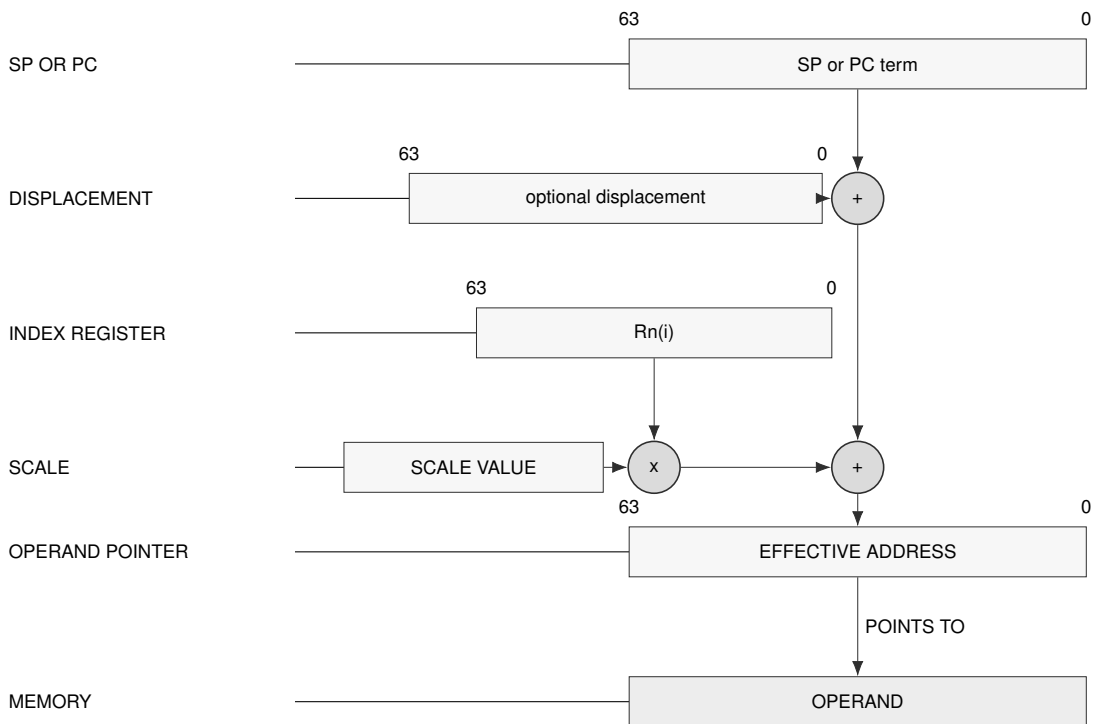
Descriptor: Byte 0 is 10001010 for SP or 10001011 for PC. Byte 1 is 0010iiii, 0000iiii, or 0001iiii for no update, postincrement, or predecrement.

Generation: offset = temporary SP or instruction-start PC + index_term * scale + displacement. A PC term names byte 0 of the current instruction.

Segment: SP forms are fixed to SS. PC forms are fixed to CS.

Payload: Two descriptor bytes plus the optional displacement selected by the compact EXT0 escape.

Update: Auto-update applies to the Rn(i) index register.

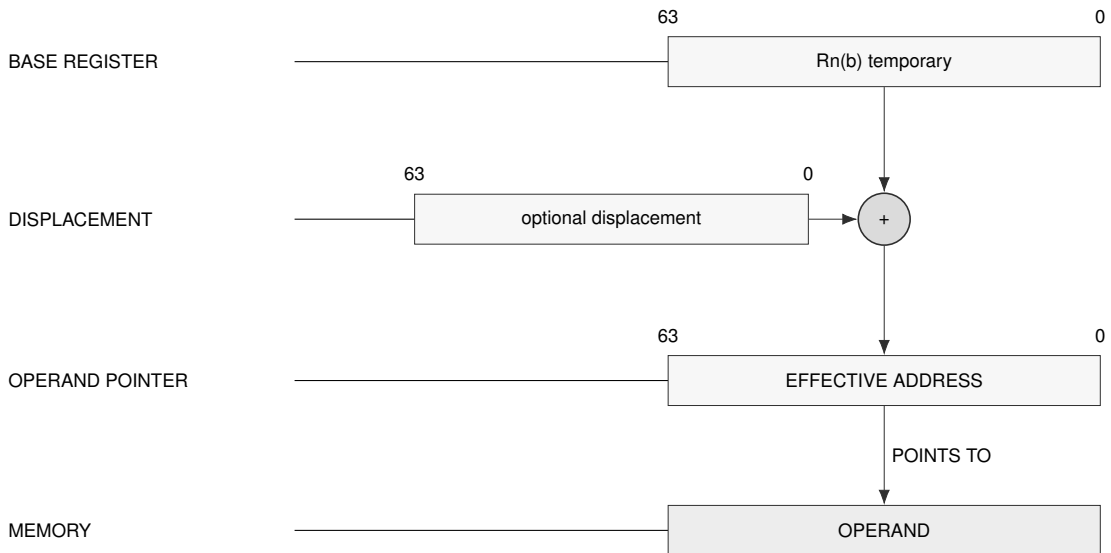


Schema — EXT0 SP/PC indexed

EXT0 Default-Segment Base Auto-Update

[Rn(b)++ + displacement]

[--Rn(b) + displacement]

Descriptor: 1bbbb100 for base postincrement; 1bbbb101 for base predecrement. One byte.**Generation:** offset = base_term + displacement.**Segment:** Uses the operation default data segment.**Payload:** One descriptor byte plus the optional displacement selected by the compact EXT0 escape.**Update:** Postincrement uses the current temporary Rn(b), then increments it by the memory access size.
Predecrement decrements temporary Rn(b) by the memory access size before use.**Schema — EXT0 default-segment base auto-update**

EA evaluation applies auto-update to a temporary operand-evaluation image. The update becomes architecturally visible only if the instruction commits.

Base auto-update changes by the EA interpretation width in bytes. Index auto-update changes by one element before scaling, so the scale affects address generation but not the architectural index increment amount.

EA evaluation begins by copying the architectural registers into a temporary operand-evaluation image. Operands are then processed in instruction order. Within one EA, the base term is evaluated before the index term and then the displacement. Auto-update changes only the temporary image while operands are being evaluated. Memory reads and writes are collected as instruction side effects, and neither those effects nor the register updates become architecturally visible until the instruction commits. A fault before commit therefore leaves both register and memory state unchanged apart from the exception-entry transition.

Postincrement forms use the current temporary value and update it afterward. Predecrement forms update the temporary value first and then use the result. A base register changes by the EA interpretation width in bytes; an index register changes by one element before scaling. Each REP iteration applies and commits these rules independently.

Size-less address-role forms INV PAGE, FLSHD CACHE, INVDCACHE, INVICACHE, WRBKDCACHE, SYNCCACHE, PREFETCH, PREFETCHNT, PTQUERY, SAVE, and RESTORE have Q interpretation width. Size-less control-target forms DJcc, IJcc, LCALL, and LJMP also have Q interpretation width. Their EXT0 index scale is eight and their base predecrement or postincrement amount is eight. Cache range loops advance their explicit address by the CPUID maintenance granule, independently of this per-EA update amount.

When one register supplies both terms in $[R1 + R1++ * scale]$, the base and index terms use the same current temporary R1 before the index update. In $[R1 + --R1 * scale]$, the base uses the current value; the index term then decrements temporary R1 and uses the updated value.

8 MEMORY ADDRESS TRANSLATION

Memory address translation is a separate pipeline after effective-address calculation. Effective-address evaluation produces an address value or operand designator; segmentation optionally turns that value into a linear address, and paging optionally turns the linear address into a memory-system address.

Instruction fetches use CS, stack accesses use SS, and ordinary data accesses use the operation default data segment unless an effective-address form explicitly selects another segment. SP and PC forms use their fixed segments.

SEGLEA exposes the linear address produced by segment pre-translation when software explicitly needs it. Its instruction description defines the point check, FLAGS result, destination suppression, and auto-update commit behavior. SEGLEA applies segment pre-translation to the computed starting point.

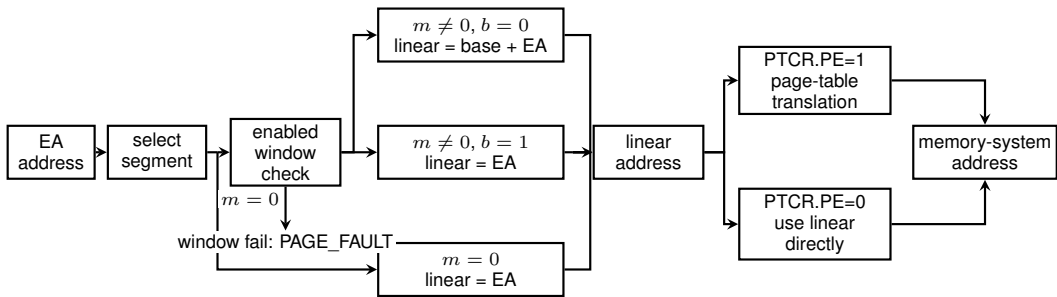


Figure 8-1. Address Translation Pipeline

Segment pre-translation interprets the selected 64-bit segment image before paging. Its base is `base_page` multiplied by 4096, and its span is $(m \ll e)$ multiplied by 4096. The limit is the mathematical sum of base and span and is checked before 64-bit truncation. A segment image with $m=0$ is a valid disabled image and passes the effective address through. An image with $m \neq 0$ is valid only when its mathematical base-plus-span is at most 2^{64} . An enabled segment either checks the effective address as an offset and adds the base, or checks an already-linear address in bounds-only mode.

Address validation begins by forming the starting EA and its segment-pretranslated linear value. The starting point must be within the selected segment. With paging enabled, the starting linear value must then be canonical and the starting leaf translation is resolved far enough to determine its AT value. That value selects the complete bounds, alignment, translation, and address-type checks described under Leaf Address Types. This ordering defines the reported result. With paging disabled, the ordinary byte-addressed range rules apply.

Complete range additions are mathematical: 64-bit wraparound never makes an out-of-range access valid. A range failure raises `PAGE_FAULT` before an architectural memory transaction begins.

The segment result is the linear address. With `PTCR.PE` set, the page-table walker consumes that address and applies the selected PTE frame and attributes. With paging disabled, the memory system receives the complete 64-bit linear address directly as a byte address. `PABITS_SEL` applies to page-table geometry, while the direct address retains all 64 bits. An invalid segment image

raises `INVALID_CONTROL_STATE` when written; a failed segment bound or canonical-address check raises `PAGE_FAULT` during access.

8.1 Multi-Page Accesses

For an `AT=0` byte range that intersects more than one 4-KiB linear page, translation, accumulated permission, and leaf address-type checks are resolved in increasing linear-page order. Each page's walk reaches its existing not-present, structural-validity, and permission result before the next page is consumed. The first page in that order that fails supplies the architectural fault and walk level.

Every covered page of a store is validated before any byte of the store becomes visible. A later-page translation, permission, address-type, or synchronous data-access fault leaves every destination byte and every leaf `D` bit unchanged. A updates completed while consuming an earlier valid entry retain the speculative-`A` rule and may remain set when a later page faults.

Instruction-record fetch uses the same increasing-address rule for all bytes required by the encoded record. A fault on a later page completes as an instruction-fetch fault before decode or operand evaluation, while earlier instruction-cache and PTE `A` effects retain their ordinary rules.

Table 8-1. Segment Pre-Translation Modes

Mode	Segment State	Check	Linear Address
disabled	<code>m = 0</code>	no segment-window check	<code>linear = EA</code>
translated window	<code>m != 0, b = 0</code>	starting point $0 \leq EA < \text{span}$; complete range selected by leaf <code>AT</code>	<code>linear = base + EA</code>
bounds-only window	<code>m != 0, b = 1</code>	starting point $\text{base} \leq EA < \text{limit}$; complete range selected by leaf <code>AT</code>	<code>linear = EA</code>

8.2 Paging Stage

Paging is enabled by `PTCR.PE` and translates the linear address produced by segment pre-translation. Before the walk can produce a translation, the linear address must be canonical for the mode selected by `PTCR.LA57`. A noncanonical address raises `PAGE_FAULT`. With paging disabled, the memory system uses the linear address directly.

Each page-table level uses a nine-bit address field to select one of 512 64-bit entries in a 4-KiB table page. For a table-page base address (`B`) and index (`i`), the selected entry is at $(B + 8i)$. The walker starts at the physical table page named by `PTCR.root_page`. Every selected entry above `L1` points to the next table page, and the selected `L1` entry is the leaf PTE. Its physical frame number is combined with linear-address bits `11..0` to form the memory-system address. The leaf `AT` bit determines whether that value is a byte address or a slot address:

$$\text{memory_system_address} = (\text{leaf_PFN} \ll 12) + \text{linear}[11 : 0].$$

The walker validates every consumed entry and accumulates its permissions while descending toward `L1`. The leaf PTE supplies the final frame number and memory attributes. The detailed entry-validity, permission, accessed, and dirty rules are specified under Page-Table Entry Format. At each consumed level, result priority is not-present first, structural validity second, and accumulated permission third; the walker resolves those results before consuming a lower level.

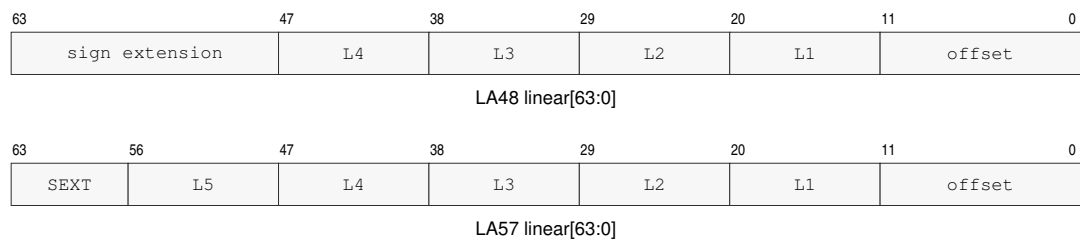


Figure 8-2. Linear-Address Paging Fields

8.3 LA48 Four-Level Paging

When `PTCR.LA57` is zero, bits 63..48 of the linear address must equal the sign extension of bit 47. Bits 47..39, 38..30, 29..21, and 20..12 select the L4, L3, L2, and L1 entries respectively. The twelve low bits are carried unchanged as the offset within the translated 4-KiB page. The L4 table is the root table named by `PTCR.root_page`.

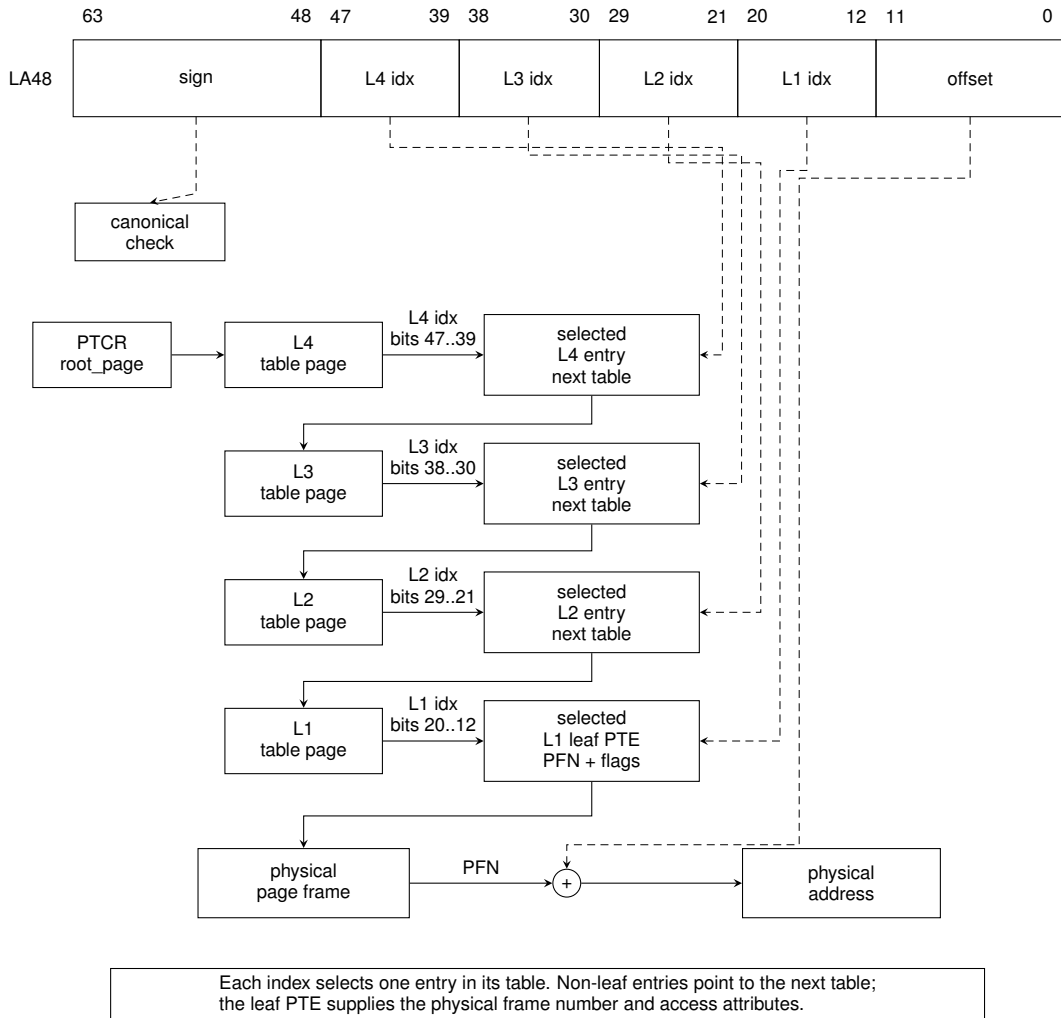


Figure 8-3. LA48 Four-Level Page Walk

8.4 LA57 Five-Level Paging

When `PTCR.LA57` is one, bits 63..57 of the linear address must equal the sign extension of bit 56. Bits 56..48 select an additional L5 entry, whose PFN identifies the L4 table. Bits 47..12 then drive the same L4-through-L1 walk used by LA48, and bits 11..0 remain the page offset. The L5 table is the root table named by `PTCR.root_page`.

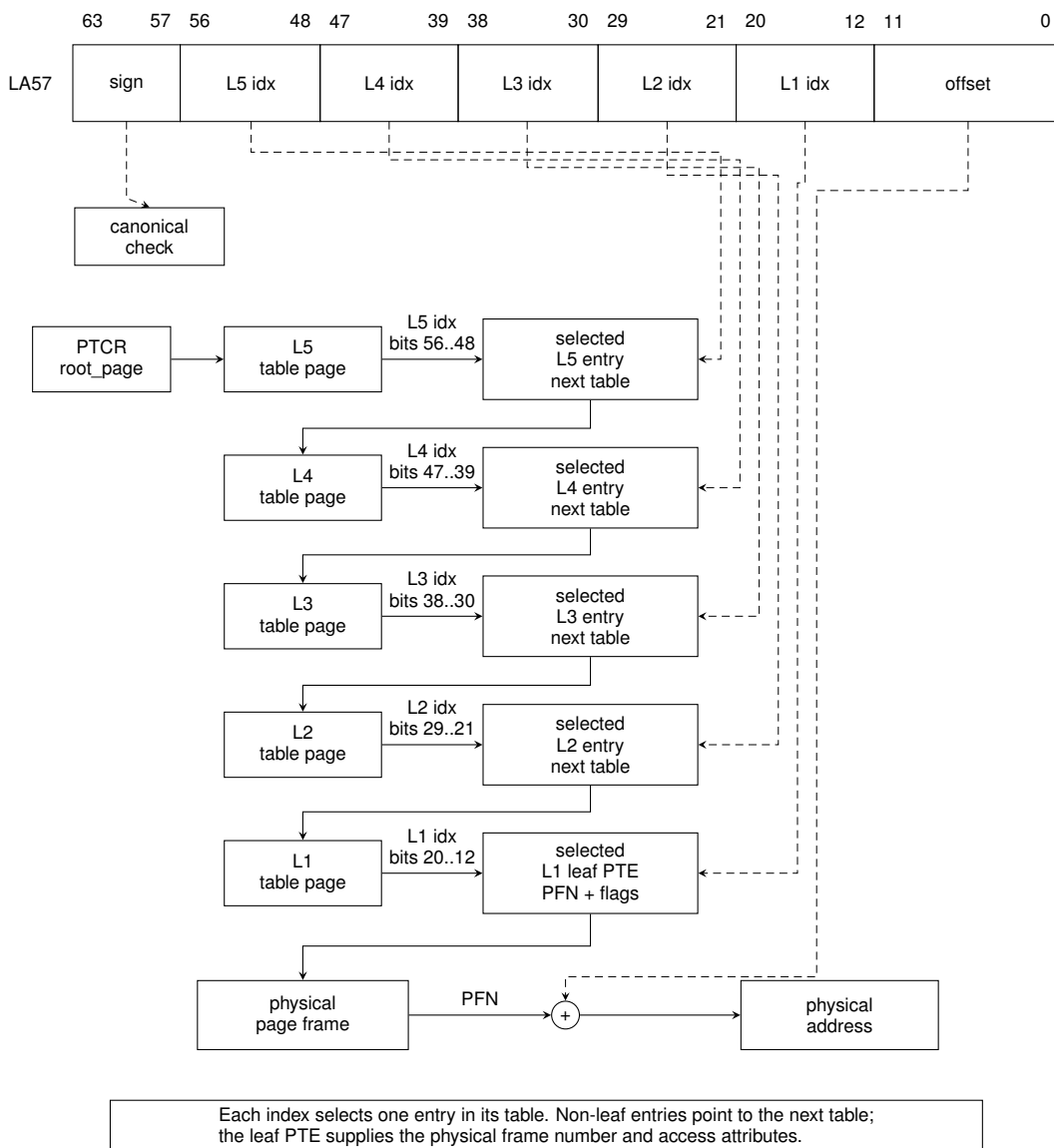


Figure 8-4. LA57 Five-Level Page Walk

8.5 Page-Table Entry Format

Every page-table entry is 64 bits. The low twelve bits contain hardware-defined traversal, permission, and memory attributes. The physical frame number occupies bits PABITS-1..12; bits above the selected physical-address width are available to software and ignored by the page-table walker.

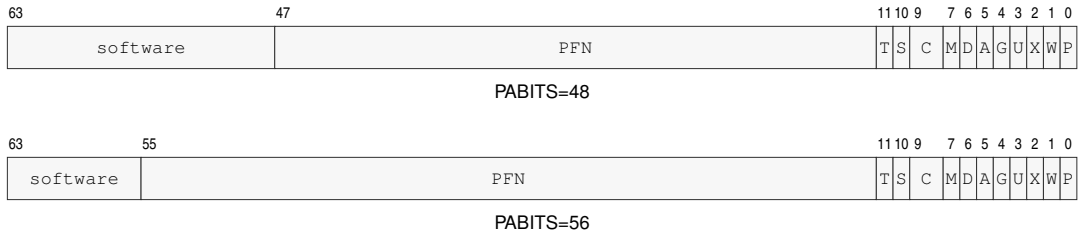


Figure 8-5. Page-Table Entry Formats

In both rows, S, C, and M abbreviate SW0, CP, and AT; the other labels match the field table.

Table 8-2. Page-Table Entry Fields

Field	Bits	Meaning
P	0	present
W	1	writable
X	2	executable
U	3	user-domain mapping
G	4	global translation
A	5	accessed; may be set on a consumed byte-addressed entry, including a speculative traversal; required one for a slot leaf
D	6	byte-addressed leaf dirty bit, set exactly when a store or atomic update commits; required one for a slot leaf
AT	7	0 byte-addressed leaf; 1 externally acknowledged slot-addressed leaf
CP	9..8	0 cacheable; 1 uncacheable; 2 write-through; 3 write-combining
SW0	10	software-defined and ignored by hardware
T	11	one for a non-leaf table pointer; zero for an L1 leaf
PFN	PABITS-1..12	next-table or mapped-page physical frame number
software	63..PABITS	software-defined and ignored by hardware

At every level above L1, P and T must both be one; D, G, and AT must be zero. At L1, P must be one and T must be zero. W, X, and U permissions are accumulated across the walk. A reserved or malformed consumed entry raises PAGE_FAULT. Supervisor current-domain access to a user mapping faults; the checked user-domain operations select the user-domain permission path explicitly.

An AT=1 leaf must have CP=1 (uncacheable), X=0, A=1, and D=1. Any AT=1 leaf with inconsistent attributes is a malformed PTE. Because A and D are required to be set in a valid slot leaf, an access never performs a speculative A update or a commit-time D update to that leaf.

When hardware changes A or D from zero to one, it performs a conditional 64-bit atomic read-modify-write of the complete PTE. The update sets only the required A or D bits and preserves

every other bit from the current 64-bit value. It succeeds only if the translation-relevant PTE value still matches the value that was validated. If a concurrent write changes that value first, the walker must re-read and revalidate the entry; it must not write back stale PTE fields.

A may be set after a structurally valid entry is consumed by a speculative walk even when the initiating instruction is later squashed or faults after that traversal. D is exact rather than speculative: a failed, squashed, permission-denied, or non-storing operation must not change D. A store or successful atomic memory update may become architecturally visible only if any required D update also succeeds. PTQUERY and VTOP retain their instruction-specific rule that they never modify A or D.

8.6 Translation-Cache Identity and Invalidation

A translation-cache entry is identified by the components below. PTCR `root_page` is not a hardware tag. While ASCR.AE is one, system software binds one ASID to one PTCR root and translation geometry; it completes the required shutdown before reusing that ASID for a different PTCR image. While AE is zero, the current untagged context is invalidated when PTCR changes.

Table 8-3. Translation-Cache Entry Identity

Identity Component	Applies To
linear page	every entry
LA57 and PABITS_SEL	every entry
ASID	non-global entry while ASCR.AE is one

Table 8-4. Local Translation-Cache Transitions

Operation	Non-Global Entries	Global Entries
WRCR PTCR with changed <code>root_page</code>	invalidate all current untagged entries when AE is zero; when AE is one software must invalidate the ASID before rebinding it	preserve only mappings whose complete leaf attributes are identical in every context
WRCR PTCR with changed LA57 or PABITS_SEL	invalidate entries using the old translation geometry	invalidate entries using the old translation geometry
WRCR ASCR changing ASID while AE remains one	select the new ASID-tagged entries	preserve
WRCR ASCR changing AE	invalidate the old untagged context or select the new tagged context as applicable	preserve
SWPT	install PTCR and invalidate the current untagged context	preserve only identical global mappings
SWPTA	install PTCR and ASCR and select the new ASID-tagged context	preserve only identical global mappings
INVPAGE	invalidate the addressed page for the current ASID, or the current untagged context when AE is zero	invalidate the addressed global page
INVASID	invalidate every entry tagged with the selected ASID	preserve
INVTLB	invalidate every local entry	invalidate every local entry

Table 8-5. Remote Translation Shutdown Protocol

Step	Required Action
1	PTE store
2	AFENCE
3	local invalidate
4	release shutdown request
5	target acquire and local invalidate
6	target release acknowledgement
7	updater acquire of every acknowledgement
8	AFENCE
9	reclaim or reuse

A global leaf ignores ASID matching. Its PFN, accumulated permissions, CP, and AT attributes must be identical in every context in which that global entry can be used. Changing any of those attributes requires invalidating the global entry on every target logical processor.

The page-table walker reads page tables as CP0 coherent normal memory. Each 64-bit PTE read observes one coherence version; it does not combine fields from different writes. An in-progress walk may complete with the old or new coherence version, so the updater does not reclaim the old mapping until the complete shutdown protocol finishes. The local invalidation instructions affect only the executing logical processor; the request and acknowledgement steps are ordinary shared-memory communication ordered by the stated release, acquire, and AFENCE operations.

8.7 Leaf Address Types and Access Ranges

The leaf AT bit selects the meaning of the low twelve linear-address bits and the memory-system address:

$$\begin{aligned} \text{AT}=0: \quad & \text{byte_address} = (\text{PFN} \ll 12) \mid \text{linear}[11 : 0], \\ \text{AT}=1: \quad & \text{slot_address} = (\text{PFN} \ll 12) \mid \text{linear}[11 : 0]. \end{aligned}$$

For AT=0, the low bits are a byte offset and a width- n access covers the half-open interval $[a, a+n)$. For AT=1, the low bits are a slot index and every B, W, L, or Q access covers exactly one addressable unit, $[a, a+1)$. Here a is the value at the applicable stage: EA for segment bounds, linear address for paging coverage, and byte or slot address after translation. All leaves consumed by an AT=0 byte range must also be byte-addressed; an address-type mismatch raises PAGE_FAULT with cause ADDRESS_TYPE.

An AT=1 transaction carries the slot address, transfer width, read/write direction, and write data. Width does not scale the slot address: for example, Q [0] and Q [1] select distinct slots, and Q [4095] remains one access to the last slot of the leaf. Slot accesses have no byte-alignment rule and no width-induced cross-page condition. Different widths at one slot remain transactions to that same slot; their device-visible relationship is defined by the applicable platform or device contract, not by normal-memory byte aliasing.

Ordinary B, W, L, and Q loads and stores are valid through an AT=1 leaf. Instruction fetch and every atomic read-modify-write through such a leaf raise ADDRESS_TYPE. A prefetch produces no slot transaction. A cache maintenance operation has no slot-side effect. PTQUERY and VTOP report the translation without issuing a slot transaction; VTOP returns the slot address for an AT=1 leaf.

Address type is selected before applying the alignment rule appropriate to that type. Consequently, an atomic whose starting leaf has $AT=1$ raises `ADDRESS_TYPE` even when the numeric slot address would be misaligned under byte-addressed rules. `ATOMIC_ALIGNMENT` applies only after an $AT=0$ leaf has been selected.

A slot store retires only after the transaction is acknowledged. The ordering and completion rules for slot transactions, including `AFENCE`, are defined by the Memory Model.

9 INSTRUCTION EXECUTION MODEL

This section defines the common execution contract used by the instruction descriptions. An individual instruction may add stricter rules, but it does not silently replace the rules below.

9.1 Architectural Result Priority

When one instruction could encounter more than one exceptional condition, the architecturally reported result is the first applicable condition in the following order. This order defines the architecturally reported result.

1. Instruction fetch and fetch translation.
2. Instruction framing and acquisition of the complete instruction record.
3. Opcode recognition.
4. Availability of every extension required by the decoded operation.
5. Fixed-field, reserved-field, and effective-address-form legality.
6. Privilege checks.
7. Selector, control-image, and other architectural-state validation.
8. Predicate evaluation for a conditional operation.
9. Every non-suppressed operand access in architectural access order.
10. Execution-stage integer and floating-point exceptions.

A false predicate suppresses the conditional operation's target-memory and implicit stack accesses. It does not suppress instruction framing, opcode and extension checks, encoding legality, privilege checks, or control-state validation. Explicit operands are accessed in instruction operand order unless an instruction states another order. For a memory-to-memory operation, the complete source read precedes access to the destination, so a source-access failure has priority over a destination-access failure.

Implicit accesses use the order stated by the instruction. In particular, PUSH captures its source before accessing the destination stack slot; POP reads the stack slot before validating or writing its destination; PUSHBP uses listed register order and POPBP uses reverse listed order; CALL validates its target before accessing the return stack; RET reads the return stack before validating its target; and the far call, far return, event return, and supervisor-entry instructions use the step order in their instruction descriptions.

9.2 Implicit Stack Operations

Ordinary implicit stack operations capture the current SS and SP before their first stack access. Every stack slot is 64 bits and is addressed through the captured SS. The ranges below are expressed in terms of the captured `old_sp`; the applicable segment, translation, and complete-range checks finish before the operation commits.

Table 9-1. Ordinary Implicit Stack Operations

Operation	Stack range	Committed SP	Slot layout or value order
PUSH, CALL	[old_sp - 8, old_sp)	old_sp-8	one source value or the following-instruction return PC at old_sp-8
POP, RET	[old_sp, old_sp + 8)	old_sp+8	one destination value or return PC read at old_sp
PUSHP, FPUSHP	[old_sp - 16, old_sp)	old_sp-16	listed pair's second register at old_sp-16 and first register at old_sp-8
POPP, FPOPP	[old_sp, old_sp + 16)	old_sp+16	listed pair's second register from old_sp, then first register from old_sp+8
LCALL	[old_sp - 16, old_sp)	old_sp-16	return PC at old_sp-16 and return CS image at old_sp-8
LRET	[old_sp, old_sp + 16)	old_sp+16	return PC at old_sp and return CS image at old_sp+8

The complete range and all source values are established before a push operation changes memory or SP. A pop operation reads its complete range and completes any applicable destination validation before changing a destination or SP. A successful operation commits all listed stack and register effects together; a preceding fault exposes none of them.

Architectural event entry and ERET use the event-frame layout in the Architectural Event Processing Model. SYSCALL and SYSRET keep their continuation in the user-return control-register bank described by the Privileged Programming Model.

9.3 Operand Evaluation and EA Defaults

Operands are decoded in instruction order. Source reads complete before the final destination write unless an atomic form says otherwise. Effective-address calculation may produce an address without reading memory.

Auto-update EA forms update a temporary operand-evaluation image. The architectural register update becomes visible only when the instruction commits.

Ordinary forms accept at most one memory operand. The explicitly encoded memory-memory exceptions are CMP, EXTSL, EXTSEQ, EXTSEQ, EXTZW, EXTZW, MOV, MOVCU, MOVUC, MOVUU, FBNDII, FBNDIX, FBNDXI, FBNDXX. Repeat behavior is available only through the repeat instructions and body-eligibility rules defined in the Streaming Execution Model.

FLAGS and FFLAGS fields that an instruction does not mention remain unchanged. Operand evaluation follows instruction operand order, and an instruction has one architectural commit point unless its description explicitly defines partial completion.

9.4 Shared Side-Effect Rules

An instruction that faults before commit leaves destination registers, destination memory, and auto-update register effects unchanged unless the instruction explicitly defines partial completion.

Atomic read-modify-write forms have a single architectural commit point for the memory update and any associated register result. Cache, TLB, and system-state instructions describe their own completion and visibility rules.

Ordinary integer ALU forms use the common one-memory-operand rule. Compare and test forms may read two operands and update FLAGS without storing their temporary result. EXTS and

EXTZ additionally provide explicitly encoded register-memory, memory-register, and memory-memory conversions; they preserve FLAGS unless a repeat rule observes a derived value. Data movement, LEA, and segment-address operations also preserve FLAGS unless their descriptions say otherwise.

Control transfers make PC and CS changes at a single control-transfer commit point. Atomic operations require an addressable memory operand. An AT=0 atomic operand must be naturally aligned and commits the read-modify-write as one architectural update; an AT=1 atomic operand raises ADDRESS_TYPE and has no byte-alignment check. TLB, page-table context, and privileged control operations require supervisor privilege. Cache-maintenance operations preserve FLAGS and define their own visibility contract. Approximate transcendental floating-point operations are available only when CPUID reports FPTRANSA and marks the instruction's accuracy-contract ID present.

10 FLAGS AND CONDITION CODES

10.1 Condition Encoding

Conditional instructions encode a four-bit condition code. The zero-valued condition is T, which is also used for canonical aliases such as JMP for Jcc.T.

The condition-code bits are the low four meaningful bits of FLAGS, as shown in the Programming Model section.

Table 10-1. Condition Code Encoding

Bits	Name	Aliases	Expression
0000	T	-	true
0001	F	-	false
0010	EQ	Z	Z == 1
0011	NE	NZ	Z == 0
0100	ULT	C	C == 1
0101	UGE	NC	C == 0
0110	MI	N	N == 1
0111	PL	NN	N == 0
1000	VS	V	V == 1
1001	VC	NV	V == 0
1010	ULE	-	C == 1 Z == 1
1011	UGT	-	C == 0 && Z == 0
1100	LT	-	N != V
1101	GE	-	N == V
1110	LE	-	Z == 1 N != V
1111	GT	-	Z == 0 && N == V

10.2 Flag Meanings

Integer condition codes are held in FLAGS. Integer instructions leave FLAGS unchanged unless their instruction description explicitly defines a FLAGS write. Unmentioned FLAGS bits are preserved unless the instruction says otherwise.

For an operand width of n bits, ordinary integer ALU results are evaluated modulo 2^n unless the instruction explicitly defines a wider intermediate.

Explicit FLAGS-writing integer instructions include CMP, TEST, ADC, SBB, INCF, DECF, BTEST, BSET, BCLR, BCHG, SETF, CLRF, WRFLAGS, CMPXCHG, and the bounds-check forms. CMPJcc and TESTJcc compute temporary condition flags for their branch decision and do not update architectural FLAGS. ADD, SUB, logic operations, INC, DEC, shifts, rotates, moves, extensions, min/max, and multiply/divide forms leave FLAGS unchanged.

Table 10-2. Integer Condition-Code Bits

Bit	Name	Meaning
Z	zero	Set when the result value is zero.
N	negative	Set from the most significant result bit at the operand size.
C	carry/borrow	Set on unsigned carry out for addition or unsigned borrow for subtraction.
V	overflow	Set on signed overflow or an explicitly reported exceptional condition.

10.3 Conditional Consumers

Conditional branches, calls, traps, sets, and repeat forms consume the condition-code table without recomputing FLAGS. A conditional consumer observes the FLAGS image produced by the last committed FLAGS-writing instruction.

A false condition suppresses the conditional action unless the instruction description explicitly defines a different commit rule, such as a repeat instruction's terminating iteration rule.

10.4 Floating-Point Interactions

Floating-point comparison instructions may update integer condition codes only when their instruction definition says so. Floating-point exception and rounding state is held in FFLAGS and FSTATUS.

Integer conditional consumers do not read FFLAGS directly; floating-point instructions that want integer control-flow consumers must materialize the selected condition into FLAGS or an Rn value.

11 MEMORY MODEL

The memory model defines instruction fetches, loads, stores, atomic read-modify-write operations, cache maintenance, and translation-cache maintenance after EA calculation and address translation.

11.1 Baseline Ordering

Normal memory is coherent. Every hardware thread observes writes to a given byte in one common coherence order, and writes by one hardware thread to that byte enter the order in program order. A naturally aligned tear-free scalar store enters that order as one write event.

Normal-memory writes are multi-copy atomic. When a write event becomes observable by a hardware thread other than the writer, it becomes observable to all such threads at the same architectural point. An implementation may still forward a hardware thread's own pending store to its later loads before that store is globally observable.

The baseline ordering is weak. Except for same-byte coherence or ordering imposed by an atomic memory-order selector or fence, loads and stores to different locations may be issued, completed, and observed in an order different from program order. Coherence and multi-copy atomicity do not by themselves order accesses to different locations.

11.2 Ordinary Load Values and Preserved Order

Each byte returned by a load comes from the initial value of that physical byte or from an actual store or successful atomic read-modify-write to that byte. A load does not obtain a value solely from a cyclic chain in which the load value is needed to produce the write that would supply it.

Among writes already globally observable when a load takes its value, the load observes the latest write in the byte's coherence order. A load may instead forward a byte from an earlier store by the same hardware thread before that store is globally observable. A store later than the load in program order cannot supply the load.

Program order is preserved from an earlier read or write to a later write when their byte ranges overlap. Two loads of the same byte, with no intervening same-thread write to that byte, do not observe the byte's coherence order moving backward. These rules apply independently to every byte of an access, including each visible portion of an unaligned access.

A load is ordered before a later memory access whose effective address depends on the loaded value. A load is also ordered before a later store when the stored value depends on the load or when execution of that store is control-dependent on the load. Control dependence alone does not order a later load; software uses an appropriate fence when that ordering is required.

11.3 PTE Cache Policies

The two-bit PTE CP field selects a cache policy for byte-addressed normal memory. Every policy uses the same coherence order, multi-copy atomicity, tearing rules, atomic operations, and fence semantics defined in this chapter. The policy changes allocation, retained cache state, and store combination; it does not create a separate value or ordering domain.

Table 11-1. Normal-Memory Cache Policies

CP	Policy	Reads and Fetches	Stores	Retained State
0	CACHEABLE_WRITE_BACK	may allocate a coherent cache copy	may retire into a coherent dirty cache copy	clean or dirty coherent cache copies
1	COHERENT_UNCACHEABLE	enter coherence without retaining a cache copy	enter coherence before retirement without retaining a dirty copy	no retained cache copy
2	COHERENT_WRITE_THROUGH	may allocate a coherent clean cache copy	enter coherence before retirement without retaining a dirty copy	clean coherent cache copies only
3	COHERENT_WRITE_COMBINING	enter coherence without retaining a cache copy	may combine adjacent byte writes before entering coherence	pending combined stores, but no retained cache copy

Aliases of one physical byte with different CP values participate in the same coherence order. A load, instruction fetch, or atomic operation through any alias observes the same physical value subject to ordinary ordering, and atomic operations serialize at the physical location. CP2 stores enter coherence before retirement and leave no dirty cache copy. CP3 combined stores preserve the byte writes and their per-location program order.

WFENCE and AFENCE complete earlier pending CP3 stores. WRBKDCACHE, FLSHDCACHE, and SYNCCACHE complete pending CP3 stores for their selected block before performing their named cache action. The data-write, cache-maintenance, rendezvous, and local instruction-cache sequence below applies without regard to the CP value used by the modified code mapping.

11.4 Slot-Addressed Transactions

An AT=1 load or store is an acknowledged slot transaction rather than a normal-memory byte access. Multiple transactions to different slot addresses may be outstanding. Transactions to the same slot address preserve program order. A store does not retire until its acknowledgement is received.

AFENCE orders completion of all earlier slot transactions before any later slot transaction and orders slot transactions with byte-addressed memory operations according to its full cumulative-fence rules. Device-specific effects of different widths at the same slot remain part of the platform or device contract.

11.5 Access Atomicity and Alignment

Unaligned ordinary memory accesses are architecturally permitted. A completed unaligned load may observe bytes from multiple coherence events, and a completed unaligned store may enter coherence as separately visible portions. Each visible store portion is coherent and multi-copy atomic.

An unaligned store is nevertheless fault-atomic for synchronous faults. If any byte of the complete store range would raise a synchronous fault, no byte of the store becomes architecturally visible.

Atomic read-modify-write instructions require natural alignment only for an AT=0 byte-addressed operand. A misaligned AT=0 atomic operand raises PAGE_FAULT with the ATOMIC_ALIGNMENT cause before any memory update or architectural result commits; it must not complete as a non-atomic update. An AT=1 operand has no byte-alignment property and every atomic operation through it instead raises ADDRESS_TYPE, regardless of the numeric slot address.

Stores to memory that is later executed as instructions are ordinary data stores until an explicit synchronization sequence makes the modified bytes visible to instruction fetch.

11.6 Cache-Maintenance Blocks

CPUID `CACHE_TOPOLOGY` reports a cache-maintenance granule G . A cache-maintenance instruction computes and translates its address-rotate EA without reading an operand value, then selects the physical byte interval $[\lfloor A/G \rfloor G, \lfloor A/G \rfloor G + G)$ containing the translated physical address A . One instruction operates on exactly that block. Because G is at most 4096 bytes, the selected physical block does not cross a 4 KiB page boundary.

FLSHDCACHE, INVDCACHE, and WRBKDCACHE apply to every data or unified cache copy of the selected block in its normal-memory coherence domain. FLSHDCACHE writes dirty bytes into normal-memory coherence and invalidates the copies. INVDCACHE discards the copies without writeback. WRBKDCACHE writes dirty bytes into normal-memory coherence and retains clean copies. Each operation completes the selected block before retirement.

INVICACHE invalidates the selected block in the executing logical processor's instruction cache, decoded instruction state, and instruction-prefetch state. SYNCCACHE first completes the data writeback for the block throughout the coherence domain, then performs the same local instruction-side invalidation, and finally serializes later instruction fetch so that it observes the synchronized bytes. To publish modified code to another logical processor, software completes SYNCCACHE on the publisher, performs a release/acquire rendezvous, and executes INVICACHE for the block on every receiving logical processor before branching to the modified bytes.

Address translation, permission, and address-type checks precede a block's cache effects. A fault leaves that instruction's selected block unchanged. An $AT=1$ mapping retains the slot-addressed rule: cache maintenance issues no slot transaction and has no cache effect.

For every scalar size below, a naturally aligned normal-memory load or store is tear-free. Unaligned accesses retain the successful-access and synchronous-fault rules above.

Natural alignment equals the operand width in bytes: B is one byte and may use any byte address; W, L, and Q are two, four, and eight bytes and require addresses divisible by two, four, and eight respectively.

11.7 Atomic Memory-Order Selectors

Table 11-2. Atomic Memory-Order Selectors

Selector	Code	Architectural Ordering Effect
RELAXED	0	Atomicity only. No ordering is imposed on unrelated memory operations.
ACQUIRE	1	Later memory operations by the same hardware thread are ordered after the atomic operation.
RELEASE	2	Earlier memory operations by the same hardware thread are ordered before the atomic operation.
ACQREL	3	Applies both acquire and release ordering.
SEQCST	4	Applies acquire-release ordering and participates in the single global sequentially consistent order.

CMPXCHG, FETCHADD, FETCHSUB, FETCHAND, FETCHOR, and FETCHXOR are the atomic read-modify-write instructions. Each performs its memory read, successful memory write, and architectural register result as one indivisible operation in the coherence order of the addressed physical location.

A successful release or stronger atomic write heads a release sequence. The sequence continues through immediately following successful atomic read-modify-write operations to the same location in coherence order and ends before any other write. An acquire or stronger atomic operation that takes its value from any member of the sequence observes the effects ordered before its head before effects ordered after the acquiring operation.

One encoded selector governs the complete CMPXCHG operation on both success and failure. A failed CMPXCHG contributes a read event. Its acquire component orders later memory operations after that read, and its release component orders earlier memory operations before that read. On success, the write event carries the selected release effect to observers of that write; on failure, the read is the operation's memory event. A failed CMPXCHG contributes no write and therefore creates no release sequence. A failed CMPXCHG.SEQCST additionally participates in the global sequentially consistent order as an SC load.

11.8 Fence Instructions

The table gives every ordered-before pair imposed by a fence on memory operations issued by the same hardware thread. An entry marked ordered places every earlier operation of the row kind before every later operation of the column kind.

Table 11-3. Fence Ordered-Before Pairs

Fence	read to read	read to write	write to read	write to write
RFENCE	ordered	baseline	baseline	baseline
WFENCE	baseline	baseline	baseline	ordered
AFENCE	ordered	ordered	ordered	ordered

A baseline entry retains the weak-ordering rules of this chapter. AFENCE is cumulative: memory effects observed before the fence are propagated before effects ordered after it, including through another hardware thread. The linked instruction entries define the matching completion behavior.

11.9 Global Sequentially Consistent Order

All AFENCE operations and all SEQCST atomic operations participate in one global total order that is consistent with each hardware thread's ordered-before relation and with per-location coherence. A failed CMPXCHG.SEQCST participates in that order as an SC load even though it performs no write.

A naturally aligned tear-free scalar load or store also participates as an SC operation when it is the only memory access between its immediately surrounding AFENCE operations in memory-event order. Thus AFENCE; access; AFENCE is the architectural sequence for an SC scalar load or store. The access and both fences occupy their required positions in the same global order.

Acquire and release operations may be strengthened with AFENCE; the resulting instruction sequence has the union of the ordering constraints imposed by its operations.

12 STREAMING EXECUTION MODEL

This section defines the architectural role of repeated scalar execution in Bedrock. The architecture exposes scalar register state and bounded repeated instruction forms.

12.1 Architectural Scalar Semantics

REP, REPcc, REPG, and any explicitly defined repeat instruction are architecturally scalar. Their visible behavior is the same as executing the repeated instruction or byte-counted instruction group in program order under the selected counter and termination rules. The architectural counter, FLAGS image, memory effects, and auto-update register effects are observed as if each committed iteration completed before the next one began.

The body is decoded when repeat state is entered and remains fixed for that repeat context. Each iteration evaluates register and memory operands by the body's ordinary rules, including the normal scalar effects of auto-update effective addresses. FLAGS and FFLAGS follow the body's instruction semantics and any repeat-specific accumulation rule. Faults remain precise at the repeated-operation boundary.

12.2 Repeat Syntax and Body Eligibility

The accepted spellings are:

- REPcc Rn, (instruction)
- REP Rn, (instruction)
- REPG Rn, { (instructions...) }
- REPGF Rn, { (instructions...) }, an assembler-only spelling that emits REPG after applying the additional eligibility checks described below

The processor decodes the body and checks its repeat eligibility before applying the zero-count rule. A malformed or unavailable body therefore faults even when the captured count is zero. Once a valid body has passed those checks, a zero count annuls it without architectural side effects.

REPGF is an assembler-checked spelling that emits REPG after applying the additional body-eligibility checks. The assembler accepts only candidate bodies that do not write the selected counter register. The emitted encoding, decoded operation, and repeat-continuation state are identical to REPG.

12.3 Counter, Condition, and Control Rules

Entering repeat state captures the selected Rn value as an unsigned remaining count. Each successfully committed iteration decrements that captured count. A condition-false REPcc iteration still commits its body and performs the decrement before repeat state is cleared. A valid zero count performs no body side effects after decoding and eligibility checks complete.

During REP and REPcc, a body read of the selected counter register observes the iteration-start architectural value, while a body write to that register is ignored. REPG executes counter-register

reads and writes in normal in-group program order, but a write does not replace the separately captured remaining count. The checked REPGF spelling rejects such a write at assembly time.

The `REPcc` observation attribute in every `REPcc`-capable instruction description selects one observation. A `flags` observation uses the complete post-body `FLAGS` image. A `result:operand` or `source:operand` observation derives logical `Z` from whether the selected-width value is zero, `N` from its most significant bit, and `C` and `V` as zero. These logical flags do not change architectural `FLAGS`. The observed value is captured before repeat counter-destination suppression. The first condition test occurs after the first body iteration. Body commit, condition capture, counter decrement, and continuation-PC selection form one precise architectural commit step. `REP` is the unconditional `T`-condition spelling, and `F` is reserved for `REPcc`.

A repeat body may not contain another repeat operation, a repeat-control instruction, or a control transfer. While repetition is active, `PC` names the current body instruction, and the initially decoded instruction or group remains fixed. Handler entry saves repeat state in `FRAME_EXT1` and clears the active architectural repeat state; `ERET` restores that state atomically with the saved `PC` and status state. `TF` and `RF` apply to the trace units defined by the Event Priority and Debug Trace subsection.

12.4 Fault and Restart Semantics

A fault during repeated execution preserves committed work and saves the state needed to resume or emulate the remainder. Event entry records the active repeat context in `FRAME_EXT1` and clears active repeat state. The Repeat Continuation subsection of the Architectural Event Processing Model defines the saved format and `ERET` restoration.

`REP` and `REPcc` atomically commit the one-instruction body, repeat-observed condition, counter decrement, and continuation-PC selection. If the body faults, that iteration does not commit and the fault is reported at the body instruction. Every committed iteration decrements the counter, including a condition-false iteration that terminates `REPcc`; earlier iterations remain committed.

For `REPG`, the unsigned body byte count must describe a group that ends on an instruction boundary. Instructions in the group commit in program order. If an instruction faults, earlier completed instructions in the current group iteration remain committed and later instructions do not execute. The fault is reported at the faulting instruction, and the repeat counter is not decremented for the incomplete group iteration; it is decremented only after the whole group iteration succeeds. The saved `PC` and repeat continuation retain the information required to reconstruct the group and continue the remaining work.

PART III

BEDROCK ARCHITECTURE

System Programming

13 PRIVILEGED EXECUTION MODEL

The privileged programming model has two independent transitions into supervisor execution. SYSCALL is an explicit supervisor call with its own entry state, working stack, and user-return bank. Architectural event delivery handles exceptions, maskable interrupts, and NMI through the common EPC/ECS/EDS entry. These paths do not share a return instruction or a memory-resident frame.

Supervisor entry changes execution context at a single architectural commit point. Invalid entry targets, segment images, saved state, or control-state writes raise the defined exception without exposing a partial transition.

13.1 Privilege and Transition State

STATUS.PM is zero in user mode and one in supervisor mode. STATUS.EA records only architectural event processing; SYSCALL does not set it. STATUS.EA is hardware managed, and WRSTATUS must preserve the current EA value. An attempted change raises INVALID_CONTROL_STATE without changing STATUS.

The other STATUS control bits are Boolean: IE enables maskable interrupts, TF controls single-step tracing, RF marks exception restart, and NI inhibits nested NMI delivery.

In supervisor mode, ordinary memory operands use current-domain access. Checked user-domain access is expressed only by the supervisor-only MOVUC, MOVCU, and MOVUU instructions. RDCR, WRCCR, translation-state changes, event-control changes, and page-table context switches require supervisor privilege unless an instruction explicitly says otherwise.

13.2 Supervisor Call Entry

SYSCALL is valid only in user mode. It first requires URCTL.V to be clear and validates SPC, SCS, SDS, SSS, and SSP. SPC must be 16-byte aligned, and SYSCALL uses its exact value. SSP is a 64-byte-aligned configured working-stack top.

After validation, SYSCALL writes the address following the instruction to URPC, saves the user SP and CS/DS/SS images, and writes FLAGS and STATUS to URCTL with V set. It then loads CS=SCS, DS=SDS, SS=SSS, SP=SSP, and PC=SPC and sets STATUS.PM. The entry does not set STATUS.EA and does not change IE, TF, RF, or NI. The complete bank update and execution-context transition are one commit.

SYSCALL performs no memory access and creates no stack frame. SSP is not modified by entry or return. Kernel code may move SP freely and may execute SYSRET from any supervisor stack position because the return context resides entirely in the UR control-register bank. An invalid entry configuration, including an already valid bank, raises INVALID_CONTROL_STATE at the SYSCALL instruction boundary.

13.3 User Return and Context Ownership

SYSRET is the only matching return instruction. It is legal only while STATUS.EA is clear and requires URCTL.V to be set. It validates the saved user STATUS, all three segment images, PC, and SP before changing architectural state. The saved user STATUS must have PM and EA

clear and valid reserved bits. A successful return restores FLAGS, STATUS, CS, DS, SS, SP, and PC and clears URCTL.V as one commit. A validation failure raises INVALID_CONTROL_STATE through the event path without modifying the bank or partially returning.

An event taken during a system call switches to ISS:ISP, FSS:FSP, or DSS:DSP according to its event kind. ERET returns to the interrupted syscall context and leaves the UR bank unchanged; SYSRET can then return to user mode.

The UR bank is per-logical-processor architectural state. If a system call blocks or is preempted and another thread may run on that logical processor, system software saves and restores the bank with the thread's supervisor context. Signal delivery, system-call restart, tracing, and initial user-process entry may inspect or establish the bank through WRCR, subject to the same validation rules.

Normative control-flow behavior is defined by SYSCALL, SYSRET, LRET, and ERET.

13.4 Architectural Event Boundary

Exceptions, maskable interrupts, and NMI use the event-entry state and an architectural event frame. Event delivery sets STATUS.EA; ERET is its only matching return. ERET is illegal with EA clear, and SYSRET is illegal with EA set. This check occurs before either instruction reads its return state. The full delivery, nesting, frame, and return rules are specified in the Architectural Event Processing Model section.

14 ARCHITECTURAL EVENT PROCESSING MODEL

An *architectural event* is a synchronous or asynchronous condition delivered through the common event-entry mechanism. Synchronous exceptions are faults or traps. Asynchronous events are maskable interrupts or NMIs. A fault and a trap are exception subtypes, not peers of exception. SYSCALL is not an architectural event: it is an explicit call into the supervisor ABI and uses the separate supervisor-call state described in the privileged programming model.

Every exception, maskable interrupt, and NMI transfers to the exact program counter in EPC after loading ECS and EDS. Hardware supplies an event code and a frame whose payload layout is fixed by the event source. Software begins at one common entry point and dispatches by the event code.

Related normative instruction entries are BKPT, ERET, and RESET.

14.1 Event Code and Sources

Every delivered event carries a 32-bit code. The high byte is the event class and the low 24 bits are an ID in that class's namespace.



Figure 14-1. Architectural Event Code

Table 14-1. Architectural Event Classes

Value	Class	ID interpretation
0x00	EXCEPTION	architecture-defined exception ID
0x01	INTERRUPT	opaque platform-assigned global interrupt ID
0x02	NMI	zero or an architecturally/platform-defined NMI source ID
0x03..0xFF	reserved	—

Table 14-2. Assigned Base Exception IDs

ID	Name	Subtype or source
0x00	DEBUG_TRACE	debug trace trap
0x02	BREAKPOINT	breakpoint trap
0x03	ILLEGAL_INSTRUCTION	illegal-instruction fault
0x08	PRIVILEGE_FAULT	privilege fault
0x09	PAGE_FAULT	address-translation, access, or atomic-alignment fault
0x0A	DIVIDE_ERROR	integer divide fault
0x0D	INVALID_CONTROL_STATE	invalid control-state fault
0x0E	FLOATING_POINT_FAULT	floating-point fault or trap
0x18	DOUBLE_FAULT	event-delivery failure
0x19	MACHINE_CHECK	corrected trap or recoverable/fatal machine-check exception

Table 14-2 (continued)

ID	Name	Subtype or source
0x1A	BUS_ERROR	synchronous precise bus-error fault

Table 14-3. Architectural Event Boundaries and Priority

Code	Event	Kind	Saved PC	Priority
EXC:00	DEBUG_TRACE	trap	next trace unit	5
EXC:02	BREAKPOINT	explicit trap	following instruction	4
EXC:03	ILLEGAL_INSTRUCTION	fault	faulting instruction	3
EXC:08	PRIVILEGE_FAULT	fault	faulting instruction	3
EXC:09	PAGE_FAULT	fault	faulting instruction	3
EXC:0a	DIVIDE_ERROR	fault	faulting instruction	3
EXC:0d	INVALID_CONTROL_STATE	fault	faulting instruction	3
EXC:0e	FLOATING_POINT_FAULT	fault	faulting instruction	3
EXC:18	DOUBLE_FAULT	delivery failure	interrupted event boundary	1
EXC:19	MACHINE_CHECK	machine check	selected by PRECISE and corrected status	2
EXC:1a	BUS_ERROR	fault	faulting instruction	3
NMI:source	NMI	asynchronous	next instruction or repeat continuation	6
INTERRUPT:platform	INTERRUPT	asynchronous	next instruction or repeat continuation	7

Table 14-4. Architectural Event Producers and Delivery State

Event	Producer	Committed State	Frame	Payload
DEBUG_TRACE	completion of a TF-selected trace unit	completed trace unit	BASIC	none
BREAKPOINT	BKPT	BKPT completed	BASIC	none
ILLEGAL_INSTRUCTION	decode or instruction-validity check	no effects of the faulting instruction	ERROR	error code
PRIVILEGE_FAULT	privilege check	no effects of the faulting instruction	BASIC	none
PAGE_FAULT	segment, translation, access, or atomic-alignment check	no effects of the faulting instruction	PAGE_FAULT	error code, fault EA, and linear address
DIVIDE_ERROR	integer divide operation	no effects of the faulting instruction	ERROR	error code
INVALID_CONTROL_STATE	control-state validation	no effects of the faulting instruction	ERROR	error code
FLOATING_POINT_FAULT	enabled floating-point exception	no destination or accrued-flag effects of the faulting instruction	ERROR	error code
DOUBLE_FAULT	architectural event-delivery failure	no partial first delivery state	AUXILIARY	delivery-failure auxiliary state
MACHINE_CHECK	processor or memory-system machine-check source	selected by PRECISE and RETRY_SAFE	AUXILIARY	machine-check auxiliary state
BUS_ERROR	synchronous acknowledged bus failure	no CPU state effects of the faulting instruction	AUXILIARY	bus-error auxiliary state

Table 14-4 (continued)

Event	Producer	Committed State	Frame	Payload
NMI	admitted non-maskable interrupt source	all earlier committed state	BASIC	none
INTERRUPT	admitted maskable interrupt source	all earlier committed state	BASIC	none

Unassigned base-profile exception IDs are reserved. External interrupt IDs do not share a namespace with exception IDs. Interrupt-controller routing, native-ID mapping, ownership, acknowledgement, trigger mode, masking, and end-of-interrupt signaling belong to the platform ABI. An interrupt ID unknown to software still reaches the common entry; the platform dispatcher applies its unhandled or spurious-interrupt policy.

A restartable fault saves the faulting instruction or its architecturally defined restart point. A trap saves the following instruction. An interrupt or NMI saves the next instruction that would otherwise execute. An event taken during repeat execution saves the active repeat continuation PC and records the remaining repeat state in the always-present `FRAME_EXT1` slot.

14.2 Event Priority and Debug Trace

When events compete at one architectural boundary, the lower priority number in the event table wins. This orders delivery failure before machine check, current synchronous faults, explicit traps, `DEBUG_TRACE`, NMI, and maskable interrupt. A lower-priority asynchronous event remains pending after a higher-priority event is selected.

A trace unit is one ordinary committed instruction, one committed REP or REPcc body iteration, or one completed REPG group iteration. Completion of a zero-count REP, REPcc, or REPG is also one trace unit. TF is sampled at the start of the unit. If sampled TF is one and sampled RF is zero, successful completion commits the unit and then delivers `DEBUG_TRACE` with the following trace-unit boundary as its saved PC.

The `TRACE` marker instruction is an ordinary instruction for this trace-unit rule and separately records the marker described by its instruction entry. The tracing terminology distinguishes the instruction, trace unit, and event.

RF is a one-shot `DEBUG_TRACE` suppressor. If sampled RF is one, successful trace-unit completion clears RF and does not produce `DEBUG_TRACE`. A synchronous fault or asynchronous event before that completion preserves RF. `WRSTATUS`, `ERET`, and `SYSRET` changes to TF or RF apply to the next trace unit. Event entry saves the old `STATUS` image and then clears live TF and RF. RF does not suppress an explicit trap such as `BKPT`.

14.3 Entry Admission and Event Depth

The event-entry state is usable only while `ECR.V` is set. A maskable interrupt is admitted only when `ECR.V`, `STATUS.IE`, and the depth limit all permit it: the current event depth must be less than `ECR.MAX_EDEPTH`. Otherwise the interrupt remains pending at the interrupt controller. A synchronous exception that requires delivery while `ECR.V` is clear cannot be deferred and enters `SHUTDOWN`. An NMI observed while `ECR.V` is clear sets the hardware-managed `ECR.NMI_P` latch and is not admitted. After `ECR.V` becomes one, hardware admits that pending NMI before

the next instruction boundary when STATUS.NI is clear. While STATUS.NI remains set, the NMI remains pending. Multiple NMI observations coalesce in ECR.NMI_P.

Event depth is zero outside event handling. A successful entry saves the old depth in the frame and increments the hidden current depth. ECR.MAX_EDEPTH equal to zero prevents maskable interrupt admission; values 1 through 15 limit only maskable interrupts and do not suppress synchronous exceptions or NMI. Depth 15 is representable but terminal: any further event requirement enters SHUTDOWN. At depth 14 or less, a delivery failure can still create a DOUBLE_FAULT frame; at depth 15 there is no representable child frame.

14.4 Supervisor Event Stacks and Nesting

The four supervisor stack pairs have fixed roles. SSS:SSP is the SYSCALL working stack. ISS:ISP, FSS:FSP, and DSS:DSP are event stack levels 1, 2, and 3.

Table 14-5. Supervisor Stack Roles

Level	Pair	Role	Use
call	SSS:SSP	supervisor call	SYSCALL working stack
1	ISS:ISP	maskable interrupt	external interrupts and IPIs
2	FSS:FSP	fault/exception	faults, traps, NMI, and machine check
3	DSS:DSP	double fault	DOUBLE_FAULT and event-delivery failure only

The processor tracks a hidden two-bit current event stack level while STATUS.EA is set. A maskable interrupt requests level 1, an exception other than DOUBLE_FAULT or an NMI requests level 2, and DOUBLE_FAULT requests level 3. If entry is from a context with EA clear, hardware loads the pair assigned to the requested level. If entry is already event-active and the request is for a higher level, hardware moves to the higher pair. A same- or lower-level nested event retains the current SS:SP and appends its frame instead of reloading a configured top and overwriting an older frame. An interrupt-handler fault therefore moves from level 1 to level 2, while an interrupt admitted during an exception remains on level 2. ERET restores the saved event-active and stack-level state.

```

if STATUS.EA == 0:
    selected_esl = requested_esl
    load the stack pair assigned to selected_esl
else if requested_esl > current_esl:
    selected_esl = requested_esl
    load the stack pair assigned to selected_esl
else:
    selected_esl = current_esl
    keep the current SS:SP

```

After stack selection, hardware rounds the event payload length up to 16 bytes, adds the fixed 64-byte frame, and subtracts that total from the active stack pointer. For a new or higher level the active pointer is ISP, FSP, or DSP; for same- or lower-level nesting it is the current SP. Hardware validates and atomically stores the entire frame before committing the new SP. The configured stack-top register is never changed.

DOUBLE_FAULT and event-delivery failure use DSS:DSP. Current event stack level is meaningful only while STATUS.EA is set. ERET validates and restores both saved depth and saved stack level.

14.5 Event Entry State

Before making an event visible, hardware validates EPC, ECS, EDS, the selected stack state, and the complete frame storage range. It then stores the frame, loads the entry segments and PC, sets STATUS.PM and STATUS.EA, and clears STATUS.IE, STATUS.TF, and STATUS.RF as one architectural commit. R0 through R15 and GS0 through GS5 are unchanged; the common entry code preserves whatever its software ABI requires.

NMI entry additionally sets STATUS.NI. Other event classes preserve NI. A further NMI observed while NI is set is recorded by setting ECR.NMI_P. ECR.NMI_P is hardware managed, so WRCCR ignores the supplied value for that bit. When entry of an NMI admitted from ECR.NMI_P commits, hardware clears ECR.NMI_P as part of the same architectural transition. A newly observed NMI may set the latch again. An entry failure does not clear the latch before committing the specified delivery-failure result. When ERET successfully restores NI to zero, a pending NMI is delivered before the next instruction boundary.

STATUS.EA is hardware-managed transition state. WRSTATUS must preserve its current value. An attempt to change EA raises INVALID_CONTROL_STATE without modifying STATUS. WRSTATUS may change IE, NI, TF, and RF; its PM and EA values must equal the current values. Clearing NI while ECR.NMI_P is one admits the pending NMI at the next instruction boundary. ERET is legal only while EA is one; SYSRET is legal only while EA is zero. A mismatch raises INVALID_CONTROL_STATE before ERET reads an event frame or SYSRET reads the user-return bank.

14.6 Architectural Event Frame

Every event frame begins with exactly eight 64-bit slots. `EVENT_INFO` contains the event code in bits 31 through 0; its upper half is zero. `FRAME_CONTROL` contains only frame shape, saved nesting state, saved `FLAGS`, and saved `STATUS`. Event classification is carried exclusively by `EVENT_INFO`.

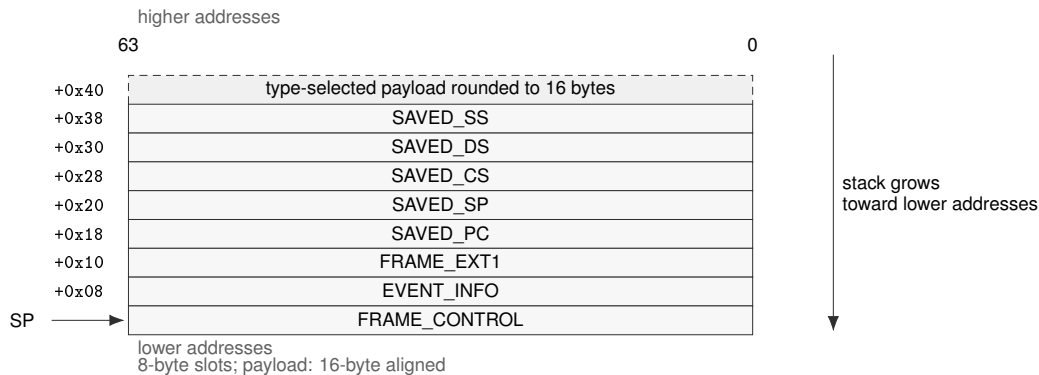


Figure 14-2. Architectural Event Frame

Table 14-6. Architectural Event Frame Fields

Offset	Slot	Meaning
+0x00	FRAME_CONTROL	frame shape, nesting metadata, saved <code>FLAGS</code> , and saved <code>STATUS</code>
+0x08	EVENT_INFO	event code in bits 31..0; bits 63..32 are zero
+0x10	FRAME_EXT1	repeat continuation when <code>repeat_saved</code> is one; otherwise zero
+0x18	SAVED_PC	saved program counter
+0x20	SAVED_SP	saved stack pointer
+0x28	SAVED_CS	saved code-segment image
+0x30	SAVED_DS	saved data-segment image
+0x38	SAVED_SS	saved stack-segment image
+0x40	ERROR_CODE	exception-specific error code; unassigned bits are zero
+0x48	FAULT_EA	numeric effective address before segment pre-translation
+0x50	FAULT_LINEAR	address after segment pre-translation
+0x58	EVENT_AUX	double-fault, machine-check, or bus-error auxiliary information

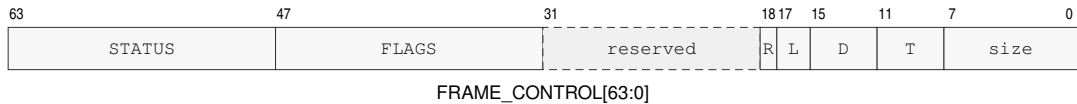


Figure 14-3. `FRAME_CONTROL` Format

D and T abbreviate depth and type.

Table 14-7. `FRAME_CONTROL` Fields

Field	Bits	Meaning
frame_size	0..7	total allocated frame size in 8-byte units
frame_type	8..11	optional payload layout code; it does not classify the event
saved_edepth	12..15	event depth before this entry
saved_esl	16..17	event stack level before this entry
repeat_saved	18	repeat continuation state is present in FRAME_EXT1
entry_flags	19..31	reserved entry flags; must be zero unless assigned by a later profile
flags	32..47	saved FLAGS image
status	48..63	saved STATUS image

ERET first validates the frame shape and nesting metadata. BASIC requires frame_size 8, ERROR requires 10, and PAGE_FAULT and AUXILIARY require 12. It also requires saved_edepth plus one to equal the current depth and requires saved_esl not to exceed the current stack level. If the saved STATUS image has EA clear, both saved values must be zero. Saved STATUS.PM identifies the interrupted privilege mode.

Only after those checks does ERET validate the saved control state, segment images, PC, SP, depth, and stack level. On success it restores FLAGS, STATUS, CS, DS, SS, SP, PC, event depth, stack level, and any saved repeat continuation as one commit. EVENT_INFO and the event-specific payload remain available to software.

14.7 Event Payloads

Payload starts at offset +0x40. Hardware allocates only the defined payload prefix, rounds it up to a 16-byte boundary, and zeros the padding. The frame-size unit remains one 8-byte slot.

Table 14-8. Architectural Event Frame Types and Sizes

Code	Type	Payload	Allocated	Total	Size	Last slot
0x0	BASIC	0	0	64	8	none
0x1	ERROR	8	16	80	10	ERROR_CODE
0x2	PAGE_FAULT	24	32	96	12	FAULT_LINEAR
0x3	AUXILIARY	32	32	96	12	EVENT_AUX

Table 14-9. Event-to-Frame-Type Assignment

Delivered event	Frame type
DEBUG_TRACE, BREAKPOINT, PRIVILEGE_FAULT	BASIC
ILLEGAL_INSTRUCTION, DIVIDE_ERROR, INVALID_CONTROL_STATE, FLOATING_POINT_FAULT	ERROR
PAGE_FAULT	PAGE_FAULT
DOUBLE_FAULT, MACHINE_CHECK, BUS_ERROR	AUXILIARY
interrupt and NMI	BASIC

The delivered event determines the frame type, and FRAME_EXT1 records repeat continuation. ERROR_CODE is defined separately by each exception and has zero in every unassigned bit. PAGE_FAULT extends that slot with effective-address and linear-address context. AUXILIARY includes all four named slots and permits EVENT_AUX to describe double-fault, machine-check, or bus-error state.

A misaligned AT=0 atomic memory operand uses the PAGE_FAULT frame with ERROR_CODE subtype ATOMIC_ALIGNMENT. FAULT_EA identifies the misaligned effective-address operand and FAULT_LINEAR contains its segment-pretranslated starting address. The fault is reported before the atomic instruction performs a memory read, memory write, or architectural result update.

14.7.1 Page-Fault Error Code

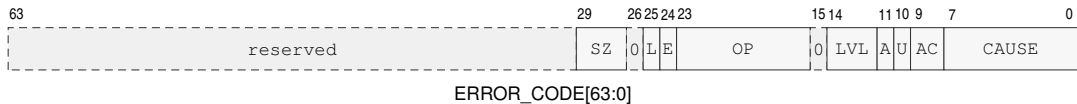


Figure 14-4. PAGE_FAULT ERROR_CODE Format

SZ, L, E, OP, LVL, and AC denote ACCESS_SIZE, FAULT_LINEAR_VALID, FAULT_EA_VALID, OPERAND, WALK_LEVEL, and ACCESS; A and U denote ATOMIC and USER_DOMAIN.

Table 14-10. PAGE_FAULT ERROR_CODE Fields

Bits	Field	Meaning
7..0	CAUSE	page-fault cause listed below
9..8	ACCESS	0 none, 1 read, 2 write, 3 execute
10	USER_DOMAIN	access selected the user-domain permission path
11	ATOMIC	access was an atomic read-modify-write
14..12	WALK_LEVEL	0 before or without a walk; 1 through 5 identify the PTE level
15	reserved	zero
23..16	OPERAND	zero-based explicit operand ordinal; 0xff for implicit fetch or stack access
24	FAULT_EA_VALID	FAULT_EA was produced
25	FAULT_LINEAR_VALID	FAULT_LINEAR was produced
26	reserved	zero
29..27	ACCESS_SIZE	0 unavailable or inapplicable; 1 B; 2 W; 3 L; 4 Q
63..30	reserved	zero

Table 14-11. PAGE_FAULT Causes

Value	Cause	Detection class
0	NOT_PRESENT	required PTE is not present
1	PERMISSION	accumulated permission excludes the access
2	MALFORMED_PTE	structurally invalid PTE
3	NONCANONICAL	noncanonical address
4	SEGMENT_BOUNDS	segment bounds failure
5	ATOMIC_ALIGNMENT	misaligned byte-addressed (AT=0) atomic operand
6	ADDRESS_TYPE	address or memory type rejects the access

Saved STATUS.PM records the originating privilege. USER_DOMAIN distinguishes the ordinary user permission path from the checked user-access path used by supervisor instructions such as MOVUC, MOVCU, and MOVUU. An atomic read-modify-write reports ACCESS=write. NOT_PRESENT and MALFORMED_PTE identify the actual PTE level. PERMISSION identifies the first level, in root-to-leaf

walk order, whose accumulated permissions exclude the requested access. A cause detected before a walk reports level zero.

ACCESS_SIZE records the architectural transfer width when one of the B, W, L, or Q widths applies. It is zero when no such width is available or applicable.

FAULT_EA is the numeric effective address before segment pre-translation. FAULT_LINEAR is the result after segment pre-translation. A value not produced is zero and has its corresponding validity bit clear.

14.7.2 Other Exception Error Codes

DIVIDE_ERROR.ERROR_CODE[7:0] is 0 for ZERO_DIVISOR and 1 for SIGNED_OVERFLOW; bits 63..8 are zero.

Table 14-12. ILLEGAL_INSTRUCTION Causes

Value	Cause	Meaning
0	INVALID_OPCODE	opcode is not assigned
1	INVALID_EA_FORM	effective-address form is not legal for the instruction
2	RESERVED_ENCODING	assigned instruction uses a reserved field value
3	UNAVAILABLE_EXTENSION	required extension is unavailable
4	EXPLICIT_ILLEGAL	architected ILLEGAL instruction
5	INSUFFICIENT_LENGTH	encoded record is shorter than the selected concrete form requires
6	INVALID_OPERAND_RELATION	writable operands use a prohibited overlap relation

Table 14-13. INVALID_CONTROL_STATE Causes

Value	Cause	Meaning
0	INVALID_SELECTOR	selector names no available resource
1	RESERVED_BITS	reserved bits are nonzero or changed illegally
2	INVALID_IMAGE	supplied architectural image is invalid
3	INVALID_TRANSITION	requested state transition is illegal

An unavailable RDPMC ID reports INVALID_SELECTOR; an illegal reserved-bit update reports RESERVED_BITS; an invalid segment, control-register, or SAVE/RESTORE image reports INVALID_IMAGE; and an illegal ERET, SYSRET, or privilege-state transition reports INVALID_TRANSITION.

FLOATING_POINT_FAULT.ERROR_CODE[4:0] is a cause bitmap: bits 4 through 0 are NV, DZ, OF, UF, and NX. All causes generated by the operation are reported together; bits 63..5 are zero.

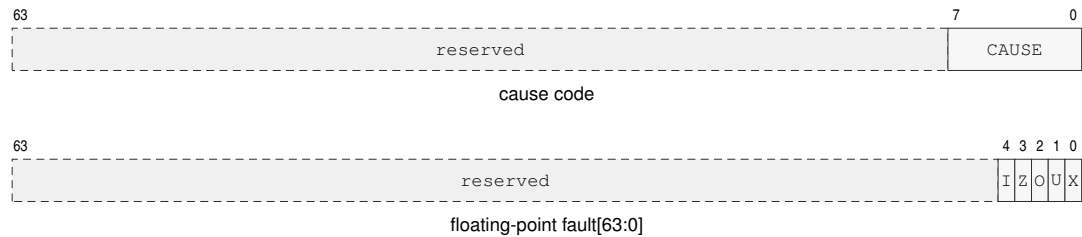


Figure 14-5. Simple Exception `ERROR_CODE` Formats

The cause-code row applies to `DIVIDE_ERROR`, `ILLEGAL_INSTRUCTION`, and `INVALID_CONTROL_STATE`. In the floating-point row, I, Z, O, U, and X abbreviate NV, DZ, OF, UF, and NX.

The bounds instructions report failure through `FLAGS.V`. Segment bounds failures use `PAGE_FAULT`. `BKPT` raises `BREAKPOINT`; `DEBUG_TRACE` is generated by the architectural trace mechanism.

14.7.3 Auxiliary Payloads

For an `AUXILIARY` frame, `ERROR_CODE` bit 26 is `EVENT_AUX_VALID`. Bits 24 and 25 are `FAULT_EA_VALID` and `FAULT_LINEAR_VALID` when the event can supply those addresses. Every unassigned bit is zero, and an auxiliary or address value not produced is zero with its validity bit clear.

For `DOUBLE_FAULT`, `ERROR_CODE[7:0]` identifies the delivery stage:

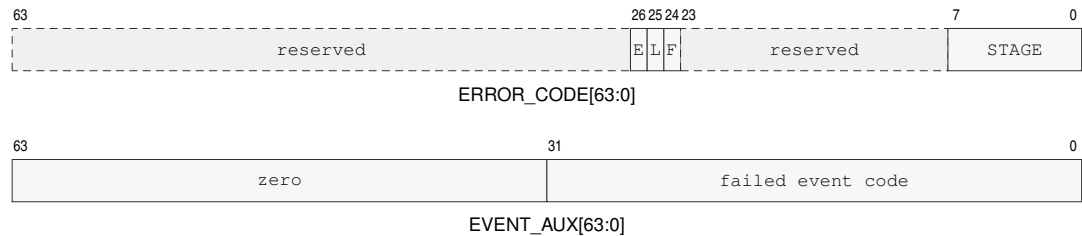


Figure 14-6. `DOUBLE_FAULT` Auxiliary Formats

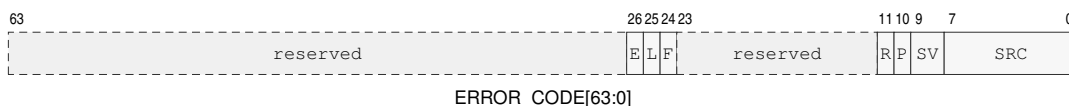
E, L, and F are `EVENT_AUX_VALID`, `FAULT_LINEAR_VALID`, and `FAULT_EA_VALID`.

Table 14-14. `DOUBLE_FAULT` Delivery Stages

Value	Stage	Failure
0	ENTRY_STATE	entry code or segment state validation
1	STACK_STATE	selected event-stack state
2	FRAME_ADDRESS	complete frame-address formation or translation
3	FRAME_STORE	complete frame storage

When valid, `EVENT_AUX[31:0]` contains the event code whose delivery failed and bits 63..32 are zero. A `FRAME_ADDRESS` or `FRAME_STORE` failure supplies `FAULT_EA` and `FAULT_LINEAR` when those values were produced.

For `MACHINE_CHECK`, the error code has this layout:

**Figure 14-7. MACHINE_CHECK ERROR_CODE Format**

E, L, and F are EVENT_AUX_VALID, FAULT_LINEAR_VALID, and FAULT_EA_VALID; P, R, SV, and SRC denote PRECISE, RETRY_SAFE, SEVERITY, and SOURCE.

Table 14-15. MACHINE_CHECK ERROR_CODE Fields

Bits	Field	Meaning
7..0	SOURCE	0 unknown; 1 core; 2 cache; 3 translation; 4 memory; 5 interconnect
9..8	SEVERITY	0 corrected; 1 recoverable; 2 fatal; 3 reserved
10	PRECISE	saved PC identifies the responsible instruction and saved CPU state precedes it
11	RETRY_SAFE	retry at that precise boundary cannot duplicate an external effect
23..12	reserved	zero
24	FAULT_EA_VALID	FAULT_EA was produced
25	FAULT_LINEAR_VALID	FAULT_LINEAR was produced
26	EVENT_AUX_VALID	EVENT_AUX was produced
63..27	reserved	zero

When valid, EVENT_AUX contains an implementation-defined syndrome. Associated effective and linear addresses are supplied when available.

RETRY_SAFE=1 requires PRECISE=1. The valid severity and continuation combinations are:

Table 14-16. MACHINE_CHECK Valid Continuation Combinations

Severity	PRECISE	RETRY_SAFE	Saved PC and continuation meaning
corrected	0	0	Trap; saved PC is the instruction following the corrected operation.
recoverable	0	0	Saved PC is the next instruction that would execute at the delivery boundary; saved CPU state is the state at that boundary. It does not identify the responsible operation.
recoverable	1	0	Saved PC identifies the responsible instruction and CPU state precedes it; retry may duplicate an external effect.
recoverable	1	1	Saved PC identifies the responsible instruction and CPU state precedes it; retry cannot duplicate an external effect.
fatal	0	0	Saved PC is the next instruction at the delivery boundary and is diagnostic only.
fatal	1	0	Saved PC identifies the responsible instruction and pre-instruction CPU state for diagnosis only.

All other combinations are invalid and are not generated. In particular, corrected machine checks never set either bit, PRECISE=0, RETRY_SAFE=1 is invalid for every severity, fatal machine checks never set RETRY_SAFE, and severity 3 is reserved. A fatal event provides no reliable continuation even when its diagnostic state is precise.

For BUS_ERROR, the error code has this layout:

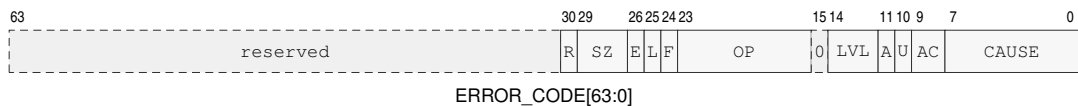


Figure 14-8. BUS_ERROR ERROR_CODE Format

E, L, and F are EVENT_AUX_VALID, FAULT_LINEAR_VALID, and FAULT_EA_VALID; SZ, OP, LVL, and AC denote ACCESS_SIZE, OPERAND, WALK_LEVEL, and ACCESS; A, U, and R denote ATOMIC, USER_DOMAIN, and RETRY_SAFE.

Table 14-17. BUS_ERROR ERROR_CODE Fields

Bits	Field	Meaning
7..0	CAUSE	0 no responder; 1 access denied; 2 timeout; 3 data error; 4 other
9..8	ACCESS	0 none, 1 read, 2 write, 3 execute
10	USER_DOMAIN	access selected the user-domain permission path
11	ATOMIC	access was an atomic read-modify-write
14..12	WALK_LEVEL	0 outside a walk; 1 through 5 identify the PTE level
15	reserved	zero
23..16	OPERAND	zero-based explicit operand ordinal; 0xff for implicit fetch or stack access
24	FAULT_EA_VALID	FAULT_EA was produced
25	FAULT_LINEAR_VALID	FAULT_LINEAR was produced
26	EVENT_AUX_VALID	EVENT_AUX was produced
29..27	ACCESS_SIZE	0 unavailable or inapplicable; 1 B; 2 W; 3 L; 4 Q
30	RETRY_SAFE	retry cannot duplicate an external effect
63..31	reserved	zero

A bus error during a page walk reports the level whose PTE transaction failed. When valid, EVENT_AUX contains the final memory-system address available for the failed transaction. For a slot-addressed access, this is the slot address and not a byte address.

14.8 Machine-Check and Bus-Error Continuation

A corrected MACHINE_CHECK is a trap, has both PRECISE and RETRY_SAFE clear, and saves the following instruction. A recoverable machine check uses the valid combinations in the auxiliary-payload table. When PRECISE is one, the saved PC identifies the responsible instruction boundary and CPU architectural state is the state from before that instruction. When it is zero, the saved PC is the next instruction that would execute at the delivery boundary and CPU state corresponds to that boundary; neither identifies the responsible operation. RETRY_SAFE may be one only with PRECISE=1 and additionally guarantees that returning to the responsible boundary cannot duplicate an external effect. A fatal machine check always clears RETRY_SAFE; PRECISE then describes diagnostic location quality and never guarantees reliable continuation.

BUS_ERROR is a synchronous precise fault. CPU architectural state remains uncommitted and the saved PC identifies the responsible instruction boundary. Its separate RETRY_SAFE bit states whether an external effect might already have occurred. Page-walk bus errors and ordinary byte-addressed-memory bus errors are retry-safe. A slot transaction reports the value determined by its acknowledgement result.

Software interprets the machine-check continuation fields before choosing a return or recovery action.

14.9 Repeat Continuation

When `FRAME_CONTROL.repeat_saved` is set, `FRAME_EXT1` records the active repeat context. When the bit is clear, `FRAME_EXT1` is zero. Event entry clears active architectural repeat state, and `ERET` restores it atomically with the saved PC and control state. During single-instruction repeat, `SAVED_PC` is the repeated body boundary. For `REPG`, software reconstructs the group start from `SAVED_PC` and the encoded byte-distance delta. `ERET` preserves the continuation unless supervisor software edits the frame before return.

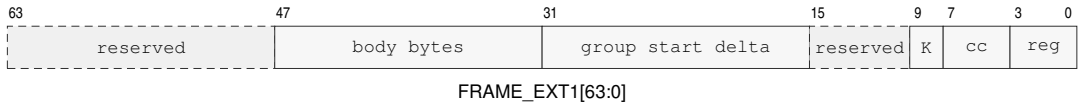


Figure 14-9. FRAME_EXT1 Repeat Continuation Format

K abbreviates `kind`.

Table 14-18. FRAME_EXT1 Repeat-Continuation Fields

Field	Bits	Meaning
<code>rep_reg</code>	0..3	general-purpose counter register R0..R15
<code>rep_cc</code>	4..7	condition code for <code>REPcc</code> ; T for unconditional <code>REP/REPG</code>
<code>repeat_kind</code>	8..9	0: <code>REP/REPcc</code> , 1: <code>REPG</code> , 2..3: reserved
<code>reserved</code>	10..15	reserved, must be zero
<code>group_start_delta</code>	16..31	unsigned byte distance from saved PC to the first grouped instruction; zero for <code>REP/REPcc</code>
<code>body_bytes</code>	32..47	grouped-repeat body length in bytes; zero for <code>REP/REPcc</code>
<code>reserved</code>	48..63	reserved, must be zero

14.10 Delivery Failure and Shutdown

Failure to validate the common entry context, establish the selected stack, or store a complete event frame is delivered as `DOUBLE_FAULT` on `DSS:DSP`. Failure while `DOUBLE_FAULT` is already being delivered enters `SHUTDOWN`. `SHUTDOWN` retires no further instructions and can be exited only by a platform reset. The frame save and all architectural entry state remain atomic, so a failed attempt never exposes a partially stored frame or partially changed execution context.

14.11 Event Reset and Context State

Table 14-19. Warm RESET Contract

Property	Architectural Rule
Scope	executing logical processor
Required privilege	supervisor
Serialization	complete prior memory effects and pending write-combining stores before reset state commits
Restart	running at BOOTPC after instruction-fetch synchronization
Other logical processors	unchanged

Table 14-20. Architectural Reset State

State	Reset Value
R0, R1, R2, R3, R4, R5, R6, R7, R8, R9, R10, R11, R12, R13, R14, R15, SP	0
FLAGS	0
STATUS	PM=1; all other STATUS bits 0
CS, DS, SS, GS0, GS1, GS2, GS3, GS4, GS5	0
PC	BOOTPC
PTCR, ASCR, ECR	0
EPC, ECS, EDS, SPC, SCS, SDS	0
URPC, URSP, URCS, URDS, URSS, URCTL	0
SSS, SSP, ISS, ISP, FSS, FSP, DSS, DSP	0
PMC, CYCLE, INSTRET, PTWALK	0
F0–F15, FSTATUS, FFLAGS, extension_state	0
hidden_repeat_state, hidden_load_reservation	invalid
hidden_current_edepth	0
hidden current event stack level	invalid
ECR.NMI_P	0
local_translation_cache, local_page_walk_cache, local_prefetch_state	invalid
coherent_data_and_instruction_cache_contents	retained as coherent contents
execution_state	running at BOOTPC
BOOTPC, BOOTCFG	platform supplied on cold reset; preserved by warm RESET

Warm RESET applies the complete generated state table to the executing logical processor only. Cold reset initializes BOOTPC and BOOTCFG before applying the same processor-state image to each logical processor selected by the platform reset event. Software initializes entry segments, exact entry PCs, and configured stack tops before setting ECR.V or entering user mode.

ECR, entry and stack registers, hidden event depth and stack level, and the user-return bank are per-logical-processor state. System software includes the applicable state when switching

supervisor contexts. Event frames are ordinary memory owned by the interrupted supervisor context.

15 CPUID FEATURE DISCOVERY

15.1 Overview

CPUID is the architectural discovery interface for optional instruction groups, processor-state save areas, implementation properties, and feature-specific parameters.

The CPUID instruction uses a selected Rn register as both query selector and result register. Software writes the selector before execution and reads the returned 64-bit value after execution.

CPUID exposes program-visible architectural information.

The normative instruction entry is CPUID.

15.2 Query Model

A query selector identifies a discovery class, leaf, and optional result index. The selector is a Q-sized value held in the selected Rn register.

CPUID is unprivileged under the base compatibility rules. For a logical processor, CPUID results remain stable until that logical processor is reset. A discovery result applies to execution on the logical processor that returned it. Software associates cached discovery state with that logical processor and uses results obtained on each logical processor before selecting its optional architectural behavior.

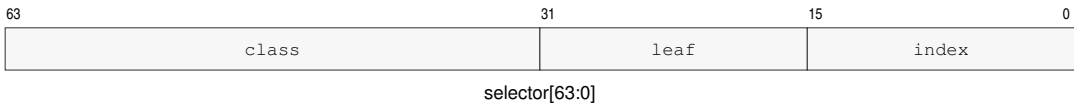
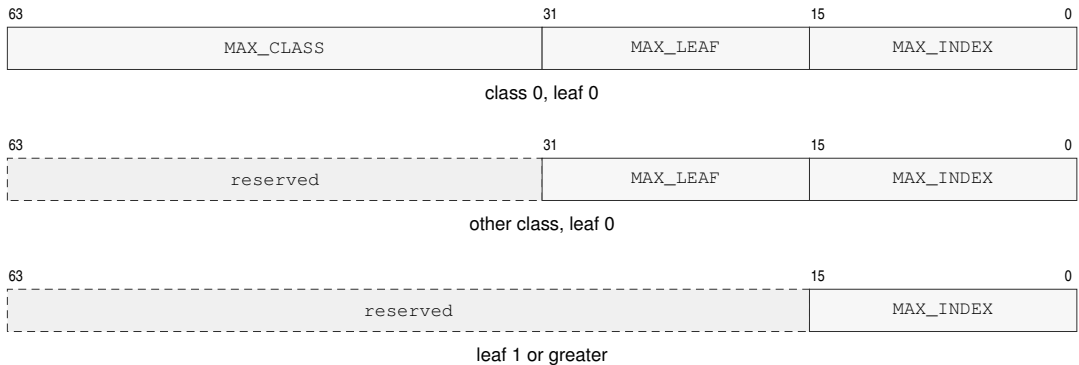


Figure 15-1. CPUID Query Selector Format

Table 15-1. CPUID Query Selector

Bits	Field	Meaning
0..15	index	result index within the selected leaf
16..31	leaf	information leaf within the selected class
32..63	class	CPUID information class

Index zero of every defined leaf is a common discovery header. Leaf-specific payload begins at index one. An unknown class, leaf, or index returns zero.

**Figure 15-2. CPUID Common Leaf Header Formats****Table 15-2. CPUID Common Leaf Header**

Query	Bits 63..32	Bits 31..16	Bits 15..0
class 0, leaf 0, index 0	MAX_CLASS	MAX_LEAF	MAX_INDEX
other class, leaf 0, index 0	reserved, zero	MAX_LEAF	MAX_INDEX
any class, leaf 1 or greater, index 0	reserved, zero	reserved, zero	MAX_INDEX

MAX_CLASS is the greatest class recognized by the implementation, MAX_LEAF is the greatest leaf recognized within the selected class, and MAX_INDEX is the greatest index recognized within the selected leaf. Each field is a maximum selector value, not a count. A leaf may be sparse below MAX_INDEX. Software ignores reserved bits in every returned value, so a later revision can assign them without invalidating existing discovery code.

15.3 Discovery Classes

Discovery classes partition architectural information by compatibility purpose. Software first checks the class and leaf directory before consuming optional feature bits.

The base class describes the mandatory scalar ISA and implementation identity. Optional groups are reported in the optional-extension directory.

15.4 Leaf Directory

Each discovery class may expose a leaf directory. A directory leaf reports which subordinate leaves are defined and how indexed leaves should be enumerated.

Table 15-3. CPUID Class and Leaf Directory

Class	Leaf	Name	Indexes	Summary
0x00000000	—	<i>Bedrock base ISA</i>	—	base identity, architecture and implementation revisions, vendor name, and processor name
0x00000000	0x0000	BASE_IDENTITY	0..18	common header, identity, revisions, vendor name, and processor name

Table 15-3 (continued)

Class	Leaf	Name	Indexes	Summary
0x00000001	—	<i>Optional extensions</i>	—	optional floating-point and transcendental groups
0x00000001	0x0000	EXTENSION_DIRECTORY	0..1	common header and optional architectural-group bits
0x00000001	0x0001	FPTRANSA_ACCURACY	0..0x0044	common header and sparse per-function accuracy contracts
0x00000002	—	<i>Implementation properties</i>	—	cache topology, performance counters, and SAVE/RESTORE area layout
0x00000002	0x0000	IMPLEMENTATION_DIRECTORY	0..0	common implementation-class header
0x00000002	0x0001	CACHE_TOPOLOGY	0..n	common header, maintenance granule, and cache-sharing descriptors
0x00000002	0x0002	PERFORMANCE_COUNTERS	0..3	common header and standard RDPMC-counter presence
0x00000002	0x0004	SAVE_AREA_LAYOUT	0..n	common header, SAVE/RESTORE sizes, bitmap, and component descriptors

15.5 Base Identity

Class 0x00000000, leaf 0x0000 reports MAX_LEAF=0 and MAX_INDEX=18 in its index-zero header. An implementation recognizing only the classes defined by this specification reports MAX_CLASS=2; an implementation that recognizes a higher implementation-defined class reports that greater selector. The base-identity payload is:

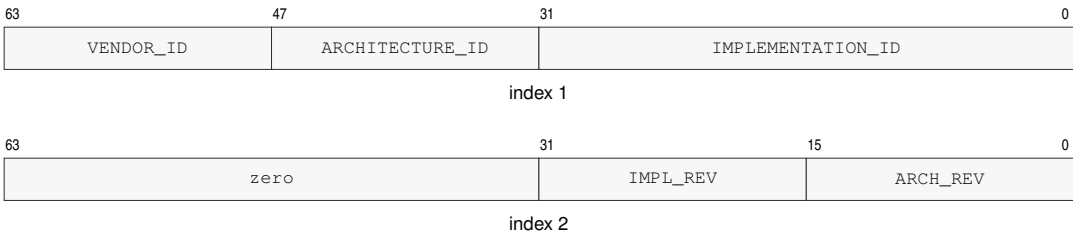


Figure 15-3. CPUID Base Identity Result Formats

Table 15-4. Base Identity Indexes

Index	Contents
1	bits 31..0 IMPLEMENTATION_ID; bits 47..32 ARCHITECTURE_ID; bits 63..48 VENDOR_ID
2	bits 15..0 ARCHITECTURE_REVISION; bits 31..16 IMPLEMENTATION_REVISION; bits 63..32 zero
3–10	64-byte vendor name
11–18	64-byte processor name

VENDOR_ID selects the vendor namespace in which IMPLEMENTATION_ID is interpreted. IMPLEMENTATION_ID is assigned within that VENDOR_ID namespace. ARCHITECTURE_ID selects

the base architectural contract, so the (VENDOR_ID, IMPLEMENTATION_ID) pair identifies an implementation and ARCHITECTURE_ID identifies the ISA contract it implements.

ARCHITECTURE_REVISION is an unsigned 16-bit, monotonically increasing compatibility level within one ARCHITECTURE_ID. A processor reporting revision N implements the base architectural behavior of every defined revision less than or equal to N . Optional instruction groups, state, and parameterized contracts are selected from their CPUID leaves and feature bits. IMPLEMENTATION_REVISION is an unsigned revision assigned by the vendor and is compared only when both VENDOR_ID and IMPLEMENTATION_ID match.

The name fields are UTF-8 byte strings in increasing index order. Within each 64-bit result, the first byte occupies bits 7..0 and subsequent bytes occupy successively higher bits. A name shorter than 64 bytes is terminated by a zero byte and all remaining bytes are zero. A 64-byte name has no terminator. Producers must not split a multi-byte UTF-8 code point at the field boundary; consumers replace an invalid sequence rather than reading beyond the field.

15.6 Optional-Extension Directory

Class 0x00000001, leaf 0x0000, index one reports optional architectural groups. The index-zero header reports MAX_LEAF=1 and MAX_INDEX=1.



Figure 15-4. Optional-Extension Directory Result

Table 15-5. Optional-Extension Feature Bits

Class	Leaf	Index	Bit	Feature	Extension
0x00000001	0x0000	1	0	FP	fpu
0x00000001	0x0000	1	1	FPTRANSA	fpu.transcendental_approx

Unlisted bits are reserved and read as zero. FPTRANSA requires FP and is set when at least one approximate transcendental contract is present.

15.7 FPTRANSA Accuracy Discovery

Class 0x00000001, leaf 0x0001 uses indexes 1 through 0x0044 as a sparse space of stable 16-bit accuracy-contract IDs. Index zero is the common header and reports MAX_INDEX=0x0044. A contract ID identifies the reference function, input domain, special-value behavior, error metric, exact anchors, and mathematical properties; it is independent of instruction encoding placement. An incompatible change receives a new ID, and a retired ID is not reused. The contracts defined here use CONTRACT_REVISION=1. Weakening a guarantee, changing the domain, or changing the error metric requires a new ID rather than a revision increment. An implementation that advertises a smaller certified ULP bound strengthens the existing contract and retains the same ID and revision.

The sparse ID space reserves 0x0001..0x000F for reduced-domain circular trigonometric functions, 0x0010..0x001F for inverse trigonometric functions, 0x0020..0x002F for hyperbolic and

inverse hyperbolic functions, 0x0030..0x003F for exponential functions, and 0x0040..0x004F for logarithmic functions. The low-nibble-zero selector of each family is unassigned, and actual contracts in each family begin at low nibble one.

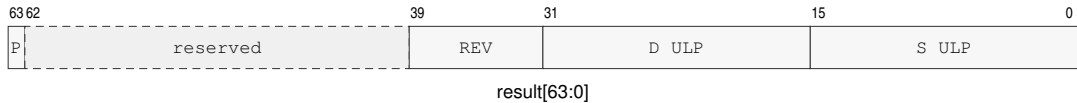


Figure 15-5. FPTRANSA Accuracy Result Format

P is PRESENT; REV is CONTRACT_REVISION; the D and S ULP fields use unsigned Q8.8.

Table 15-6. FPTRANSA Accuracy Result

Bits	Field	Meaning
0..15	S_MAX_ULP_Q8_8	certified worst-case S-format ULP bound, rounded upward to unsigned Q8.8
16..31	D_MAX_ULP_Q8_8	certified worst-case D-format ULP bound, rounded upward to unsigned Q8.8
32..39	CONTRACT_REVISION	value 1 for the contracts defined here
40..62	reserved	zero
63	PRESENT	the instruction and both S and D forms are available

For a certified bound K , the field value is $\lceil 256K \rceil$ and the advertised bound is the field divided by 256. Thus 0.5, 1, 1.5, 2, and 4 ULP encode as 0x0080, 0x0100, 0x0180, 0x0200, and 0x0400. A present contract has both fields in 0x0001..0x0400. Software that does not recognize the returned revision uses its fallback path.

Software first checks FP and FPTRANSA in EXTENSION_DIRECTORY, then queries the intended contract ID in FPTRANSA_ACCURACY. It uses the instruction only when PRESENT is set, the returned CONTRACT_REVISION is 1, and the S or D bound required by the call site is no larger than its quality threshold; otherwise it uses a software fallback. For example, a D-format FLOG2A path requiring at most 2 ULP accepts selector 0x0000000100010043 only when the returned D field is at most 0x0200.

The selector is $(0x00000001 \ll 32) | (0x0001 \ll 16) | \text{contract-id}$. For example, FSINA uses selector 0x0000000100010001, and FLOG2A uses 0x0000000100010043. An implementation that certifies FSINA at 1.5 ULP for S and 2 ULP for D returns 0x8000000102000180.

PRESENT=1 guarantees that the instruction and both S and D forms are architecturally usable under the returned contract. The S and D bounds are worst-case bounds over every input in the contract domain and apply to every execution on the logical processor that returned the CPUID result until reset. Executing the instruction with PRESENT=0 raises ILLEGAL_INSTRUCTION before operand or floating-point-state access.

Table 15-7. FPTRANSA Accuracy Contracts

Contract ID	Mnemonic	Reference	Domain	ISA maximum
0x0001	FSINA	$\sin(x)$ in radians	finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP

Table 15-7 (continued)

Contract ID	Mnemonic	Reference	Domain	ISA maximum
0x0002	FCOSA	$\cos(x)$ in radians	finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0003	FTANA	$\tan(x)$ in radians	finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0004	FSINCOSA	paired $\sin(x)$ and $\cos(x)$ in radians	finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0011	FASINA	$\text{asin}(x)$ in radians	finite x with $\text{abs}(x) \leq 1$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0012	FACOSA	$\text{acos}(x)$ in radians	finite x with $\text{abs}(x) \leq 1$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0013	FATANA	$\text{atan}(x)$ in radians	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0021	FSINHA	$\sinh(x)$	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0022	FCOSHA	$\cosh(x)$	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0023	FTANHA	$\tanh(x)$	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0024	FATANHA	$\text{atanh}(x)$	finite x with $\text{abs}(x) \leq 1$ after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0031	FETOXA	e^x	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0032	FETOXM1A	$e^x - 1$ without an intermediate rounded exponential	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0033	FTWOTOXA	2^x	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0034	FTENTOXA	10^x	all finite values and signed infinity after DAZ	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0041	FLOGNA	$\ln(x)$	positive finite values and positive infinity after DAZ; signed zero is the DZ boundary	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0042	FLOGNP1A	$\ln(1 + x)$ without an intermediate rounded addition	finite $x \geq -1$ and positive infinity after DAZ; -1 is the DZ boundary	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0043	FLOG2A	$\log_2(x)$	positive finite values and positive infinity after DAZ; signed zero is the DZ boundary	$S \leq 4$ ULP; $D \leq 4$ ULP
0x0044	FLOG10A	$\log_{10}(x)$	positive finite values and positive infinity after DAZ; signed zero is the DZ boundary	$S \leq 4$ ULP; $D \leq 4$ ULP

15.8 Implementation-Class Directory

Class 0x00000002, leaf 0x0000 consists of the common header. It reports `MAX_LEAF=4` and `MAX_INDEX=0`.

15.9 Cache-Topology Discovery

Class 0x00000002, leaf 0x0001 reports the cache-maintenance granule and the cache instances visible to the current logical processor. Index zero is the common header. Index one is the maintenance-properties result, and indexes 2 through MAX_INDEX are a dense array of cache descriptors. With *N* descriptors, MAX_INDEX=1+N.

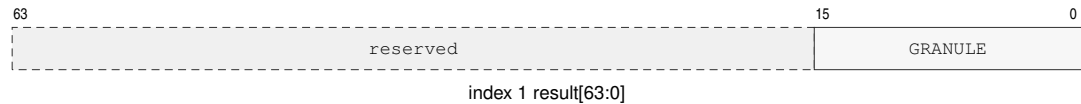


Figure 15-6. Cache-Maintenance Properties Result

GRANULE is MAINTENANCE_GRANULE_BYTES.

Table 15-8. Cache-Maintenance Properties

Index	Bits	Field	Meaning
1	15..0	MAINTENANCE_GRANULE_BYTES	one-block cache-maintenance granule in bytes
1	63..16	reserved	zero

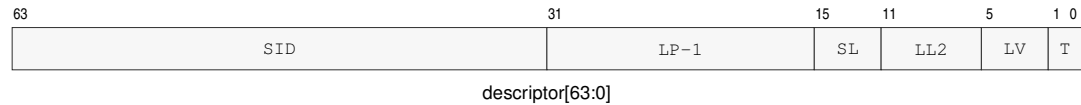


Figure 15-7. Cache-Topology Descriptor Format

SID, LP-1, SL, LL2, LV, and T denote SHARING_ID, SHARING_LP_COUNT_MINUS1, SHARING_LEVEL, LINE_LOG2, LEVEL, and TYPE.

Table 15-9. Cache-Topology Descriptor

Bits	Field	Meaning
1..0	TYPE	1 data; 2 instruction; 3 unified
5..2	LEVEL	cache level, in the range 1 through 15
11..6	LINE_LOG2	cache-line bytes are 1 shifted left by this value
15..12	SHARING_LEVEL	nested cache-sharing scope; zero is one logical processor
31..16	SHARING_LP_COUNT_MINUS1	logical-processor count in the sharing scope, minus one
63..32	SHARING_ID	identifier of the processor set at the reported sharing level

MAINTENANCE_GRANULE_BYTES is a nonzero power of two no greater than 4096 and has the same value on every logical processor in one normal-memory coherence domain. Every reported cache-line size divides the maintenance granule. A descriptor has TYPE 1, 2, or 3 and a nonzero LEVEL. Descriptors are ordered by increasing cache level, cache type, sharing level, and sharing ID.

SHARING_LEVEL values describe strictly nested processor sets. The pair (SHARING_LEVEL, SHARING_ID) is equal exactly when two descriptors name the same logical-processor set. At sharing level zero, SHARING_LP_COUNT_MINUS1 is zero. At a greater sharing level, the count field equals the number of logical processors in that set minus one. The tuple of cache type, cache level, sharing level, and sharing ID is unique within the leaf.

15.10 Performance-Counter Discovery

Class 0x00000002, leaf 0x0002 uses indexes 1 through 65535 as RDPMC counter IDs. Index zero is the common header. Result bit 0 is PRESENT; bits 63..1 are reserved and read as zero. CYCLE, INSTRET, and PTWALK IDs 1, 2, and 3 are mandatory and therefore always report present; the index-zero header reports MAX_INDEX=3 when no implementation-defined counter has a greater ID. An implementation-defined ID may be used only when its query reports present.

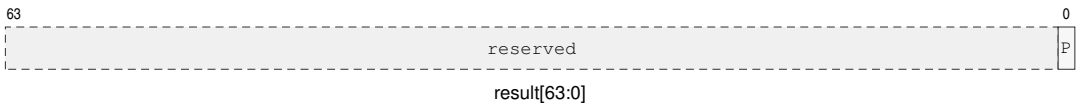


Figure 15-8. Performance-Counter Presence Result

P is PRESENT.

15.11 SAVE-Area Layout Discovery

Class 0x00000002, leaf 0x0004 describes the memory image used by SAVE and RESTORE. Index zero is the common header. Indexes one and two describe the complete image and its fixed block. Each available extension component contributes descriptor A followed by descriptor B, so for N components $\text{MAX_INDEX}=2+2N$. Component descriptors are ordered by increasing COMPONENT_ID.

Table 15-10. SAVE-AREA-LAYOUT Indexes

Index	Contents
0	common CPUID leaf header
1	SAVE_AREA_SIZE_BYTES in bits 63..0
2	FIXED_SIZE_BYTES[15:0], COMPONENT_COUNT[31:16], BITMAP_WORDS[47:32], FMT[51:48]; bits 63..52 are zero
3+2i	component descriptor A for zero-based component ordinal i
4+2i	component descriptor B for zero-based component ordinal i

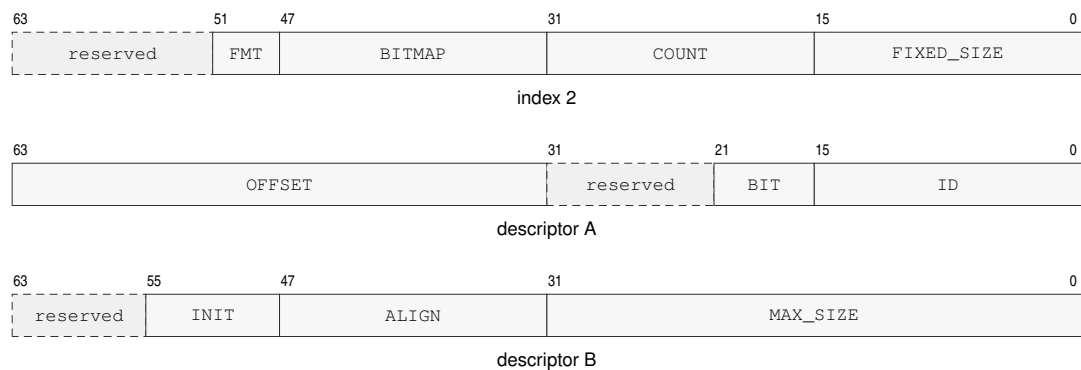


Figure 15-9. SAVE-Area Layout CPUID Result Formats

Diagram labels abbreviate the field names in the descriptor tables below; BITMAP, COUNT, and FIXED_SIZE are BITMAP_WORDS, COMPONENT_COUNT, and FIXED_SIZE_BYTES.

Table 15-11. SAVE Component Descriptor A

Bits	Field	Meaning
15..0	COMPONENT_ID	stable component identifier
21..16	BITMAP_BIT	bit selecting this component
31..22	reserved	zero
63..32	OFFSET_BYTES	offset from the save-area base

Table 15-12. SAVE Component Descriptor B

Bits	Field	Meaning
31..0	MAX_SIZE_BYTES	allocated component size
47..32	ALIGNMENT_BYTES	required component alignment
55..48	INIT_POLICY	1 restores the component-defined initial state when its bitmap bit is clear
63..56	reserved	zero

Table 15-13. Floating-Point SAVE Component

Component Offset	State	Contents
0x000..0x07f	F0_F15	F0 through F15 in ascending register order
0x080..0x087	FFLAGS	FFLAGS in bits 15..0; remaining bits zero
0x088..0x08f	FSTATUS	FSTATUS in bits 15..0; remaining bits zero

SAVE_AREA_SIZE_BYTES is the complete byte range validated and accessed by SAVE or RESTORE. It is at least FIXED_SIZE_BYTES, covers the end of every component, and is a multiple of 64 bytes. FIXED_SIZE_BYTES is 0x00c0, BITMAP_WORDS is 1, and FMT is zero for the save-image format defined by this specification.

Each descriptor names one available extension component. Component IDs and bitmap bits are unique. Component regions begin at or after the fixed block, meet their reported alignment, and

do not overlap. A set state-block bitmap bit selects the descriptor carrying the same `BITMAP_BIT`. `INIT_POLICY=1` means that `RESTORE` reconstructs the component-defined initial state when that component's bitmap bit is clear.

When the FP extension is present, component ID 1 uses bitmap bit 0, offset `0x00c0`, size `0x00c0`, and alignment 64. Its initial state is F0 through F15 equal to positive zero, `FFLAGS` equal to zero, and `FSTATUS` equal to zero.

15.12 Bit Fields

`CPUID` bit fields are feature predicates unless the leaf description says they encode a numeric value. Reserved result bits are ignored by software.

A set feature bit only permits software to use the corresponding architectural behavior; the instruction's normal privilege, operand, and compatibility checks still apply.

The FP and `FPTRANSA` predicates report the base floating-point and approximate transcendental floating-point groups. Numeric fields use names that describe their unit. `SAVE_AREA_SIZE_BYTES`, `OFFSET_BYTES`, `MAX_SIZE_BYTES`, and `ALIGNMENT_BYTES` are byte counts.

16 PROCESSOR-STATE SAVE AND RESTORE

SAVE and RESTORE use a CPUID-defined memory image whose base is 4 KiB aligned. The fixed block contains the base header, state-block bitmap, R0 through R15, and GS0 through GS5 images.

The header carries FLAGS, STATUS, format, and GS-validity state. GS_VALID selects which GS images are meaningful. SAVE may omit an architecturally clean extension component by clearing its bitmap bit, and RESTORE uses that bitmap as authoritative when reconstructing extension state.

SAVE writes FMT=0, sets GS_VALID=0x3f, and records GS0 through GS5. RESTORE replaces each GS_n whose corresponding GS_VALID bit is set and preserves each GS_n whose bit is clear. It validates every selected segment image before changing architectural state.

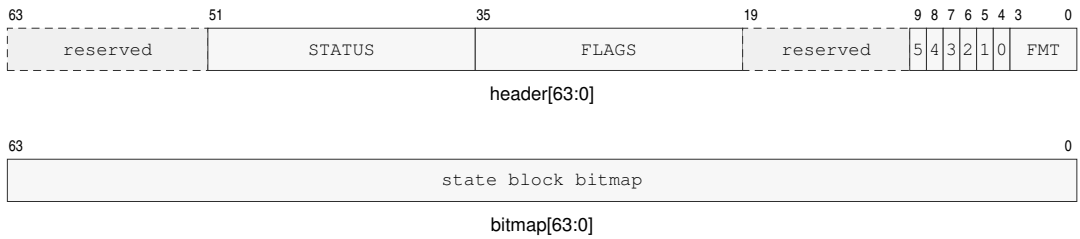
SAVE sets bitmap bits only for components described by its CPUID SAVE_AREA_LAYOUT leaf. RESTORE accepts only the defined format and only bitmap bits that name CPUID-described components for that format. RESTORE raises INVALID_CONTROL_STATE before changing architectural state when the format is unrecognized or a set bitmap bit does not name a CPUID-described component for that format.

An extension component is architecturally clean when its state is known to equal its component-defined initial state. Reset and RESTORE of a clear component bit make that component clean. A committed write that can make a component differ from its initial state makes it modified; RESTORE of a non-initial component image also makes it modified. An implementation may mark a component clean whenever it establishes that the complete component again equals its initial state. SAVE does not change clean or modified state. SAVE may clear a bitmap bit only for a clean component. RESTORE of a clear, described component bit installs that component's initial state.

User-mode RESTORE ignores saved supervisor-only STATUS state. Supervisor RESTORE validates the saved STATUS image by the ordinary privileged write rules before making it visible.

Software obtains every extension component's offset and maximum size from CPUID SAVE_AREA_LAYOUT. Each component starts on a 64-byte boundary. When floating-point state is enabled, its component contains F0 through F15, FFLAGS, and FSTATUS in the layout reported in the CPUID chapter. The floating-point component's initial state is positive zero in F0 through F15 and zero in FFLAGS and FSTATUS.

The normative instruction entries are SAVE and RESTORE.

**Figure 16-1. SAVE/RESTORE Base Header**

The labels 5 through 0 are the six `GS_VALID` bits. `SAVE` writes `GS_VALID=0x3f` and `FMT=0`.

Table 16-1. SAVE/RESTORE Fixed Base Block

offset from the 4 KiB-aligned save-area base					offsets increase downward			
0x000-0x00f	BASE HEADER 0x000 · 8 bytes				STATE BLOCK BITMAP 0x008 · 8 bytes			
0x010-0x04f	R0	R1	R2	R3	R4	R5	R6	R7
0x050-0x08f	R8	R9	R10	R11	R12	R13	R14	R15
0x090-0x0bf	GS0	GS1	GS2	GS3	GS4	GS5	RESTORE uses slots selected by GS_VALID	
0x0c0+	EXTENSION STATE COMPONENTS CPUID-defined slots · each component starts on a 64-byte boundary							

Each R_n and GS_n cell is one 8-byte slot.

R_n=0x010+8n; GS_n=0x090+8n.

PART IV

BEDROCK ARCHITECTURE

Instruction Set Reference

READING AN INSTRUCTION DESCRIPTION

Entry Organization

Each instruction entry presents its normative Operation, then Assembler Syntax, Attributes, and structured status metadata. Detailed Semantics follows when more space is needed, before the concrete encoding forms. Brief, non-normative summaries appear only in the instruction-set summary tables. Each mnemonic begins on a new page so its form diagrams, field explanations, and side-effect text stay together as one reference unit.

Mnemonics and Suffixes

The B, W, L, and Q suffixes select 8-, 16-, 32-, and 64-bit integer sizes; S and D select 32- and 64-bit floating-point scalar sizes where defined.

When an instruction form encodes a size selector *z*, its definition maps *z* to the suffix set shown for that form. A mnemonic without a size suffix has a single architecturally defined size or no data-size operand. Conditional mnemonic forms use the condition names listed in the Flags and Condition Codes chapter.

CMPJcc, DJcc, IJcc, Jcc, MOVcc, SETcc, TESTJcc, and FMOVcc place the condition suffix in the mnemonic. A form with only one architectural size omits the size suffix.

Operand and Status Notation

R_n names a general-purpose register; SP and PC are special registers outside that encoding namespace. An <ea> operand names a compact effective-address field and any appended EA payload.

FLAGS, STATUS, FFLAGS, and FSTATUS are architectural state registers. Instruction descriptions state whether a form updates, preserves, reads, or ignores each status field.

In instruction pseudocode, Operand Read applies the addressing mode and operation size. Operand Write stores the selected-size result; a sub-Q R_n write preserves upper bits unless the instruction defines another rule. EA Address computes an address without reading memory. A control-target memory form reads its declared-width target, while a control-target register or immediate form supplies the target value directly. A selector may name an R_n register, a segment register, or an encoded immediate. FLAGS ZNCV refers to the integer Z, N, C, and V bits, while FFLAGS names the accrued floating-point exception and status bits.

Encoding Forms

In an encoding diagram, fixed 0 and 1 fields identify opcode bits, and the notes below decode lettered fields. An <ea> field is the compact effective-address field and may append EA payload bytes in operand order.

For an extended encoding, the four-bit L field selects a 3+L-byte instruction record. Required bytes is the space consumed by the opcode and selected operand payloads. Every larger encoded length through 18 bytes is valid and the trailing bytes are uninterpreted padding. The LEN *n*, spelling records an explicit total length.

17 GENERAL INSTRUCTIONS

17.1 Summary

Table 17-1. General Instructions Summary

Mnemonic	Brief description
ABS	Performs the absolute value operation.
ADC	Performs the add with carry operation.
ADD	Adds the source operand to the destination operand.
AFENCE	Order prior memory operations before later memory operations.
AND	Computes the bitwise AND of the source and destination operands.
BCHG	Performs the bit change operation.
BCLR	Performs the bit clear operation.
BKPT	Raise the breakpoint exception.
BNDSII	Checks signed value membership in the inclusive-inclusive interval [lo, hi].
BNDSIX	Checks signed value membership in the inclusive-exclusive interval [lo, hi).
BNDSXI	Checks signed value membership in the exclusive-inclusive interval (lo, hi].
BNDSXX	Checks signed value membership in the exclusive-exclusive interval (lo, hi).
BNDUII	Checks unsigned value membership in the inclusive-inclusive interval [lo, hi].
BNDUIX	Checks unsigned value membership in the inclusive-exclusive interval [lo, hi).
BNDUXI	Checks unsigned value membership in the exclusive-inclusive interval (lo, hi].
BNDUXX	Checks unsigned value membership in the exclusive-exclusive interval (lo, hi).
BSET	Performs the bit set operation.
BTEST	Performs the bit test operation.
CALL	Performs the call operation.
CALLcc	Performs a call when the condition is true.
CLMUL	Performs the carryless multiply operation.
CLMULH	Performs the carryless multiply high operation.
CLR	Performs the clear operation.
CLRF	Clears selected integer FLAGS bits.
CLS	Performs the count leading set bits operation.
CLZ	Performs the count leading zeros operation.
CMP	Computes a comparison result and updates condition codes without storing the temporary value.
CMPJcc	Compares two Rn registers and branches on the selected condition without writing FLAGS.
CMPXCHG	Performs the compare and exchange operation.
CPUID	Queries architectural discovery information through a selected Rn register.
CTS	Performs the count trailing set bits operation.
CTZ	Performs the count trailing zeros operation.
DEC	Performs the decrement operation.
DECF	Decrements the destination operand and updates integer FLAGS.
DIVMODS	Performs the signed divide with remainder operation.
DIVMODU	Performs the unsigned divide with remainder operation.
DIVS	Performs the signed divide operation.
DIVU	Performs the unsigned divide operation.
DJcc	Performs the decrement and jump conditional operation.

Table 17-1 (continued)

Mnemonic	Brief description
IJcc	Performs the increment and jump conditional operation.
EXTRACT	Extracts a selected-width value from a concatenated pair of Rn registers.
EXTSL	Performs the sign extend to long operation.
EXTSQ	Performs the sign extend to quad operation.
EXTSW	Performs the sign extend to word operation.
EXTZL	Performs the zero extend to long operation.
EXTZQ	Performs the zero extend to quad operation.
EXTZW	Performs the zero extend to word operation.
FETCHADD	Performs the atomic fetch add operation.
FETCHAND	Performs the atomic fetch and operation.
FETCHOR	Performs the atomic fetch or operation.
FETCHSUB	Performs the atomic fetch subtract operation.
FETCHXOR	Performs the atomic fetch xor operation.
FLSHDCACHE	Write back and invalidate one cache-maintenance block.
HALT	Halt the executing logical processor until an asynchronous architectural event is admitted.
ILLEGAL	Raise illegal-instruction exception.
INC	Performs the increment operation.
INCF	Increments the destination operand and updates integer FLAGS.
INVASID	Invalidate TLB entries for ASID.
INVDCACHE	Invalidate one data-cache maintenance block.
INVICACHE	Invalidate one local instruction-cache maintenance block.
INVPAGE	Invalidate TLB entry for linear page.
INVTLB	Invalidate all TLB entries.
ERET	Return from an architectural event frame.
JMP	Transfers control to the target address.
Jcc	Transfers control when the encoded condition is true.
LCALL	Pushes a far return frame and atomically transfers control to a new CS:PC pair.
LEA	Performs the load effective address operation.
LJMP	Atomically transfers control to a new CS:PC pair.
LRET	Pops a far return frame and atomically restores CS:PC.
MAXS	Performs the signed maximum operation.
MAXU	Performs the unsigned maximum operation.
MINS	Performs the signed minimum operation.
MINU	Performs the unsigned minimum operation.
MODS	Performs the signed modulo operation.
MODU	Performs the unsigned modulo operation.
MOV	Copies the source operand to the destination operand.
MOVcc	Performs the move conditional operation.
MOVCU	Copies from a current-domain source to user-domain memory.
MOVNT	Stores an Rn value to memory with a non-temporal access hint.
MOVUC	Copies from user-domain memory to a current-domain destination.
MOVUU	Copies from user-domain memory to user-domain memory.
MUL	Multiplies the destination by the source and keeps the low half.
MULHS	Performs the signed multiply high operation.
MULHSU	Performs the signed by unsigned multiply high operation.

Table 17-1 (continued)

Mnemonic	Brief description
MULHU	Performs the unsigned multiply high operation.
NEG	Performs the negate operation.
NOP	No architectural state change.
NOT	Performs the bitwise not operation.
OR	Computes the bitwise OR of the source and destination operands.
PARITY	Computes operand parity and writes the predicate to an Rn register.
POP	Pops one 64-bit stack value into an Rn or SREG destination.
POPCNT	Performs the population count operation.
POPP	Pops one canonical Rn register pair selected by an inline pair index.
PREFETCH	Hint that EA will be read soon; no architectural fault unless policy says otherwise.
PREFETCHNT	Hint that EA will be read with non-temporal locality.
PTQUERY	Non-destructively reads the raw PTE selected at one page-table level.
PUSH	Pushes one Rn value, SREG image, or the current CS image onto the stack.
PUSHP	Pushes one canonical Rn register pair selected by an inline pair index.
RDCR	Read a privileged control register.
RDFLAGS	Read the integer condition-code flags into an Rn register.
RDPMC	Read the 64-bit performance counter selected by a constant counter ID.
RDSEG	Read an SREG or the current CS image into an Rn register.
RDSTATUS	Read the processor STATUS register into an Rn register.
REPG	Repeats the following straight-line instruction body using an Rn counter.
REPcc	Repeats the next instruction while the counter and condition remain active.
RESET	Perform warm reset; preserve BOOTPC and BOOTCFG.
RESTORE	Restore processor state from a CPUID-defined save area.
RET	Performs the return operation.
REVBYTE	Performs the reverse bytes operation.
RFENCE	Order prior reads before later reads.
ROL	Performs the rotate left operation.
ROR	Performs the rotate right operation.
SAR	Performs the shift arithmetic right operation.
SAVE	Save modified processor state to a CPUID-defined save area.
SBB	Performs the subtract with borrow operation.
SEGLEA	Performs the segment load effective address operation.
SET	Writes integer one to the destination operand.
SETcc	Performs the set conditional operation.
SETF	Sets selected integer FLAGS bits.
SHL	Performs the shift left operation.
SHR	Performs the shift right operation.
SUB	Subtracts the source operand from the destination operand.
SWPT	Performs the switch page table operation.
SWPTA	Switches the page table and enables address-space context matching.
SYNCCACHE	Synchronize one cache-maintenance block for local instruction fetch.
SYSCALL	Enter the supervisor-call path through SPC/SCS/SDS and the configured call stack.
SYSRET	Return to user mode from the user-return control-register bank.
TEST	Computes a comparison result and updates condition codes without storing the temporary value.

Table 17-1 (continued)

Mnemonic	Brief description
TESTJcc	Tests two Rn registers and branches on the selected condition without writing FLAGS.
TRACE	Records a marker and instruction boundary in the implementation trace stream.
VTOP	Non-destructively queries the current translation of a linear address.
WAIT	Pause execution for a finite implementation-selected interval.
WFENCE	Order prior writes before later writes.
WRBKDCACHE	Write back one data-cache maintenance block.
WRCR	Write a privileged control register from an Rn register.
WRFLAGS	Write the integer condition-code flags from an Rn register.
WRSEG	Write an SREG segment image from an Rn register.
WRSTATUS	Write the processor STATUS register from an Rn register.
XCHG	Performs the exchange operation.
XOR	Computes the bitwise XOR of the source and destination operands.
YIELD	Hints to the scheduler or implementation that the current thread may yield.

ABS

Absolute Value

ABS

Operation: Replaces the selected-width destination value with its two’s-complement absolute value. The most-negative value wraps to itself and does not trap.

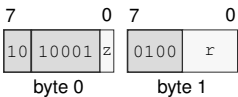
Assembler Syntax: ABS.X(z:L/Q) Rn(r)
ABS.X(z:B/W) <ea>
ABS.X(z:L/Q) <ea>

Attributes: Class = integer
Family = integer unary
Privilege = unprivileged
Length = 2-13 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

ABS.X(z:L/Q) Rn(r)
short · 2 bytes · unprivileged

Instruction Format:



Format — Instruction format for ABS.X(z:L/Q) Rn(r)

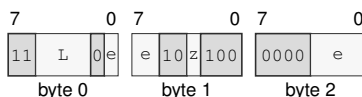
Instruction Fields:

- Size field** z—Selects L/Q.
- Register field** r—Selects the operand.

ABS.X(z:B/W) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for ABS.X(z:B/W) <ea>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

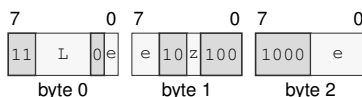
Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects B/W.

ABS.X(z:L/Q) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for ABS.X(z:L/Q) <ea>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register except $R_N(r)$; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects L/Q.

ADC**Add With Carry****ADC**

Operation: Adds the selected-width source value and the incoming C flag to the destination, writes the truncated sum, and updates Z, N, C, and V from the complete addition.

Assembler Syntax: `ADC.X(z:B/W/L/Q) Rn(s), Rn(d)`
`ADC.X(z:B/W/L/Q) Rn(s), <ea>(e)`
`ADC.X(z:B/W/L/Q) <ea>(e), Rn(s)`

Attributes: Class = integer
Family = integer alu
Privilege = unprivileged
Length = 3-14 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	result == 0
N	most_significant_bit(result)
C	unsigned carry out
V	signed overflow

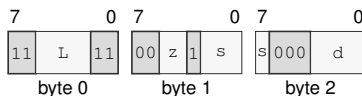
Let n be the selected width, d and s the unsigned operand bit patterns, and c the incoming C flag. The written result is

$$r = (d + s + c) \bmod 2^n.$$

Z is set when $r = 0$, and N receives the most-significant bit of r . C reports an unsigned carry out of either addition. V reports signed overflow for the complete $d + s + c$ operation.

Instruction Forms:

ADC.X(z:B/W/L/Q) Rn(s), Rn(d)
 medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for ADC.X(z:B/W/L/Q) Rn(s), Rn(d)

Instruction Fields:

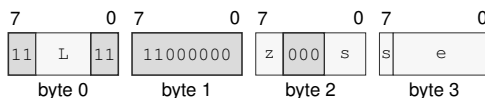
Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

ADC.X(z:B/W/L/Q) Rn(s), <ea>(e)
 long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for ADC.X(z:B/W/L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

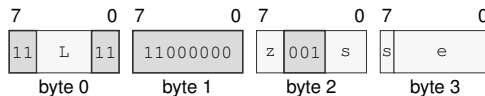
Allowed: Register except Rn(r); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

ADC.X(z:B/W/L/Q) <ea>(e), Rn(s)

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for ADC.X(z:B/W/L/Q) <ea>(e), Rn(s)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register except Rn(r); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

ADD**Add****ADD**

Operation: Performs selected-width modular addition of the source and destination values and writes the low result bits back to the destination.

Assembler Syntax:

```

ADD.Q 8, SP
ADD.X(z:L/Q) Rn(s), Rn(d)
ADD.Q imm8(i), SP
ADD.Q <imm16s>, SP
ADD.Q <imm32s>, SP
ADD.X(z:B/W) Rn(s), <ea>(e)
ADD.X(z:L/Q) Rn(s), <ea>(e)
ADD.X(z:B/W) <ea>(e), Rn(d)
ADD.X(z:L/Q) <ea>(e), Rn(d)

```

Attributes:

```

Class = integer
Family = integer alu
Privilege = unprivileged
Length = 1-13 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

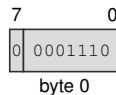
```

Instruction Forms:

```

ADD.Q 8, SP
extrashort · 1 byte · unprivileged

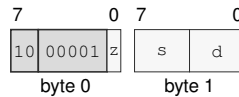
```

Instruction Format:

Format — Instruction format for ADD.Q 8, SP

ADD.X(z:L/Q) Rn(s), Rn(d)
 short · 2 bytes · unprivileged

Instruction Format:



Format — Instruction format for ADD.X(z:L/Q) Rn(s), Rn(d)

Instruction Fields:

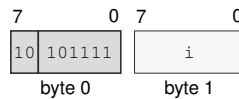
Size field z—Selects L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

ADD.Q imm8(i), SP
 short · 2 bytes · unprivileged

Instruction Format:



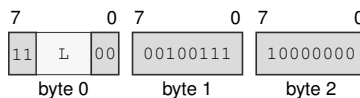
Format — Instruction format for ADD.Q imm8(i), SP

Instruction Fields:

Immediate field i—Encodes the immediate value.

ADD.Q <imm16s>, SP
 medium · 5 bytes · unprivileged

Instruction Format:



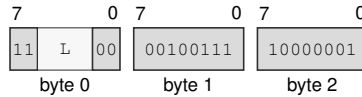
Format — Instruction format for ADD.Q <imm16s>, SP

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

ADD.Q <imm32s>, SP
 medium · 7 bytes · unprivileged

Instruction Format:



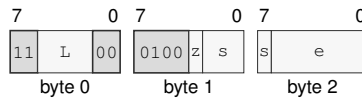
Format — Instruction format for ADD.Q <imm32s>, SP

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

ADD.X(z:B/W) Rn(s), <ea>(e)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for ADD.X(z:B/W) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

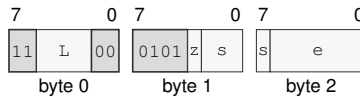
Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

ADD.X(z:L/Q) Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for ADD.X(z:L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

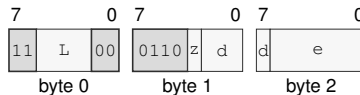
Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

ADD.X(z:B/W) <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for ADD.X(z:B/W) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

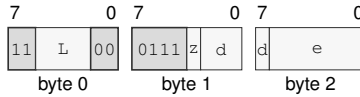
EA availability

Allowed: Register except Rn(r); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

ADD.X(z:L/Q) <ea>(e), Rn(d)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for ADD.X(z:L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

AFENCE

Address Fence

AFENCE

Operation:

Acts as a full cumulative fence. Every earlier memory read and write is ordered before every later memory read and write on the same hardware thread, and memory effects observed before the fence are propagated before effects ordered after it. It also requires completion of earlier slot-addressed transactions before later slot-addressed transactions. Every AFENCE participates in the single global sequentially consistent order. It performs no memory access and changes no register or flag.

Assembler Syntax:

AFENCE

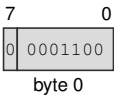
Attributes:

Class = system
Family = core control
Privilege = any
Length = 1 byte
Repeat = none

Instruction Forms:

AFENCE
extrashort · 1 byte · any

Instruction Format:



Format — Instruction format for AFENCE

AND**Bitwise And****AND**

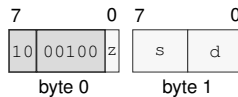
Operation: Writes the selected-width bitwise conjunction of the source and destination values to the destination.

Assembler Syntax: `AND.X(z:L/Q) Rn(s), Rn(d)`
`AND.X(z:B/W) Rn(s), <ea>(e)`
`AND.X(z:L/Q) Rn(s), <ea>(e)`
`AND.X(z:B/W) <ea>(e), Rn(d)`
`AND.X(z:L/Q) <ea>(e), Rn(d)`

Attributes: Class = integer
Family = integer alu
Privilege = unprivileged
Length = 2-13 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

`AND.X(z:L/Q) Rn(s), Rn(d)`
short · 2 bytes · unprivileged

Instruction Format:

Format — Instruction format for AND.X(z:L/Q) Rn(s), Rn(d)

Instruction Fields:

Size field *z*—Selects L/Q.

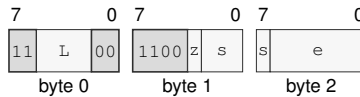
Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

AND, X(z:B/W) Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for AND.X(z:B/W) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

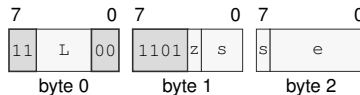
Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

AND, X(z:L/Q) Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for AND.X(z:L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

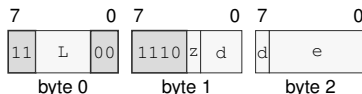
EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

AND.X(z:B/W) <ea>(e), Rn(d)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for AND.X(z:B/W) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

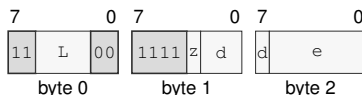
EA availability

Allowed: Register except Rn(r); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

AND.X(z:L/Q) <ea>(e), Rn(d)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for AND.X(z:L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BCHG**Bit Change****BCHG**

Operation: Complements the destination bit selected by the low six bits of the bit index and sets Z when that bit was previously clear. A memory destination is read by an ordinary selected-width load and updated by an ordinary selected-width store.

Assembler Syntax: `BCHG Rn(b), <ea>(e)`
`BCHG imm6(i), <ea>(e)`

Attributes: Class = integer
Family = integer bitfield
Privilege = unprivileged
Length = 4-14 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

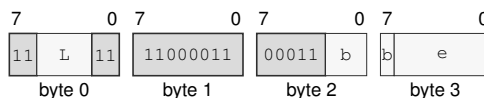
Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	old_bit == 0
N	unchanged
C	unchanged
V	unchanged

Instruction Forms:

BCHG Rn(b), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for BCHG Rn(b), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field b—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

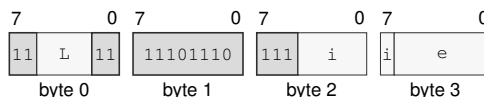
EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BCHG imm6(i), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for BCHG imm6(i), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BCLR**Bit Clear****BCLR**

Operation: Clears the destination bit selected by the low six bits of the bit index and sets Z when that bit was previously clear. A memory destination is read by an ordinary selected-width load and updated by an ordinary selected-width store.

Assembler Syntax: BCLR Rn(b), <ea>(e)
BCLR imm6(i), <ea>(e)

Attributes: Class = integer
Family = integer bitfield
Privilege = unprivileged
Length = 4-14 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

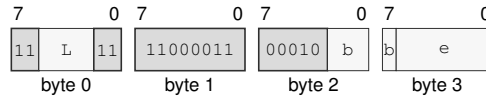
Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	old_bit == 0
N	unchanged
C	unchanged
V	unchanged

Instruction Forms:

BCLR Rn(b), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for BCLR Rn(b), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field b—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

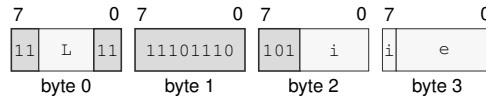
EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BCLR imm6(i), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for BCLR imm6(i), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BKPT

Breakpoint

BKPT

Operation: Raises BREAKPOINT and saves the next instruction address without executing that instruction before event entry commits.

Assembler Syntax: BKPT

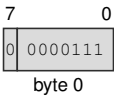
Attributes: Class = system
Family = core control
Privilege = any
Length = 1 byte
Repeat = none

Exceptions: BREAKPOINT: the instruction executes

Instruction Forms:

BKPT
extrashort · 1 byte · any

Instruction Format:



Format — Instruction format for BKPT

BND\$II**Signed Bounds Check Inclusive-Inclusive****BND\$II**

Operation: BND\$II treats both bounds as inclusive. It sets V when signed(value) is outside [signed(lo), signed(hi)]; Z, N, and C are unchanged.

Assembler Syntax: BND\$II.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

Attributes:
 Class = integer
 Family = bounds
 Privilege = unprivileged
 Length = 5-15 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = flags

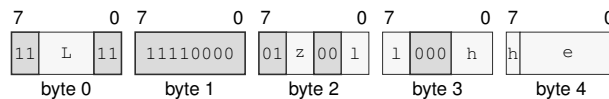
Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	signed(value) is outside the selected interval

Instruction Forms:

BND\$II.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

extralong · 5-15 bytes · unprivileged

Instruction Format:

Format — Instruction format for BND\$II.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field l—Selects the operand 1.

Register field h—Selects the operand 3.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BNDSIX

Signed Bounds Check Inclusive-Exclusive

BNDSIX

Operation:

BNDSIX treats the low bound as inclusive and the high bound as exclusive. It sets V when signed(value) is outside [signed(lo), signed(hi)); Z, N, and C are unchanged.

Assembler Syntax:

BNDSIX.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

Attributes:

Class = integer
Family = bounds
Privilege = unprivileged
Length = 5-15 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

Detailed Semantics

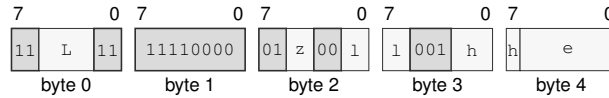
FLAGS Effects

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	signed(value) is outside the selected interval

Instruction Forms:

BNDXIX.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

extralong · 5-15 bytes · unprivileged

Instruction Format:

Format — Instruction format for **BNDXIX.X**(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field l—Selects the operand 1.

Register field h—Selects the operand 3.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BNDSXI

Signed Bounds Check Exclusive-Inclusive

BNDSXI

Operation:

BNDSXI treats the low bound as exclusive and the high bound as inclusive. It sets V when signed(value) is outside (signed(lo), signed(hi)); Z, N, and C are unchanged.

Assembler Syntax:

BNDSXI.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

Attributes:

Class = integer
Family = bounds
Privilege = unprivileged
Length = 5-15 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

Detailed Semantics

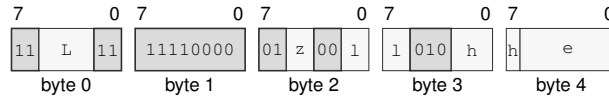
FLAGS Effects

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	signed(value) is outside the selected interval

Instruction Forms:

BNDXIX.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

extralong · 5-15 bytes · unprivileged

Instruction Format:

Format — Instruction format for **BNDXIX.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)**

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field l—Selects the operand 1.

Register field h—Selects the operand 3.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BNDSXX

Signed Bounds Check Exclusive-Exclusive

BNDSXX

Operation:

BNDSXX treats both bounds as exclusive. It sets V when signed(value) is outside (signed(lo), signed(hi)); Z, N, and C are unchanged.

Assembler Syntax:

BNDSXX.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

Attributes:

Class = integer
Family = bounds
Privilege = unprivileged
Length = 5-15 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

Detailed Semantics

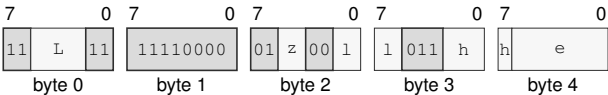
FLAGS Effects

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	signed(value) is outside the selected interval

Instruction Forms:

BNDSXX.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)
extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for BNDSXX.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

Instruction Fields:

- Length field** L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** l—Selects the operand 1.
- Register field** h—Selects the operand 3.
- Effective Address field** e—Specifies the source operand.
- EA availability**
- Allowed:** Register; Memory; Immediate; EXT0
- Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

BNDUII**Unsigned Bounds Check Inclusive-Inclusive****BNDUII**

Operation: BNDUII treats both bounds as inclusive. It sets V when unsigned(value) is outside [unsigned(lo), unsigned(hi)]; Z, N, and C are unchanged.

Assembler Syntax: `BNDUII.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)`

Attributes:

- Class = integer
- Family = bounds
- Privilege = unprivileged
- Length = 5-15 bytes
- Repeat = REP, REPcc, REPG
- REPcc observation = flags

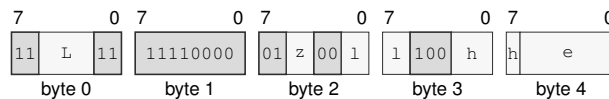
Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	unsigned(value) is outside the selected interval

Instruction Forms:

`BNDUII.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)`

extralong · 5-15 bytes · unprivileged

Instruction Format:

Format — Instruction format for BNDUII.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field l—Selects the operand 1.

Register field h—Selects the operand 3.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BNDUIX

Unsigned Bounds Check Inclusive-Exclusive

BNDUIX

Operation: BNDUIX treats the low bound as inclusive and the high bound as exclusive. It sets V when unsigned(value) is outside [unsigned(lo), unsigned(hi)); Z, N, and C are unchanged.

Assembler Syntax: BNDUIX.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

Attributes:
Class = integer
Family = bounds
Privilege = unprivileged
Length = 5-15 bytes
Repeat = REP, REPCC, REPG
REPCC observation = flags

Detailed Semantics

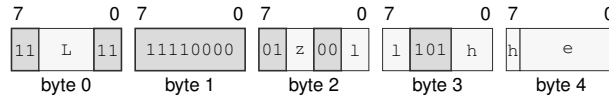
FLAGS Effects

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	unsigned(value) is outside the selected interval

Instruction Forms:

BNDUIX.X(*z*:B/W/L/Q) *Rn*(*l*), <*ea*>(*e*), *Rn*(*h*)

extralong · 5-15 bytes · unprivileged

Instruction Format:

Format — Instruction format for **BNDUIX.X**(*z*:B/W/L/Q) *Rn*(*l*), <*ea*>(*e*), *Rn*(*h*)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *l*—Selects the operand 1.

Register field *h*—Selects the operand 3.

Effective Address field *e*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BNDUXI

Unsigned Bounds Check Exclusive-Inclusive

BNDUXI

Operation: BNDUXI treats the low bound as exclusive and the high bound as inclusive. It sets V when unsigned(value) is outside (unsigned(lo), unsigned(hi)]; Z, N, and C are unchanged.

Assembler Syntax: BNDUXI.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

Attributes:
Class = integer
Family = bounds
Privilege = unprivileged
Length = 5-15 bytes
Repeat = REP, REPCC, REPG
REPCC observation = flags

Detailed Semantics

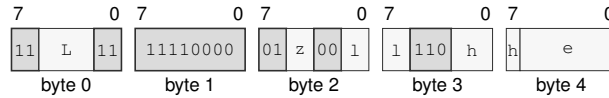
FLAGS Effects

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	unsigned(value) is outside the selected interval

Instruction Forms:

BNDUXI.X(*z*:B/W/L/Q) *Rn*(*l*), <*ea*>(*e*), *Rn*(*h*)

extralong · 5-15 bytes · unprivileged

Instruction Format:

Format — Instruction format for **BNDUXI.X**(*z*:B/W/L/Q) *Rn*(*l*), <*ea*>(*e*), *Rn*(*h*)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *l*—Selects the operand 1.

Register field *h*—Selects the operand 3.

Effective Address field *e*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BNDUXX

Unsigned Bounds Check Exclusive-Exclusive

BNDUXX

Operation:

BNDUXX treats both bounds as exclusive. It sets V when unsigned(value) is outside (unsigned(lo), unsigned(hi)); Z, N, and C are unchanged.

Assembler Syntax:

BNDUXX.X(z:B/W/L/Q) Rn(l), <ea>(e), Rn(h)

Attributes:

Class = integer
Family = bounds
Privilege = unprivileged
Length = 5-15 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

Detailed Semantics

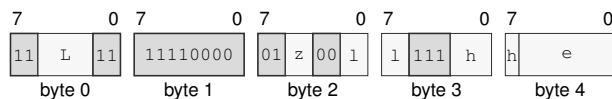
FLAGS Effects

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	unsigned(value) is outside the selected interval

Instruction Forms:

BNDUXX.X(*z*:B/W/L/Q) *Rn*(*l*), <*ea*>(*e*), *Rn*(*h*)

extralong · 5-15 bytes · unprivileged

Instruction Format:

Format — Instruction format for **BNDUXX.X(*z*:B/W/L/Q) *Rn*(*l*), <*ea*>(*e*), *Rn*(*h*)**

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *l*—Selects the operand 1.

Register field *h*—Selects the operand 3.

Effective Address field *e*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BSET**Bit Set****BSET**

Operation: Sets the destination bit selected by the low six bits of the bit index and sets Z when that bit was previously clear. A memory destination is read by an ordinary selected-width load and updated by an ordinary selected-width store.

Assembler Syntax: BSET Rn(b), <ea>(e)
BSET imm6(i), <ea>(e)

Attributes: Class = integer
Family = integer bitfield
Privilege = unprivileged
Length = 4-14 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

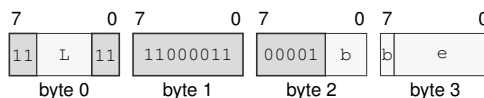
Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	old_bit == 0
N	unchanged
C	unchanged
V	unchanged

Instruction Forms:

BSET Rn(b), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for BSET Rn(b), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field b—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

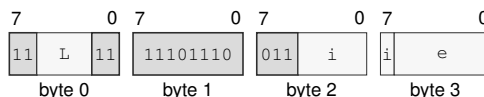
EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BSET imm6(i), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for BSET imm6(i), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BTEST**Bit Test****BTEST**

Operation: Tests the destination bit selected by the low six bits of the bit index and sets Z when that bit is clear without changing the destination.

Assembler Syntax: BTEST Rn(b), <ea>(e)
BTEST imm6(i), <ea>(e)

Attributes: Class = integer
Family = integer bitfield
Privilege = unprivileged
Length = 4-14 bytes
Repeat = REP, REPCC, REPG
REPCC observation = flags

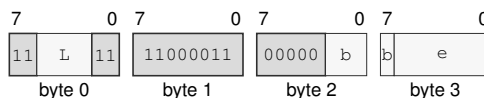
Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	old_bit == 0
N	unchanged
C	unchanged
V	unchanged

Instruction Forms:

BTEST Rn(b), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for BTEST Rn(b), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field b—Selects the source operand.

Effective Address field e—Specifies the source operand.

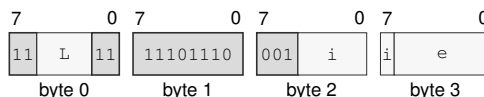
EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

BTEST imm6(i), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for BTEST imm6(i), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

CALL

Call

CALL

Operation:	Pushes the next instruction address as one 64-bit stack value and transfers control to the target. An EA memory form reads a 64-bit target; an EA register form supplies its 64-bit value directly. The stack update and control transfer use the ordinary SS:SP and target-validation rules.
Assembler Syntax:	CALL <imm16s> CALL <imm32s> CALL <ea>(e)
Attributes:	Class = control flow Family = control flow Privilege = unprivileged Length = 4-14 bytes Repeat = none

Detailed Semantics

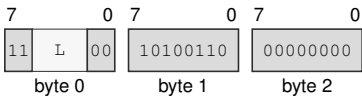
1. Evaluate the target operand in the old architectural context and validate the target under the current CS.
2. Probe the 8-byte return-address store through the old SS:SP context.
3. Commit the return-address store, decremented SP, and new PC as one successful control-transfer operation.

For CALLcc, a false condition is evaluated before target-memory or stack access and suppresses both. Encoding, extension, privilege, and control-state validation still occur. A fault before the final commit changes neither the stack nor PC.

Instruction Forms:

CALL <imm16s>
medium · 5 bytes · unprivileged

Instruction Format:



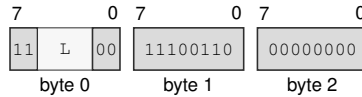
Format — Instruction format for CALL <imm16s>

Instruction Fields:

Length field L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

CALL <imm32s>

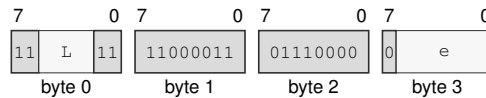
medium · 7 bytes · unprivileged

Instruction Format:**Format — Instruction format for CALL <imm32s>****Instruction Fields:**

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

CALL <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for CALL <ea>(e)****Instruction Fields:**

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field *e*—Specifies the control target.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

CALLcc

Call Conditional

CALLcc

Operation:

When the encoded condition is true, pushes the next instruction address as one 64-bit stack value and transfers control to the target; otherwise execution continues without changing the stack.

Assembler Syntax:

CALLcc <imm16s>
CALLcc <imm32s>
CALLcc <ea>(e)

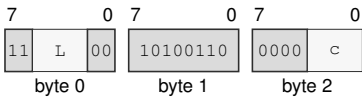
Attributes:

Class = control flow
Family = control flow
Privilege = unprivileged
Length = 4-14 bytes
Repeat = none

Instruction Forms:

CALLcc <imm16s>
medium · 5 bytes · unprivileged

Instruction Format:



Format — Instruction format for CALLcc <imm16s>

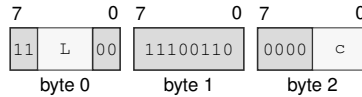
Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Condition field *c*—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

CALLcc <imm32s>

medium · 7 bytes · unprivileged

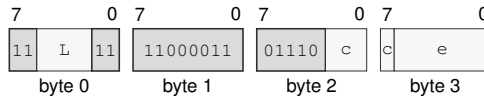
Instruction Format:**Format — Instruction format for CALLcc <imm32s>****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

CALLcc <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for CALLcc <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

CLMUL

Carryless Multiply

CLMUL

Operation: Forms the carryless polynomial product of the selected-width source and destination values and writes its low half to the destination.

Assembler Syntax: `CLMUL.X(z:B/W/L/Q) <ea>(e), Rn(d)`

Attributes:
 Class = integer
 Family = integer mul div
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = REP, REPCC, REPG
 REPCC observation = result dst

Detailed Semantics

Treat each selected-width operand as a polynomial over $GF(2)$, with operand bit i as the coefficient of x^i . Their double-width product is

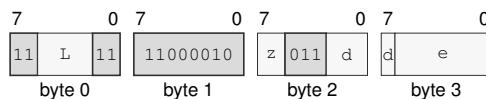
$$P = \bigoplus_{i=0}^{n-1} s_i (d \ll i).$$

Addition in this expression is exclusive-or, so no carry propagates between bit positions. CLMUL writes the low n bits of P to the destination.

Instruction Forms:

`CLMUL.X(z:B/W/L/Q) <ea>(e), Rn(d)`
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for `CLMUL.X(z:B/W/L/Q) <ea>(e), Rn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

CLMULH**Carryless Multiply High****CLMULH**

Operation: Forms the carryless polynomial product of the selected-width source and destination values and writes its high half to the destination.

Assembler Syntax: `CLMULH.Q <ea>(e), Rn(d)`

Attributes:
 Class = integer
 Family = integer mul div
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Detailed Semantics

Treat each selected-width operand as a polynomial over $GF(2)$, with operand bit i as the coefficient of x^i . Their double-width product is

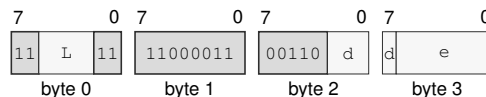
$$P = \bigoplus_{i=0}^{n-1} s_i(d \ll i).$$

Addition in this expression is exclusive-or, so no carry propagates between bit positions. CLMULH writes the high n bits of P to the destination.

Instruction Forms:

`CLMULH.Q <ea>(e), Rn(d)`

long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for CLMULH.Q <ea>(e), Rn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

CLR

Clear

CLR

Operation: Writes zero to the selected-width destination.

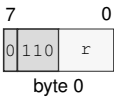
Assembler Syntax: CLR.Q Rn(r)
CLR.L Rn(r)
CLR.X(z:B/W) <ea>
CLR.X(z:L/Q) <ea>

Attributes: Class = integer
Family = integer unary
Privilege = unprivileged
Length = 1-13 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

CLR.Q Rn(r)
extrashort · 1 byte · unprivileged

Instruction Format:



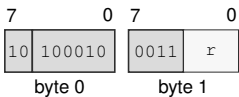
Format — Instruction format for CLR.Q Rn(r)

Instruction Fields:

Register field r—Selects the operand.

CLR.L Rn(r)
short · 2 bytes · unprivileged

Instruction Format:



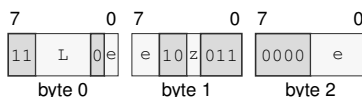
Format — Instruction format for CLR.L Rn(r)

Instruction Fields:

Register field r—Selects the operand.

CLR.X(z:B/W) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for CLR.X(z:B/W) <ea>****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the destination operand.

EA availability

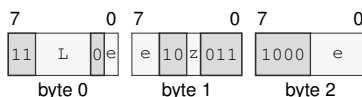
Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects B/W.

CLR.X(z:L/Q) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for CLR.X(z:L/Q) <ea>****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the destination operand.

EA availability

Allowed: Register except $R_N(r)$; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects L/Q.

CLRf

Clear Flags

CLRf

Operation: CLRf clears the selected architectural integer FLAGS bits. The immediate operand is a 4-bit mask: bit 3 selects Z, bit 2 selects N, bit 1 selects C, and bit 0 selects V. Selected bits are written to zero; unselected FLAGS bits are unchanged. The assembler may provide symbolic mask aliases such as Z, N, C, V, and combinations using |. CLC is an assembler alias for CLRf C.

Assembler Syntax: CLRf flags_bitmap(m)

Attributes: Class = system
Family = system registers
Privilege = unprivileged
Length = 3 bytes
Repeat = none

Detailed Semantics

FLAGS Effects

Flag	Effect
Z	cleared when mask bit 3 is one; otherwise unchanged
N	cleared when mask bit 2 is one; otherwise unchanged
C	cleared when mask bit 1 is one; otherwise unchanged
V	cleared when mask bit 0 is one; otherwise unchanged

Instruction Forms:

CLRf flags_bitmap(m)
medium · 3 bytes · unprivileged

Instruction Format:



Format — Instruction format for CLRf flags_bitmap(m)

Instruction Fields:

Length field *L*—Encodes the total instruction length as 3 + *L* bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Immediate field *m*—Encodes the immediate value.

CLS**Count Leading Set Bits****CLS**

Operation: Counts the leading one bits within the selected operand size and writes the count to the destination register. A zero operand returns 0; an operand whose selected bits are all one returns the operand width in bits.

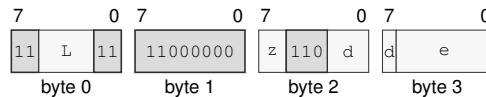
Assembler Syntax: `CLS.X(z:B/W/L/Q) <ea>(e), Rn(d)`

Attributes:
 Class = integer
 Family = integer bitfield
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Instruction Forms:

`CLS.X(z:B/W/L/Q) <ea>(e), Rn(d)`

long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for CLS.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

CLZ

Count Leading Zeros

CLZ

Operation: Counts the leading zero bits within the selected operand size and writes the count to the destination register. A zero operand returns the operand width in bits.

Assembler Syntax: CLZ.X(z:B/W/L/Q) <ea>(e), Rn(d)

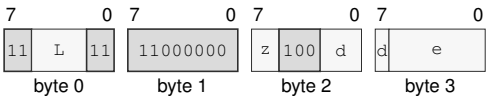
Attributes:

- Class = integer
- Family = integer bitfield
- Privilege = unprivileged
- Length = 4-14 bytes
- Repeat = REP, REPcc, REPG
- REPcc observation = result dst

Instruction Forms:

CLZ.X(z:B/W/L/Q) <ea>(e), Rn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for CLZ.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.
- EA availability**
- Allowed:** Register; Memory; Immediate; EXT0
- Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

CMP**Compare****CMP**

Operation: Subtracts the source from the destination only to derive Z, N, C, and V; the temporary difference is not written to either operand.

Assembler Syntax: `CMP.X(z:L/Q) Rn(s), Rn(d)`
`CMP.X(z:B/W) Rn(s), <ea>(e)`
`CMP.X(z:L/Q) Rn(s), <ea>(e)`
`CMP.X(z:B/W) <ea>(e), Rn(d)`
`CMP.X(z:L/Q) <ea>(e), Rn(d)`
`CMP.X(z:B/W/L/Q) <ea>(s), <ea>(d)`

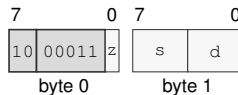
Attributes: Class = integer
Family = integer alu
Privilege = unprivileged
Length = 2-18 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	result == 0
N	most_significant_bit(result)
C	unsigned borrow
V	signed overflow

Instruction Forms:

`CMP.X(z:L/Q) Rn(s), Rn(d)`
short · 2 bytes · unprivileged

Instruction Format:

Format — Instruction format for `CMP.X(z:L/Q) Rn(s), Rn(d)`

Instruction Fields:

Size field *z*—Selects L/Q.

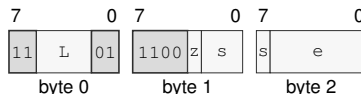
Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

$\text{CMP.X}(z:B/W) \text{ Rn}(s), \langle ea \rangle(e)$

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for $\text{CMP.X}(z:B/W) \text{ Rn}(s), \langle ea \rangle(e)$

Instruction Fields:

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z —Selects B/W.

Register field s —Selects the source operand.

Effective Address field e —Specifies the source operand.

EA availability

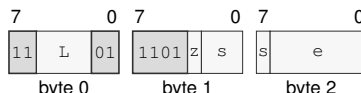
Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

$\text{CMP.X}(z:L/Q) \text{ Rn}(s), \langle ea \rangle(e)$

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for $\text{CMP.X}(z:L/Q) \text{ Rn}(s), \langle ea \rangle(e)$

Instruction Fields:

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z —Selects L/Q.

Register field s —Selects the source operand.

Effective Address field e —Specifies the source operand.

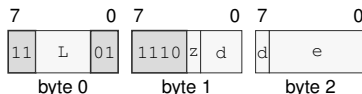
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`CMP.X(z:B/W) <ea>(e), Rn(d)`
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for `CMP.X(z:B/W) <ea>(e), Rn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

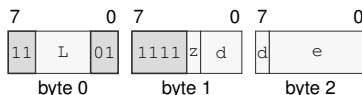
EA availability

Allowed: Register except $R_n(r)$; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`CMP.X(z:L/Q) <ea>(e), Rn(d)`
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for `CMP.X(z:L/Q) <ea>(e), Rn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

EA availability

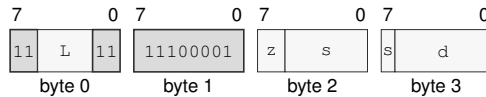
Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`CMP.X(z:B/W/L/Q) <ea>(s), <ea>(d)`

long · 4-18 bytes · unprivileged

Instruction Format:



Format — Instruction format for `CMP.X(z:B/W/L/Q) <ea>(s), <ea>(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Effective Address field *s*—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field *d*—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

CMPJcc**Compare And Jump Conditional****CMPJcc**

Operation: CMPJcc computes the same temporary compare result as CMP for the selected operand size, evaluates the encoded condition from that temporary result, and branches to the relative target when the condition is true. The computed condition is consumed only by the branch decision; architectural FLAGS are not read or written. Conditions T and F are reserved and are not valid encodings.

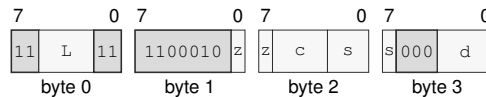
Assembler Syntax: `CMPJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm8s>`
`CMPJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm16s>`

Attributes:
 Class = control flow
 Family = control flow
 Privilege = unprivileged
 Length = 5-6 bytes
 Repeat = none

Instruction Forms:

`CMPJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm8s>`

long · 5 bytes · unprivileged

Instruction Format:

Format — Instruction format for CMPJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm8s>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

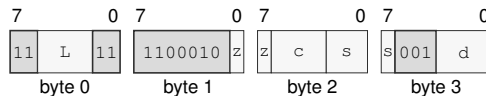
Register field s—Selects the operand 1.

Register field d—Selects the operand 2.

`CMPJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm16s>`

long · 6 bytes · unprivileged

Instruction Format:



Format — Instruction format for `CMPJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm16s>`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Condition field *c*—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field *s*—Selects the operand 1.

Register field *d*—Selects the operand 2.

CMPXCHG**Compare And Exchange****CMPXCHG**

Operation: Compares memory with the expected Rn register and conditionally stores the desired Rn register. After the operation, expected contains the observed old memory value. Z=1 means the store happened; Z=0 means memory was left unchanged.

Assembler Syntax: `CMPXCHG.X(z:B/W/L/Q)/order(o) Rn(x), Rn(d), <ea>(e)`

Attributes:
 Class = system
 Family = atomics
 Privilege = unprivileged
 Length = 5-15 bytes
 Repeat = none

Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	store succeeded
N	cleared
C	cleared
V	cleared

The memory read, optional memory write, expected-register update, and flag update form one atomic read-modify-write operation. The encoded memory-order selector governs the complete operation on both comparison outcomes. On failure, the operation contributes the observed memory read as its ordering event: acquire orders later memory operations after that read, release orders earlier memory operations before that read, and sequential consistency places that read in the global SC order. On success, the memory write carries the same selected order to observers of that write. A failed comparison contributes no memory write and creates no release sequence.

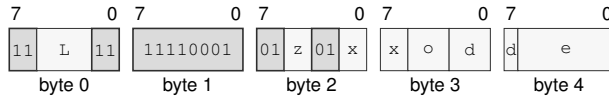
- If memory equals the original expected value, the desired value is stored and Z is set.
- Otherwise memory is unchanged and Z is cleared.
- In both cases the expected register receives the value originally read from memory, and N, C, and V are cleared.

A memory-access fault exposes none of these updates.

Instruction Forms:

`CMPXCHG.X(z:B/W/L/Q)/order(o) Rn(x), Rn(d), <ea>(e)`

extralong · 5-15 bytes · unprivileged

Instruction Format:

Format — Instruction format for `CMPXCHG.X(z:B/W/L/Q)/order(o) Rn(x), Rn(d), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *x*—Selects the operand 1.

Memory-order field *o*—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.

Register field *d*—Selects the operand 2.

Effective Address field *e*—Specifies the read/write operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

CPUID**CPU Identification****CPUID**

Operation: Queries architectural discovery information. The selected R_n register contains the CPUID query selector before execution and receives the selected 64-bit result after execution.

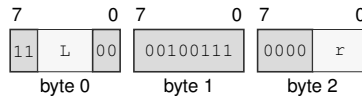
Assembler Syntax: CPUID $R_n(r)$

Attributes:
 Class = system
 Family = system registers
 Privilege = unprivileged
 Length = 3 bytes
 Repeat = none

Instruction Forms:

CPUID $R_n(r)$

medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for CPUID $R_n(r)$

Instruction Fields:

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field r —Selects the operand.

CTS

Count Trailing Set Bits

CTS

Operation: Counts the trailing one bits within the selected operand size and writes the count to the destination register. A zero operand returns 0; an operand whose selected bits are all one returns the operand width in bits.

Assembler Syntax: CTS.X(z:B/W/L/Q) <ea>(e), Rn(d)

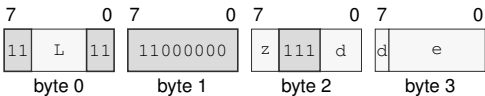
Attributes:

- Class = integer
- Family = integer bitfield
- Privilege = unprivileged
- Length = 4-14 bytes
- Repeat = REP, REPcc, REPG
- REPcc observation = result dst

Instruction Forms:

CTS.X(z:B/W/L/Q) <ea>(e), Rn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for CTS.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.
- EA availability**
- Allowed:** Register; Memory; Immediate; EXT0
- Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

CTZ**Count Trailing Zeros****CTZ**

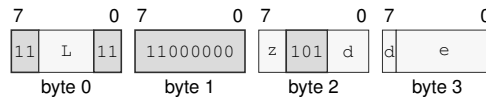
Operation: Counts the trailing zero bits within the selected operand size and writes the count to the destination register. A zero operand returns the operand width in bits.

Assembler Syntax: `CTZ.X(z:B/W/L/Q) <ea>(e), Rn(d)`

Attributes:
 Class = integer
 Family = integer bitfield
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Instruction Forms:

`CTZ.X(z:B/W/L/Q) <ea>(e), Rn(d)`
 long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for CTZ.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

DEC

Decrement

DEC

Operation: Subtracts one from the selected-width destination and writes the modular result without changing FLAGS.

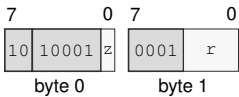
Assembler Syntax: DEC.X(z:L/Q) Rn(r)
DEC.X(z:B/W) <ea>
DEC.X(z:L/Q) <ea>

Attributes: Class = integer
Family = integer unary
Privilege = unprivileged
Length = 2-13 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

DEC.X(z:L/Q) Rn(r)
short · 2 bytes · unprivileged

Instruction Format:



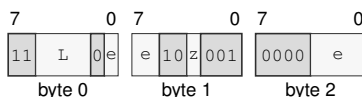
Format — Instruction format for DEC.X(z:L/Q) Rn(r)

Instruction Fields:

- Size field** z—Selects L/Q.
- Register field** r—Selects the operand.

DEC.X(z:B/W) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for DEC.X(z:B/W) <ea>****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand.

EA availability

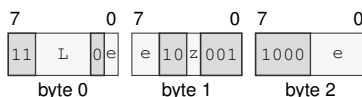
Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects B/W.

DEC.X(z:L/Q) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for DEC.X(z:L/Q) <ea>****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register except $R_N(r)$; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects L/Q.

DECF

Decrement And Set Flags

DECF

Operation:

Subtracts one from the selected-width destination, writes the modular result, and updates Z, N, C, and V.

Assembler Syntax:

DECF.X(z:B/W/L/Q) Rn(r)

Attributes:

Class = integer
Family = integer unary
Privilege = unprivileged
Length = 3 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

Detailed Semantics

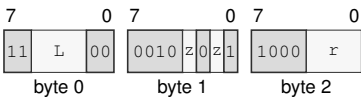
FLAGS Effects

Flag	Effect
Z	result == 0
N	most_significant_bit(result)
C	unsigned borrow
V	signed overflow

Instruction Forms:

DECF.X(z:B/W/L/Q) Rn(r)
medium · 3 bytes · unprivileged

Instruction Format:



Format — Instruction format for DECF.X(z:B/W/L/Q) Rn(r)

Instruction Fields:

- Length field** L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** r—Selects the operand.

DIVMODS**Signed Divide With Remainder****DIVMODS**

Operation:	The quotient register Rn(q) supplies the dividend before the operation, then receives the quotient. The remainder register Rn(r) receives the remainder.
Assembler Syntax:	<code>DIVMODS.X(z:B/W/L/Q) <ea>(e), Rn(q), Rn(r)</code>
Attributes:	Class = integer Family = integer mul div Privilege = unprivileged Length = 5-15 bytes Repeat = REP, REPcc, REPG REPcc observation = result quotient
Exceptions:	DIVIDE_ERROR: the divisor is zero or the most-negative value is divided by minus one ILLEGAL_INSTRUCTION: quotient and remainder designate the same register; cause is INVALID_OPERAND_RELATION

Detailed Semantics

Let n be the selected operand width, d the signed two's-complement dividend, and s the signed two's-complement divisor. The quotient and remainder are defined by

$$q = \text{trunc}(d/s), \quad r = d - qs.$$

Thus a nonzero remainder has the sign of the dividend. The original quotient operand supplies the dividend. On success the quotient operand receives q , and the separate remainder operand receives r .

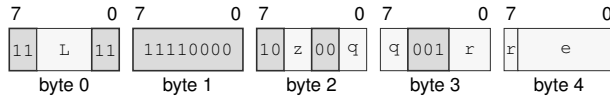
A zero divisor and the most-negative value divided by -1 raise `DIVIDE_ERROR`; no destination is changed.

Instruction Forms:

`DIVMODS.X(z:B/W/L/Q) <ea>(e), Rn(q), Rn(r)`

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for `DIVMODS.X(z:B/W/L/Q) <ea>(e), Rn(q), Rn(r)`

Instruction Fields:

Length field `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field `z`—Selects B/W/L/Q.

Register field `q`—Selects the operand 2.

Register field `r`—Selects the operand 3.

Effective Address field `e`—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Destination overlap—When quotient = remainder designate the same architectural register, the instruction raises `ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION` before architectural effects.

DIVMODU**Unsigned Divide With Remainder****DIVMODU**

Operation: The quotient register $Rn(q)$ supplies the dividend before the operation, then receives the quotient. The remainder register $Rn(r)$ receives the remainder.

Assembler Syntax: `DIVMODU.X(z:B/W/L/Q) <ea>(e), Rn(q), Rn(r)`

Attributes:
 Class = integer
 Family = integer mul div
 Privilege = unprivileged
 Length = 5-15 bytes
 Repeat = REP, REPCC, REPG
 REPCC observation = result quotient

Exceptions:
 DIVIDE_ERROR: the divisor is zero
 ILLEGAL_INSTRUCTION: quotient and remainder designate the same register; cause is INVALID_OPERAND_RELATION

Detailed Semantics

Let n be the selected operand width, d the unsigned dividend, and s the unsigned divisor. The quotient and remainder are defined by

$$q = \lfloor d/s \rfloor, \quad r = d - qs.$$

For a nonzero divisor, the remainder satisfies $0 \leq r < s$. The original quotient operand supplies the dividend. On success the quotient operand receives q , and the separate remainder operand receives r .

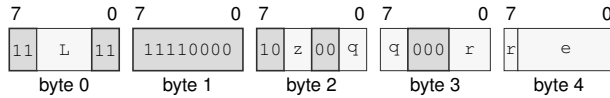
A zero divisor raises DIVIDE_ERROR; no destination is changed.

Instruction Forms:

`DIVMODU.X(z:B/W/L/Q) <ea>(e), Rn(q), Rn(r)`

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for `DIVMODU.X(z:B/W/L/Q) <ea>(e), Rn(q), Rn(r)`

Instruction Fields:

Length field `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field `z`—Selects B/W/L/Q.

Register field `q`—Selects the operand 2.

Register field `r`—Selects the operand 3.

Effective Address field `e`—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Destination overlap—When quotient = remainder designate the same architectural register, the instruction raises `ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION` before architectural effects.

DIVS**Signed Divide****DIVS**

Operation:	Interprets both selected-width operands as signed integers, divides the destination by the source with truncation toward zero, and writes the quotient. Division by zero and the most-negative value divided by minus one raise <code>DIVIDE_ERROR</code> .
Assembler Syntax:	<code>DIVS.X(z:B/W/L/Q) <ea>(e), Rn(d)</code>
Attributes:	Class = integer Family = integer mul div Privilege = unprivileged Length = 4-14 bytes Repeat = REP, REPcc, REPG REPcc observation = result dst
Exceptions:	<code>DIVIDE_ERROR</code> : the divisor is zero or the most-negative value is divided by minus one

Detailed Semantics

Let n be the selected operand width, d the signed two's-complement dividend, and s the signed two's-complement divisor. The quotient and remainder are defined by

$$q = \text{trunc}(d/s), \quad r = d - qs.$$

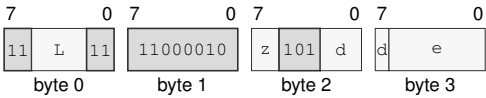
Thus a nonzero remainder has the sign of the dividend. The destination supplies the dividend and receives q .

A zero divisor and the most-negative value divided by -1 raise `DIVIDE_ERROR`; no destination is changed.

Instruction Forms:

DIVS.X(z:B/W/L/Q) <ea>(e), Rn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for DIVS.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

DIVU**Unsigned Divide****DIVU**

Operation: Divides the selected-width unsigned destination value by the unsigned source value and writes the quotient. A zero divisor raises `DIVIDE_ERROR`.

Assembler Syntax: `DIVU.X(z:B/W/L/Q) <ea>(e), Rn(d)`

Attributes:
 Class = integer
 Family = integer mul div
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Exceptions: `DIVIDE_ERROR`: the divisor is zero

Detailed Semantics

Let n be the selected operand width, d the unsigned dividend, and s the unsigned divisor. The quotient and remainder are defined by

$$q = \lfloor d/s \rfloor, \quad r = d - qs.$$

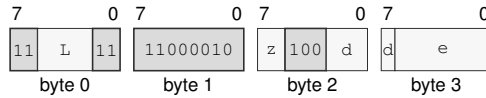
For a nonzero divisor, the remainder satisfies $0 \leq r < s$. The destination supplies the dividend and receives q .

A zero divisor raises `DIVIDE_ERROR`; no destination is changed.

Instruction Forms:

`DIVU.X(z:B/W/L/Q) <ea>(e), Rn(d)`

long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for `DIVU.X(z:B/W/L/Q) <ea>(e), Rn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

DJcc**Decrement And Jump Conditional****DJcc**

Operation: Decrements the selected counter and transfers control only when the new count is nonzero and the encoded condition is true. The target EA has Q interpretation width: a memory form reads a 64-bit target and a register or immediate form supplies its value directly.

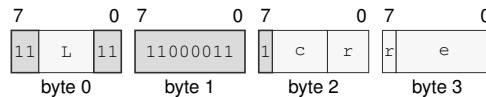
Assembler Syntax: `DJcc Rn(r), <ea>(e)`

Attributes:
 Class = control flow
 Family = control flow
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = none

Instruction Forms:

`DJcc Rn(r), <ea>(e)`

long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for DJcc Rn(r), <ea>(e)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Condition field *c*—Selects the condition code. Allowed values: T, EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field *r*—Selects the source operand.

Effective Address field *e*—Specifies the control target.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

IJcc

Increment And Jump Conditional

IJcc

Operation: Increments the index and transfers control only when the new index differs from the bound and the encoded condition is true. The target EA has Q interpretation width: a memory form reads a 64-bit target and a register or immediate form supplies its value directly.

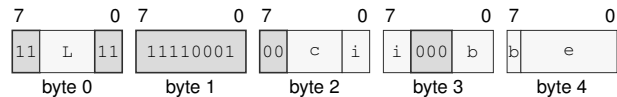
Assembler Syntax: IJcc Rn(i), Rn(b), <ea>(e)

Attributes:
Class = control flow
Family = control flow
Privilege = unprivileged
Length = 5-15 bytes
Repeat = none

Instruction Forms:

IJcc Rn(i), Rn(b), <ea>(e)
extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for IJcc Rn(i), Rn(b), <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
 - Condition field** c—Selects the condition code. Allowed values: T, EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.
 - Register field** i—Selects the operand 1.
 - Register field** b—Selects the operand 2.
 - Effective Address field** e—Specifies the control target.
- EA availability**
Allowed: Register; Memory; Immediate; EXT0
Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTRACT

Extract From Register Pair

EXTRACT

Operation: Concatenates $Rn(\text{high})$ as the upper half and the original $Rn(\text{low}/\text{dest})$ value as the lower half, shifts the concatenated value right by an unsigned 7-bit immediate offset, and writes the low selected-size result back to $Rn(\text{low}/\text{dest})$.

Assembler Syntax: `EXTRACT.X(z:B/W/L/Q) imm7(i), Rn(h), Rn(l)`

Attributes:
 Class = integer
 Family = integer bitfield
 Privilege = unprivileged
 Length = 4 bytes
 Repeat = REP, REPCc, REPG
 REPCc observation = result low_dst

Detailed Semantics

Let n be the selected operand width and form the $2n$ -bit concatenation

$$T = \text{concat}(Rn(\text{high})[n-1:0], Rn(\text{low})[n-1:0]).$$

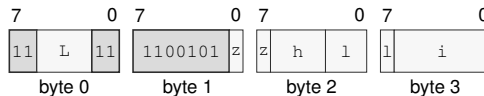
The immediate is an unsigned seven-bit shift count. The destination receives the low n bits of T after a logical right shift by that count. A count greater than or equal to $2n$ produces zero. The high register is not changed.

Instruction Forms:

`EXTRACT.X(z:B/W/L/Q) imm7(i), Rn(h), Rn(l)`

long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for `EXTRACT.X(z:B/W/L/Q) imm7(i), Rn(h), Rn(l)`

Instruction Fields:

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z —Selects B/W/L/Q.

Register field h —Selects the operand 2.

Register field l —Selects the operand 3.

Immediate field i —Encodes the immediate value.

EXTSL

Sign Extend To Long

EXTSL

Operation: Sign-extends the source operand from the instruction size to long size. The low source-size bytes of the source are copied into the destination, and the remaining high destination bits are filled with the source sign bit.

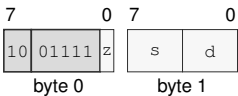
Assembler Syntax: EXTSL.X(z:B/W) Rn(s), Rn(d)
EXTSL.B Rn(s), <ea>(e)
EXTSL.B <ea>(e), Rn(d)
EXTSL.W Rn(s), <ea>(e)
EXTSL.W <ea>(e), Rn(d)
EXTSL.X(z:B/W) <ea>(s), <ea>(d)

Attributes: Class = integer
Family = integer unary
Privilege = unprivileged
Length = 2-18 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

EXTSL.X(z:B/W) Rn(s), Rn(d)
short · 2 bytes · unprivileged

Instruction Format:



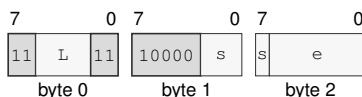
Format — Instruction format for EXTSL.X(z:B/W) Rn(s), Rn(d)

Instruction Fields:

- Size field** z—Selects B/W.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

EXTSL.B $R_n(s)$, <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSL.B $R_n(s)$, <ea>(e)****Instruction Fields:**

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s —Selects the source operand.

Effective Address field e —Specifies the destination operand.

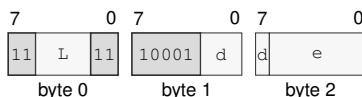
EA availability

Allowed: Register except $R_n(x)$; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSL.B <ea>(e), $R_n(d)$

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSL.B <ea>(e), $R_n(d)$** **Instruction Fields:**

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d —Selects the destination operand.

Effective Address field e —Specifies the source operand.

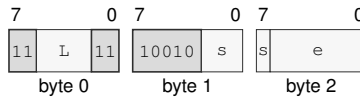
EA availability

Allowed: Register except $R_n(x)$; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSL.W Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSL.W Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

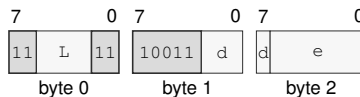
EA availability

Allowed: Register except Rn(r); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSL.W <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSL.W <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

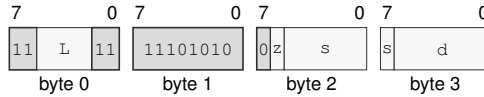
EA availability

Allowed: Register except Rn(r); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSL.X(z:B/W) <ea>(s), <ea>(d)
 long · 4-18 bytes · unprivileged

Instruction Format:



Format — Instruction format for EXTSL.X(z:B/W) <ea>(s), <ea>(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Effective Address field s—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field d—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSQ

Sign Extend To Quad

EXTSQ

Operation: Sign-extends the source operand from the instruction size to quad size. The low source-size bytes of the source are copied into the destination, and the remaining high destination bits are filled with the source sign bit.

Assembler Syntax:

```
EXTSQ.L Rn(s), Rn(d)
EXTSQ.B Rn(s), <ea>(e)
EXTSQ.B <ea>(e), Rn(d)
EXTSQ.W Rn(s), <ea>(e)
EXTSQ.W <ea>(e), Rn(d)
EXTSQ.L Rn(s), <ea>(e)
EXTSQ.L <ea>(e), Rn(d)
EXTSQ.B <ea>(s), <ea>(d)
EXTSQ.W <ea>(s), <ea>(d)
EXTSQ.L <ea>(s), <ea>(d)
```

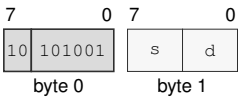
Attributes:

Class = integer
Family = integer unary
Privilege = unprivileged
Length = 2-18 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

EXTSQ.L Rn(s), Rn(d)
short · 2 bytes · unprivileged

Instruction Format:



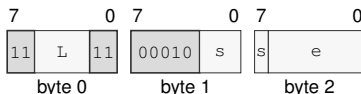
Format — Instruction format for EXTSQ.L Rn(s), Rn(d)

Instruction Fields:

- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

EXTSQ.B Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSQ.B Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

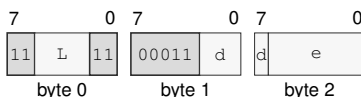
EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSQ.B <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSQ.B <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

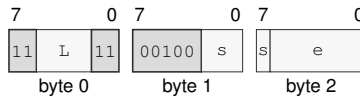
EA availability

Allowed: Register except Rn(x); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSQ.W Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSQ.W Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

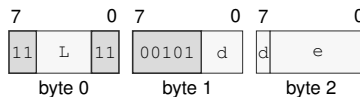
EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSQ.W <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSQ.W <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

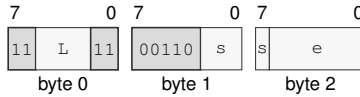
EA availability

Allowed: Register except Rn(r); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSQ.L Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSQ.L Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

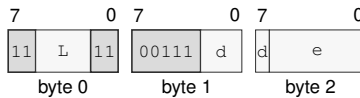
EA availability

Allowed: Register except Rn(x); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSQ.L <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSQ.L <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

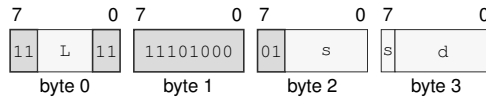
EA availability

Allowed: Register except Rn(x); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSQ.B <ea>(s), <ea>(d)

long · 4-18 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSQ.B <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field s—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field d—Specifies the destination operand.

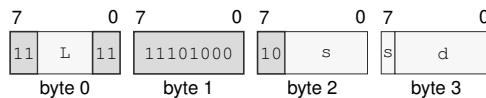
EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSQ.W <ea>(s), <ea>(d)

long · 4-18 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSQ.W <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field s—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field d—Specifies the destination operand.

EA availability

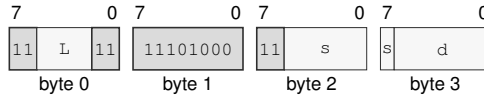
Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSQ.L <ea>(s), <ea>(d)

long · 4-18 bytes · unprivileged

Instruction Format:



Format — Instruction format for EXTSQ.L <ea>(s), <ea>(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field *s*—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field *d*—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSW

Sign Extend To Word

EXTSW

Operation: Sign-extends the source operand from byte size to word size. The low byte of the source is copied into the destination, and the remaining high destination bits are filled with the source sign bit.

Assembler Syntax: `EXTSW.B Rn(s), <ea>(e)`
`EXTSW.B <ea>(e), Rn(d)`
`EXTSW.B <ea>(s), <ea>(d)`

Attributes: Class = integer
Family = integer unary
Privilege = unprivileged
Length = 3-18 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

`EXTSW.B Rn(s), <ea>(e)`
medium · 3-13 bytes · unprivileged

Instruction Format:



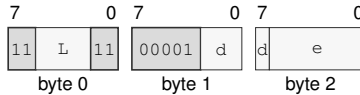
Format — Instruction format for EXTSW.B Rn(s), <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Register field** s—Selects the source operand.
- Effective Address field** e—Specifies the destination operand.
- EA availability**
- Allowed:** Register; Memory; EXT0
- Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSW.B <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSW.B <ea>(e), Rn(d)****Instruction Fields:**

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

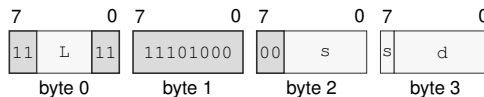
EA availability

Allowed: Register except $R_n(x)$; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTSW.B <ea>(s), <ea>(d)

long · 4-18 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTSW.B <ea>(s), <ea>(d)****Instruction Fields:**

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field *s*—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field *d*—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZL

Zero Extend To Long

EXTZL

Operation: Zero-extends the source operand from the instruction size to long size. The low source-size bytes of the source are copied into the destination, and the remaining high destination bits are filled with zero.

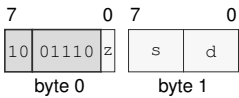
Assembler Syntax: `EXTZL.X(z:B/W) Rn(s), Rn(d)`
`EXTZL.B Rn(s), <ea>(e)`
`EXTZL.B <ea>(e), Rn(d)`
`EXTZL.W Rn(s), <ea>(e)`
`EXTZL.W <ea>(e), Rn(d)`
`EXTZL.X(z:B/W) <ea>(s), <ea>(d)`

Attributes: Class = integer
Family = integer unary
Privilege = unprivileged
Length = 2-18 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

`EXTZL.X(z:B/W) Rn(s), Rn(d)`
short · 2 bytes · unprivileged

Instruction Format:



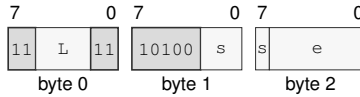
Format — Instruction format for `EXTZL.X(z:B/W) Rn(s), Rn(d)`

Instruction Fields:

- Size field** `z`—Selects B/W.
- Register field** `s`—Selects the source operand.
- Register field** `d`—Selects the destination operand.

EXTZL.B $R_n(s)$, <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZL.B $R_n(s)$, <ea>(e)****Instruction Fields:**

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s —Selects the source operand.

Effective Address field e —Specifies the destination operand.

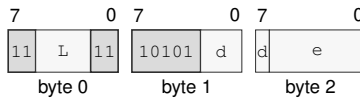
EA availability

Allowed: Register except $R_n(x)$; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZL.B <ea>(e), $R_n(d)$

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZL.B <ea>(e), $R_n(d)$** **Instruction Fields:**

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d —Selects the destination operand.

Effective Address field e —Specifies the source operand.

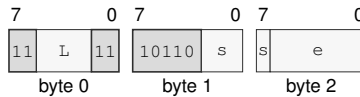
EA availability

Allowed: Register except $R_n(x)$; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZL.W Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZL.W Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

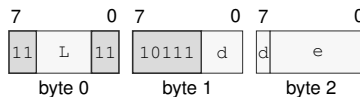
EA availability

Allowed: Register except Rn(r); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZL.W <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZL.W <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

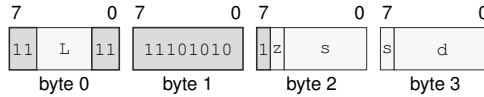
EA availability

Allowed: Register except Rn(r); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZL.X(z:B/W) <ea>(s), <ea>(d)
 long · 4-18 bytes · unprivileged

Instruction Format:



Format — Instruction format for EXTZL.X(z:B/W) <ea>(s), <ea>(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Effective Address field s—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field d—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZQ

Zero Extend To Quad

EXTZQ

Operation: Zero-extends the source operand from the instruction size to quad size. The low source-size bytes of the source are copied into the destination, and the remaining high destination bits are filled with zero.

Assembler Syntax:

```
EXTZQ.L Rn(s), Rn(d)
EXTZQ.B Rn(s), <ea>(e)
EXTZQ.B <ea>(e), Rn(d)
EXTZQ.W Rn(s), <ea>(e)
EXTZQ.W <ea>(e), Rn(d)
EXTZQ.L Rn(s), <ea>(e)
EXTZQ.L <ea>(e), Rn(d)
EXTZQ.B <ea>(s), <ea>(d)
EXTZQ.W <ea>(s), <ea>(d)
EXTZQ.L <ea>(s), <ea>(d)
```

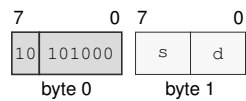
Attributes:

Class = integer
Family = integer unary
Privilege = unprivileged
Length = 2-18 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

EXTZQ.L Rn(s), Rn(d)
short · 2 bytes · unprivileged

Instruction Format:



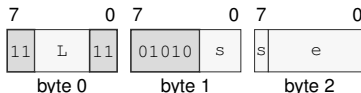
Format — Instruction format for EXTZQ.L Rn(s), Rn(d)

Instruction Fields:

- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

EXTZQ.B Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZQ.B Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

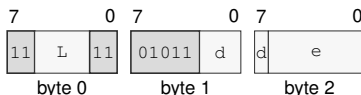
EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZQ.B <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZQ.B <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

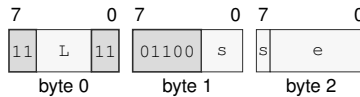
EA availability

Allowed: Register except Rn(x); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZQ.W Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZQ.W Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

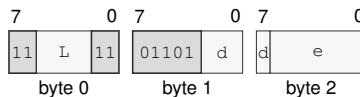
EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZQ.W <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZQ.W <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

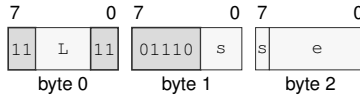
EA availability

Allowed: Register except Rn(r); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZQ.L Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZQ.L Rn(s), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

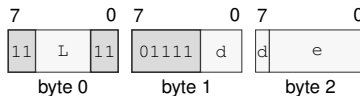
EA availability

Allowed: Register except Rn(x); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZQ.L <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZQ.L <ea>(e), Rn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

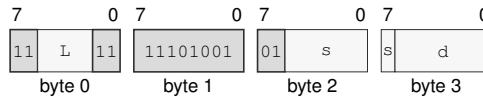
EA availability

Allowed: Register except Rn(x); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZQ.B <ea>(s), <ea>(d)

long · 4-18 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZQ.B <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field s—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field d—Specifies the destination operand.

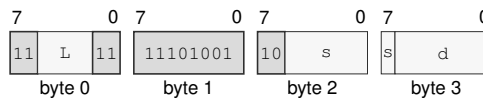
EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZQ.W <ea>(s), <ea>(d)

long · 4-18 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZQ.W <ea>(s), <ea>(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field s—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field d—Specifies the destination operand.

EA availability

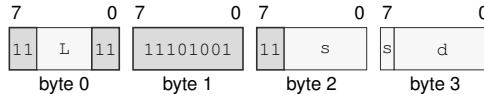
Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZQ.L <ea>(s), <ea>(d)

long · 4-18 bytes · unprivileged

Instruction Format:



Format — Instruction format for EXTZQ.L <ea>(s), <ea>(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field s—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field d—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZW

Zero Extend To Word

EXTZW

Operation: Zero-extends the source operand from byte size to word size. The low byte of the source is copied into the destination, and the remaining high destination bits are filled with zero.

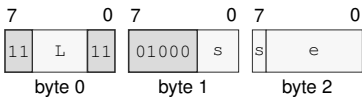
Assembler Syntax: EXTZW.B Rn(s), <ea>(e)
EXTZW.B <ea>(e), Rn(d)
EXTZW.B <ea>(s), <ea>(d)

Attributes: Class = integer
Family = integer unary
Privilege = unprivileged
Length = 3-18 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

EXTZW.B Rn(s), <ea>(e)
medium · 3-13 bytes · unprivileged

Instruction Format:



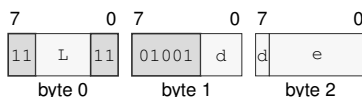
Format — Instruction format for EXTZW.B Rn(s), <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Register field** s—Selects the source operand.
- Effective Address field** e—Specifies the destination operand.
- EA availability**
- Allowed:** Register; Memory; EXT0
- Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZW.B <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZW.B <ea>(e), Rn(d)****Instruction Fields:**

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

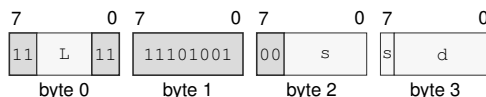
EA availability

Allowed: Register except $R_n(x)$; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

EXTZW.B <ea>(s), <ea>(d)

long · 4-18 bytes · unprivileged

Instruction Format:**Format — Instruction format for EXTZW.B <ea>(s), <ea>(d)****Instruction Fields:**

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field *s*—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field *d*—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FETCHADD

Atomic Fetch Add

FETCHADD

Operation: Atomically replaces the memory operand with its selected-width sum with the source and returns the previous memory value in the source register.

Assembler Syntax: `FETCHADD.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

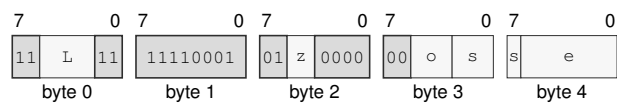
Attributes:

- Class = system
- Family = atomics
- Privilege = unprivileged
- Length = 5-15 bytes
- Repeat = none

Instruction Forms:

`FETCHADD.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`
extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FETCHADD.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Memory-order field *o*—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FETCHAND

Atomic Fetch And

FETCHAND

Operation: Atomically replaces the memory operand with its bitwise conjunction with the source and returns the previous memory value in the source register.

Assembler Syntax: `FETCHAND.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

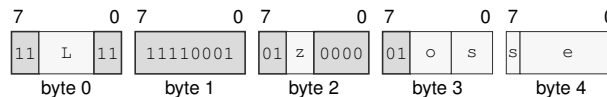
Attributes:
 Class = system
 Family = atomics
 Privilege = unprivileged
 Length = 5-15 bytes
 Repeat = none

Instruction Forms:

`FETCHAND.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FETCHAND.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Memory-order field *o*—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FETCHOR

Atomic Fetch Or

FETCHOR

Operation: Atomically replaces the memory operand with its bitwise disjunction with the source and returns the previous memory value in the source register.

Assembler Syntax: `FETCHOR.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

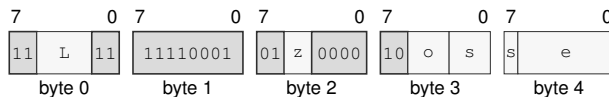
Attributes:
 Class = system
 Family = atomics
 Privilege = unprivileged
 Length = 5-15 bytes
 Repeat = none

Instruction Forms:

`FETCHOR.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FETCHOR.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Memory-order field *o*—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FETCHSUB

Atomic Fetch Subtract

FETCHSUB

Operation: Atomically replaces the memory operand with its selected-width difference from the source and returns the previous memory value in the source register.

Assembler Syntax: `FETCHSUB.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

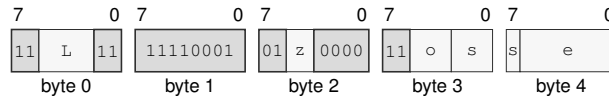
Attributes:
 Class = system
 Family = atomics
 Privilege = unprivileged
 Length = 5-15 bytes
 Repeat = none

Instruction Forms:

`FETCHSUB.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FETCHSUB.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Memory-order field *o*—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FETCHXOR

Atomic Fetch Xor

FETCHXOR

Operation: Atomically replaces the memory operand with its bitwise exclusive-or with the source and returns the previous memory value in the source register.

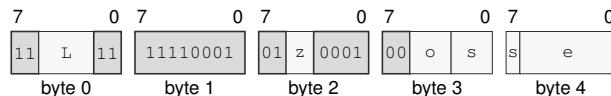
Assembler Syntax: `FETCHXOR.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

Attributes:
 Class = system
 Family = atomics
 Privilege = unprivileged
 Length = 5-15 bytes
 Repeat = none

Instruction Forms:

`FETCHXOR.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

extralong · 5-15 bytes · unprivileged

Instruction Format:

Format — Instruction format for `FETCHXOR.X(z:B/W/L/Q)/order(o) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Memory-order field *o*—Selects the memory ordering. Allowed values: RELAXED, ACQUIRE, RELEASE, ACQREL, SEQCST.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FLSHDCACHE

Flush Data Cache

FLSHDCACHE

Operation: Computes and translates the address-role EA without reading an operand value, selects its CPUID-sized maintenance block, writes dirty bytes from every data or unified cache copy in the coherence domain into normal-memory coherence, invalidates those copies, and retires after the block is complete.

Assembler Syntax: FLSHDCACHE <ea>(e)

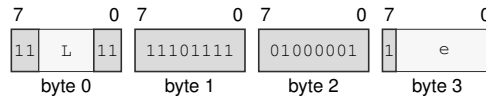
Attributes:
 Class = system
 Family = cache
 Privilege = supervisor
 Length = 4-14 bytes
 Repeat = none

Instruction Forms:

FLSHDCACHE <ea>(e)

long · 4-14 bytes · supervisor

Instruction Format:



Format — Instruction format for FLSHDCACHE <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the address operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

HALT

Halt

HALT

Operation:	Retires with the next instruction as its continuation and stops instruction fetch on the executing logical processor until an asynchronous architectural event is admitted and delivered.
Assembler Syntax:	HALT
Attributes:	Class = system Family = core control Privilege = supervisor Length = 2 bytes Repeat = none

Detailed Semantics

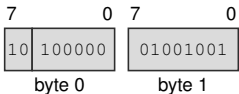
HALT retires with the following instruction as its continuation, then applies the Architectural Event Processing Model's event priority and admission rules at that instruction boundary. If the model selects an event at that boundary, it is delivered with the continuation as its saved PC before the logical processor enters the halted state.

Otherwise, the executing logical processor stops instruction fetch. When an asynchronous architectural event later becomes admissible, the logical processor leaves the halted state by delivering that event with the continuation as its saved PC. Event delivery precedes execution of the continuation. Only the executing logical processor enters the halted state.

Instruction Forms:

HALT
short · 2 bytes · supervisor

Instruction Format:



Format — Instruction format for HALT

ILLEGAL**Illegal Instruction****ILLEGAL**

Operation: Raises ILLEGAL_INSTRUCTION with the EXPLICIT_ILLEGAL cause at the current instruction boundary without retiring or changing architectural destination state before event entry.

Assembler Syntax: ILLEGAL

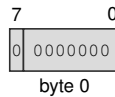
Attributes: Class = system
Family = core control
Privilege = any
Length = 1 byte
Repeat = none

Exceptions: ILLEGAL_INSTRUCTION: the instruction executes; cause is EXPLICIT_ILLEGAL

Instruction Forms:

ILLEGAL

extrashort · 1 byte · any

Instruction Format:

Format — Instruction format for ILLEGAL

INC

Increment

INC

Operation: Adds one to the selected-width destination and writes the modular result without changing FLAGS.

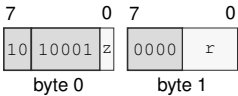
Assembler Syntax: `INC.X(z:L/Q) Rn(r)`
`INC.X(z:B/W) <ea>`
`INC.X(z:L/Q) <ea>`

Attributes: Class = integer
Family = integer unary
Privilege = unprivileged
Length = 2-13 bytes
Repeat = REP, REPCC, REPG
REPCC observation = result dst

Instruction Forms:

`INC.X(z:L/Q) Rn(r)`
short · 2 bytes · unprivileged

Instruction Format:



Format — Instruction format for INC.X(z:L/Q) Rn(r)

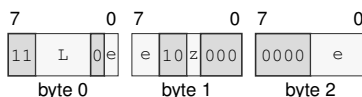
Instruction Fields:

- Size field** z—Selects L/Q.
- Register field** r—Selects the operand.

INC.X(z:B/W) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for INC.X(z:B/W) <ea>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

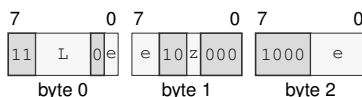
Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects B/W.

INC.X(z:L/Q) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for INC.X(z:L/Q) <ea>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register except $R_N(r)$; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects L/Q.

INCF

Increment And Set Flags

INCF

Operation: Adds one to the selected-width destination, writes the modular result, and updates Z, N, C, and V.

Assembler Syntax: `INCF.X(z:B/W/L/Q) Rn(r)`

Attributes: Class = integer
Family = integer unary
Privilege = unprivileged
Length = 3 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

Detailed Semantics

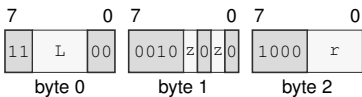
FLAGS Effects

Flag	Effect
Z	result == 0
N	most_significant_bit(result)
C	unsigned carry out
V	signed overflow

Instruction Forms:

`INCF.X(z:B/W/L/Q) Rn(r)`
medium · 3 bytes · unprivileged

Instruction Format:



Format — Instruction format for `INCF.X(z:B/W/L/Q) Rn(r)`

Instruction Fields:

Length field `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field `z`—Selects B/W/L/Q.

Register field `r`—Selects the operand.

INVASID**Invalidate TLB By ASID****INVASID**

Operation: Invalidates every non-global cached translation for the selected 16-bit ASID on the current hardware thread, discards matching in-progress results, and prevents later use of stale entries.

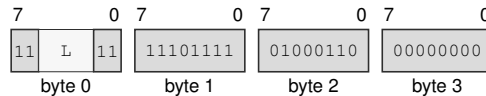
Assembler Syntax: INVASID <imm16>

Attributes:
 Class = system
 Family = tlb and context
 Privilege = supervisor
 Length = 6 bytes
 Repeat = none

Instruction Forms:

INVASID <imm16>

long · 6 bytes · supervisor

Instruction Format:

Format — Instruction format for INVASID <imm16>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

INVDCACHE

Invalidate Data Cache

INVDCACHE

Operation:

Computes and translates the address-role EA without reading an operand value, selects its CUID-sized maintenance block, discards every data or unified cache copy in the coherence domain without writeback, and retires after the block is complete.

Assembler Syntax:

INVDCACHE <ea>(e)

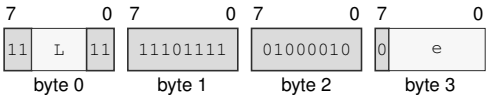
Attributes:

Class = system
Family = cache
Privilege = supervisor
Length = 4-14 bytes
Repeat = none

Instruction Forms:

INVDCACHE <ea>(e)
long · 4-14 bytes · supervisor

Instruction Format:



Format — Instruction format for INVDCACHE <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Effective Address field** e—Specifies the address operand.
- EA availability**
- Allowed:** Memory; Immediate; EXT0
- Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

INVICACHE

Invalidate Instruction Cache

INVICACHE

Operation: Computes and translates the address-role EA without fetching from it, selects its CUID-sized maintenance block, invalidates the executing logical processor's matching instruction-cache, decoded, and prefetch state, and serializes later instruction fetch.

Assembler Syntax: INVICACHE <ea>(e)

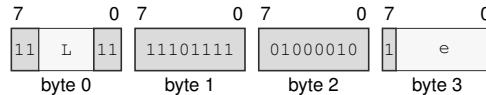
Attributes:
 Class = system
 Family = cache
 Privilege = supervisor
 Length = 4-14 bytes
 Repeat = none

Instruction Forms:

INVICACHE <ea>(e)

long · 4-14 bytes · supervisor

Instruction Format:



Format — Instruction format for INVICACHE <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the address operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

INVPAGE

Invalidate TLB Page

INVPAGE

Operation: Computes the source linear page without reading memory and invalidates matching global or current-ASID translations and in-progress results before retirement.

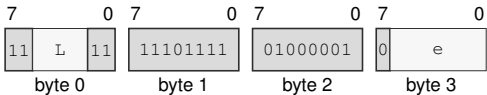
Assembler Syntax: INVPAGE <ea>(e)

Attributes: Class = system
Family = tlb and context
Privilege = supervisor
Length = 4-14 bytes
Repeat = none

Instruction Forms:

INVPAGE <ea>(e)
long · 4-14 bytes · supervisor

Instruction Format:



Format — Instruction format for INVPAGE <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Effective Address field** e—Specifies the address operand.
- EA availability**
 - Allowed:** Register; Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

INVTLB**Invalidate TLB****INVTLB**

Operation: Invalidates all cached instruction and data translations and discards in-progress results on the current hardware thread before later accesses may proceed.

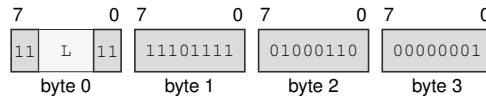
Assembler Syntax: INVTLB

Attributes:
 Class = system
 Family = tlb and context
 Privilege = supervisor
 Length = 4 bytes
 Repeat = none

Instruction Forms:

INVTLB

long · 4 bytes · supervisor

Instruction Format:

Format — Instruction format for INVTLB

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

ERET**Event Return****ERET**

Operation:	Return from the active architectural event after validating the saved event frame and return state. ERET is valid only while event-active state is set.
Assembler Syntax:	ERET
Attributes:	Class = system Family = core control Privilege = supervisor Length = 1 byte Repeat = none
Exceptions:	INVALID_CONTROL_STATE: event-active state is clear or the event frame or return image fails validation

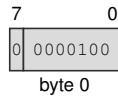
Detailed Semantics

1. Require supervisor mode and active architectural event state; otherwise raise INVALID_CONTROL_STATE.
2. Read and decode FRAME_CONTROL at SS:SP without changing SP. Accept only the defined BASIC, ERROR, PAGE_FAULT, and AUXILIARY sizes, with all reserved bits zero.
3. Validate the saved event depth and stack level. A saved state with EA clear must carry zero saved event depth and stack level.
4. Read the complete selected frame and validate FLAGS, STATUS, CS, DS, SS, SP, and an executable canonical PC without changing architectural state.
5. When repeat state is saved, validate the repeat context in the always-present FRAME_EXT1 slot; otherwise require FRAME_EXT1 to be zero and prepare cleared repeat state. Frame type remains determined only by the event.
6. Atomically restore the validated state, event-depth state, and repeat state.
7. After restoration, admit a pending NMI before the next instruction boundary when STATUS.NI is clear and ECR.NMI_P is set.

Instruction Forms:

ERET

extrashort · 1 byte · supervisor

Instruction Format:**Format — Instruction format for ERET**

JMP

Jump

JMP

Operation:

Validates the target under the current control context and transfers execution to it. For the extended EA form, JMP.L reads the low 32-bit target from its register or memory EA and zero-extends it to the 64-bit near target; JMP.Q uses the complete 64-bit target.

Assembler Syntax:

JMP imm8s(i)
JMP <imm16s>
JMP <imm32s>
JMP.X(z:L/Q) <ea>(e)

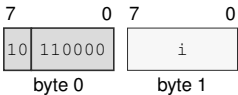
Attributes:

Class = control flow
Family = control flow
Privilege = unprivileged
Length = 2-14 bytes
Repeat = none

Instruction Forms:

JMP imm8s(i)
short · 2 bytes · unprivileged

Instruction Format:



Format — Instruction format for JMP imm8s(i)

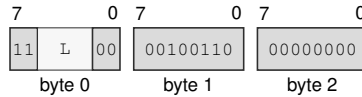
Instruction Fields:

Immediate field i—Encodes the immediate value.

JMP <imm16s>

medium · 5 bytes · unprivileged

Instruction Format:



Format — Instruction format for JMP <imm16s>

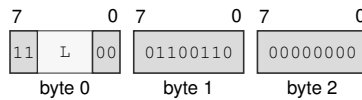
Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

JMP <imm32s>

medium · 7 bytes · unprivileged

Instruction Format:



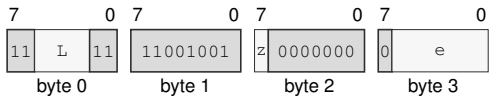
Format — Instruction format for JMP <imm32s>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

JMP.X(z:L/Q) <ea>(e)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for JMP.X(z:L/Q) <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Effective Address field e—Specifies the control target.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Jcc**Jump Conditional****Jcc**

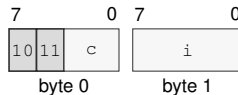
Operation: Transfers execution to the target when the encoded condition is true and otherwise continues at the next instruction. Compact direct-relative forms select a W- or L-width displacement. Extended EA forms use L/Q. Jcc.L reads the low 32-bit target from its register or memory EA and zero-extends it to the 64-bit near target; Jcc.Q uses the complete 64-bit target.

Assembler Syntax: `Jcc imm8s(i)`
`Jcc <imm16s>`
`Jcc <imm32s>`
`Jcc.X(z:L/Q) <ea>(e)`

Attributes: Class = control flow
Family = control flow
Privilege = unprivileged
Length = 2-14 bytes
Repeat = none

Instruction Forms:

`Jcc imm8s(i)`
short · 2 bytes · unprivileged

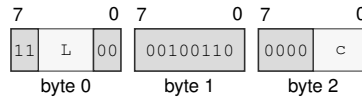
Instruction Format:**Format — Instruction format for Jcc imm8s(i)****Instruction Fields:**

Condition field *c*—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Immediate field *i*—Encodes the immediate value.

Jcc <imm16s>

medium · 5 bytes · unprivileged

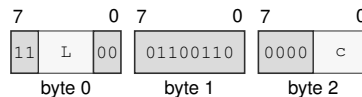
Instruction Format:**Format — Instruction format for Jcc <imm16s>****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Jcc <imm32s>

medium · 7 bytes · unprivileged

Instruction Format:**Format — Instruction format for Jcc <imm32s>****Instruction Fields:**

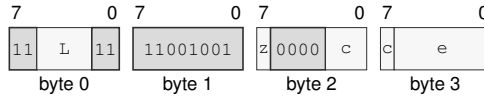
Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Jcc.X(z:L/Q) <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for Jcc.X(z:L/Q) <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Effective Address field e—Specifies the control target.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

LCALL

Long Call

LCALL

Operation: Long call is the CS-changing call instruction. It first constructs the far return frame on SS:SP, then commits the new CS and PC together. The two-operand form takes the new CS image from Rn. Its target EA has Q interpretation width: a memory form reads a 64-bit offset and a register or immediate form supplies its value directly.

Assembler Syntax: `LCALL Rn(r), <ea>(e)`

Attributes:
Class = control flow
Family = control flow
Privilege = unprivileged
Length = 4-14 bytes
Repeat = none

Detailed Semantics

1. Evaluate the target operand completely in the old architectural context.
2. Validate the complete new CS image.
3. Validate the target offset against the new CS and probe executable translation at the resulting linear address.
4. Probe both 8-byte far-return-frame stores through the old SS context.
5. Commit the two frame stores, decremented SP, new CS, and new PC as one successful control-transfer operation.

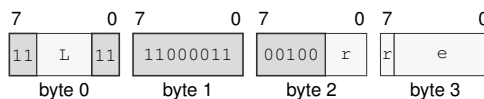
Fault atomicity. No frame word, SP, CS, or PC update is visible when any prior step faults.

The far-return frame is 16 bytes in the old SS context: the return PC occupies offset 0 and the return CS image occupies offset 8. An invalid new CS image raises `INVALID_CONTROL_STATE`; target translation or either frame-store translation failure raises `PAGE_FAULT`.

Instruction Forms:

LCALL Rn(r), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for LCALL Rn(r), <ea>(e)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field r—Selects the source operand.

Effective Address field e—Specifies the control target.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

LEA

Load Effective Address

LEA

Operation: Computes the source effective address without reading the addressed memory and writes that address to the destination.

Assembler Syntax: `LEA.X(z:B/W/L/Q) <ea>, Rn`

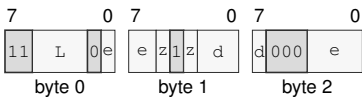
Attributes:

- Class = data movement
- Family = ea utility
- Privilege = unprivileged
- Length = 3-13 bytes
- Repeat = REP, REPCC, REPG
- REPCC observation = result dst

Instruction Forms:

`LEA.X(z:B/W/L/Q) <ea>, Rn`
medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for `LEA.X(z:B/W/L/Q) <ea>, Rn`

Instruction Fields:

- Length field** `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Effective Address field** `e`—Specifies the address operand.
- EA availability**
- Allowed:** Register; Memory; Immediate; EXT0
- Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.
- Size field** `z`—Selects B/W/L/Q.
- Register field** `d`—Selects a register operand.

LJMP**Long Jump****LJMP**

Operation: Long jump is the CS-changing jump instruction. It commits the new CS and PC as a single architectural control-transfer step. The two-operand form takes the new CS image from Rn. Its target EA has Q interpretation width: a memory form reads a 64-bit offset and a register or immediate form supplies its value directly.

Assembler Syntax: LJMP Rn(r), <ea>(e)

Attributes:
 Class = control flow
 Family = control flow
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = none

Detailed Semantics

1. Evaluate the target operand completely in the old architectural context.
2. Validate the complete new CS image.
3. Validate the target offset against the new CS and probe executable translation at the resulting linear address.
4. Commit new CS and new PC as one successful control-transfer operation.

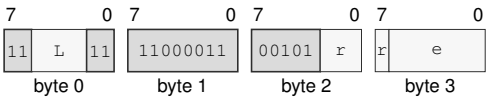
Fault atomicity. CS and PC remain unchanged when operand evaluation or target validation faults.

An invalid new CS image raises `INVALID_CONTROL_STATE`; target translation failure raises `PAGE_FAULT`.

Instruction Forms:

LJMP Rn(r), <ea>(e)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for LJMP Rn(r), <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
 - Register field** r—Selects the source operand.
 - Effective Address field** e—Specifies the control target.
- EA availability**
Allowed: Register; Memory; Immediate; EXT0
Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

LRET**Long Return****LRET**

Operation: Long return reads the far return frame from SS:SP and commits CS and PC together. The frame contains the return PC Q value at the current SP and the return CS Q value at SP+8; after both values are fetched, SP is advanced by 16 bytes and the control state is committed.

Assembler Syntax: LRET

Attributes: Class = control flow
Family = control flow
Privilege = unprivileged
Length = 1 byte
Repeat = none

Detailed Semantics

1. Read both 8-byte far-return-frame words through the old SS context without changing SP.
2. Validate the complete return CS image.
3. Validate the return PC offset against the return CS and probe executable translation at the resulting linear address.
4. Commit advanced SP, return CS, and return PC as one successful control-transfer operation.

Fault atomicity. SP, CS, and PC remain unchanged when a frame read or return-target validation faults.

The 16-byte far-return frame is read through the old SS context: the return PC occupies offset 0 and the return CS image occupies offset 8. A frame translation or return-target translation failure raises `PAGE_FAULT`; an invalid return CS image raises `INVALID_CONTROL_STATE`.

Instruction Forms:

LRET
extrashort · 1 byte · unprivileged

Instruction Format:



Format — Instruction format for LRET

MAXS**Signed Maximum****MAXS**

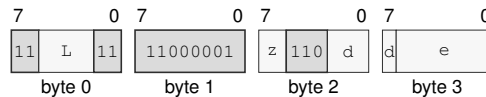
Operation: Compares the selected-width operands as signed integers and writes the greater value to the destination.

Assembler Syntax: `MAXS.X(z:B/W/L/Q) <ea>(e), Rn(d)`
`MAXS.X(z:B/W/L/Q) Rn(s), <ea>(e)`

Attributes:
 Class = integer
 Family = integer alu
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Instruction Forms:

`MAXS.X(z:B/W/L/Q) <ea>(e), Rn(d)`
 long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for MAXS.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

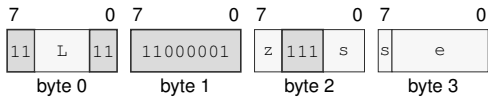
EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MAXS.X(z:B/W/L/Q) Rn(s), <ea>(e)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for MAXS.X(z:B/W/L/Q) Rn(s), <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** s—Selects the source operand.
- Effective Address field** e—Specifies the read/write operand.
- EA availability**
 - Allowed:** Register except Rn(r); Memory; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

MAXU**Unsigned Maximum****MAXU**

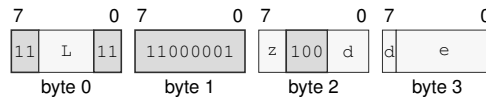
Operation: Compares the selected-width operands as unsigned integers and writes the greater value to the destination.

Assembler Syntax: `MAXU.X(z:B/W/L/Q) <ea>(e), Rn(d)`
`MAXU.X(z:B/W/L/Q) Rn(s), <ea>(e)`

Attributes:
 Class = integer
 Family = integer alu
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Instruction Forms:

`MAXU.X(z:B/W/L/Q) <ea>(e), Rn(d)`
 long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for MAXU.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

EA availability

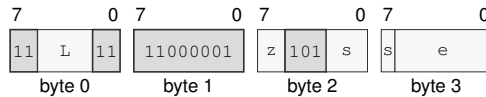
Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MAXU.X(z:B/W/L/Q) Rn(s), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for MAXU.X(z:B/W/L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register except Rn(r); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MINS**Signed Minimum****MINS**

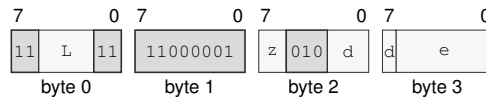
Operation: Compares the selected-width operands as signed integers and writes the lesser value to the destination.

Assembler Syntax: `MINS.X(z:B/W/L/Q) <ea>(e), Rn(d)`
`MINS.X(z:B/W/L/Q) Rn(s), <ea>(e)`

Attributes: Class = integer
 Family = integer alu
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Instruction Forms:

`MINS.X(z:B/W/L/Q) <ea>(e), Rn(d)`
 long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for `MINS.X(z:B/W/L/Q) <ea>(e), Rn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

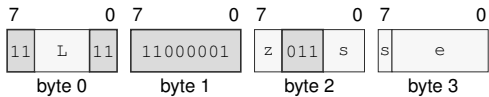
EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MINS.X(z:B/W/L/Q) Rn(s), <ea>(e)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for MINS.X(z:B/W/L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register except Rn(r); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MINU**Unsigned Minimum****MINU**

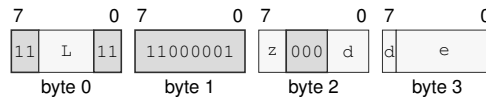
Operation: Compares the selected-width operands as unsigned integers and writes the lesser value to the destination.

Assembler Syntax: `MINU.X(z:B/W/L/Q) <ea>(e), Rn(d)`
`MINU.X(z:B/W/L/Q) Rn(s), <ea>(e)`

Attributes:
 Class = integer
 Family = integer alu
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Instruction Forms:

`MINU.X(z:B/W/L/Q) <ea>(e), Rn(d)`
 long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for MINU.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

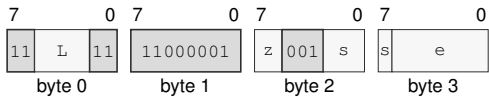
EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MINU.X(z:B/W/L/Q) Rn(s), <ea>(e)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for MINU.X(z:B/W/L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register except Rn(r); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MODS**Signed Modulo****MODS**

Operation: Interprets both selected-width operands as signed integers and writes the remainder associated with truncation-toward-zero division. Invalid signed division cases raise `DIVIDE_ERROR`.

Assembler Syntax: `MODS.X(z:B/W/L/Q) <ea>(e), Rn(d)`

Attributes:
 Class = integer
 Family = integer mul div
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = `REP`, `REPcc`, `REPG`
`REPcc` observation = `result dst`

Exceptions: `DIVIDE_ERROR`: the divisor is zero or the most-negative value is divided by minus one

Detailed Semantics

Let n be the selected operand width, d the signed two's-complement dividend, and s the signed two's-complement divisor. The quotient and remainder are defined by

$$q = \text{trunc}(d/s), \quad r = d - qs.$$

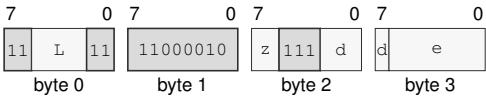
Thus a nonzero remainder has the sign of the dividend. The destination supplies the dividend and receives r .

A zero divisor and the most-negative value divided by -1 raise `DIVIDE_ERROR`; no destination is changed.

Instruction Forms:

MODS.X(z:B/W/L/Q) <ea>(e), Rn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for MODS.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MODU**Unsigned Modulo****MODU**

Operation: Writes the remainder of selected-width unsigned destination-by-source division. A zero divisor raises `DIVIDE_ERROR`.

Assembler Syntax: `MODU.X(z:B/W/L/Q) <ea>(e), Rn(d)`

Attributes:
 Class = integer
 Family = integer mul div
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = `REP`, `REPcc`, `REPG`
`REPcc` observation = `result dst`

Exceptions: `DIVIDE_ERROR`: the divisor is zero

Detailed Semantics

Let n be the selected operand width, d the unsigned dividend, and s the unsigned divisor. The quotient and remainder are defined by

$$q = \lfloor d/s \rfloor, \quad r = d - qs.$$

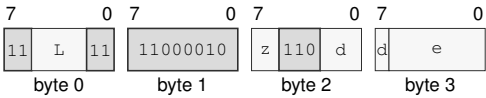
For a nonzero divisor, the remainder satisfies $0 \leq r < s$. The destination supplies the dividend and receives r .

A zero divisor raises `DIVIDE_ERROR`; no destination is changed.

Instruction Forms:

MODU.X(z:B/W/L/Q) <ea>(e), Rn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for MODU.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MOV**Move****MOV**

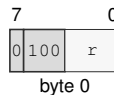
Operation: Reads the source using its addressing mode and selected size and writes that value to the destination.

Assembler Syntax: `MOV.Q Rn(r), SP`
`MOV.Q SP, Rn(r)`
`MOV.X(z:L/Q) Rn(s), Rn(d)`
`MOV.X(z:B/W) Rn(s), <ea>(e)`
`MOV.X(z:L/Q) Rn(s), <ea>(e)`
`MOV.X(z:B/W) <ea>(e), Rn(d)`
`MOV.X(z:L/Q) <ea>(e), Rn(d)`
`MOV.X(z:B/W/L/Q) <ea>(s), <ea>(d)`

Attributes: Class = data movement
Family = data movement
Privilege = unprivileged
Length = 1-18 bytes
Repeat = REP, REPcc, REPG
REPcc observation = `result dst`

Instruction Forms:

`MOV.Q Rn(r), SP`
extrashort · 1 byte · unprivileged

Instruction Format:

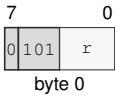
Format — Instruction format for MOV.Q Rn(r), SP

Instruction Fields:

Register field *r*—Selects the source operand.

MOV.Q SP, Rn(r)
extrashort · 1 byte · unprivileged

Instruction Format:



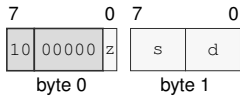
Format — Instruction format for MOV.Q SP, Rn(r)

Instruction Fields:

Register field r—Selects the destination operand.

MOV.X(z:L/Q) Rn(s), Rn(d)
short · 2 bytes · unprivileged

Instruction Format:



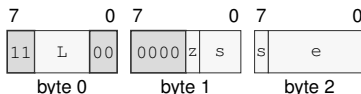
Format — Instruction format for MOV.X(z:L/Q) Rn(s), Rn(d)

Instruction Fields:

- Size field** z—Selects L/Q.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

`MOV.X(z:B/W) Rn(s), <ea>(e)`
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for `MOV.X(z:B/W) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the destination operand.

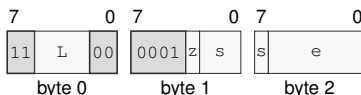
EA availability

Allowed: Register except SP; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`MOV.X(z:L/Q) Rn(s), <ea>(e)`
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for `MOV.X(z:L/Q) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects L/Q.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the destination operand.

EA availability

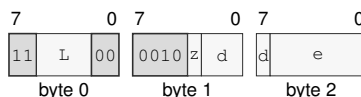
Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MOV.X(z:B/W) <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for MOV.X(z:B/W) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

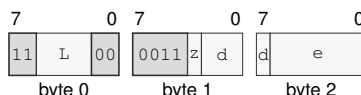
Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MOV.X(z:L/Q) <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for MOV.X(z:L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

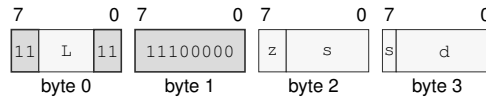
Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MOV.X(z:B/W/L/Q) <ea>(s), <ea>(d)

long · 4-18 bytes · unprivileged

Instruction Format:



Format — Instruction format for MOV.X(z:B/W/L/Q) <ea>(s), <ea>(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Effective Address field s—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field d—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MOVcc

Move Conditional

MOVcc

Operation: Copies the source value to the destination only when the encoded condition is true.

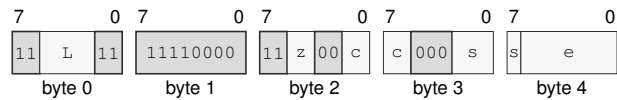
Assembler Syntax: MOVcc.X(z:B/W/L/Q) Rn(s), <ea>(e)
MOVcc.X(z:B/W/L/Q) <ea>(e), Rn(d)

Attributes: Class = control flow
Family = control flow
Privilege = unprivileged
Length = 5-15 bytes
Repeat = REP, REPG

Instruction Forms:

MOVcc.X(z:B/W/L/Q) Rn(s), <ea>(e)
extralong · 5-15 bytes · unprivileged

Instruction Format:



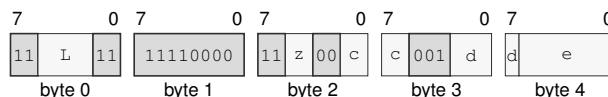
Format — Instruction format for MOVcc.X(z:B/W/L/Q) Rn(s), <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
 - Size field** z—Selects B/W/L/Q.
 - Condition field** c—Selects the condition code.
 - Register field** s—Selects the source operand.
 - Effective Address field** e—Specifies the destination operand.
- EA availability**
Allowed: Register; Memory; EXT0
Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`MOVcc.X(z:B/W/L/Q) <ea>(e), Rn(d)`
 extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for `MOVcc.X(z:B/W/L/Q) <ea>(e), Rn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Condition field *c*—Selects the condition code.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MOVCU

Move Current To User

MOVCU

Operation: MOVCU is a supervisor-only checked user-access move. The source is a current-domain memory operand, register, or immediate. If the destination operand is memory, that access uses the user domain.

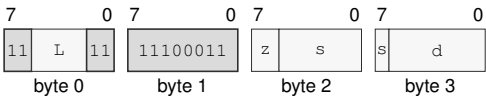
Assembler Syntax: MOVCU.X(z:B/W/L/Q) <ea>(s), <ea>(d)

Attributes: Class = system
Family = data movement
Privilege = supervisor
Length = 4-18 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

MOVCU.X(z:B/W/L/Q) <ea>(s), <ea>(d)
long · 4-18 bytes · supervisor

Instruction Format:



Format — Instruction format for MOVCU.X(z:B/W/L/Q) <ea>(s), <ea>(d)

Instruction Fields:

Length field L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Effective Address field s—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field d—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MOVNT**Move Non-Temporal****MOVNT**

Operation: MOVNT is a store-only non-temporal move. The source is an Rn register and the destination must be a memory operand.

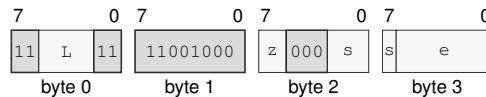
Assembler Syntax: `MOVNT.X(z:B/W/L/Q) Rn(s), <ea>(e)`

Attributes:
 Class = data movement
 Family = cache
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = source src

Instruction Forms:

`MOVNT.X(z:B/W/L/Q) Rn(s), <ea>(e)`

long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for MOVNT.X(z:B/W/L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MOVUC

Move User To Current

MOVUC

Operation: MOVUC is a supervisor-only checked user-access move. If the source operand is memory, that access uses the user domain. The destination is a current-domain memory operand or an Rn register.

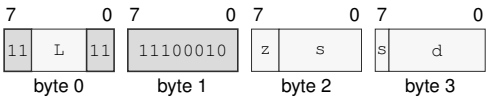
Assembler Syntax: MOVUC.X(z:B/W/L/Q) <ea>(s), <ea>(d)

Attributes: Class = system
Family = data movement
Privilege = supervisor
Length = 4-18 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

MOVUC.X(z:B/W/L/Q) <ea>(s), <ea>(d)
long · 4-18 bytes · supervisor

Instruction Format:



Format — Instruction format for MOVUC.X(z:B/W/L/Q) <ea>(s), <ea>(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Effective Address field** s—Specifies the source operand.
- EA availability**
Allowed: Memory; EXT0
- Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.
- Effective Address field** d—Specifies the destination operand.
- EA availability**
Allowed: Register; Memory; EXT0
- Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

MOVUU**Move User To User****MOVUU**

Operation: MOVUU is a supervisor-only checked user-access move. Both source and destination operands must be memory operands, and both memory accesses use the user domain.

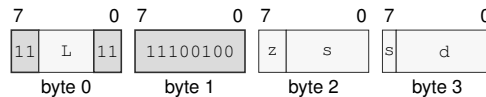
Assembler Syntax: `MOVUU.X(z:B/W/L/Q) <ea>(s), <ea>(d)`

Attributes:
 Class = system
 Family = data movement
 Privilege = supervisor
 Length = 4-18 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Instruction Forms:

`MOVUU.X(z:B/W/L/Q) <ea>(s), <ea>(d)`

long · 4-18 bytes · supervisor

Instruction Format:

Format — Instruction format for MOVUU.X(z:B/W/L/Q) <ea>(s), <ea>(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W/L/Q.

Effective Address field *s*—Specifies the source operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field *d*—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

MUL

Multiply Low

MUL

Operation:

MUL multiplies the selected-width operands and writes the low N bits of the product.

Assembler Syntax:

MUL.X(z:B/W/L/Q) <ea>(e), Rn(d)

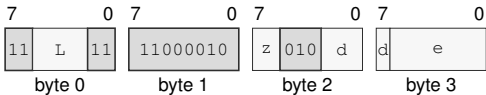
Attributes:

Class = integer
Family = integer mul div
Privilege = unprivileged
Length = 4-14 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

MUL.X(z:B/W/L/Q) <ea>(e), Rn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for MUL.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.
- EA availability**
 - Allowed:** Register; Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

MULHS**Signed Multiply High****MULHS**

Operation: Multiplies the selected-width operands as signed integers and writes the high half of the double-width product.

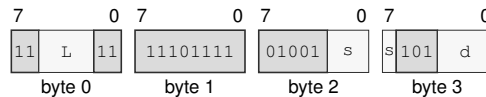
Assembler Syntax: `MULHS.Q Rn(s), Rn(d)`

Attributes:
 Class = integer
 Family = integer mul div
 Privilege = unprivileged
 Length = 4 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Instruction Forms:

`MULHS.Q Rn(s), Rn(d)`

long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for MULHS.Q Rn(s), Rn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

MULHSU

Signed By Unsigned Multiply High

MULHSU

Operation:

Multiplies a signed destination by an unsigned source at the selected width and writes the high half of the double-width product.

Assembler Syntax:

MULHSU.Q Rn(s), Rn(d)

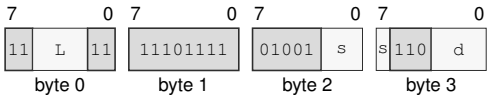
Attributes:

Class = integer
Family = integer mul div
Privilege = unprivileged
Length = 4 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

MULHSU.Q Rn(s), Rn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for MULHSU.Q Rn(s), Rn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

MULHU**Unsigned Multiply High****MULHU**

Operation: Multiplies the selected-width operands as unsigned integers and writes the high half of the double-width product.

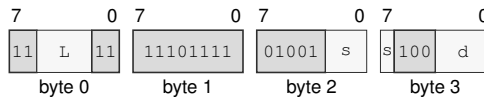
Assembler Syntax: `MULHU.Q Rn(s), Rn(d)`

Attributes:
 Class = integer
 Family = integer mul div
 Privilege = unprivileged
 Length = 4 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Instruction Forms:

`MULHU.Q Rn(s), Rn(d)`

long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for MULHU.Q Rn(s), Rn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

NEG

Negate

NEG

Operation: Writes the selected-width two’s-complement negation of the destination value.

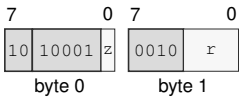
Assembler Syntax: `NEG.X(z:L/Q) Rn(r)`
`NEG.X(z:B/W) <ea>`
`NEG.X(z:L/Q) <ea>`

Attributes: Class = integer
Family = integer unary
Privilege = unprivileged
Length = 2-13 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

`NEG.X(z:L/Q) Rn(r)`
short · 2 bytes · unprivileged

Instruction Format:



Format — Instruction format for NEG.X(z:L/Q) Rn(r)

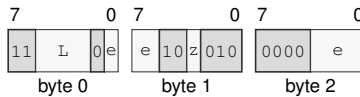
Instruction Fields:

- Size field** `z`—Selects L/Q.
- Register field** `r`—Selects the operand.

NEG.X(z:B/W) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for NEG.X(z:B/W) <ea>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

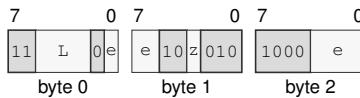
Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects B/W.

NEG.X(z:L/Q) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for NEG.X(z:L/Q) <ea>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register except $R_N(r)$; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects L/Q.

NOP

No Operation

NOP

Operation: Retires normally without changing architectural state.

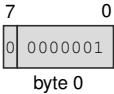
Assembler Syntax: NOP

Attributes: Class = system
Family = core control
Privilege = any
Length = 1 byte
Repeat = none

Instruction Forms:

NOP
extrashort · 1 byte · any

Instruction Format:



Format — Instruction format for NOP

NOT**Bitwise Not****NOT**

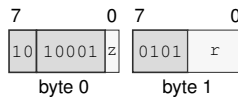
Operation: Writes the selected-width bitwise complement of the destination value.

Assembler Syntax: `NOT.X(z:L/Q) Rn(r)`
`NOT.X(z:B/W) <ea>`
`NOT.X(z:L/Q) <ea>`

Attributes: Class = integer
Family = integer unary
Privilege = unprivileged
Length = 2-13 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

`NOT.X(z:L/Q) Rn(r)`
short · 2 bytes · unprivileged

Instruction Format:

Format — Instruction format for NOT.X(z:L/Q) Rn(r)

Instruction Fields:

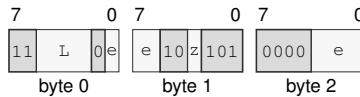
Size field z—Selects L/Q.

Register field r—Selects the operand.

NOT.X(z:B/W) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for NOT.X(z:B/W) <ea>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

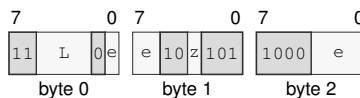
Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects B/W.

NOT.X(z:L/Q) <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for NOT.X(z:L/Q) <ea>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register except $R_n(r)$; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Size field z—Selects L/Q.

OR**Bitwise Or****OR**

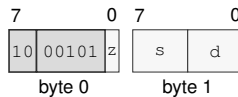
Operation: Writes the selected-width bitwise disjunction of the source and destination values to the destination.

Assembler Syntax: `OR.X(z:L/Q) Rn(s), Rn(d)`
`OR.X(z:B/W) Rn(s), <ea>(e)`
`OR.X(z:L/Q) Rn(s), <ea>(e)`
`OR.X(z:B/W) <ea>(e), Rn(d)`
`OR.X(z:L/Q) <ea>(e), Rn(d)`

Attributes: Class = integer
Family = integer alu
Privilege = unprivileged
Length = 2-13 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

`OR.X(z:L/Q) Rn(s), Rn(d)`
short · 2 bytes · unprivileged

Instruction Format:

Format — Instruction format for **OR.X(z:L/Q) Rn(s), Rn(d)**

Instruction Fields:

Size field *z*—Selects L/Q.

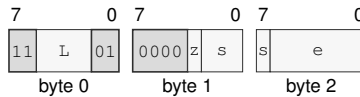
Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

OR.X(z:B/W) Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for OR.X(z:B/W) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

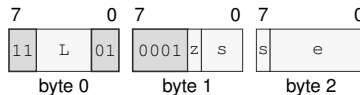
Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

OR.X(z:L/Q) Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for OR.X(z:L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

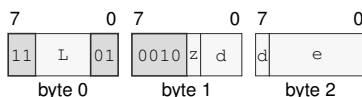
EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

OR.X(z:B/W) <ea>(e), Rn(d)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for OR.X(z:B/W) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

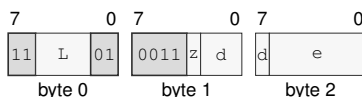
EA availability

Allowed: Register except Rn(r); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

OR.X(z:L/Q) <ea>(e), Rn(d)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for OR.X(z:L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

PARITY

Parity Test

PARITY

Operation:

Computes the parity of the selected operand size and writes popcount(src) & 1 to the destination register. The written value is 1 for odd parity and 0 for even parity. FLAGS are unchanged.

Assembler Syntax:

PARITY.X(z:B/W/L/Q) <ea>(e), Rn(d)

Attributes:

Class = integer

Family = integer bitfield

Privilege = unprivileged

Length = 4-14 bytes

Repeat = REP, REPcc, REPG

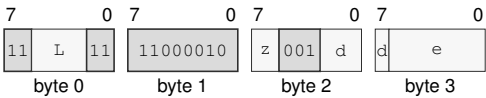
REPcc observation = result dst

Instruction Forms:

PARITY.X(z:B/W/L/Q) <ea>(e), Rn(d)

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for PARITY.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.
- EA availability**
- Allowed:** Register; Memory; Immediate; EXT0
- Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

POP**Pop****POP**

Operation: POP loads one 64-bit stack value into a general register or SREG destination. The SREG form accepts DS, SS, and GS0-GS5 and is unprivileged. A candidate segment image is validated before either the destination SREG or SP is changed. POP SS reads through the old SS and makes the new SS visible only when the destination and updated SP commit together.

Assembler Syntax: POP Rn(r)
POP SREG(s)

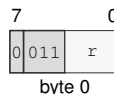
Attributes: Class = data movement
Family = data movement
Privilege = unprivileged
Length = 1-2 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result reg

Detailed Semantics

POP reads and validates the stack value without changing architectural state. For an SREG destination, segment-image validation completes before commit. It then updates the destination and SP together; any preceding access or validation fault leaves both unchanged. POP SS performs the read using the old SS and exposes the new SS only at the final commit.

Instruction Forms:

POP Rn(r)
extrashort · 1 byte · unprivileged

Instruction Format:

Format — Instruction format for POP Rn(r)

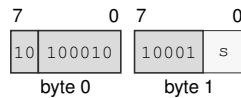
Instruction Fields:

Register field r—Selects the operand.

POP SREG(s)

short · 2 bytes · unprivileged

Instruction Format:



Format — Instruction format for POP SREG(s)

Instruction Fields:

Bits field s—Encodes the bits value.

POPCNT

Population Count

POPCNT

Operation: Counts set bits within the selected B, W, L, or Q source width and writes that count to the destination.

Assembler Syntax: `POPCNT.X(z:B/W/L/Q) <ea>(e), Rn(d)`

Attributes:

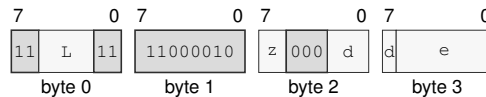
- Class = integer
- Family = integer bitfield
- Privilege = unprivileged
- Length = 4-14 bytes
- Repeat = REP, REPcc, REPG
- REPcc observation = result dst

Instruction Forms:

`POPCNT.X(z:B/W/L/Q) <ea>(e), Rn(d)`

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for POPCNT.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

POPP

Pop Register Pairs

POPP

Operation:	Pops the canonical register pair selected by the pair ID. The canonical pair sequence is pair 0 (R14, R15), pair 1 (R12, R13), pair 2 (R10, R11), pair 3 (R8, R9), pair 4 (R6, R7), pair 5 (R4, R5), pair 6 (R2, R3), and pair 7 (R0, R1). All eight pair IDs are valid and each selects one pair.
Assembler Syntax:	POPP pair_id(i)
Attributes:	Class = data movement Family = data movement Privilege = unprivileged Length = 1 byte Repeat = REP, REPG

Detailed Semantics

POPP selects one canonical pair using the unsigned 3-bit pair index and reads the complete two-slot stack range through the old SS:SP context before either destination register or SP is changed. For a listed pair (*first*, *second*), the second register is read from old SP and the first register is read from old SP plus 8. Both destination registers and the SP update to old SP plus 16 commit together. A preceding fault changes neither destination register nor SP.

Multiple POPP instructions can be used in reverse canonical epilogue order to restore multiple pairs.

Instruction Forms:

POPP pair_id(i)
extrashort · 1 byte · unprivileged

Instruction Format:



Format — Instruction format for POPP pair_id(i)

Instruction Fields:

Immediate field *i*—Encodes the immediate value.

PREFETCH

Prefetch

PREFETCH

Operation: Computes the source address without an architectural read and issues an optional temporal read-prefetch hint. An AT=1 mapping produces no slot transaction. Ignoring the hint has no architectural effect except under an explicitly configured prefetch-fault policy.

Assembler Syntax: `PREFETCH <ea>(e)`

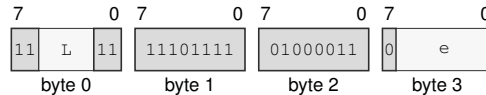
Attributes:
 Class = system
 Family = cache
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = REP, REPG

Instruction Forms:

`PREFETCH <ea>(e)`

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for PREFETCH <ea>(e)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field *e*—Specifies the address operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

PREFETCHNT

Prefetch Non-Temporal

PREFETCHNT

Operation: Computes the source address without an architectural read and issues an optional non-temporal read-prefetch hint. An AT=1 mapping produces no slot transaction. Ignoring the hint has no architectural effect except under an explicitly configured prefetch-fault policy.

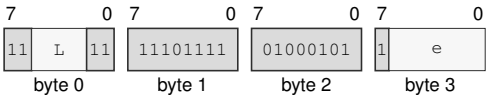
Assembler Syntax: PREFETCHNT <ea>(e)

Attributes:
Class = system
Family = cache
Privilege = unprivileged
Length = 4-14 bytes
Repeat = REP, REPG

Instruction Forms:

PREFETCHNT <ea>(e)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for PREFETCHNT <ea>(e)

Instruction Fields:

- Length field L**—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Effective Address field e**—Specifies the address operand.
- EA availability**
Allowed: Memory; Immediate; EXT0
Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

PTQUERY**Page Table Query****PTQUERY**

Operation: PTQUERY treats the computed source effective address as a linear address and walks the page tables configured by PTCR down to the requested level. Levels 1 through 5 name L1 through L5; level 5 is unavailable in LA48 mode. If every preceding table pointer is valid, the destination receives the complete raw 64-bit PTE at the requested level, including software-defined bits. Z reports whether that returned entry is structurally valid for its level, and N, C, and V report accumulated U, W, and X permissions only when Z is set. A malformed requested entry is therefore observable in the destination even though Z is clear. Failure before the requested entry is read returns zero and clears all four flags. The query is independent of PTCR.PE and never sets PTE A or D, fills or modifies a TLB entry, or raises PAGE_FAULT for walk failure.

Assembler Syntax: PTQUERY pt_level(i), <ea>(e), Rn(d)

Attributes:
 Class = system
 Family = tlb and context
 Privilege = supervisor
 Length = 4-14 bytes
 Repeat = none

Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	requested PTE was read and is structurally valid for its level
N	valid queried path is user-domain accessible; cleared on failure
C	valid queried path is writable; cleared on failure
V	valid queried path is executable; cleared on failure

The requested level is the low three bits of the level operand. Values outside 1 through 5 raise ILLEGAL_INSTRUCTION; level 5 is reported as an ordinary unsuccessful query in LA48 mode.

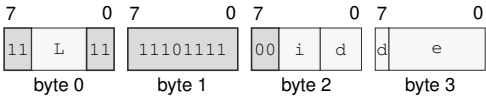
1. Compute the linear address without reading the source memory and reject a noncanonical address as an unsuccessful query.
2. Starting at the PTCR root, read one 64-bit PTE per level. Every preceding entry must be a valid table pointer with P, T, D, G, and AT in their required states.
3. Accumulate U, W, and X permissions along the traversed path. Stop after reading the requested level, even when that requested entry is structurally malformed.
4. Return the raw requested PTE and set Z only when it is structurally valid. N, C, and V report the accumulated U, W, and X permissions only while Z is set.

Failure before the requested entry is read returns zero and clears all four flags. The walk ignores PTCR.PE, does not set PTE A or D, does not modify a TLB, and does not raise PAGE_FAULT for walk failure.

Instruction Forms:

PTQUERY pt_level(i), <ea>(e), Rn(d)
long · 4-14 bytes · supervisor

Instruction Format:



Format — Instruction format for PTQUERY pt_level(i), <ea>(e), Rn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Immediate field** i—Encodes the immediate value.
- Register field** d—Selects the operand 3.
- Effective Address field** e—Specifies the address operand.
- EA availability**
 - Allowed:** Register; Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

PUSH

Push

PUSH

Operation: PUSH stores one 64-bit general-register value, selected SREG image, or current CS image on the stack. The SREG form accepts DS, SS, and GS0-GS5. PUSH SS captures the old SS image and uses that same old SS as the stack context for the store.

Assembler Syntax: PUSH CS
 PUSH Rn(r)
 PUSH SREG(s)

Attributes: Class = data movement
 Family = data movement
 Privilege = unprivileged
 Length = 1-2 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = source reg

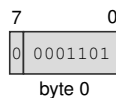
Detailed Semantics

PUSH captures the source value and current SS:SP before changing architectural state. After the destination stack slot has been validated, it commits the 64-bit store and decremented SP together. A fault before commit leaves memory and SP unchanged. For PUSH SS, both the captured value and the stack access use the old SS image.

Instruction Forms:

PUSH CS
 extrashort · 1 byte · unprivileged

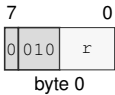
Instruction Format:



Format — Instruction format for PUSH CS

PUSH Rn(r)
extrashort · 1 byte · unprivileged

Instruction Format:



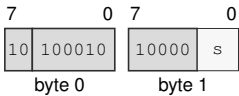
Format — Instruction format for PUSH Rn(r)

Instruction Fields:

Register field r—Selects the operand.

PUSH SREG(s)
short · 2 bytes · unprivileged

Instruction Format:



Format — Instruction format for PUSH SREG(s)

Instruction Fields:

Bits field s—Encodes the bits value.

PUSHP

Push Register Pairs

PUSHP

Operation: Pushes the canonical register pair selected by the pair ID. The canonical pair sequence is pair 0 (R14, R15), pair 1 (R12, R13), pair 2 (R10, R11), pair 3 (R8, R9), pair 4 (R6, R7), pair 5 (R4, R5), pair 6 (R2, R3), and pair 7 (R0, R1). All eight pair IDs are valid and each selects one pair.

Assembler Syntax: `PUSHP pair_id(i)`

Attributes:
 Class = data movement
 Family = data movement
 Privilege = unprivileged
 Length = 1 byte
 Repeat = REP, REPG

Detailed Semantics

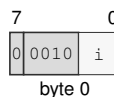
PUSHP selects one canonical pair using the unsigned 3-bit pair index and captures both register values and the old SS:SP context. It validates the complete two-slot stack range before either slot or SP is changed. For a listed pair (*first*, *second*), the second register is stored at old SP minus 16 and the first register is stored at old SP minus 8. Both stores and the SP update to old SP minus 16 commit together. A preceding fault changes neither slot nor SP.

Multiple PUSHP instructions can be used in canonical prologue order to save multiple pairs.

Instruction Forms:

`PUSHP pair_id(i)`
 extrashort · 1 byte · unprivileged

Instruction Format:



Format — Instruction format for PUSHP pair_id(i)

Instruction Fields:

Immediate field *i*—Encodes the immediate value.

RDCR

Read Control Register

RDCR

Operation: RDCR reads a selected control register in supervisor mode.

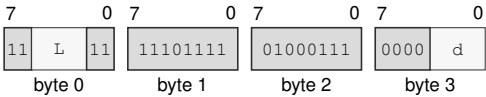
Assembler Syntax: RDCR <imm16>, Rn(d)

Attributes:
Class = system
Family = system registers
Privilege = supervisor
Length = 6 bytes
Repeat = none

Instruction Forms:

RDCR <imm16>, Rn(d)
long · 6 bytes · supervisor

Instruction Format:



Format — Instruction format for RDCR <imm16>, Rn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Register field** d—Selects the destination operand.

RDFLAGS**Read Flags****RDFLAGS**

Operation: RDFLAGS reads the architectural 16-bit FLAGS register, zero-extends it to 64 bits, and writes the result to the destination Rn register.

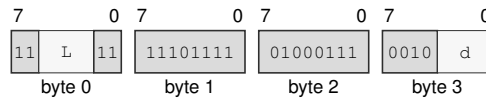
Assembler Syntax: RDFLAGS Rn(d)

Attributes:
 Class = system
 Family = system registers
 Privilege = unprivileged
 Length = 4 bytes
 Repeat = none

Instruction Forms:

RDFLAGS Rn(d)

long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for RDFLAGS Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d—Selects the operand.

RDPMC

Read Performance Counter

RDPMC

Operation: RDPMC reads a mandatory or implementation-discovered performance counter from any privilege level. Counter ID zero and every other unimplemented counter ID raise INVALID_CONTROL_STATE with the INVALID_SELECTOR cause.

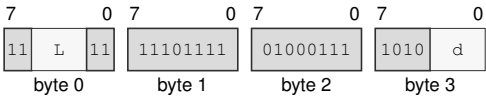
Assembler Syntax: RDPMC <imm16>, Rn(d)

Attributes: Class = system
Family = system registers
Privilege = unprivileged
Length = 6 bytes
Repeat = none

Instruction Forms:

RDPMC <imm16>, Rn(d)
long · 6 bytes · unprivileged

Instruction Format:



Format — Instruction format for RDPMC <imm16>, Rn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Register field** d—Selects the destination operand.

RDSEG**Read Segment Register****RDSEG**

Operation: RDSEG reads the 64-bit image of an SREG-class segment register into the destination Rn register. The RDSEG CS form reads the current CS image.

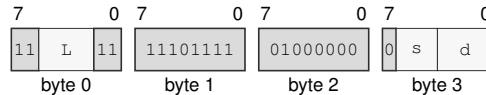
Assembler Syntax: RDSEG SREG(s), Rn(d)
RDSEG CS, Rn(d)

Attributes: Class = system
Family = system registers
Privilege = unprivileged
Length = 4 bytes
Repeat = none

Instruction Forms:

RDSEG SREG(s), Rn(d)

long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for RDSEG SREG(s), Rn(d)

Instruction Fields:

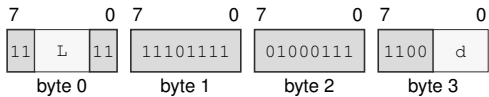
Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Bits field s—Encodes the bits value.

Register field d—Selects the destination operand.

RDSEG CS, Rn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for RDSEG CS, Rn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Register field** d—Selects the destination operand.

RDSTATUS

Read Status

RDSTATUS

Operation: RDSTATUS reads the architectural 16-bit STATUS register, zero-extends it to 64 bits, and writes the result to the destination Rn register. Reading STATUS is unprivileged; writing STATUS remains supervisor-only.

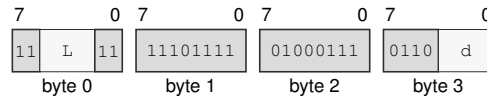
Assembler Syntax: RDSTATUS Rn(d)

Attributes:
 Class = system
 Family = system registers
 Privilege = unprivileged
 Length = 4 bytes
 Repeat = none

Instruction Forms:

RDSTATUS Rn(d)
 long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for RDSTATUS Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d—Selects the operand.

REPG**Grouped Repeat****REPG**

Operation: REPG takes an Rn counter and an unsigned 16-bit body byte count. The body starts at the next instruction after REPG and extends for exactly body_bytes bytes. body_bytes is an unsigned byte count. A zero body byte count is invalid. A nonzero body byte count is valid only when the selected byte range decodes as a whole number of complete instructions and ends exactly at an instruction boundary. The grouped body is decoded when repeat state is entered and remains the repeated body until the repeat context exits.

Assembler Syntax: REPG Rn(r), { (instructions...) }
REPGF Rn(r), { (instructions...) }

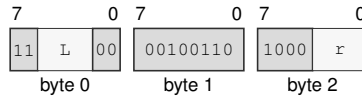
Attributes: Class = control flow
Family = control flow
Privilege = unprivileged
Length = 3 bytes
Repeat = none

Detailed Semantics

The grouped body is decoded and each body instruction is checked for REPG repeatability before the zero-count rule is applied. If the counter is zero, the decoded group is annulled and has no architectural side effects. If the counter is nonzero, one successful whole-group iteration decrements it by one. The update occurs only after the whole grouped body iteration succeeds. A normal REPG body may write the selected counter register; such writes are ordinary architectural writes in the grouped instruction stream and do not modify the captured repeat remaining-count value. At a successful group-iteration boundary, the repeat counter update has final priority for the selected counter register. The REPGF assembler-checked alias rejects bodies that write the selected counter register. Completed prior body instructions in a faulting iteration remain committed, later body instructions do not execute, and the saved repeat continuation records the selected counter, condition T, repeat kind, group_start_delta, and body_bytes in FRAME_EXT1. FLAGS and FFLAGS follow the rules of the last completed grouped instruction; FFLAGS from completed floating-point operations are accumulated by bitwise inclusive-or.

Instruction Forms:

```
REPG Rn(r), { (instructions...) }
medium · 3 bytes · unprivileged
```

Instruction Format:**Format — Instruction format for REPG Rn(r), { (instructions...) }****Instruction Fields:**

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field *r*—Selects the source operand.

REPcc**Repeat Conditional****REPcc**

Operation: REPcc installs repeat state for the following single instruction. The selected counter register is interpreted as an unsigned remaining count. The body instruction is decoded before zero-count annulment, and body repeatability is checked before annulment. REP is the unconditional spelling and uses condition T. Condition F is reserved and is not a valid encoding. While repeat state is active, architectural PC points at the body instruction boundary. Exceptions and architectural events save repeat state in FRAME_EXT1 and clear architectural repeat state before entering the handler; ERET restores the saved repeat state with the saved PC and status state.

Assembler Syntax: REPcc Rn(r), (instruction)
REP Rn(r), (instruction)

Attributes: Class = control flow
Family = control flow
Privilege = unprivileged
Length = 3 bytes
Repeat = none

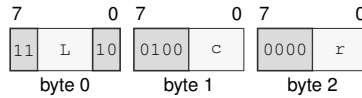
Detailed Semantics

The body instruction is fetched, decoded, and checked for repeatability before the zero-count rule is applied. If the counter is zero, the decoded body is annulled and has no architectural side effects. If the counter is nonzero, the first body iteration executes before any condition check. The repeat condition is the body instruction's repeat-observed value, observed during body execution rather than sampled by REP outside the body. Body commit, condition observation, counter decrement, and continuation PC selection are one precise architectural commit step. Every completed body iteration decrements the counter by one, including the terminating condition-false iteration. Writes by the body to the selected counter register have no architectural effect while repeat state is active.

Instruction Forms:

REPcc Rn(r), (instruction)

medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for REPcc Rn(r), (instruction)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Condition field *c*—Selects the condition code. Allowed values: T, EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field *r*—Selects the source operand.

RESET

Reset

RESET

Operation:	RESET serializes earlier memory effects, applies the complete architectural warm-reset state to the executing logical processor, and resumes execution at BOOTPC. BOOTPC and BOOTCFG are preserved.
Assembler Syntax:	RESET
Attributes:	Class = system Family = core control Privilege = supervisor Length = 2 bytes Repeat = none

Detailed Semantics

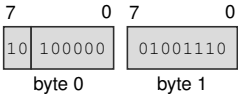
RESET is supervisor-only and affects only the executing logical processor. Before the reset-state transition commits, all earlier memory effects and pending write-combining stores from that logical processor complete. No younger instruction effect becomes visible.

The instruction installs the complete Architectural Reset State table. R0 through R15 and SP become zero, PC becomes BOOTPC, STATUS contains only PM set, and BOOTPC and BOOTCFG retain their values. Architectural repeat state, the load reservation, translation and page-walk caches, decoded instruction state, and prefetch state are invalidated. Coherent data- and instruction-cache contents remain coherent. Instruction-fetch synchronization completes before the first instruction at BOOTPC is fetched.

Instruction Forms:

RESET
short · 2 bytes · supervisor

Instruction Format:



Format — Instruction format for RESET

RESTORE**Restore Processor State****RESTORE**

Operation: The save area uses a fixed architectural layout discovered through CPUID. The save-area base must be 4 KiB aligned. RESTORE is the matching restore operation for SAVE and may restore base architectural state as well as enabled extension state from the same save image. An unrecognized format or any set bitmap bit that lacks a CPUID component descriptor raises `INVALID_CONTROL_STATE` before architectural state changes. User-mode RESTORE ignores the saved STATUS image; supervisor RESTORE applies the normal STATUS write rules.

Assembler Syntax: `RESTORE <ea>(e)`

Attributes:

- Class = system
- Family = tlb and context
- Privilege = unprivileged
- Length = 4-14 bytes
- Repeat = none

Detailed Semantics

RESTORE treats its address-role EA as the base of the save area without first reading an EA-sized operand. It restores R0–R15 and FLAGS from the fixed base block. A set `GS_VALID` bit selects the corresponding GS image for validation and restoration; a clear bit preserves the current GS_n. User-mode RESTORE ignores the saved STATUS image, while supervisor RESTORE applies the normal STATUS write rules. A set state-block bitmap bit restores its CPUID-described extension component. A clear bit for a described component installs that component's initial state and makes the component clean; a restored non-initial image makes the component modified.

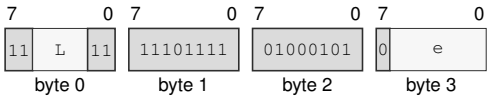
Before reading any extension component or changing architectural state, RESTORE validates the complete base header and bitmap. An unrecognized format, a nonzero reserved bit, or any set bitmap bit without a matching CPUID component descriptor raises `INVALID_CONTROL_STATE`.

The complete `SAVE_AREA_SIZE_BYTES` range reported by CPUID, every selected GS image, and every selected extension component are validated before architectural state is committed. Any address, access, or validation fault leaves the architectural register state and save area unchanged.

Instruction Forms:

RESTORE <ea>(e)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for RESTORE <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Effective Address field** e—Specifies the address operand.
- EA availability**
 - Allowed:** Register; Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

RET**Return****RET**

Operation: Reads one 64-bit return address from SS:SP, advances SP by eight bytes, validates the target, and transfers control to it.

Assembler Syntax: RET

Attributes: Class = control flow
Family = control flow
Privilege = unprivileged
Length = 1 byte
Repeat = none

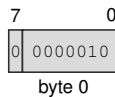
Detailed Semantics

1. Read the 8-byte return address through the old SS:SP context without changing SP.
2. Validate that address as an executable target under the current CS.
3. Commit advanced SP and the new PC as one successful control-transfer operation.

A stack-read or target-validation fault leaves SP and PC unchanged.

Instruction Forms:

RET
extrashort · 1 byte · unprivileged

Instruction Format:

Format — Instruction format for RET

REVBYTE

Reverse Bytes

REVBYTE

Operation: Reverses the byte order within the selected operand width and writes the result to the destination.

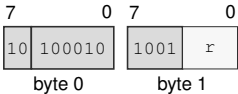
Assembler Syntax: REVBYTE.W Rn(r)
REVBYTE.L Rn(r)
REVBYTE.Q Rn(r)
REVBYTE.W <ea>
REVBYTE.L <ea>
REVBYTE.Q <ea>

Attributes: Class = integer
Family = integer alu
Privilege = unprivileged
Length = 2-13 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

REVBYTE.W Rn(r)
short · 2 bytes · unprivileged

Instruction Format:



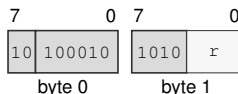
Format — Instruction format for REVBYTE.W Rn(r)

Instruction Fields:

Register field r—Selects the operand.

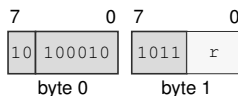
REVBYTE.L Rn(r)

short · 2 bytes · unprivileged

Instruction Format:**Format — Instruction format for REVBYTE.L Rn(r)****Instruction Fields:****Register field** r—Selects the operand.

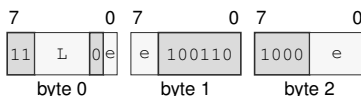
REVBYTE.Q Rn(r)

short · 2 bytes · unprivileged

Instruction Format:**Format — Instruction format for REVBYTE.Q Rn(r)****Instruction Fields:****Register field** r—Selects the operand.

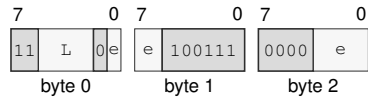
REVBYTE.W <ea>

medium · 3-13 bytes · unprivileged

Instruction Format:**Format — Instruction format for REVBYTE.W <ea>****Instruction Fields:****Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.**Effective Address field** e—Specifies the read/write operand.**EA availability****Allowed:** Register except Rn(r); Memory; EXT0**Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

REVBYTE.L <ea>
medium · 3-13 bytes · unprivileged

Instruction Format:



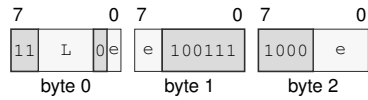
Format — Instruction format for REVBYTE.L <ea>

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Effective Address field** e—Specifies the read/write operand.
- EA availability**
 - Allowed:** Register except $R_N(r)$; Memory; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

REVBYTE.Q <ea>
medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for REVBYTE.Q <ea>

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Effective Address field** e—Specifies the read/write operand.
- EA availability**
 - Allowed:** Register except $R_N(r)$; Memory; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

RFENCE**Read Fence****RFENCE**

Operation: Prevents retirement until every earlier memory read is ordered before every later memory read on the same hardware thread. It performs no memory access and changes no register or flag.

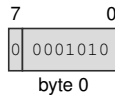
Assembler Syntax: RFENCE

Attributes:
 Class = system
 Family = core control
 Privilege = any
 Length = 1 byte
 Repeat = none

Instruction Forms:

RFENCE

extrashort · 1 byte · any

Instruction Format:

Format — Instruction format for RFENCE

ROL

Rotate Left

ROL

Operation: The effective count is the count operand modulo the selected operand width: B uses mod 8, W uses mod 16, L uses mod 32, and Q uses mod 64. If the effective count is zero, the destination is unchanged. FLAGS are unchanged for all counts.

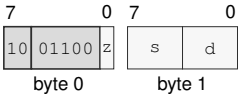
Assembler Syntax: `ROL.X(z:L/Q) Rn(s), Rn(d)`
`ROL.X(z:B/W) Rn(s), <ea>(e)`
`ROL.X(z:L/Q) Rn(s), <ea>(e)`
`ROL.X(z:B/W/L/Q) imm6(i), <ea>(e)`

Attributes: Class = integer
Family = integer bitfield
Privilege = unprivileged
Length = 2-14 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

`ROL.X(z:L/Q) Rn(s), Rn(d)`
short · 2 bytes · unprivileged

Instruction Format:



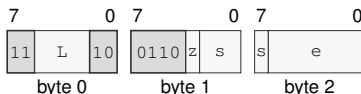
Format — Instruction format for ROL.X(z:L/Q) Rn(s), Rn(d)

Instruction Fields:

- Size field** z—Selects L/Q.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

`ROL.X(z:B/W) Rn(s), <ea>(e)`
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for `ROL.X(z:B/W) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand.

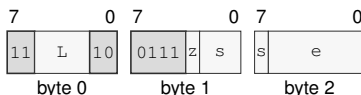
EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`ROL.X(z:L/Q) Rn(s), <ea>(e)`
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for `ROL.X(z:L/Q) Rn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects L/Q.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand.

EA availability

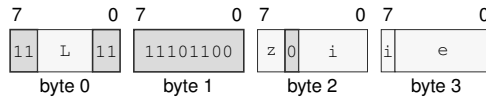
Allowed: Register except $R_n(x)$; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

ROL.X(z:B/W/L/Q) imm6(i), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for ROL.X(z:B/W/L/Q) imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

ROR**Rotate Right****ROR**

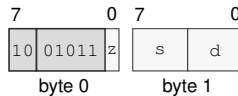
Operation: The effective count is the count operand modulo the selected operand width: B uses mod 8, W uses mod 16, L uses mod 32, and Q uses mod 64. If the effective count is zero, the destination is unchanged. FLAGS are unchanged for all counts.

Assembler Syntax: `ROR.X(z:L/Q) Rn(s), Rn(d)`
`ROR.X(z:B/W) Rn(s), <ea>(e)`
`ROR.X(z:L/Q) Rn(s), <ea>(e)`
`ROR.X(z:B/W/L/Q) imm6(i), <ea>(e)`

Attributes: Class = integer
Family = integer bitfield
Privilege = unprivileged
Length = 2-14 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

`ROR.X(z:L/Q) Rn(s), Rn(d)`
short · 2 bytes · unprivileged

Instruction Format:

Format — Instruction format for ROR.X(z:L/Q) Rn(s), Rn(d)

Instruction Fields:

Size field *z*—Selects L/Q.

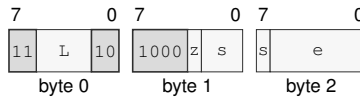
Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

ROR.X(z:B/W) Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for ROR.X(z:B/W) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

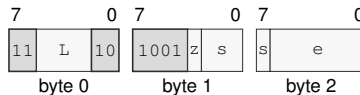
Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

ROR.X(z:L/Q) Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for ROR.X(z:L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

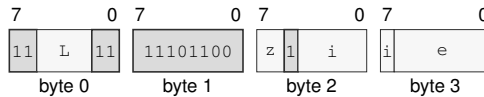
Allowed: Register except Rn(r); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

ROR.X(z:B/W/L/Q) imm6(i), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for ROR.X(z:B/W/L/Q) imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SAR

Shift Arithmetic Right

SAR

Operation: The effective count is the count operand modulo the selected operand width: B uses mod 8, W uses mod 16, L uses mod 32, and Q uses mod 64. If the effective count is zero, the destination is unchanged. FLAGS are unchanged for all counts.

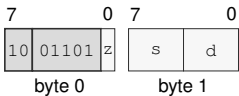
Assembler Syntax: SAR.X(z:L/Q) Rn(s), Rn(d)
SAR.X(z:B/W) Rn(s), <ea>(e)
SAR.X(z:L/Q) Rn(s), <ea>(e)
SAR.X(z:B/W/L/Q) imm6(i), <ea>(e)

Attributes: Class = integer
Family = integer bitfield
Privilege = unprivileged
Length = 2-14 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

SAR.X(z:L/Q) Rn(s), Rn(d)
short · 2 bytes · unprivileged

Instruction Format:



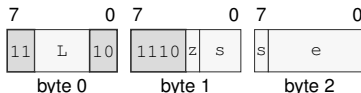
Format — Instruction format for SAR.X(z:L/Q) Rn(s), Rn(d)

Instruction Fields:

- Size field** z—Selects L/Q.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

`SAR.X(z:B/W) Rn(s), <ea>(e)`
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for SAR.X(z:B/W) Rn(s), <ea>(e)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand.

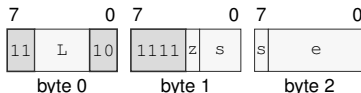
EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`SAR.X(z:L/Q) Rn(s), <ea>(e)`
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for SAR.X(z:L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects L/Q.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the read/write operand.

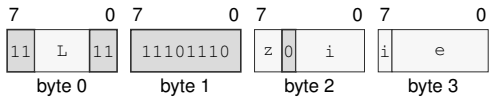
EA availability

Allowed: Register except $R_n(r)$; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SAR.X(z:B/W/L/Q) imm6(i), <ea>(e)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for SAR.X(z:B/W/L/Q) imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SAVE**Save Processor State****SAVE**

Operation: SAVE writes modified enabled extension components to their CPUID-defined 64-byte-aligned offsets and may clear bitmap bits for architecturally clean components. The fixed base block records R0-R15, GS0-GS5, FLAGS, and STATUS with FMT=0 and GS_VALID=0x3f. Software allocates CPUID SAVE_AREA_SIZE_BYTES bytes and locates each component using its reported offset and size. The save-area base must be 4 KiB aligned.

Assembler Syntax: SAVE <ea>(e)

Attributes:
 Class = system
 Family = tlb and context
 Privilege = unprivileged
 Length = 4-14 bytes
 Repeat = none

Detailed Semantics

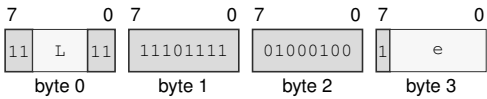
SAVE treats its address-role EA as the base of the save area without first reading an EA-sized operand. It writes the fixed base block, FMT=0, GS_VALID=0x3f, the state-block bitmap, R0-R15, GS0-GS5, FLAGS, and STATUS according to the Processor-State Save and Restore layout. Enabled modified extension components are written to their CPUID-defined offsets; a component known to equal its initial state may be omitted and reported clear in the bitmap. SAVE does not change a component's clean or modified state.

The complete SAVE_AREA_SIZE_BYTES range reported by CPUID is validated before any byte is written. Any address or access fault leaves the architectural register state and save area unchanged.

Instruction Forms:

SAVE <ea>(e)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for SAVE <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Effective Address field** e—Specifies the address operand.
- EA availability**
 - Allowed:** Register; Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

SBB**Subtract With Borrow****SBB**

Operation: Subtracts the selected-width source value and incoming C borrow from the destination, writes the truncated difference, and updates Z, N, C, and V.

Assembler Syntax: `SBB.X(z:B/W/L/Q) Rn(s), Rn(d)`
`SBB.X(z:B/W/L/Q) Rn(s), <ea>(e)`
`SBB.X(z:B/W/L/Q) <ea>(e), Rn(s)`

Attributes: Class = integer
Family = integer alu
Privilege = unprivileged
Length = 3-14 bytes
Repeat = REP, REPCC, REPG
REPCC observation = flags

Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	result == 0
N	most_significant_bit(result)
C	unsigned borrow
V	signed overflow

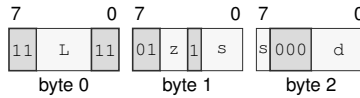
Let n be the selected width, d and s the unsigned operand bit patterns, and b the incoming borrow represented by the C flag. The written result is

$$r = (d - s - b) \bmod 2^n.$$

Z is set when $r = 0$, and N receives the most-significant bit of r . C reports an unsigned borrow from either subtraction. V reports signed overflow for the complete $d - s - b$ operation.

Instruction Forms:

SBB.X(z:B/W/L/Q) Rn(s), Rn(d)
 medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for SBB.X(z:B/W/L/Q) Rn(s), Rn(d)

Instruction Fields:

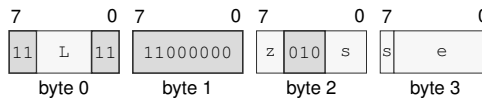
Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

SBB.X(z:B/W/L/Q) Rn(s), <ea>(e)
 long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for SBB.X(z:B/W/L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

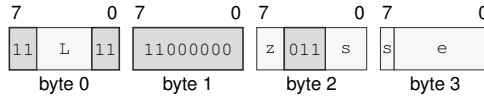
EA availability

Allowed: Register except Rn(r); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SBB.X(z:B/W/L/Q) <ea>(e), Rn(s)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for SBB.X(z:B/W/L/Q) <ea>(e), Rn(s)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field s—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Register except Rn(r); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SEGLEA

Segment Load Effective Address

SEGLEA

Operation: SEGLEA computes the source effective-address point and applies the selected segment's pre-translation to that point. The size suffix selects the index scale used during effective-address calculation. On success, SEGLEA writes the segment result to the destination and clears V. A failed segment-bound check sets V, suppresses the destination write, commits effective-address auto-update effects, and completes without an exception.

Assembler Syntax: SEGLEA.X(z:B/W/L/Q) <ea>(e), Rn(d)

Attributes: Class = data movement
Family = ea utility
Privilege = unprivileged
Length = 4-14 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

Detailed Semantics

FLAGS Effects

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	cleared on success; set when the segment-point bounds check fails

Let a be the source effective address before a memory read. From the selected segment image, define

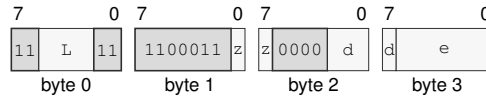
$$b = \text{segment}[63 : 12] \ll 12, \quad s = (\text{mantissa} \ll \text{exponent}) \ll 12.$$

A zero mantissa disables bounds and returns a . In translated mode, a must satisfy $0 \leq a < s$ and the result is $b + a$. In bounds-only mode, a must satisfy $b \leq a < b + s$ and the result remains a . V is set for an out-of-bounds address, in which case the destination is not written.

Instruction Forms:

SEGLEA.X(z:B/W/L/Q) <ea>(e), Rn(d)

long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for SEGLEA.X(z:B/W/L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the address operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SET

Set True

SET

Operation: Writes integer one to the selected-width destination.

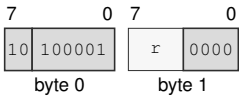
Assembler Syntax: SET Rn(r)

Attributes: Class = control flow
Family = control flow
Privilege = unprivileged
Length = 2 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

SET Rn(r)
short · 2 bytes · unprivileged

Instruction Format:



Format — Instruction format for SET Rn(r)

Instruction Fields:

Register field r—Selects the operand.

SETcc**Set Conditional****SETcc**

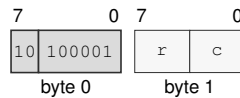
Operation: Writes one when the encoded condition is true and zero when it is false.

Assembler Syntax: SETcc Rn(r)

Attributes:
 Class = control flow
 Family = control flow
 Privilege = unprivileged
 Length = 2 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Instruction Forms:

SETcc Rn(r)
 short · 2 bytes · unprivileged

Instruction Format:

Format — Instruction format for SETcc Rn(r)

Instruction Fields:

Register field r—Selects the operand.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

SETF

Set Flags

SETF

Operation: SETF sets the selected architectural integer FLAGS bits. The immediate operand is a 4-bit mask: bit 3 selects Z, bit 2 selects N, bit 1 selects C, and bit 0 selects V. Selected bits are written to one; unselected FLAGS bits are unchanged. The assembler may provide symbolic mask aliases such as Z, N, C, V, and combinations using |. STC is an assembler alias for SETF C.

Assembler Syntax: SETF flags_bitmap(m)

Attributes: Class = system
Family = system registers
Privilege = unprivileged
Length = 3 bytes
Repeat = none

Detailed Semantics

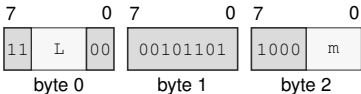
FLAGS Effects

Flag	Effect
Z	set when mask bit 3 is one; otherwise unchanged
N	set when mask bit 2 is one; otherwise unchanged
C	set when mask bit 1 is one; otherwise unchanged
V	set when mask bit 0 is one; otherwise unchanged

Instruction Forms:

SETF flags_bitmap(m)
medium · 3 bytes · unprivileged

Instruction Format:



Format — Instruction format for SETF flags_bitmap(m)

Instruction Fields:

Length field *L*—Encodes the total instruction length as 3 + *L* bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Immediate field *m*—Encodes the immediate value.

SHL**Shift Left****SHL**

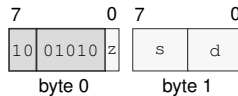
Operation: The effective count is the count operand modulo the selected operand width: B uses mod 8, W uses mod 16, L uses mod 32, and Q uses mod 64. If the effective count is zero, the destination is unchanged. FLAGS are unchanged for all counts.

Assembler Syntax: SHL.X(z:L/Q) Rn(s), Rn(d)
 SHL.X(z:B/W) Rn(s), <ea>(e)
 SHL.X(z:L/Q) Rn(s), <ea>(e)
 SHL.X(z:B/W/L/Q) imm6(i), <ea>(e)

Attributes: Class = integer
 Family = integer bitfield
 Privilege = unprivileged
 Length = 2-14 bytes
 Repeat = REP, REPcc, REPG
 REPcc observation = result dst

Instruction Forms:

SHL.X(z:L/Q) Rn(s), Rn(d)
 short · 2 bytes · unprivileged

Instruction Format:

Format — Instruction format for SHL.X(z:L/Q) Rn(s), Rn(d)

Instruction Fields:

Size field z—Selects L/Q.

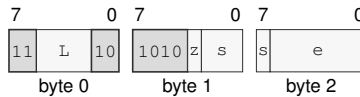
Register field s—Selects the source operand.

Register field d—Selects the destination operand.

SHL.X(z:B/W) Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for SHL.X(z:B/W) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

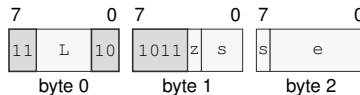
Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SHL.X(z:L/Q) Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for SHL.X(z:L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

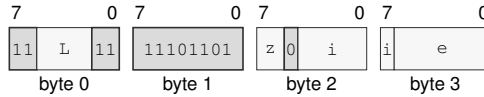
Allowed: Register except Rn(r); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SHL.X(z:B/W/L/Q) imm6(i), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for SHL.X(z:B/W/L/Q) imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SHR

Shift Right

SHR

Operation:

The effective count is the count operand modulo the selected operand width: B uses mod 8, W uses mod 16, L uses mod 32, and Q uses mod 64. If the effective count is zero, the destination is unchanged. FLAGS are unchanged for all counts.

Assembler Syntax:

SHR.X(z:L/Q) Rn(s), Rn(d)
SHR.X(z:B/W) Rn(s), <ea>(e)
SHR.X(z:L/Q) Rn(s), <ea>(e)
SHR.X(z:B/W/L/Q) imm6(i), <ea>(e)

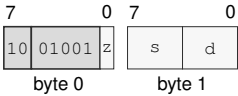
Attributes:

Class = integer
Family = integer bitfield
Privilege = unprivileged
Length = 2-14 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

SHR.X(z:L/Q) Rn(s), Rn(d)
short · 2 bytes · unprivileged

Instruction Format:



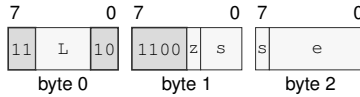
Format — Instruction format for SHR.X(z:L/Q) Rn(s), Rn(d)

Instruction Fields:

- Size field** z—Selects L/Q.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

SHR.X(z:B/W) Rn(s), <ea>(e)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for SHR.X(z:B/W) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

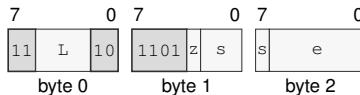
EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SHR.X(z:L/Q) Rn(s), <ea>(e)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for SHR.X(z:L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

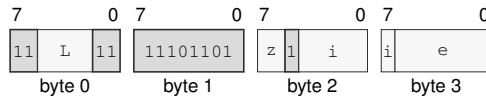
Allowed: Register except Rn(r); Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SHR.X(z:B/W/L/Q) imm6(i), <ea>(e)

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for SHR.X(z:B/W/L/Q) imm6(i), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Immediate field i—Encodes the immediate value.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SUB**Subtract****SUB**

Operation: Performs selected-width modular subtraction of the source from the destination and writes the low result bits back to the destination.

Assembler Syntax:

```

SUB.Q 8, SP
SUB.X(z:L/Q) Rn(s), Rn(d)
SUB.Q imm8(i), SP
SUB.Q <imm16s>, SP
SUB.Q <imm32s>, SP
SUB.X(z:B/W) Rn(s), <ea>(e)
SUB.X(z:L/Q) Rn(s), <ea>(e)
SUB.X(z:B/W) <ea>(e), Rn(d)
SUB.X(z:L/Q) <ea>(e), Rn(d)

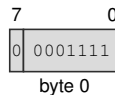
```

Attributes:

- Class = integer
- Family = integer alu
- Privilege = unprivileged
- Length = 1-13 bytes
- Repeat = REP, REPcc, REPG
- REPcc observation = result dst

Instruction Forms:

SUB.Q 8, SP
extrashort · 1 byte · unprivileged

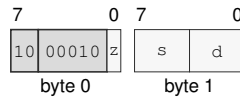
Instruction Format:

Format — Instruction format for SUB.Q 8, SP

SUB.X(z:L/Q) Rn(s), Rn(d)

short · 2 bytes · unprivileged

Instruction Format:



Format — Instruction format for SUB.X(z:L/Q) Rn(s), Rn(d)

Instruction Fields:

Size field z—Selects L/Q.

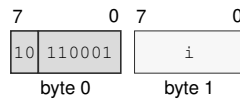
Register field s—Selects the source operand.

Register field d—Selects the destination operand.

SUB.Q imm8(i), SP

short · 2 bytes · unprivileged

Instruction Format:



Format — Instruction format for SUB.Q imm8(i), SP

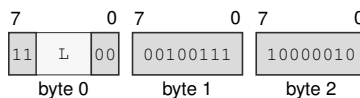
Instruction Fields:

Immediate field i—Encodes the immediate value. Allowed values: 1-255.

SUB.Q <imm16s>, SP

medium · 5 bytes · unprivileged

Instruction Format:



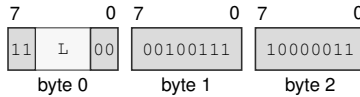
Format — Instruction format for SUB.Q <imm16s>, SP

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

SUB.Q <imm32s>, SP
 medium · 7 bytes · unprivileged

Instruction Format:



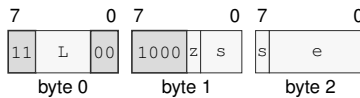
Format — Instruction format for SUB.Q <imm32s>, SP

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

SUB.X(z:B/W) Rn(s), <ea>(e)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for SUB.X(z:B/W) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

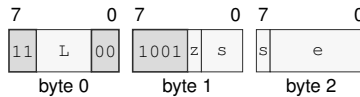
EA availability

Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

$\text{SUB.X}(z:L/Q) \text{ Rn}(s), \langle ea \rangle(e)$
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for $\text{SUB.X}(z:L/Q) \text{ Rn}(s), \langle ea \rangle(e)$

Instruction Fields:

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z —Selects L/Q .

Register field s —Selects the source operand.

Effective Address field e —Specifies the read/write operand.

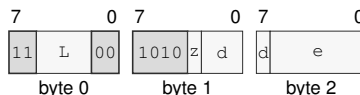
EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

$\text{SUB.X}(z:B/W) \langle ea \rangle(e), \text{Rn}(d)$
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for $\text{SUB.X}(z:B/W) \langle ea \rangle(e), \text{Rn}(d)$

Instruction Fields:

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z —Selects B/W .

Register field d —Selects the destination operand.

Effective Address field e —Specifies the source operand.

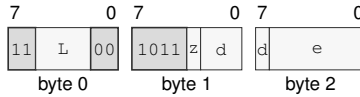
EA availability

Allowed: Register except $\text{Rn}(r)$; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SUB.X(z:L/Q) <ea>(e), Rn(d)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for SUB.X(z:L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

SWPT

Switch Page Table

SWPT

Operation:

Replaces PTCR with the supplied page-table control value and clears ASCR as one architectural page-table switch.

Assembler Syntax:

SWPT Rn(p)

Attributes:

Class = system

Family = tlb and context

Privilege = supervisor

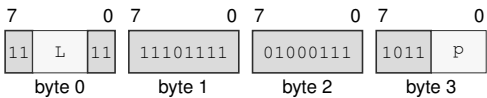
Length = 4 bytes

Repeat = none

Instruction Forms:

SWPT Rn(p)
long · 4 bytes · supervisor

Instruction Format:



Format — Instruction format for SWPT Rn(p)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Register field** p—Selects the operand.

SWPTA

Switch Page Table With Context

SWPTA

Operation: Updates PTCR from the first Rn register, writes the low 16 bits of the second Rn register into ASCR[31:16], and sets ASCR.AE.

Assembler Syntax: SWPTA Rn(p), Rn(a)

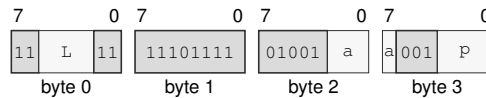
Attributes:
 Class = system
 Family = tlb and context
 Privilege = supervisor
 Length = 4 bytes
 Repeat = none

Instruction Forms:

SWPTA Rn(p), Rn(a)

long · 4 bytes · supervisor

Instruction Format:



Format — Instruction format for SWPTA Rn(p), Rn(a)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field a—Selects the destination operand.

Register field p—Selects the source operand.

SYNCCACHE

Synchronize Caches

SYNCCACHE

Operation:

Computes and translates the address-role EA, writes dirty data for its CPUID-sized maintenance block into normal-memory coherence throughout the coherence domain, invalidates the executing logical processor's matching instruction, decoded, and prefetch state, and serializes later instruction fetch.

Assembler Syntax:

SYNCCACHE <ea>(e)

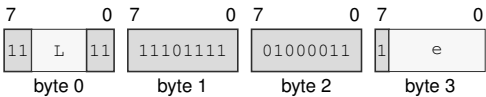
Attributes:

Class = system
Family = cache
Privilege = supervisor
Length = 4-14 bytes
Repeat = none

Instruction Forms:

SYNCCACHE <ea>(e)
long · 4-14 bytes · supervisor

Instruction Format:



Format — Instruction format for SYNCCACHE <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Effective Address field** e—Specifies the address operand.
- EA availability**
 - Allowed:** Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

SYSCALL**System Call****SYSCALL**

Operation:	Enter supervisor mode through the explicit SPC/SCS/SDS supervisor-call path. SYSCALL saves the user continuation in the UR control-register bank, switches to SSS:SSP, and does not create an event frame or set event-active state.
Assembler Syntax:	SYSCALL
Attributes:	Class = system Family = core control Privilege = any Length = 1 byte Repeat = none
Exceptions:	INVALID_CONTROL_STATE: the user-return bank is already valid or the configured entry context fails validation

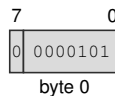
Detailed Semantics

Execution is valid only outside supervisor and event-active state and only when URCTL.V is clear. SCS, SDS, and SSS are validated as supervisor segment images; SPC must be an exact 16-byte-aligned executable address under SCS, and SSP an exact 64-byte-aligned stack top under SSS.

On success, the instruction atomically saves the user continuation in URPC, URSP, URCS, URDS, URSS, and URCTL, then loads CS, DS, SS, SP, and PC from the supervisor-call bank and sets STATUS.PM. It preserves IE, TF, RF, NI, and EA, performs no memory access, and creates no stack frame.

Instruction Forms:

SYSCALL
 extrashort · 1 byte · any

Instruction Format:**Format — Instruction format for SYSCALL**

SYSRET

System Return

SYSRET

Operation:	Return to user mode from the UR control-register bank. SYSRET is the matching return instruction for SYSCALL and is valid only outside architectural event processing.
Assembler Syntax:	SYSRET
Attributes:	Class = system Family = core control Privilege = supervisor Length = 1 byte Repeat = none
Exceptions:	INVALID_CONTROL_STATE: event-active state is set or the user-return bank fails validation

Detailed Semantics

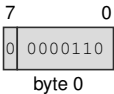
Execution requires supervisor mode outside event-active state and a valid URCTL bank. The instruction validates the saved FLAGS, STATUS, CS, DS, SS, SP, and executable PC before changing architectural state.

On success, FLAGS, STATUS, CS, DS, SS, SP, and PC are restored as one commit and URCTL.V is cleared. A validation failure leaves both the current state and the user-return bank unchanged.

Instruction Forms:

SYSRET
extrashort · 1 byte · supervisor

Instruction Format:



Format — Instruction format for SYSRET

TEST**Test****TEST**

Operation: Computes the selected-width bitwise conjunction only to set Z and N, clears C and V, and does not write the temporary result.

Assembler Syntax: `TEST.X(z:L/Q) Rn(s), Rn(d)`
`TEST.X(z:B/W) Rn(s), <ea>(e)`
`TEST.X(z:L/Q) Rn(s), <ea>(e)`
`TEST.X(z:B/W) <ea>(e), Rn(d)`
`TEST.X(z:L/Q) <ea>(e), Rn(d)`

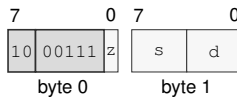
Attributes: Class = integer
Family = integer alu
Privilege = unprivileged
Length = 2-13 bytes
Repeat = REP, REPcc, REPG
REPcc observation = flags

Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	result == 0
N	most_significant_bit(result)
C	0
V	0

Instruction Forms:

`TEST.X(z:L/Q) Rn(s), Rn(d)`
short · 2 bytes · unprivileged

Instruction Format:

Format — Instruction format for `TEST.X(z:L/Q) Rn(s), Rn(d)`

Instruction Fields:

Size field *z*—Selects L/Q.

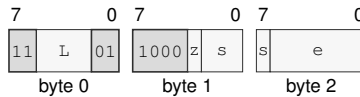
Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

TEST.X(z:B/W) Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for TEST.X(z:B/W) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field s—Selects the source operand.

Effective Address field e—Specifies the source operand.

EA availability

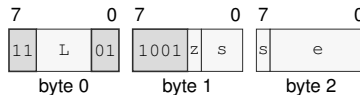
Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

TEST.X(z:L/Q) Rn(s), <ea>(e)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for TEST.X(z:L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the source operand.

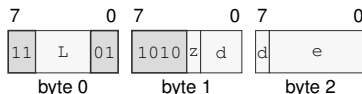
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

TEST.X(z:B/W) <ea>(e), Rn(d)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for TEST.X(z:B/W) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

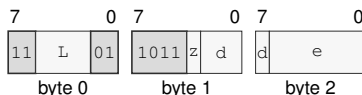
EA availability

Allowed: Register except Rn(r); Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

TEST.X(z:L/Q) <ea>(e), Rn(d)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for TEST.X(z:L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

TESTJcc

Test And Jump Conditional

TESTJcc

Operation: TESTJcc computes the same temporary logical-test result as TEST for the selected operand size, evaluates the encoded condition from that temporary result, and branches to the relative target when the condition is true. The computed condition is consumed only by the branch decision; architectural FLAGS are not read or written. Conditions T and F are reserved and are not valid encodings.

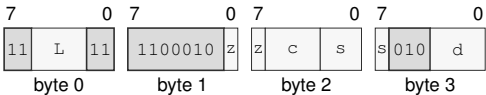
Assembler Syntax: TESTJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm8s>
TESTJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm16s>

Attributes: Class = control flow
Family = control flow
Privilege = unprivileged
Length = 5-6 bytes
Repeat = none

Instruction Forms:

TESTJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm8s>
long · 5 bytes · unprivileged

Instruction Format:



Format — Instruction format for TESTJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm8s>

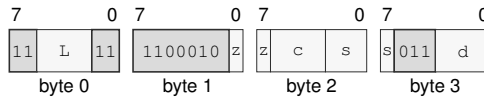
Instruction Fields:

- Length field** L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects B/W/L/Q.
- Condition field** c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.
- Register field** s—Selects the operand 1.
- Register field** d—Selects the operand 2.

TESTJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm16s>

long · 6 bytes · unprivileged

Instruction Format:



Format — Instruction format for TESTJcc.X(z:B/W/L/Q) Rn(s), Rn(d), <imm16s>

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W/L/Q.

Condition field c—Selects the condition code. Allowed values: EQ, NE, ULT, UGE, MI, PL, VS, VC, ULE, UGT, LT, GE, LE, GT.

Register field s—Selects the operand 1.

Register field d—Selects the operand 2.

TRACE

Trace Stream Marker

TRACE

Operation: When the implementation trace stream is enabled, appends the low 16-bit marker and current instruction boundary to that stream; otherwise it completes without changing architectural state. The tracing terminology distinguishes this marker instruction from a TF-selected trace unit and the `DEBUG_TRACE` architectural event.

Assembler Syntax: `TRACE <imm16>`

Attributes:

- Class = control flow
- Family = control flow
- Privilege = unprivileged
- Length = 5 bytes
- Repeat = none

Instruction Forms:

`TRACE <imm16>`
medium · 5 bytes · unprivileged

Instruction Format:



Format — Instruction format for `TRACE <imm16>`

Instruction Fields:

Length field `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

VTOP**Virtual To Physical Query****VTOP**

Operation: VTOP translates the supplied linear address with the current PTCR configuration without performing an architectural memory access. With paging disabled, the address is returned unchanged. With paging enabled, the instruction performs the configured LA48 or LA57 walk. Translation failures are reported through the integer flags instead of raising `PAGE_FAULT`. A successful query returns the byte address for an `AT=0` leaf or the slot address for an `AT=1` leaf and reports the accumulated U, W, and X permissions. The query issues no slot transaction and never sets PTE A or D, fills or modifies a TLB entry, or otherwise changes translation state.

Assembler Syntax: `VTOP Rn(v), Rn(p)`

Attributes:
 Class = system
 Family = tlb and context
 Privilege = supervisor
 Length = 4 bytes
 Repeat = none

Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	translation query succeeded
N	successful translation is user-domain accessible; cleared on failure
C	successful translation is writable; cleared on failure
V	successful translation is executable; cleared on failure

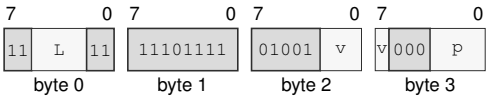
The source address is checked for canonicity before the page-table walk. The walk begins at the PTCR root and follows the active LA48 or LA57 hierarchy using the virtual page number.

Every nonleaf entry must be a valid table pointer. U, W, and X are accumulated across the walk. A valid L1 leaf supplies the physical frame number, which is combined with the original 12-bit page offset. Success sets Z and reports effective U, W, and X in N, C, and V. Any canonicity or walk failure returns zero and clears all four flags without raising `PAGE_FAULT`, setting PTE A or D, or modifying the TLB.

Instruction Forms:

VTOP Rn(v), Rn(p)
long · 4 bytes · supervisor

Instruction Format:



Format — Instruction format for VTOP Rn(v), Rn(p)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Register field** v—Selects the source operand.
- Register field** p—Selects the destination operand.

WAIT**Wait****WAIT**

Operation: Retires normally with the next instruction as its continuation and permits a finite implementation-selected delay, including zero, before subsequent execution.

Assembler Syntax: WAIT

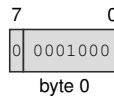
Attributes:
 Class = system
 Family = core control
 Privilege = unprivileged
 Length = 1 byte
 Repeat = none

Detailed Semantics

WAIT is a PAUSE-like execution hint. It retires as an ordinary instruction with the following instruction as its continuation. The implementation selects a finite delay before subsequent execution; the selected delay may be zero. The architectural result of WAIT is normal retirement to the recorded continuation.

Instruction Forms:

WAIT
 extrashort · 1 byte · unprivileged

Instruction Format:

Format — Instruction format for WAIT

WFENCE

Write Fence

WFENCE

Operation:

Prevents retirement until every earlier memory write is ordered before every later memory write on the same hardware thread. It performs no memory access and changes no register or flag.

Assembler Syntax:

WFENCE

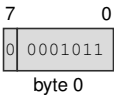
Attributes:

Class = system
Family = core control
Privilege = any
Length = 1 byte
Repeat = none

Instruction Forms:

WFENCE
extrashort · 1 byte · any

Instruction Format:



Format — Instruction format for WFENCE

WRBKDCACHE**Write Back Data Cache****WRBKDCACHE**

Operation: Computes and translates the address-role EA without reading an operand value, selects its CPUID-sized maintenance block, writes dirty bytes from every data or unified cache copy in the coherence domain into normal-memory coherence, retains clean copies, and retires after the block is complete.

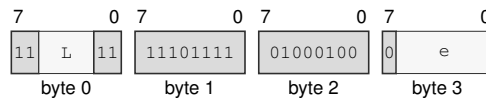
Assembler Syntax: WRBKDCACHE <ea>(e)

Attributes:
 Class = system
 Family = cache
 Privilege = supervisor
 Length = 4-14 bytes
 Repeat = none

Instruction Forms:

WRBKDCACHE <ea>(e)

long · 4-14 bytes · supervisor

Instruction Format:

Format — Instruction format for WRBKDCACHE <ea>(e)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Effective Address field *e*—Specifies the address operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

WRCR

Write Control Register

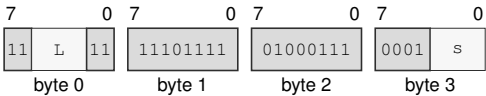
WRCR

Operation:	WRCR validates and atomically writes the selected control register in supervisor mode. The WRCR Selector Rules table defines each writable image, validation condition, and translation or event-state side effect.
Assembler Syntax:	WRCR Rn(s), <imm16>
Attributes:	Class = system Family = system registers Privilege = supervisor Length = 6 bytes Repeat = none
Exceptions:	INVALID_CONTROL_STATE: the selector, reserved bits, image, or requested transition fails the WRCR selector rule

Instruction Forms:

WRCR Rn(s), <imm16>
long · 6 bytes · supervisor

Instruction Format:



Format — Instruction format for WRCR Rn(s), <imm16>

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Register field** s—Selects the source operand.

WRFLAGS

Write Flags

WRFLAGS

Operation: WRFLAGS writes the low 16 bits from the source Rn register into the architectural FLAGS register. Reserved FLAGS bits must be zero.

Assembler Syntax: WRFLAGS Rn(s)

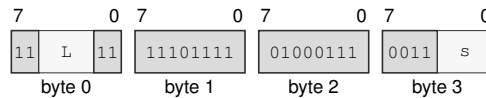
Attributes:
 Class = system
 Family = system registers
 Privilege = unprivileged
 Length = 4 bytes
 Repeat = none

Instruction Forms:

WRFLAGS Rn(s)

long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for WRFLAGS Rn(s)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s—Selects the operand.

WRSEG

Write Segment Register

WRSEG

Operation:	WRSEG validates a 64-bit segment register image from the source Rn register and writes it to the selected SREG.
Assembler Syntax:	WRSEG Rn(d) , SREG(s)
Attributes:	Class = system Family = system registers Privilege = unprivileged Length = 4 bytes Repeat = none

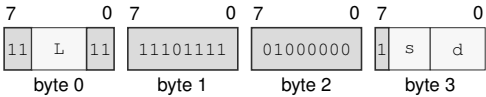
Detailed Semantics

The complete source image is validated before architectural state changes. An invalid image raises INVALID_CONTROL_STATE; the selected segment register remains unchanged when validation fails.

Instruction Forms:

WRSEG Rn(d) , SREG(s)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for WRSEG Rn(d), SREG(s)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Bits field** s—Encodes the bits value.
- Register field** d—Selects the source operand.

WRSTATUS

Write Status

WRSTATUS

Operation: WRSTATUS validates the low 16 bits from the source Rn register and atomically updates IE, NI, TF, and RF. Reserved bits must be zero, and the supplied PM and EA values must equal the current values. Clearing NI with a pending NMI admits that NMI at the next instruction boundary.

Assembler Syntax: WRSTATUS Rn(s)

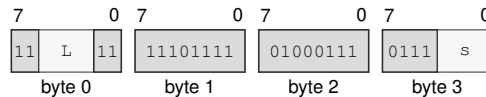
Attributes:
 Class = system
 Family = system registers
 Privilege = supervisor
 Length = 4 bytes
 Repeat = none

Exceptions: INVALID_CONTROL_STATE: reserved bits are nonzero or the supplied PM or EA value differs from the current value

Instruction Forms:

WRSTATUS Rn(s)
 long · 4 bytes · supervisor

Instruction Format:



Format — Instruction format for WRSTATUS Rn(s)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s—Selects the operand.

XCHG

Exchange

XCHG

Operation: Exchanges the two selected-width operand values as one instruction. A memory form performs an ordinary selected-width load followed by an ordinary selected-width store, with both architectural destinations committed by the instruction.

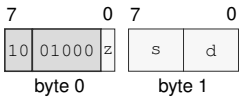
Assembler Syntax: XCHG.X(z:L/Q) Rn(s), Rn(d)
XCHG.X(z:B/W) Rn(s), <ea>(e)
XCHG.X(z:L/Q) Rn(s), <ea>(e)
XCHG.X(z:B/W) <ea>(e), Rn(d)
XCHG.X(z:L/Q) <ea>(e), Rn(d)

Attributes: Class = data movement
Family = data movement
Privilege = unprivileged
Length = 2-13 bytes
Repeat = REP, REPG

Instruction Forms:

XCHG.X(z:L/Q) Rn(s), Rn(d)
short · 2 bytes · unprivileged

Instruction Format:



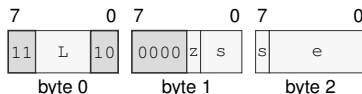
Format — Instruction format for XCHG.X(z:L/Q) Rn(s), Rn(d)

Instruction Fields:

- Size field** z—Selects L/Q.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.
- Destination overlap**—When lhs = rhs designate the same architectural register, the final value equals that register's initial value.

XCHG.X(z:B/W) Rn(s), <ea>(e)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for XCHG.X(z:B/W) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

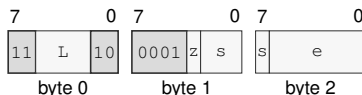
Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Destination overlap—When lhs = rhs designate the same architectural register, the final value equals that register's initial value.

XCHG.X(z:L/Q) Rn(s), <ea>(e)
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for XCHG.X(z:L/Q) Rn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field s—Selects the source operand.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Memory; EXT0

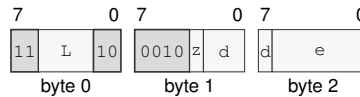
Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Destination overlap—When lhs = rhs designate the same architectural register, the final value equals that register's initial value.

XCHG.X(z:B/W) <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for XCHG.X(z:B/W) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects B/W.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Register except Rn(r); Memory; EXT0

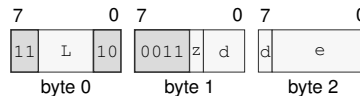
Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Destination overlap—When lhs = rhs designate the same architectural register, the final value equals that register's initial value.

XCHG.X(z:L/Q) <ea>(e), Rn(d)

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for XCHG.X(z:L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects L/Q.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the read/write operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Destination overlap—When lhs = rhs designate the same architectural register, the final value equals that register's initial value.

XOR

Bitwise Xor

XOR

Operation: Writes the selected-width bitwise exclusive-or of the source and destination values to the destination.

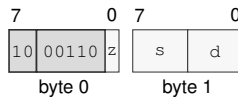
Assembler Syntax: `XOR.X(z:L/Q) Rn(s), Rn(d)`
`XOR.X(z:B/W) Rn(s), <ea>(e)`
`XOR.X(z:L/Q) Rn(s), <ea>(e)`
`XOR.X(z:B/W) <ea>(e), Rn(d)`
`XOR.X(z:L/Q) <ea>(e), Rn(d)`

Attributes: Class = integer
Family = integer alu
Privilege = unprivileged
Length = 2-13 bytes
Repeat = REP, REPcc, REPG
REPcc observation = result dst

Instruction Forms:

`XOR.X(z:L/Q) Rn(s), Rn(d)`
short · 2 bytes · unprivileged

Instruction Format:



Format — Instruction format for XOR.X(z:L/Q) Rn(s), Rn(d)

Instruction Fields:

Size field *z*—Selects L/Q.

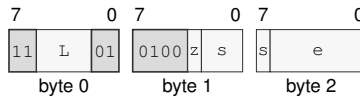
Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

$\text{XOR.X}(z:B/W) \text{ Rn}(s), \langle ea \rangle(e)$

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for $\text{XOR.X}(z:B/W) \text{ Rn}(s), \langle ea \rangle(e)$

Instruction Fields:

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z —Selects B/W.

Register field s —Selects the source operand.

Effective Address field e —Specifies the read/write operand.

EA availability

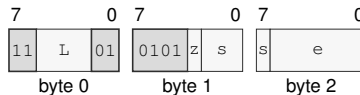
Allowed: Register; Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

$\text{XOR.X}(z:L/Q) \text{ Rn}(s), \langle ea \rangle(e)$

medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for $\text{XOR.X}(z:L/Q) \text{ Rn}(s), \langle ea \rangle(e)$

Instruction Fields:

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z —Selects L/Q.

Register field s —Selects the source operand.

Effective Address field e —Specifies the read/write operand.

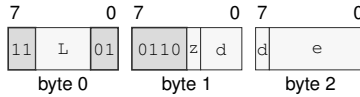
EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`XOR.X(z:B/W) <ea>(e), Rn(d)`
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for XOR.X(z:B/W) <ea>(e), Rn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects B/W.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

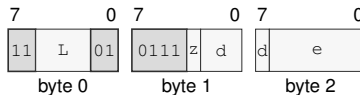
EA availability

Allowed: Register except $R_n(r)$; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`XOR.X(z:L/Q) <ea>(e), Rn(d)`
 medium · 3-13 bytes · unprivileged

Instruction Format:



Format — Instruction format for XOR.X(z:L/Q) <ea>(e), Rn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects L/Q.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

YIELD

Yield

YIELD

Operation:

Notifies the implementation that the current hardware thread may be deprioritized or descheduled. YIELD affects scheduling only; memory accesses retain the ordinary instruction-ordering rules, and execution resumes at the next instruction.

Assembler Syntax:

YIELD

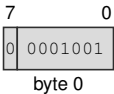
Attributes:

Class = system
Family = core control
Privilege = any
Length = 1 byte
Repeat = none

Instruction Forms:

YIELD
extrashort · 1 byte · any

Instruction Format:



Format — Instruction format for YIELD

18 FLOATING-POINT INSTRUCTIONS

18.1 Common Floating-Point Semantics

The S and D formats are IEEE-754 binary32 and binary64. S uses sign bit 31, exponent bits 30..23, and fraction bits 22..0. D uses sign bit 63, exponent bits 62..52, and fraction bits 51..0. An all-ones exponent and nonzero fraction is a NaN. The most significant fraction bit distinguishes a quiet NaN from a signaling NaN: zero is signaling and one is quiet. The architectural default quiet NaNs are 0x7fc00000 for S and 0x7ff8000000000000 for D. Figure 5-4 shows both encodings in their 64-bit Fn register view.

An S operation reads bits 31..0 of each Fn operand. Writing an S result to Fn writes the result to bits 31..0 and writes zero to bits 63..32. A D operation reads and writes all 64 bits. Operations use the selected format for their intermediate and final values. FMADD, FMSUB, FNMADD, and FNMSUB form the complete multiply-add expression with unbounded intermediate range and precision and round only the final result. Other arithmetic operations round once after computing their exact mathematical result.

18.1.1 Input, NaN, Rounding, and Result Order

Each ordinary floating-point operation applies the following order.

1. Read all operands and classify zero, subnormal, normal, infinity, quiet NaN, and signaling NaN.
2. If `FSTATUS.DAZ` is set, replace each subnormal input with a zero having the same sign.
3. Determine signaling-NaN and operation-specific exception causes and form the exact mathematical result.
4. For a NaN result, select the first signaling NaN in source-operand order, or the first quiet NaN when there is no signaling NaN, and set its quiet bit. If the operation produces a NaN without a NaN operand, use the default quiet NaN. When `FSTATUS.DN` is set, replace the selected NaN with the default quiet NaN.
5. Round a numeric result to the selected format using `FSTATUS.RM`, except where an instruction names a fixed rounding direction.
6. Detect overflow and tininess after rounding. Underflow accrues when the rounded result is tiny and inexact.
7. If `FSTATUS.FTZ` is set and the rounded result is a nonzero subnormal, replace it with a zero of the same sign and generate both UF and NX.
8. Test all generated causes against their `FSTATUS` enable bits and then perform the architectural commit.

A signaling NaN generates NV. A quiet NaN does not generate NV unless the operation itself is invalid. When two NaN operands have the same priority, source-operand order determines the

selected sign and payload. Quieting sets the quiet bit and preserves the remaining selected sign and payload bits.

Overflow generates OF and NX. Nearest-even produces signed infinity. Toward zero produces the signed maximum finite value. Toward positive produces positive infinity for a positive result and the negative maximum finite value for a negative result. Toward negative produces negative infinity for a negative result and the positive maximum finite value for a positive result.

18.1.2 Exception Causes and Commit

Cause	Generated when
NV	an input is a signaling NaN; an arithmetic operation has no defined numeric result, including infinity minus the same infinity, zero times infinity, zero divided by zero, infinity divided by infinity, or square root of a negative nonzero value; or a floating-to-integer conversion is invalid
DZ	a finite nonzero value is divided by zero
OF	the rounded numeric magnitude exceeds the selected format's finite range
UF	the result is tiny after rounding and inexact, or FTZ flushes a nonzero subnormal result
NX	rounding or a valid conversion changes the exact value, or OF or FTZ generates NX

If any generated cause has its matching FSTATUS enable bit set, the instruction raises `FLOATING_POINT_FAULT`. Its error-code cause bitmap contains every cause generated by the operation. The destination, integer FLAGS, and FFLAGS remain unchanged, and arithmetic instructions never modify FSTATUS.

If no generated cause is enabled, the instruction commits its destination and integer FLAGS effects and ORs every generated cause into FFLAGS. Fields not named by the instruction remain unchanged. An instruction-specific rule may exclude a cause, as the FPTRANSA contracts exclude NX.

18.1.3 Comparison, Minimum, and Maximum

FCMP and FTEST write Z, N, C, V as follows.

Relation	Z	N	C	V
greater	0	0	0	0
equal	1	0	0	0
less	0	1	1	0
unordered	0	0	0	1

A quiet NaN produces unordered without NV. A signaling NaN produces unordered and NV. FTEST compares its operand with positive zero under these rules.

For FMIN and FMAX, when exactly one operand is a NaN the numeric operand is the result. When both operands are NaNs, the common NaN-selection rule supplies the result. A signaling NaN generates NV even when the other operand is numeric. Between positive and negative zero, FMIN selects negative zero and FMAX selects positive zero.

18.1.4 Conversions

For Fn-to-Fn FCVT or FCVTU, the `.S` form converts D to S and the `.D` form converts S to D. The two mnemonics have identical behavior for this conversion family. Rn-to-Fn FCVT interprets all 64 source bits as a signed integer; FCVTU interprets them as an unsigned integer. Integer-to-floating conversion uses `FSTATUS.RM`.

Fn-to-Rn conversion truncates toward zero. FCVT produces a signed 64-bit result and FCVTU produces an unsigned 64-bit result. A valid conversion that discards a fractional part generates NX. An invalid conversion generates NV without OF or NX. When NV is not enabled, the committed invalid result is:

Instruction	Input	Result
FCVT	negative overflow or negative infinity	0x8000000000000000
FCVT	positive overflow or positive infinity	0x7fffffffffffffff
FCVT	NaN	0x0000000000000000
FCVTU	negative value or negative infinity	0x0000000000000000
FCVTU	positive overflow or positive infinity	0xfffffffffffffff
FCVTU	NaN	0x0000000000000000

18.2 Summary

Table 18-1. Floating-Point Instructions Summary

Mnemonic	Brief description
FABS	Performs the floating-point absolute value operation.
FADD	Adds the source operand to the destination operand.
FBNDII	Checks floating-point value membership in the inclusive-inclusive interval [lo, hi].
FBNDIX	Checks floating-point value membership in the inclusive-exclusive interval [lo, hi).
FBNDXI	Checks floating-point value membership in the exclusive-inclusive interval (lo, hi].
FBNDXX	Checks floating-point value membership in the exclusive-exclusive interval (lo, hi).
FCEIL	Performs the floating-point ceiling operation.
FCLASS	Performs the floating-point classify operation.
FCLR	Performs the floating-point clear operation.
FCMP	Performs the floating-point compare operation.
FCOPYSIGN	Performs the floating-point copy sign operation.
FCVT	Performs the floating-point convert signed operation.
FCVTU	Performs the floating-point convert unsigned operation.
FDIV	Performs the floating-point divide operation.
FFLOOR	Performs the floating-point floor operation.
FGETEXP	Performs the floating-point get exponent operation.
FGETMAN	Performs the floating-point get mantissa operation.
FINT	Performs the floating-point integral value operation.
FINTRZ	Performs the floating-point integral toward zero operation.
FMADD	Performs the floating-point fused multiply add operation.
FMAX	Performs the floating-point maximum operation.
FMIN	Performs the floating-point minimum operation.
FMOD	Performs the floating-point modulo operation.

Table 18-1 (continued)

Mnemonic	Brief description
FMOV	Copies the source operand to the destination operand.
FMOVCR	Loads a named architectural binary64 constant selected by an unsigned 16-bit ID.
FMOVcc	Performs the floating-point conditional move operation.
FPOPP	Pops one canonical floating-point register pair selected by an inline pair index.
FPUSHP	Pushes one canonical floating-point register pair selected by an inline pair index.
FMSUB	Performs the floating-point fused multiply subtract operation.
FMUL	Performs the floating-point multiply operation.
FNEG	Performs the floating-point negate operation.
FMADD	Performs the floating-point fused negative multiply add operation.
FNMSUB	Performs the floating-point fused negative multiply subtract operation.
FREM	Performs the floating-point remainder operation.
FROUND	Performs the floating-point round operation.
FSCALE	Performs the floating-point scale operation.
FSQRT	Performs the floating-point square root operation.
FSUB	Subtracts the source operand from the destination operand.
FTEST	Performs the floating-point test operation.
FTRUNC	Performs the floating-point truncate operation.
FXCHG	Performs the floating-point exchange operation.
RDFFLAGS	Read the accrued floating-point exception flags into a register.
RDFSTATUS	Read the FPU status/control register into an Rn register.
WRFFLAGS	Write the accrued floating-point exception flags from a register.
WFSTATUS	Write the FPU status/control register from an Rn register.

FABS**Floating-Point Absolute Value****FABS**

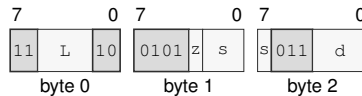
Operation: Clears the sign of the selected floating-point source value and writes the resulting magnitude to the destination.

Assembler Syntax: `FABS.X(z:S/D) Fn(s), Fn(d)`
`FABS.X(z:S/D) <ea>(e), Fn(d)`
`FABS.X(z:S/D) Fn(s), <ea>(e)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 3-14 bytes
Feature = FP
Repeat = REP, REPG

Instruction Forms:

`FABS.X(z:S/D) Fn(s), Fn(d)`
medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for FABS.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

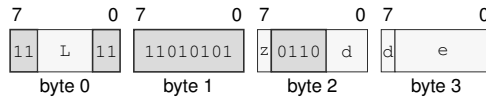
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FABS.X(z:S/D) <ea>(e), Fn(d)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FABS.X(z:S/D) <ea>(e), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

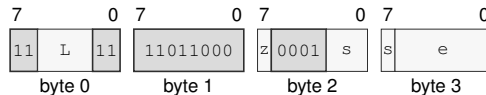
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FABS.X(z:S/D) Fn(s), <ea>(e)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FABS.X(z:S/D) Fn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FADD**Floating-Point Add****FADD**

Operation: Adds the selected-format floating-point source and destination values and writes the rounded result under the architectural floating-point environment.

Assembler Syntax: `FADD.X(z:S/D) Fn(s), Fn(d)`
`FADD.X(z:S/D) <ea>(e), Fn(d)`

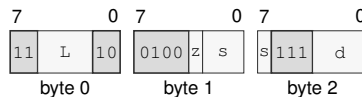
Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 3-14 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	may accrue

Instruction Forms:

`FADD.X(z:S/D) Fn(s), Fn(d)`
medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for `FADD.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

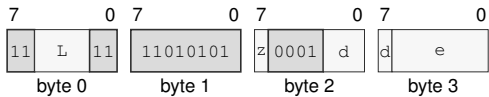
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FADD.X(z:S/D) <ea>(e), Fn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FADD.X(z:S/D) <ea>(e), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.
- EA availability**
 - Allowed:** Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

FBNDII**Floating-Point Bounds Check Inclusive-Inclusive****FBNDII**

Operation: FBNDII treats both bounds as inclusive. It sets V when value is unordered or outside [lo, hi]; Z, N, and C are unchanged.

Assembler Syntax: FBNDII.X(z:S/D) Fn(l), Fn(v), Fn(h)
 FBNDII.X(z:S/D) Fn(l), <ea>(v), Fn(h)
 FBNDII.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

Attributes: Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 4-18 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	set when the value is out of bounds or unordered

FFLAGS Effects

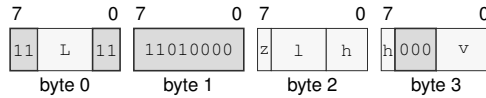
Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

Instruction Forms:

FBNDII.X(z:S/D) Fn(l), Fn(v), Fn(h)

long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FBNDII.X(z:S/D) Fn(l), Fn(v), Fn(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects the operand 1.

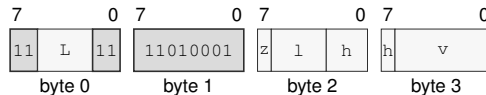
Register field h—Selects the operand 3.

Register field v—Selects the operand 2.

FBNDII.X(z:S/D) Fn(l), <ea>(v), Fn(h)

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FBNDII.X(z:S/D) Fn(l), <ea>(v), Fn(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects the operand 1.

Register field h—Selects the operand 3.

Effective Address field v—Specifies the source operand.

EA availability

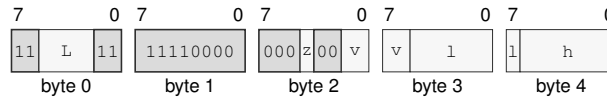
Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FBNDII.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

extralong · 5-18 bytes · unprivileged

Instruction Format:



Format — Instruction format for FBNDII.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *v*—Selects the operand 2.

Effective Address field *l*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field *h*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FBNDIX

Floating-Point Bounds Check Inclusive-Exclusive

FBNDIX

Operation:

FBNDIX treats the low bound as inclusive and the high bound as exclusive. It sets V when value is unordered or outside [lo, hi); Z, N, and C are unchanged.

Assembler Syntax:

FBNDIX.X(z:S/D) Fn(l), Fn(v), Fn(h)
FBNDIX.X(z:S/D) Fn(l), <ea>(v), Fn(h)
FBNDIX.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

Attributes:

Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4-18 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics

FLAGS Effects

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	set when the value is out of bounds or unordered

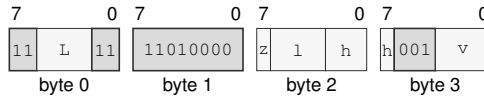
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

Instruction Forms:

FBNDIX.X(z:S/D) Fn(l), Fn(v), Fn(h)

long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for FBNDIX.X(z:S/D) Fn(l), Fn(v), Fn(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

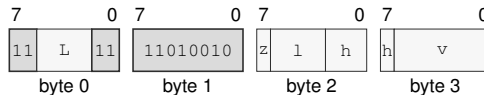
Register field l—Selects the operand 1.

Register field h—Selects the operand 3.

Register field v—Selects the operand 2.

FBNDIX.X(z:S/D) Fn(l), <ea>(v), Fn(h)

long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for FBNDIX.X(z:S/D) Fn(l), <ea>(v), Fn(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects the operand 1.

Register field h—Selects the operand 3.

Effective Address field v—Specifies the source operand.

EA availability

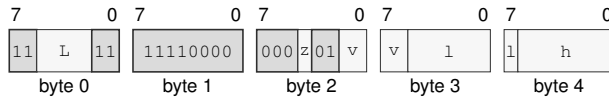
Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FBNDIX.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

extralong · 5-18 bytes · unprivileged

Instruction Format:



Format — Instruction format for FBNDIX.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *v*—Selects the operand 2.

Effective Address field *l*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field *h*—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FBNDXI**Floating-Point Bounds Check Exclusive-Inclusive****FBNDXI**

Operation: FBNDXI treats the low bound as exclusive and the high bound as inclusive. It sets V when value is unordered or outside (lo, hi]; Z, N, and C are unchanged.

Assembler Syntax: FBNDXI.X(z:S/D) Fn(l), Fn(v), Fn(h)
 FBNDXI.X(z:S/D) Fn(l), <ea>(v), Fn(h)
 FBNDXI.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

Attributes: Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 4-18 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	set when the value is out of bounds or unordered

FFLAGS Effects

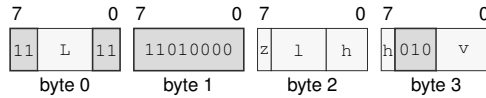
Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

Instruction Forms:

FBNDXI.X(z:S/D) Fn(l), Fn(v), Fn(h)

long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FBNDXI.X(z:S/D) Fn(l), Fn(v), Fn(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects the operand 1.

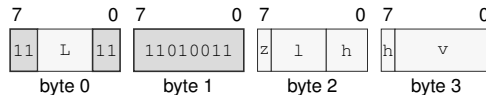
Register field h—Selects the operand 3.

Register field v—Selects the operand 2.

FBNDXI.X(z:S/D) Fn(l), <ea>(v), Fn(h)

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FBNDXI.X(z:S/D) Fn(l), <ea>(v), Fn(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects the operand 1.

Register field h—Selects the operand 3.

Effective Address field v—Specifies the source operand.

EA availability

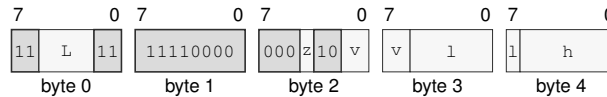
Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FBNDXI.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

extralong · 5-18 bytes · unprivileged

Instruction Format:



Format — Instruction format for FBNDXI.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field v—Selects the operand 2.

Effective Address field 1—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field h—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FBNDXX

Floating-Point Bounds Check Exclusive-Exclusive

FBNDXX

Operation: FBNDXX treats both bounds as exclusive. It sets V when value is unordered or outside (lo, hi); Z, N, and C are unchanged.

Assembler Syntax: FBNDXX.X(z:S/D) Fn(l), Fn(v), Fn(h)
FBNDXX.X(z:S/D) Fn(l), <ea>(v), Fn(h)
FBNDXX.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4-18 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics

FLAGS Effects

Flag	Effect
Z	preserved
N	preserved
C	preserved
V	set when the value is out of bounds or unordered

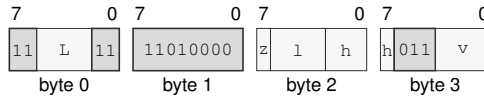
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

Instruction Forms:

FBNDXX.X(z:S/D) Fn(l), Fn(v), Fn(h)

long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for FBNDXX.X(z:S/D) Fn(l), Fn(v), Fn(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

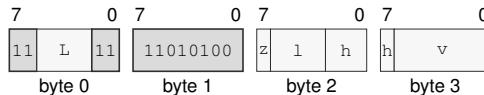
Register field l—Selects the operand 1.

Register field h—Selects the operand 3.

Register field v—Selects the operand 2.

FBNDXX.X(z:S/D) Fn(l), <ea>(v), Fn(h)

long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for FBNDXX.X(z:S/D) Fn(l), <ea>(v), Fn(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects the operand 1.

Register field h—Selects the operand 3.

Effective Address field v—Specifies the source operand.

EA availability

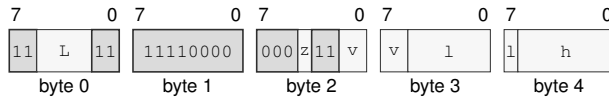
Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FBNDXX.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

extralong · 5-18 bytes · unprivileged

Instruction Format:



Format — Instruction format for FBNDXX.X(z:S/D) <ea>(l), Fn(v), <ea>(h)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field v—Selects the operand 2.

Effective Address field l—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

Effective Address field h—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FCEIL**Floating-Point Ceiling****FCEIL**

Operation: Rounds the selected floating-point source toward positive infinity to an integral floating-point value.

Assembler Syntax: `FCEIL.X(z:S/D) Fn(s), Fn(d)`
`FCEIL.X(z:S/D) <ea>(e), Fn(d)`
`FCEIL.X(z:S/D) Fn(s), <ea>(e)`

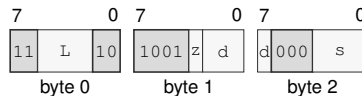
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 3-14 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	may accrue

Instruction Forms:

`FCEIL.X(z:S/D) Fn(s), Fn(d)`
 medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for FCEIL.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

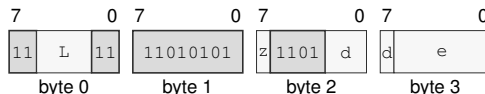
Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Register field *s*—Selects the source operand.

FCEIL.X(z:S/D) <ea>(e), Fn(d)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FCEIL.X(z:S/D) <ea>(e), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

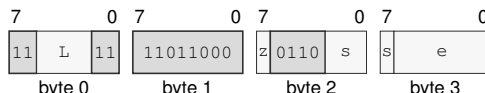
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FCEIL.X(z:S/D) Fn(s), <ea>(e)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FCEIL.X(z:S/D) Fn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FCLASS**Floating-Point Classify****FCLASS**

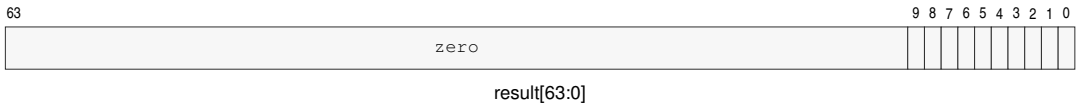
Operation: Examines the selected floating-point encoding without raising a floating-point exception and writes a one-hot class bitmap to the integer destination.

Assembler Syntax: `FCLASS.X(z:S/D) Fn(s), Rn(d)`

Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics

The destination is a one-hot bitmap selected directly from the sign, exponent, fraction, and NaN quiet bit of the S or D source encoding.

**FCLASS Result Bitmap**

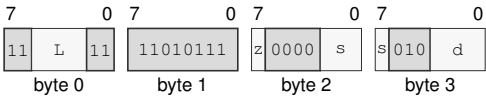
Bit	Class
0	negative infinity
1	negative normal
2	negative subnormal
3	negative zero
4	positive zero
5	positive subnormal
6	positive normal
7	positive infinity
8	signaling NaN
9	quiet NaN

Bits 63..10 of the integer destination are zero. No floating-point exception flag is changed.

Instruction Forms:

FCLASS.X(z:S/D) Fn(s), Rn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FCLASS.X(z:S/D) Fn(s), Rn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

FCLR**Floating-Point Clear****FCLR**

Operation: Writes positive floating-point zero in the selected format to the destination.

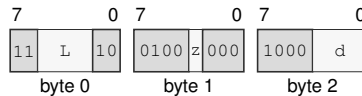
Assembler Syntax: FCLR Fn(d)

Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 3 bytes
 Feature = FP
 Repeat = REP, REPG

Instruction Forms:

FCLR Fn(d)

medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for FCLR Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects the operand size.

Register field d—Selects the operand.

FCMP**Floating-Point Compare****FCMP**

Operation: Compares the selected-format operands and writes Z,N,C,V as greater=0000, equal=1000, less=0110, or unordered=0001. A quiet NaN is unordered; a signaling NaN is unordered and generates NV.

Assembler Syntax: FCMP.X(z:S/D) Fn(s), Fn(d)
FCMP.X(z:S/D) <ea>(e), Fn(d)

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 3-14 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	set only when equal
N	set only when less
C	set only when less
V	set when unordered

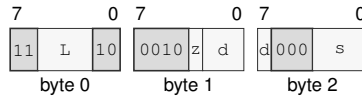
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

Instruction Forms:

FCMP.X(z:S/D) Fn(s), Fn(d)

medium · 3 bytes · unprivileged

Instruction Format:**Format — Instruction format for FCMP.X(z:S/D) Fn(s), Fn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

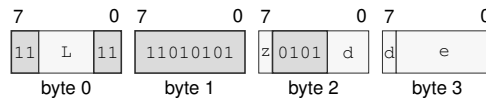
Size field z—Selects S/D.

Register field d—Selects the destination operand.

Register field s—Selects the source operand.

FCMP.X(z:S/D) <ea>(e), Fn(d)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for FCMP.X(z:S/D) <ea>(e), Fn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FCOPYSIGN

Floating-Point Copy Sign

FCOPYSIGN

Operation:

Combines the magnitude bits of the destination with the sign bit of the source and writes the selected-format result.

Assembler Syntax:

FCOPYSIGN.X(z:S/D) Fn(s), Fn(m), Fn(d)

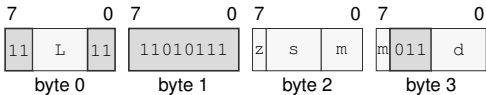
Attributes:

Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4 bytes
Feature = FP
Repeat = REP, REPG

Instruction Forms:

FCOPYSIGN.X(z:S/D) Fn(s), Fn(m), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FCOPYSIGN.X(z:S/D) Fn(s), Fn(m), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** s—Selects the operand 1.
- Register field** m—Selects the operand 2.
- Register field** d—Selects the operand 3.

FCVT**Floating-Point Convert Signed****FCVT**

Operation: Converts between supported signed integer and floating-point forms, or between the supported floating-point widths, according to the operand classes.

Assembler Syntax: `FCVT.X(z:S/D) Fn(s), Fn(d)`
`FCVT.X(z:S/D) Fn(s), Rn(d)`
`FCVT.X(z:S/D) Rn(s), Fn(d)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	may accrue

The operand classes select one of three conversion families.

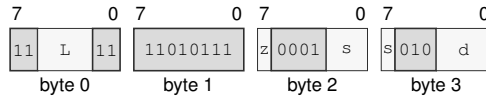
- Fn to Fn .S converts D to S, while .D converts S to D.
- Fn to Rn truncates toward zero and writes a signed 64-bit integer. Negative overflow and negative infinity produce 0x8000000000000000; positive overflow and positive infinity produce 0x7fffffffffffffff; NaN produces zero.
- Rn to Fn converts the full signed 64-bit source to the selected floating-point format using FSTATUS.RM.

An invalid Fn-to-Rn conversion generates NV without OF or NX. A valid conversion that discards a fractional part generates NX. Fn-to-Fn and Rn-to-Fn conversion use the common floating-point rounding, exception, and commit rules.

Instruction Forms:

FCVT.X(z:S/D) Fn(s), Fn(d)

long · 4 bytes · unprivileged

Instruction Format:**Format — Instruction format for FCVT.X(z:S/D) Fn(s), Fn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

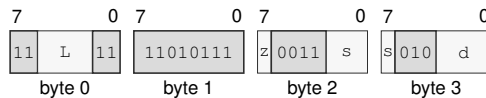
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FCVT.X(z:S/D) Fn(s), Rn(d)

long · 4 bytes · unprivileged

Instruction Format:**Format — Instruction format for FCVT.X(z:S/D) Fn(s), Rn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

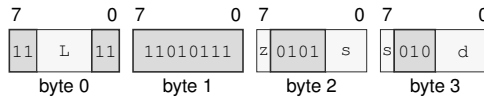
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FCVT.X(z:S/D) Rn(s), Fn(d)
 long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FCVT.X(z:S/D) Rn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FCVTU

Floating-Point Convert Unsigned

FCVTU

Operation:

Converts between supported unsigned integer and floating-point forms, or between the supported floating-point widths, according to the operand classes.

Assembler Syntax:

FCVTU.X(z:S/D) Fn(s), Fn(d)
FCVTU.X(z:S/D) Fn(s), Rn(d)
FCVTU.X(z:S/D) Rn(s), Fn(d)

Attributes:

Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics

FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	may accrue

The operand classes select one of three conversion families.

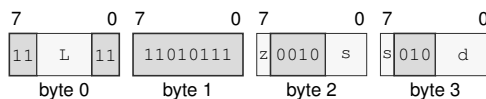
- Fn to Fn .S converts D to S, while .D converts S to D.
- Fn to Rn truncates toward zero and writes an unsigned 64-bit integer. A negative value or negative infinity produces zero; positive overflow or positive infinity produces 0xffffffffffffffff; NaN produces zero.
- Rn to Fn converts the full unsigned 64-bit source range to the selected floating-point format using FSTATUS.RM.

An invalid Fn-to-Rn conversion generates NV without OF or NX. A valid conversion that discards a fractional part generates NX. Fn-to-Fn and Rn-to-Fn conversion use the common floating-point rounding, exception, and commit rules.

Instruction Forms:

FCVTU.X(z:S/D) Fn(s), Fn(d)

long · 4 bytes · unprivileged

Instruction Format:**Format — Instruction format for FCVTU.X(z:S/D) Fn(s), Fn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

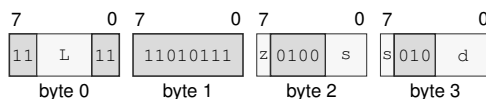
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FCVTU.X(z:S/D) Fn(s), Rn(d)

long · 4 bytes · unprivileged

Instruction Format:**Format — Instruction format for FCVTU.X(z:S/D) Fn(s), Rn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

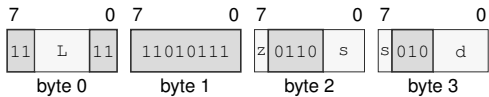
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FCVTU.X(z:S/D) Rn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FCVTU.X(z:S/D) Rn(s), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** s—Selects the source operand.
- Register field** d—Selects the destination operand.

FDIV**Floating-Point Divide****FDIV**

Operation: Divides the selected-format floating-point destination value by the source and writes the rounded result under the architectural floating-point environment.

Assembler Syntax: `FDIV.X(z:S/D) Fn(s), Fn(d)`
`FDIV.X(z:S/D) <ea>(e), Fn(d)`

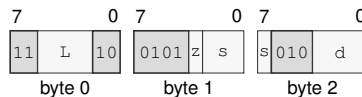
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 3-14 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	may accrue
UF	may accrue
NX	may accrue

Instruction Forms:

`FDIV.X(z:S/D) Fn(s), Fn(d)`
 medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for `FDIV.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

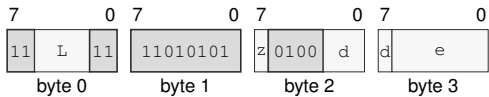
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FDIV.X(z:S/D) <ea>(e), Fn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FDIV.X(z:S/D) <ea>(e), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.
- EA availability**
 - Allowed:** Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

FFLOOR**Floating-Point Floor****FFLOOR**

Operation: Rounds the selected floating-point source toward negative infinity to an integral floating-point value.

Assembler Syntax: `FFLOOR.X(z:S/D) Fn(s), Fn(d)`
`FFLOOR.X(z:S/D) <ea>(e), Fn(d)`
`FFLOOR.X(z:S/D) Fn(s), <ea>(e)`

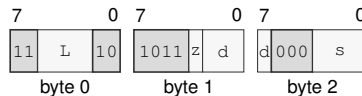
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 3-14 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	may accrue

Instruction Forms:

`FFLOOR.X(z:S/D) Fn(s), Fn(d)`
 medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for FFLOOR.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

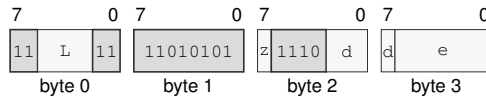
Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Register field *s*—Selects the source operand.

FFLOOR.X(z:S/D) <ea>(e), Fn(d)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FFLOOR.X(z:S/D) <ea>(e), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

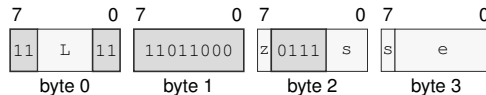
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FFLOOR.X(z:S/D) Fn(s), <ea>(e)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FFLOOR.X(z:S/D) Fn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FGETEXP**Floating-Point Get Exponent****FGETEXP**

Operation: Extracts the unbiased exponent classification of the selected floating-point source and writes the specified floating-point result for finite and special values.

Assembler Syntax: `FGETEXP.X(z:S/D) <ea>(e), Fn(d)`
`FGETEXP.X(z:S/D) Fn(s), Fn(d)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4-14 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

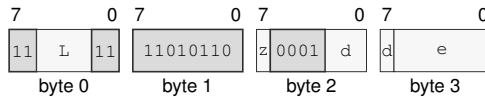
Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

For S format, exponent field zero is treated as biased exponent one and the bias 127 is subtracted. For D format, exponent field zero is likewise treated as one and bias 1023 is subtracted. The resulting signed integer exponent is converted to the selected floating-point destination format. This rule gives zero and subnormal inputs the minimum normal exponent and leaves infinity and NaN encodings at the exponent produced by their all-ones field.

Instruction Forms:

FGETEXP.X(z:S/D) <ea>(e), Fn(d)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for FGETEXP.X(z:S/D) <ea>(e), Fn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

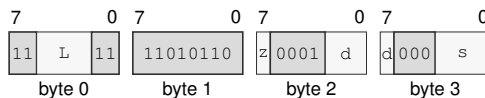
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FGETEXP.X(z:S/D) Fn(s), Fn(d)

long · 4 bytes · unprivileged

Instruction Format:**Format — Instruction format for FGETEXP.X(z:S/D) Fn(s), Fn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Register field s—Selects the source operand.

FGETMAN**Floating-Point Get Mantissa****FGETMAN**

Operation: Extracts the normalized significand of the selected floating-point source and writes the specified floating-point result for finite and special values.

Assembler Syntax: `FGETMAN.X(z:S/D) <ea>(e), Fn(d)`
`FGETMAN.X(z:S/D) Fn(s), Fn(d)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4-14 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

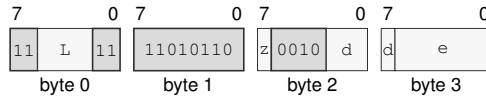
Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	may accrue

For a normal finite source, the result preserves the sign and fraction and replaces the exponent with the bias, producing a magnitude in $[1, 2)$. A zero, subnormal, infinity, or NaN source retains its source encoding, subject to the architectural signaling-NaN and default-NaN rules. The S and D forms use their native exponent and fraction widths.

Instruction Forms:

FGETMAN.X(z:S/D) <ea>(e), Fn(d)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for FGETMAN.X(z:S/D) <ea>(e), Fn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

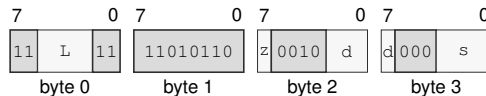
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FGETMAN.X(z:S/D) Fn(s), Fn(d)

long · 4 bytes · unprivileged

Instruction Format:**Format — Instruction format for FGETMAN.X(z:S/D) Fn(s), Fn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Register field s—Selects the source operand.

FINT**Floating-Point Integral Value****FINT**

Operation: Rounds the selected floating-point source to an integral floating-point value using the active FSTATUS rounding mode.

Assembler Syntax: `FINT.X(z:S/D) <ea>(e), Fn(d)`
`FINT.X(z:S/D) Fn(s), Fn(d)`
`FINT.X(z:S/D) Fn(s), <ea>(e)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4-14 bytes
Feature = FP
Repeat = REP, REPG

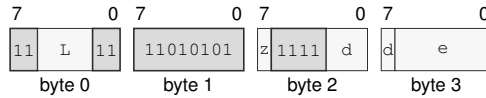
Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	may accrue

Instruction Forms:

`FINT.X(z:S/D) <ea>(e), Fn(d)`

long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for `FINT.X(z:S/D) <ea>(e), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

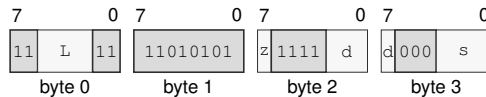
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`FINT.X(z:S/D) Fn(s), Fn(d)`

long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for `FINT.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

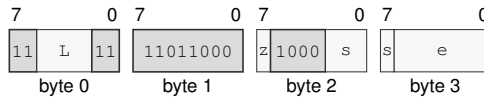
Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Register field *s*—Selects the source operand.

FINT.X(z:S/D) Fn(s), <ea>(e)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FINT.X(z:S/D) Fn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FINTRZ

Floating-Point Integral Toward Zero

FINTRZ

Operation: Rounds the selected floating-point source toward zero to an integral floating-point value.

Assembler Syntax: FINTRZ.X(z:S/D) <ea>(e), Fn(d)
FINTRZ.X(z:S/D) Fn(s), Fn(d)
FINTRZ.X(z:S/D) Fn(s), <ea>(e)

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4-14 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics

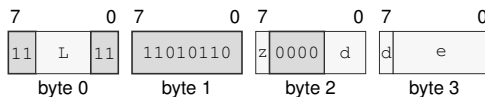
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	may accrue

Instruction Forms:

FINTRZ.X(z:S/D) <ea>(e), Fn(d)

long · 4-14 bytes · unprivileged

Instruction Format:**Format — Instruction format for FINTRZ.X(z:S/D) <ea>(e), Fn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

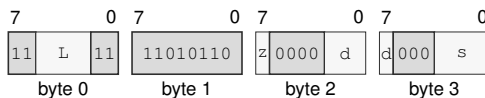
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FINTRZ.X(z:S/D) Fn(s), Fn(d)

long · 4 bytes · unprivileged

Instruction Format:**Format — Instruction format for FINTRZ.X(z:S/D) Fn(s), Fn(d)****Instruction Fields:**

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

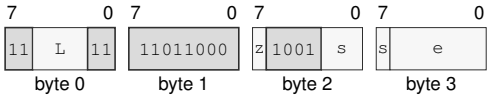
Size field z—Selects S/D.

Register field d—Selects the destination operand.

Register field s—Selects the source operand.

FINTRZ.X(z:S/D) Fn(s), <ea>(e)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FINTRZ.X(z:S/D) Fn(s), <ea>(e)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** s—Selects the source operand.
- Effective Address field** e—Specifies the destination operand.
- EA availability**
Allowed: Memory; EXT0
Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FMADD**Floating-Point Fused Multiply Add****FMADD**

Operation: Computes the selected-format fused multiply-add with one final rounding and writes the result to the destination.

Assembler Syntax: `FMADD.X(z:S/D) Fn(l), Fn(r), Fn(d)`
`FMADD.X(z:S/D) <ea>(l), Fn(r), Fn(d)`
`FMADD.X(z:S/D) Fn(l), <ea>(r), Fn(d)`

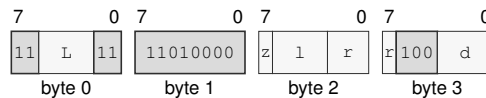
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 4-15 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	may accrue

Instruction Forms:

`FMADD.X(z:S/D) Fn(l), Fn(r), Fn(d)`
 long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for FMADD.X(z:S/D) Fn(l), Fn(r), Fn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *l*—Selects the operand 1.

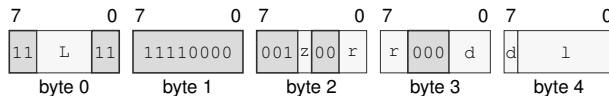
Register field *r*—Selects the operand 2.

Register field *d*—Selects the operand 3.

FMADD.X(z:S/D) <ea>(l), Fn(r), Fn(d)

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for FMADD.X(z:S/D) <ea>(l), Fn(r), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field r—Selects the operand 2.

Register field d—Selects the operand 3.

Effective Address field l—Specifies the source operand.

EA availability

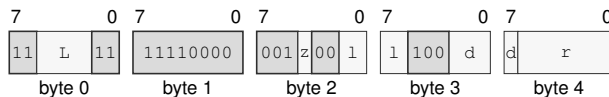
Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FMADD.X(z:S/D) Fn(l), <ea>(r), Fn(d)

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for FMADD.X(z:S/D) Fn(l), <ea>(r), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects the operand 1.

Register field d—Selects the operand 3.

Effective Address field r—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FMAX**Floating-Point Maximum****FMAX**

Operation: Selects the greater numeric operand; one NaN selects the numeric operand, two NaNs use the common NaN-selection rule, and a positive-zero/negative-zero pair selects positive zero. A signaling NaN generates NV.

Assembler Syntax: `FMAX.X(z:S/D) Fn(s), Fn(d)`
`FMAX.X(z:S/D) <ea>(e), Fn(d)`

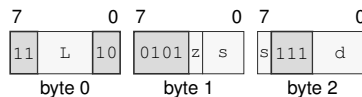
Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 3-14 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

Instruction Forms:

`FMAX.X(z:S/D) Fn(s), Fn(d)`
medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for `FMAX.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

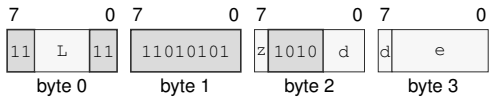
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FMAX.X(z:S/D) <ea>(e), Fn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FMAX.X(z:S/D) <ea>(e), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.
- EA availability**
 - Allowed:** Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

FMIN**Floating-Point Minimum****FMIN**

Operation: Selects the lesser numeric operand; one NaN selects the numeric operand, two NaNs use the common NaN-selection rule, and a positive-zero/negative-zero pair selects negative zero. A signaling NaN generates NV.

Assembler Syntax: `FMIN.X(z:S/D) Fn(s), Fn(d)`
`FMIN.X(z:S/D) <ea>(e), Fn(d)`

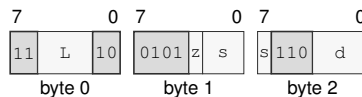
Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 3-14 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

Instruction Forms:

`FMIN.X(z:S/D) Fn(s), Fn(d)`
medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for FMIN.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

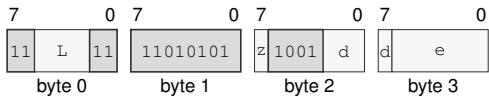
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FMIN.X(z:S/D) <ea>(e), Fn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FMIN.X(z:S/D) <ea>(e), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.
- EA availability**
 - Allowed:** Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

FMOD**Floating-Point Modulo****FMOD**

Operation: Computes the IEEE-style floating-point modulus defined by a quotient rounded toward zero and writes the selected-format remainder.

Assembler Syntax: `FMOD.X(z:S/D) <ea>(e), Fn(d)`
`FMOD.X(z:S/D) Fn(s), Fn(d)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4-14 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

After DAZ processing, let x be the original destination and y the source. For finite valid operands, choose q as x/y truncated toward zero and compute

$$r = x - qy$$

with exact intermediate arithmetic. A zero result has the sign of x .

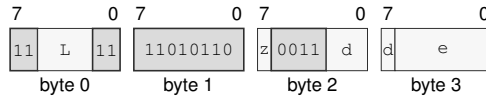
A signaling NaN accrues NV; a NaN input produces the selected quiet NaN. Infinite x or zero y accrues NV and produces the architectural default quiet NaN. Infinite y with finite x returns x . Enabled traps raise `FLOATING_POINT_FAULT` before the destination write.

Instruction Forms:

`FMOD.X(z:S/D) <ea>(e), Fn(d)`

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FMOD.X(z:S/D) <ea>(e), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

EA availability

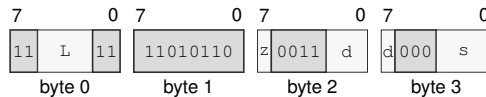
Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`FMOD.X(z:S/D) Fn(s), Fn(d)`

long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FMOD.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Register field *s*—Selects the source operand.

FMOV**Floating-Point Move****FMOV**

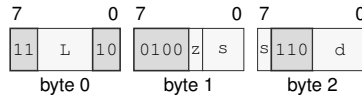
Operation: Copies the selected-format floating-point source value to the destination.

Assembler Syntax: `FMOV.X(z:S/D) Fn(s), Fn(d)`
`FMOV.X(z:S/D) <ea>(e), Fn(d)`
`FMOV.X(z:S/D) Fn(s), <ea>(e)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 3-14 bytes
Feature = FP
Repeat = REP, REPG

Instruction Forms:

`FMOV.X(z:S/D) Fn(s), Fn(d)`
medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for FMOV.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

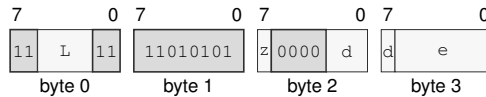
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

`FMOV.X(z:S/D) <ea>(e), Fn(d)`
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FMOV.X(z:S/D) <ea>(e), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

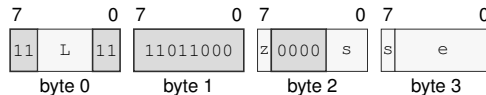
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`FMOV.X(z:S/D) Fn(s), <ea>(e)`
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FMOV.X(z:S/D) Fn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FMOVCR**Floating-Point Move Constant****FMOVCR**

Operation: FMOVCR maps an unsigned 16-bit constant ID to a fixed IEEE-754 binary64 bit pattern and writes that pattern to the destination F register. The mapping includes common numeric, transcendental, infinity, NaN, and format-boundary constants. It is independent of FSTATUS rounding, DAZ, FTZ, and default-NaN modes. Loading a signaling NaN is a bitwise move and does not itself raise NV; a later consuming floating-point operation applies its normal NaN rules.

Assembler Syntax: FMOVCR.X(z:S/D) <imm16>, Fn(d)

Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 6 bytes
 Feature = FP
 Repeat = REP, REPG

Table 18-2. FMOVCR Constant IDs (IEEE-754 binary64)

ID	Constant	Result bits
0x0000	positive zero	0x0000000000000000
0x0001	negative zero	0x8000000000000000
0x0002	positive one	0x3ff0000000000000
0x0003	negative one	0xbff0000000000000
0x0004	positive one half	0x3fe0000000000000
0x0005	negative one half	0xbfe0000000000000
0x0006	positive two	0x4000000000000000
0x0007	negative two	0xc000000000000000
0x0008	positive ten	0x4024000000000000
0x0009	negative ten	0xc024000000000000
0x0010	pi	0x400921fb54442d18
0x0011	pi over 2	0x3ff921fb54442d18
0x0012	pi over 4	0x3fe921fb54442d18
0x0013	two pi	0x401921fb54442d18
0x0014	reciprocal pi	0x3fd45f306dc9c883
0x0015	two over pi	0x3fe45f306dc9c883
0x0016	sqrt 2	0x3ff6a09e667f3bcd
0x0017	reciprocal sqrt 2	0x3fe6a09e667f3bcc
0x0020	e	0x4005bf0a8b145769
0x0021	log2 e	0x3ff71547652b82fe
0x0022	log10 e	0x3fdbcb7b1526e50e
0x0023	ln 2	0x3fe62e42fefa39ef
0x0024	ln 10	0x40026bb1bbb55516
0x0025	log2 10	0x400a934f0979a371
0x0026	log10 2	0x3fd34413509f79ff
0x0100	positive infinity	0x7ff0000000000000

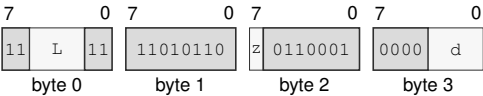
Table 18-2 (continued)

ID	Constant	Result bits
0x0101	negative infinity	0xfff0000000000000
0x0102	canonical positive quiet nan	0x7ff8000000000000
0x0103	canonical negative quiet nan	0xfff8000000000000
0x0104	canonical positive signaling nan	0x7ff0000000000001
0x0105	canonical negative signaling nan	0xfff0000000000001
0x0110	maximum positive finite	0x7fefffffffffffffff
0x0111	maximum negative finite	0xffefffffffffffffff
0x0112	minimum positive normal	0x0010000000000000
0x0113	minimum negative normal	0x8010000000000000
0x0114	minimum positive subnormal	0x0000000000000001
0x0115	minimum negative subnormal	0x8000000000000001
0x0116	machine epsilon	0x3cb0000000000000
0x0117	maximum positive subnormal	0x000fffffffffffffff
0x0118	maximum negative subnormal	0x800fffffffffffffff

Instruction Forms:

FMOVCR.X(z:S/D) <imm16>, Fn(d)
long · 6 bytes · unprivileged

Instruction Format:



Format — Instruction format for FMOVCR.X(z:S/D) <imm16>, Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.

FMOVcc**Floating-Point Conditional Move****FMOVcc**

Operation: Copies the selected-format floating-point source value to the destination only when the encoded integer condition is true.

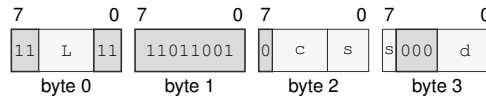
Assembler Syntax: `FMOVcc Fn(s), Fn(d)`
`FMOVcc.X(z:S/D) <ea>(e), Fn(d)`
`FMOVcc.X(z:S/D) Fn(s), <ea>(e)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4-14 bytes
Feature = FP
Repeat = REP, REPG

Instruction Forms:

`FMOVcc Fn(s), Fn(d)`

long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for FMOVcc Fn(s), Fn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

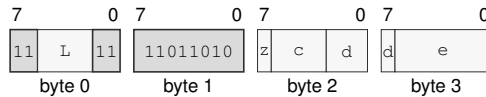
Condition field *c*—Selects the condition code.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

$\text{FMOVcc.X}(z:S/D) \text{ <ea>}(e), \text{Fn}(d)$
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for $\text{FMOVcc.X}(z:S/D) \text{ <ea>}(e), \text{Fn}(d)$

Instruction Fields:

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z —Selects S/D.

Condition field c —Selects the condition code.

Register field d —Selects the destination operand.

Effective Address field e —Specifies the source operand.

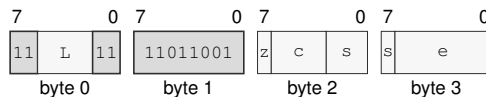
EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

$\text{FMOVcc.X}(z:S/D) \text{Fn}(s), \text{<ea>}(e)$
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for $\text{FMOVcc.X}(z:S/D) \text{Fn}(s), \text{<ea>}(e)$

Instruction Fields:

Length field L —Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z —Selects S/D.

Condition field c —Selects the condition code.

Register field s —Selects the source operand.

Effective Address field e —Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FPOPP**Floating-Point Pop Register Pair****FPOPP**

Operation: Pops the canonical floating-point register pair selected by the pair ID. The canonical pair sequence is pair 0 (F14, F15), pair 1 (F12, F13), pair 2 (F10, F11), pair 3 (F8, F9), pair 4 (F6, F7), pair 5 (F4, F5), pair 6 (F2, F3), and pair 7 (F0, F1). All eight pair IDs are valid and each selects one pair.

Assembler Syntax: `FPOPP pair_id(i)`

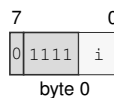
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 1 byte
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics

FPOPP selects one canonical floating-point pair using the unsigned 3-bit pair index and processes that pair's registers in reverse listed order. The complete two-slot stack range is validated and read before either floating-point register or SP is changed. Each register pop loads the 64-bit stack slot at SP and then increments SP by 8.

Instruction Forms:

`FPOPP pair_id(i)`
 extrashort · 1 byte · unprivileged

Instruction Format:

Format — Instruction format for FPOPP pair_id(i)

Instruction Fields:

Immediate field *i*—Encodes the immediate value.

FPUSHP

Floating-Point Push Register Pair

FPUSHP

Operation:	Pushes the canonical floating-point register pair selected by the pair ID. The canonical pair sequence is pair 0 (F14, F15), pair 1 (F12, F13), pair 2 (F10, F11), pair 3 (F8, F9), pair 4 (F6, F7), pair 5 (F4, F5), pair 6 (F2, F3), and pair 7 (F0, F1). All eight pair IDs are valid and each selects one pair.
Assembler Syntax:	FPUSHP pair_id(i)
Attributes:	Class = fpu Family = fpu Privilege = unprivileged Length = 1 byte Feature = FP Repeat = REP, REPG

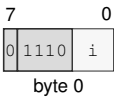
Detailed Semantics

FPUSHP selects one canonical floating-point pair using the unsigned 3-bit pair index and processes that pair's registers in the listed order. The complete two-slot stack range is validated before either slot or SP is changed. Each register push decrements SP by 8 and then stores the 64-bit floating-point register value at the new SP.

Instruction Forms:

FPUSHP pair_id(i)
extrashort · 1 byte · unprivileged

Instruction Format:



Format — Instruction format for FPUSHP pair_id(i)

Instruction Fields:

Immediate field i—Encodes the immediate value.

FMSUB**Floating-Point Fused Multiply Subtract****FMSUB**

Operation: Computes the selected-format fused multiply-subtract with one final rounding and writes the result to the destination.

Assembler Syntax: `FMSUB.X(z:S/D) Fn(l), Fn(r), Fn(d)`
`FMSUB.X(z:S/D) <ea>(l), Fn(r), Fn(d)`
`FMSUB.X(z:S/D) Fn(l), <ea>(r), Fn(d)`

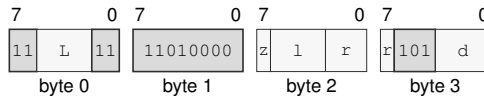
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 4-15 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	may accrue

Instruction Forms:

`FMSUB.X(z:S/D) Fn(l), Fn(r), Fn(d)`
 long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for `FMSUB.X(z:S/D) Fn(l), Fn(r), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *l*—Selects the operand 1.

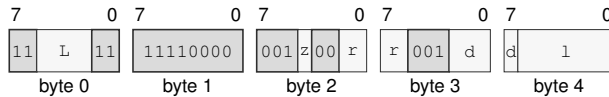
Register field *r*—Selects the operand 2.

Register field *d*—Selects the operand 3.

FMSUB.X(z:S/D) <ea>(l), Fn(r), Fn(d)

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for FMSUB.X(z:S/D) <ea>(l), Fn(r), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field r—Selects the operand 2.

Register field d—Selects the operand 3.

Effective Address field l—Specifies the source operand.

EA availability

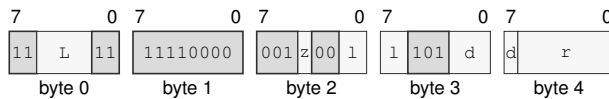
Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FMSUB.X(z:S/D) Fn(l), <ea>(r), Fn(d)

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for FMSUB.X(z:S/D) Fn(l), <ea>(r), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects the operand 1.

Register field d—Selects the operand 3.

Effective Address field r—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FMUL**Floating-Point Multiply****FMUL**

Operation: Multiplies the selected-format floating-point source and destination values and writes the rounded result.

Assembler Syntax: `FMUL.X(z:S/D) Fn(s), Fn(d)`
`FMUL.X(z:S/D) <ea>(e), Fn(d)`

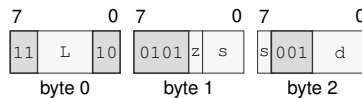
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 3-14 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	may accrue

Instruction Forms:

`FMUL.X(z:S/D) Fn(s), Fn(d)`
 medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for FMUL.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

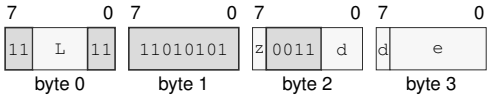
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FMUL.X(z:S/D) <ea>(e), Fn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FMUL.X(z:S/D) <ea>(e), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.
- EA availability**
 - Allowed:** Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

FNEG**Floating-Point Negate****FNEG**

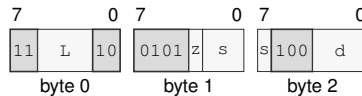
Operation: Complements the sign bit of the selected floating-point source value and writes the result.

Assembler Syntax: `FNEG.X(z:S/D) Fn(s), Fn(d)`
`FNEG.X(z:S/D) <ea>(e), Fn(d)`
`FNEG.X(z:S/D) Fn(s), <ea>(e)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 3-14 bytes
Feature = FP
Repeat = REP, REPG

Instruction Forms:

`FNEG.X(z:S/D) Fn(s), Fn(d)`
medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for FNEG.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

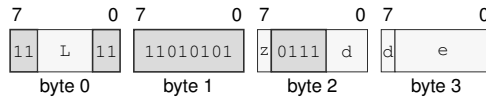
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FNEG.X(z:S/D) <ea>(e), Fn(d)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FNEG.X(z:S/D) <ea>(e), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

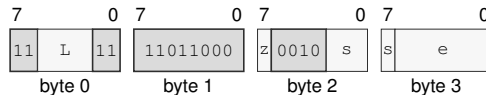
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FNEG.X(z:S/D) Fn(s), <ea>(e)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FNEG.X(z:S/D) Fn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FNMADD**Floating-Point Fused Negative Multiply Add****FNMADD**

Operation: Computes the negated selected-format fused multiply-add with one final rounding and writes the result.

Assembler Syntax: `FNMADD.X(z:S/D) Fn(l), Fn(r), Fn(d)`
`FNMADD.X(z:S/D) <ea>(l), Fn(r), Fn(d)`
`FNMADD.X(z:S/D) Fn(l), <ea>(r), Fn(d)`

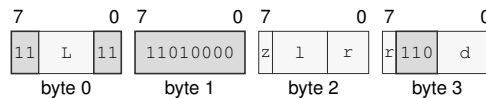
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 4-15 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	may accrue

Instruction Forms:

`FNMADD.X(z:S/D) Fn(l), Fn(r), Fn(d)`
 long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for `FNMADD.X(z:S/D) Fn(l), Fn(r), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *l*—Selects the operand 1.

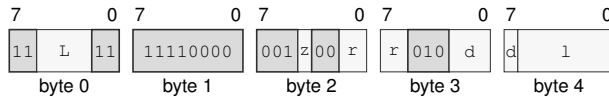
Register field *r*—Selects the operand 2.

Register field *d*—Selects the operand 3.

FNMADD.X(z:S/D) <ea>(l), Fn(r), Fn(d)

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for FNMADD.X(z:S/D) <ea>(l), Fn(r), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field r—Selects the operand 2.

Register field d—Selects the operand 3.

Effective Address field 1—Specifies the source operand.

EA availability

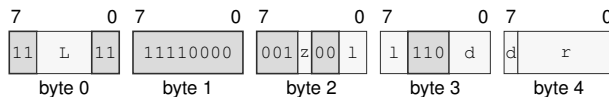
Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FNMADD.X(z:S/D) Fn(l), <ea>(r), Fn(d)

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for FNMADD.X(z:S/D) Fn(l), <ea>(r), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects the operand 1.

Register field d—Selects the operand 3.

Effective Address field r—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FNMSUB**Floating-Point Fused Negative Multiply Subtract****FNMSUB**

Operation: Computes the negated selected-format fused multiply-subtract with one final rounding and writes the result.

Assembler Syntax: `FNMSUB.X(z:S/D) Fn(l), Fn(r), Fn(d)`
`FNMSUB.X(z:S/D) <ea>(l), Fn(r), Fn(d)`
`FNMSUB.X(z:S/D) Fn(l), <ea>(r), Fn(d)`

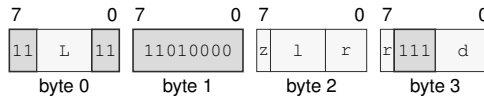
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 4-15 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	may accrue

Instruction Forms:

`FNMSUB.X(z:S/D) Fn(l), Fn(r), Fn(d)`
 long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for `FNMSUB.X(z:S/D) Fn(l), Fn(r), Fn(d)`

Instruction Fields:

Length field `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field `z`—Selects S/D.

Register field `l`—Selects the operand 1.

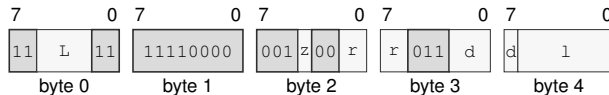
Register field `r`—Selects the operand 2.

Register field `d`—Selects the operand 3.

FNMSUB.X(z:S/D) <ea>(l), Fn(r), Fn(d)

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for FNMSUB.X(z:S/D) <ea>(l), Fn(r), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field r—Selects the operand 2.

Register field d—Selects the operand 3.

Effective Address field l—Specifies the source operand.

EA availability

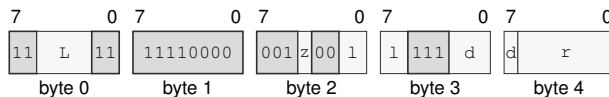
Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FNMSUB.X(z:S/D) Fn(l), <ea>(r), Fn(d)

extralong · 5-15 bytes · unprivileged

Instruction Format:



Format — Instruction format for FNMSUB.X(z:S/D) Fn(l), <ea>(r), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field l—Selects the operand 1.

Register field d—Selects the operand 3.

Effective Address field r—Specifies the source operand.

EA availability

Allowed: Register; Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FREM**Floating-Point Remainder****FREM**

Operation: Computes the IEEE-style floating-point remainder using a quotient rounded to the nearest integer with ties to even.

Assembler Syntax: `FREM.X(z:S/D) <ea>(e), Fn(d)`
`FREM.X(z:S/D) Fn(s), Fn(d)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4-14 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

After DAZ processing, let x be the original destination and y the source. For finite valid operands, choose q as x/y rounded to the nearest integer with ties to even and compute

$$r = x - qy$$

with exact intermediate arithmetic. A zero result has the sign of x .

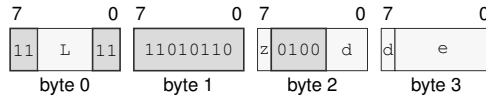
A signaling NaN accrues NV; a NaN input produces the selected quiet NaN. Infinite x or zero y accrues NV and produces the architectural default quiet NaN. Infinite y with finite x returns x . Enabled traps raise `FLOATING_POINT_FAULT` before the destination write.

Instruction Forms:

`FREM.X(z:S/D) <ea>(e), Fn(d)`

long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FREM.X(z:S/D) <ea>(e), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

EA availability

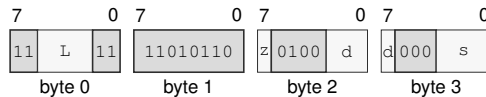
Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`FREM.X(z:S/D) Fn(s), Fn(d)`

long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FREM.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Register field *s*—Selects the source operand.

FROUND**Floating-Point Round****FROUND**

Operation: Rounds the selected floating-point source to the nearest integral value in the same format using nearest-even, independently of FSTATUS.RM.

Assembler Syntax: `FROUND.X(z:S/D) Fn(s), Fn(d)`
`FROUND.X(z:S/D) <ea>(e), Fn(d)`
`FROUND.X(z:S/D) Fn(s), <ea>(e)`

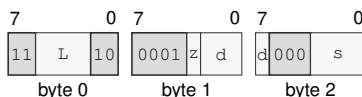
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 3-14 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	may accrue

Instruction Forms:

`FROUND.X(z:S/D) Fn(s), Fn(d)`
 medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for `FROUND.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

Length field `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

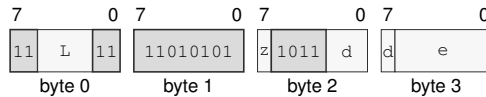
Size field `z`—Selects S/D.

Register field `d`—Selects the destination operand.

Register field `s`—Selects the source operand.

`FROUND.X(z:S/D) <ea>(e), Fn(d)`
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FROUND.X(z:S/D) <ea>(e), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

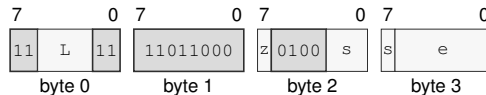
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`FROUND.X(z:S/D) Fn(s), <ea>(e)`
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FROUND.X(z:S/D) Fn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FSCALE**Floating-Point Scale****FSCALE**

Operation: Scales the selected floating-point destination by an integral power of two derived from the source and writes the rounded result.

Assembler Syntax: `FSCALE.X(z:S/D) <ea>(e), Fn(d)`
`FSCALE.X(z:S/D) Fn(s), Fn(d)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4-14 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	may accrue

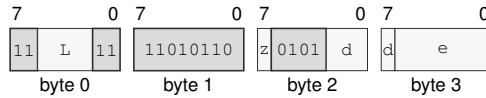
Let x be the first source after DAZ processing and let k be the second source rounded to an integer using the current rounding mode. The mathematical result before format rounding is

$$x \times 2^k.$$

A NaN operand is propagated according to the common floating-point rules. Multiplying zero or infinity by the power of two preserves that value and its sign. Conversion of the scale operand to k can accrue NX; final rounding can accrue OF, UF, or NX. Enabled exceptions trap before the destination is written.

Instruction Forms:

FSCALE.X(z:S/D) <ea>(e), Fn(d)
 long · 4-14 bytes · unprivileged

Instruction Format:

Format — Instruction format for FSCALE.X(z:S/D) <ea>(e), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

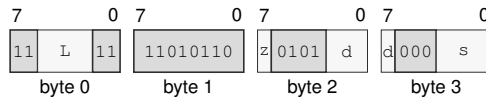
Effective Address field e—Specifies the source operand.

EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FSCALE.X(z:S/D) Fn(s), Fn(d)
 long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for FSCALE.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Register field s—Selects the source operand.

FSQRT**Floating-Point Square Root****FSQRT**

Operation: Computes the selected-format square root of the source and writes the rounded result.

Assembler Syntax: FSQRT.X(z:S/D) Fn(s), Fn(d)
 FSQRT.X(z:S/D) <ea>(e), Fn(d)
 FSQRT.X(z:S/D) Fn(s), <ea>(e)

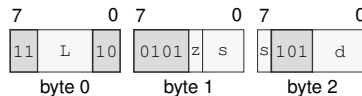
Attributes: Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 3-14 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	may accrue

Instruction Forms:

FSQRT.X(z:S/D) Fn(s), Fn(d)
 medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for FSQRT.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

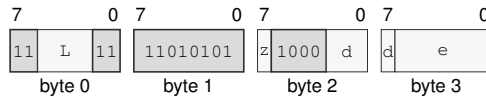
Size field z—Selects S/D.

Register field s—Selects the source operand.

Register field d—Selects the destination operand.

FSQRT.X($z:S/D$) <ea>(e), Fn(d)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FSQRT.X($z:S/D$) <ea>(e), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Effective Address field e—Specifies the source operand.

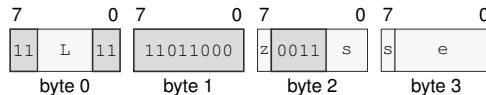
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FSQRT.X($z:S/D$) Fn(s), <ea>(e)
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FSQRT.X($z:S/D$) Fn(s), <ea>(e)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field s—Selects the source operand.

Effective Address field e—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FSUB**Floating-Point Subtract****FSUB**

Operation: Subtracts the selected-format floating-point source from the destination and writes the rounded result.

Assembler Syntax: `FSUB.X(z:S/D) Fn(s), Fn(d)`
`FSUB.X(z:S/D) <ea>(e), Fn(d)`

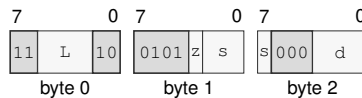
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 3-14 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	may accrue

Instruction Forms:

`FSUB.X(z:S/D) Fn(s), Fn(d)`
 medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for FSUB.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

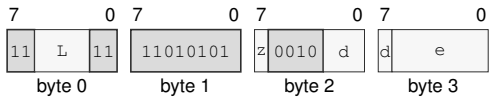
Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Register field *d*—Selects the destination operand.

FSUB.X(z:S/D) <ea>(e), Fn(d)
long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for FSUB.X(z:S/D) <ea>(e), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Effective Address field** e—Specifies the source operand.
- EA availability**
 - Allowed:** Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

FTEST**Floating-Point Test****FTEST**

Operation: Compares the selected-format operand with positive zero and writes Z,N,C,V as greater=0000, equal=1000, less=0110, or unordered=0001. A quiet NaN is unordered; a signaling NaN is unordered and generates NV.

Assembler Syntax: `FTEST.X(z:S/D) <ea>(e)`
`FTEST.X(z:S/D) Fn(s)`

Attributes: Class = fpu
Family = fpu
Privilege = unprivileged
Length = 4-14 bytes
Feature = FP
Repeat = REP, REPG

Detailed Semantics**FLAGS Effects**

Flag	Effect
Z	set only when equal
N	set only when less
C	set only when less
V	set when unordered

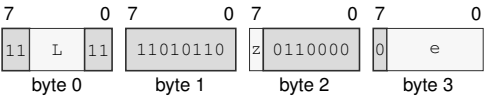
FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

Instruction Forms:

FTEST.X(z:S/D) <ea>(e)
long · 4-14 bytes · unprivileged

Instruction Format:



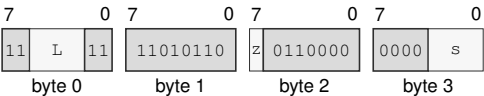
Format — Instruction format for FTEST.X(z:S/D) <ea>(e)

Instruction Fields:

- Length field L**—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field z**—Selects S/D.
- Effective Address field e**—Specifies the source operand.
- EA availability**
 - Allowed:** Register except $R_n(r)$; Memory; Immediate; EXT0
 - Encoding:** See Compact EA Encoding and the Extended EA Descriptor profiles.

FTEST.X(z:S/D) Fn(s)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FTEST.X(z:S/D) Fn(s)

Instruction Fields:

- Length field L**—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field z**—Selects S/D.
- Register field s**—Selects the operand.

FTRUNC**Floating-Point Truncate****FTRUNC**

Operation: Rounds the selected floating-point source toward zero to an integral value in the same floating-point format.

Assembler Syntax: `FTRUNC.X(z:S/D) Fn(s), Fn(d)`
`FTRUNC.X(z:S/D) <ea>(e), Fn(d)`
`FTRUNC.X(z:S/D) Fn(s), <ea>(e)`

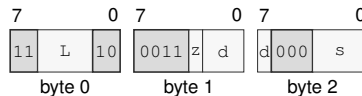
Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 3-14 bytes
 Feature = FP
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	may accrue

Instruction Forms:

`FTRUNC.X(z:S/D) Fn(s), Fn(d)`
 medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for `FTRUNC.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

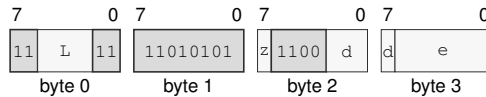
Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Register field *s*—Selects the source operand.

`FTRUNC.X(z:S/D) <ea>(e), Fn(d)`
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FTRUNC.X(z:S/D) <ea>(e), Fn(d)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *d*—Selects the destination operand.

Effective Address field *e*—Specifies the source operand.

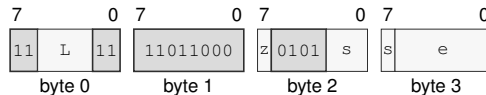
EA availability

Allowed: Memory; Immediate; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

`FTRUNC.X(z:S/D) Fn(s), <ea>(e)`
 long · 4-14 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FTRUNC.X(z:S/D) Fn(s), <ea>(e)`

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field *z*—Selects S/D.

Register field *s*—Selects the source operand.

Effective Address field *e*—Specifies the destination operand.

EA availability

Allowed: Memory; EXT0

Encoding: See Compact EA Encoding and the Extended EA Descriptor profiles.

FXCHG**Floating-Point Exchange****FXCHG**

Operation: Exchanges the two selected-format floating-point register values.

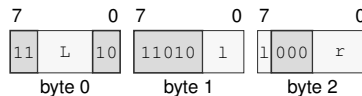
Assembler Syntax: `FXCHG Fn(l), Fn(r)`

Attributes:
 Class = fpu
 Family = fpu
 Privilege = unprivileged
 Length = 3 bytes
 Feature = FP
 Repeat = REP, REPG

Instruction Forms:

`FXCHG Fn(l), Fn(r)`

medium · 3 bytes · unprivileged

Instruction Format:

Format — Instruction format for FXCHG Fn(l), Fn(r)

Instruction Fields:

Length field *L*—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field *l*—Selects the source operand.

Register field *r*—Selects the destination operand.

Destination overlap—When *lhs* = *rhs* designate the same architectural register, the final value equals that register's initial value.

RDFFLAGS

Read Floating-Point Flags

RDFFLAGS

Operation: RDFFLAGS reads the architectural FFLAGS register, zero-extends it to the destination register, and writes only that destination. RDFFLAGS requires the floating-point extension.

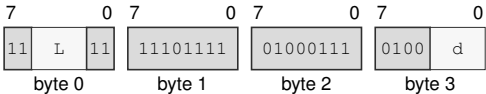
Assembler Syntax: RDFFLAGS Rn(d)

Attributes:
Class = system
Family = system registers
Privilege = unprivileged
Length = 4 bytes
Feature = FP
Repeat = none

Instruction Forms:

RDFFLAGS Rn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for RDFFLAGS Rn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Register field** d—Selects the operand.

RDFSTATUS**Read Floating-Point Status****RDFSTATUS**

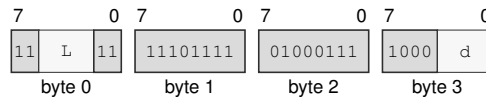
Operation: RDFSTATUS reads the architectural 16-bit FSTATUS register, zero-extends it to 64 bits, and writes the result to the destination Rn register. FSTATUS contains floating-point exception trap enables, rounding mode, and non-default mode bits. FFLAGS holds accrued floating-point exception flags. RDFSTATUS requires the floating-point extension.

Assembler Syntax: RDFSTATUS Rn(d)

Attributes:
 Class = system
 Family = system registers
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FP
 Repeat = none

Instruction Forms:

RDFSTATUS Rn(d)
 long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for RDFSTATUS Rn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field d—Selects the operand.

WRFFLAGS

Write Floating-Point Flags

WRFFLAGS

Operation: WRFFLAGS writes the architectural FFLAGS register from the low source bits. Reserved FFLAGS bits must be zero. WRFFLAGS requires the floating-point extension.

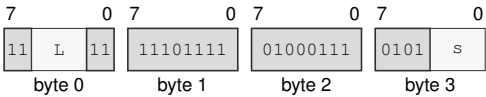
Assembler Syntax: WRFFLAGS Rn(s)

Attributes: Class = system
Family = system registers
Privilege = unprivileged
Length = 4 bytes
Feature = FP
Repeat = none

Instruction Forms:

WRFFLAGS Rn(s)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for WRFFLAGS Rn(s)

Instruction Fields:

- Length field** L—Encodes the total instruction length as 3 + L bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Register field** s—Selects the operand.

WRFSTATUS**Write Floating-Point Status****WRFSTATUS**

Operation: WRFSTATUS writes only the architectural 16-bit FSTATUS register from the low 16 bits of the source Rn register. Reserved FSTATUS bits must be zero. WRFSTATUS requires the floating-point extension.

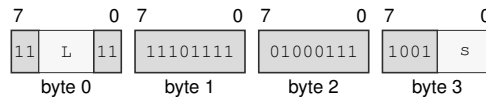
Assembler Syntax: WRFSTATUS Rn(s)

Attributes:
 Class = system
 Family = system registers
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FP
 Repeat = none

Instruction Forms:

WRFSTATUS Rn(s)

long · 4 bytes · unprivileged

Instruction Format:

Format — Instruction format for WRFSTATUS Rn(s)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Register field s—Selects the operand.

19 APPROXIMATE FLOATING-POINT TRANSCENDENTAL INSTRUCTIONS

19.1 FPTRANSA ULP Error Model

For an approximate transcendental instruction, x' is the source after `FSTATUS.DAZ`, $y = f(x')$ is the exact real reference value named by its accuracy contract, and a is the approximate result before `FSTATUS.FTZ`. For a format with precision p and minimum normal exponent $e_{\min}$, define

$$\text{ulp}_F(y) = \begin{cases} 2^{\max(\lfloor \log_2 |y| \rfloor - p + 1, e_{\min} - p + 1)}, & y \neq 0, \\ 2^{e_{\min} - p + 1}, & y = 0. \end{cases}$$

The measured error is $|a - y| / \text{ulp}_F(y)$. The bound applies only to finite reference values within the selected format's finite range; NaN, infinity, and overflow follow the instruction's special-value rules. Every present contract guarantees at most 4 ULP for both S and D, corresponding to at least 21 and 50 worst-case relative precision bits respectively for nonzero normal results. CPUID may advertise a smaller certified bound.

FPTRANSA applies DAZ before forming the exact reference input and measures its advertised ULP error before FTZ. It ignores `FSTATUS.RM` and formats its implementation-defined approximation with nearest-even rounding. The approximation is deterministic for the same implementation, input, format, and relevant `FSTATUS` mode, but need not be bit-identical across implementations. DN and FTZ retain their ordinary meanings. FPTRANSA may accrue NV, DZ, OF, and UF as its instruction contract requires, but it leaves the accrued NX flag unchanged and never takes an NX trap. Overflow means that the exact result magnitude exceeds the selected format's maximum finite value. Underflow means that the exact nonzero result magnitude is below the selected format's minimum normal value.

19.2 Summary

Table 19-1. Approximate Floating-Point Transcendental Instructions Summary

Mnemonic	Brief description
FACOSA	Performs the bounded approximate floating-point arc cosine operation.
FASINA	Performs the bounded approximate floating-point arc sine operation.
FATANA	Performs the bounded approximate floating-point arc tangent operation.
FATANHA	Performs the bounded approximate floating-point hyperbolic arc tangent operation.
FCOSA	Performs the bounded approximate floating-point cosine operation.
FCOSHA	Performs the bounded approximate floating-point hyperbolic cosine operation.
FETOXA	Performs the bounded approximate floating-point e to x operation.
FETOXM1A	Performs the bounded approximate floating-point e to x minus one operation.
FLOG10A	Performs the bounded approximate floating-point log base 10 operation.
FLOG2A	Performs the bounded approximate floating-point log base 2 operation.
FLOGNA	Performs the bounded approximate floating-point natural log operation.
FLOGNP1A	Performs the bounded approximate floating-point natural log plus one operation.
FSINA	Performs the bounded approximate floating-point sine operation.
FSINCOSA	Performs the bounded approximate floating-point sine and cosine operation.
FSINHA	Performs the bounded approximate floating-point hyperbolic sine operation.
FTANA	Performs the bounded approximate floating-point tangent operation.

Table 19-1 (continued)

Mnemonic	Brief description
FTANHA	Performs the bounded approximate floating-point hyperbolic tangent operation.
FTENTOXA	Performs the bounded approximate floating-point 10 to x operation.
FTWOTOXA	Performs the bounded approximate floating-point 2 to x operation.

FACOSA**Approximate Floating-Point Arc Cosine****FACOSA**

Operation: Computes the $\text{acos}(x)$ in radians reference over finite x with $\text{abs}(x) \leq 1$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FACOSA.X(z:S/D) Fn(s), Fn(d)`

Attributes:

- Class = fpu
- Family = fpu transcendental approx
- Privilege = unprivileged
- Length = 4 bytes
- Feature = FPTRANSA
- Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0012. If PRESENT is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\text{acos}(x)$ in radians on finite x with $\text{abs}(x) \leq 1$ after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source equals 1, the result is positive zero.

Exact anchors

- $\text{acos}(+1) = +0$

Required properties

- monotonically decreasing on $[-1, 1]$
- result is in $[0, \pi]$

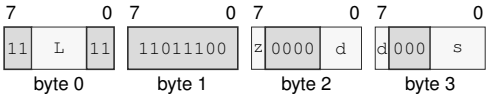
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

`FACOSA.X(z:S/D) Fn(s), Fn(d)`
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FACOSA.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

- Length field** `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** `z`—Selects S/D.
- Register field** `d`—Selects the destination operand.
- Register field** `s`—Selects the source operand.

FASINA**Approximate Floating-Point Arc Sine****FASINA**

Operation: Computes the $\text{asin}(x)$ in radians reference over finite x with $\text{abs}(x) \leq 1$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FASINA.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0011. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\text{asin}(x)$ in radians on finite x with $\text{abs}(x) \leq 1$ after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, the result is the source.

Exact anchors

- $\text{asin}(+0) = +0$
- $\text{asin}(-0) = -0$

Required properties

- odd
- monotonically increasing on $[-1, 1]$
- result is in $[-\pi/2, \pi/2]$

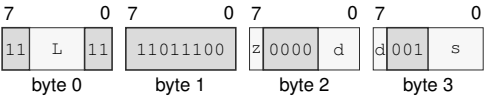
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

`FASINA.X(z:S/D) Fn(s), Fn(d)`
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FASINA.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

- Length field** `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** `z`—Selects S/D.
- Register field** `d`—Selects the destination operand.
- Register field** `s`—Selects the source operand.

FATANA**Approximate Floating-Point Arc Tangent****FATANA**

Operation: Computes the $\text{atan}(x)$ in radians reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FATANA.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0013. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\text{atan}(x)$ in radians on all finite values and signed infinity after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, the result is the source.

Exact anchors

- $\text{atan}(+0) = +0$
- $\text{atan}(-0) = -0$

Required properties

- odd
- monotonically increasing
- finite results are in $[-\pi/2, \pi/2]$

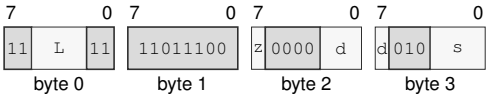
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

FATANA.X(z:S/D) Fn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FATANA.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Register field** s—Selects the source operand.

FATANHA**Approximate Floating-Point Hyperbolic Arc Tangent****FATANHA**

Operation: Computes the $\operatorname{atanh}(x)$ reference over finite x with $\operatorname{abs}(x) \leq 1$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FATANHA.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0024. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\operatorname{atanh}(x)$ on finite x with $\operatorname{abs}(x) \leq 1$ after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, the result is the source.

Exact anchors

- $\operatorname{atanh}(+0) = +0$
- $\operatorname{atanh}(-0) = -0$
- $\operatorname{atanh}(\text{signed } 1) = \text{signed infinity}$

Required properties

- odd
- monotonically increasing on $(-1, 1)$

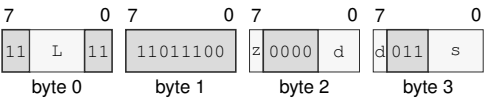
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

FATANHA.X(z:S/D) Fn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FATANHA.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Register field** s—Selects the source operand.

FCOSA**Approximate Floating-Point Cosine****FCOSA**

Operation: Computes the $\cos(x)$ in radians reference over finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FCOSA.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	preserved
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0002. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\cos(x)$ in radians on finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite or $\text{abs}(\text{the source}) > \pi / 4$, NV is accrued and the result is the architectural default quiet NaN.
- When the source is zero, the result is positive one.

Exact anchors

- $\cos(+0) = +1$
- $\cos(-0) = +1$

Required properties

- even
- nondecreasing on $[-\pi/4, 0]$ and nonincreasing on $[0, \pi/4]$
- result is in $[\sqrt{2}/2, 1]$

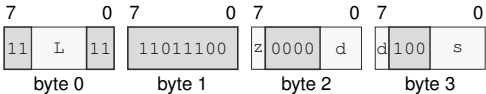
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

FCOSA.X(z:S/D) Fn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FCOSA.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Register field** s—Selects the source operand.

FCOSHA**Approximate Floating-Point Hyperbolic Cosine****FCOSHA**

Operation: Computes the $\cosh(x)$ reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FCOSHA.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	preserved
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0022. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\cosh(x)$ on all finite values and signed infinity after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite, the result is +infinity.
- When the source is zero, the result is positive one.

Exact anchors

- $\cosh(+0) = +1$
- $\cosh(-0) = +1$
- $\cosh(\text{infinity}) = +\text{infinity}$

Required properties

- even
- nonincreasing below zero and nondecreasing above zero
- result is at least 1

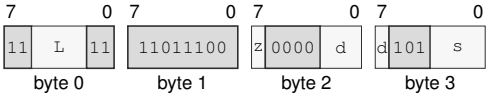
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

FCOSHA.X(z:S/D) Fn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FCOSHA.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Register field** s—Selects the source operand.

FETOXA**Approximate Floating-Point e To x****FETOXA**

Operation: Computes the e^{**x} reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FETOXA.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0031. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is e^{**x} on all finite values and signed infinity after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is +infinity, the result is +infinity.
- When the source is -infinity, the result is positive zero.
- When the source is zero, the result is positive one.

Exact anchors

- $e^{**\text{signed } 0} = +1$
- $e^{**+\text{infinity}} = +\text{infinity}$
- $e^{**-\text{infinity}} = +0$

Required properties

- strictly increasing for finite inputs
- result is nonnegative

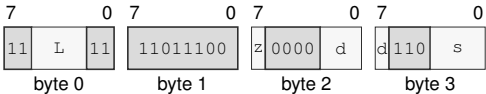
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

FETOXA.X(z:S/D) Fn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FETOXA.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Register field** s—Selects the source operand.

FETOXM1A**Approximate Floating-Point e To x Minus One****FETOXM1A**

Operation: Computes the $(e^{**x}) - 1$ without an intermediate rounded exponential reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: FETOXM1A.X(z:S/D) Fn(s), Fn(d)

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0032. If PRESENT is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $(e^{**x}) - 1$ without an intermediate rounded exponential on all finite values and signed infinity after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is +infinity, the result is +infinity.
- When the source is -infinity, the result is -1.
- When the source is zero, the result is the source.

Exact anchors

- $\text{expm1}(+0) = +0$
- $\text{expm1}(-0) = -0$
- $\text{expm1}(-\text{infinity}) = -1$

Required properties

- strictly increasing for finite inputs
- result is at least -1

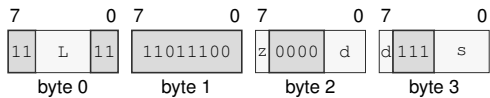
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

FETOXM1A.X(z:S/D) Fn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FETOXM1A.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

Length field L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field z—Selects S/D.

Register field d—Selects the destination operand.

Register field s—Selects the source operand.

FLOG10A**Approximate Floating-Point Log Base 10****FLOG10A**

Operation: Computes the $\log_{10}(x)$ reference over positive finite values and positive infinity after DAZ; signed zero is the DZ boundary and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FLOG10A.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0044. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\log_{10}(x)$ on positive finite values and positive infinity after DAZ; signed zero is the DZ boundary. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, DZ is accrued and the result is -infinity.
- When the source < 0 , NV is accrued and the result is the architectural default quiet NaN.
- When the source is +infinity, the result is +infinity.
- When the source equals 1, the result is positive zero.

Exact anchors

- $\log_{10}(+1) = +0$
- $\log_{10}(\text{signed } 0) = -\text{infinity}$
- $\log_{10}(+\text{infinity}) = +\text{infinity}$

Required properties

- strictly increasing on the positive domain

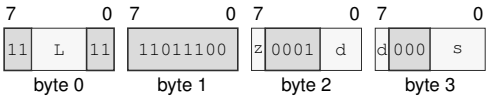
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

`FLOG10A.X(z:S/D) Fn(s), Fn(d)`
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FLOG10A.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

Length field `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field `z`—Selects S/D.

Register field `d`—Selects the destination operand.

Register field `s`—Selects the source operand.

FLOG2A**Approximate Floating-Point Log Base 2****FLOG2A**

Operation: Computes the $\log_2(x)$ reference over positive finite values and positive infinity after DAZ; signed zero is the DZ boundary and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FLOG2A.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0043. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\log_2(x)$ on positive finite values and positive infinity after DAZ; signed zero is the DZ boundary. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, DZ is accrued and the result is -infinity.
- When the source < 0 , NV is accrued and the result is the architectural default quiet NaN.
- When the source is +infinity, the result is +infinity.
- When the source equals 1, the result is positive zero.

Exact anchors

- $\log_2(+1) = +0$
- $\log_2(\text{signed } 0) = -\text{infinity}$
- $\log_2(+\text{infinity}) = +\text{infinity}$

Required properties

- strictly increasing on the positive domain

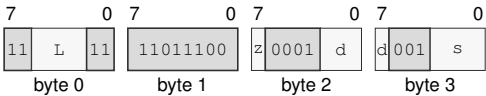
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

`FLOG2A.X(z:S/D) Fn(s), Fn(d)`
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FLOG2A.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

Length field `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field `z`—Selects S/D.

Register field `d`—Selects the destination operand.

Register field `s`—Selects the source operand.

FLOGNA**Approximate Floating-Point Natural Log****FLOGNA**

Operation: Computes the $\ln(x)$ reference over positive finite values and positive infinity after DAZ; signed zero is the DZ boundary and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FLOGNA.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	may accrue
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0041. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\ln(x)$ on positive finite values and positive infinity after DAZ; signed zero is the DZ boundary. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is zero, DZ is accrued and the result is -infinity.
- When the source < 0 , NV is accrued and the result is the architectural default quiet NaN.
- When the source is +infinity, the result is +infinity.
- When the source equals 1, the result is positive zero.

Exact anchors

- $\ln(+1) = +0$
- $\ln(\text{signed } 0) = -\text{infinity}$
- $\ln(+\text{infinity}) = +\text{infinity}$

Required properties

- strictly increasing on the positive domain

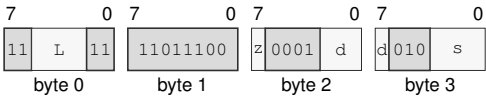
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

`FLOGNA.X(z:S/D) Fn(s), Fn(d)`
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FLOGNA.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

Length field `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field `z`—Selects S/D.

Register field `d`—Selects the destination operand.

Register field `s`—Selects the source operand.

FLOGNP1A Approximate Floating-Point Natural Log Plus One FLOGNP1A

Operation: Computes the $\ln(1 + x)$ without an intermediate rounded addition reference over finite $x \geq -1$ and positive infinity after DAZ; -1 is the DZ boundary and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: FLOGNP1A.X(z:S/D) Fn(s), Fn(d)

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics

FFLAGS Effects

Flag	Effect
NV	may accrue
DZ	may accrue
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0042. If PRESENT is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\ln(1 + x)$ without an intermediate rounded addition on finite $x \geq -1$ and positive infinity after DAZ; -1 is the DZ boundary. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source equals -1, DZ is accrued and the result is -infinity.
- When the source < -1 , NV is accrued and the result is the architectural default quiet NaN.
- When the source is +infinity, the result is +infinity.
- When the source is zero, the result is the source.

Exact anchors

- $\log_1 p(+0) = +0$
- $\log_1 p(-0) = -0$
- $\log_1 p(-1) = -\text{infinity}$

Required properties

- strictly increasing on (-1, +infinity)

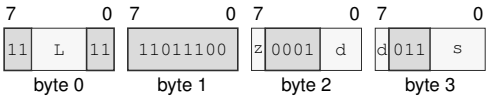
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

`FLOGNP1A.X(z:S/D) Fn(s), Fn(d)`
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FLOGNP1A.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

- Length field** `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** `z`—Selects S/D.
- Register field** `d`—Selects the destination operand.
- Register field** `s`—Selects the source operand.

FSINA**Approximate Floating-Point Sine****FSINA**

Operation: Computes the $\sin(x)$ in radians reference over finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FSINA.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0001. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\sin(x)$ in radians on finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite or $\text{abs}(\text{the source}) > \pi / 4$, NV is accrued and the result is the architectural default quiet NaN.
- When the source is zero, the result is the source.

Exact anchors

- $\sin(+0) = +0$
- $\sin(-0) = -0$

Required properties

- odd
- monotonically increasing on the contract domain
- result is in $[-\sqrt{2}/2, \sqrt{2}/2]$

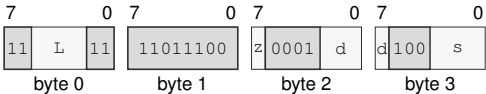
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

FSINA.X(z:S/D) Fn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FSINA.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Register field** s—Selects the source operand.

FSINCOSA**Approximate Floating-Point Sine And Cosine****FSINCOSA**

Operation: Computes the paired $\sin(x)$ and $\cos(x)$ in radians reference over finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FSINCOSA.X(z:S/D) Fn(s), Fn(d), Fn(c)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 5 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Exceptions:
 ILLEGAL_INSTRUCTION: `sin_dst` and `cos_dst` designate the same register; cause is INVALID_OPERAND_RELATION
 FLOATING_POINT_FAULT: an enabled floating-point exception is raised

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0004. If PRESENT is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is paired $\sin(x)$ and $\cos(x)$ in radians on finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the two destinations name the same F register, ILLEGAL_INSTRUCTION is raised before operands or floating-point state are read.
- When the source is infinite or its magnitude exceeds $\pi / 4$, NV is accrued and both results are the architectural default quiet NaN.
- When the source is zero, the sine result preserves the source zero and the cosine result is positive one.

Exact anchors

- $\sin(+0) = +0$ and $\cos(+0) = +1$

- $\sin(-0) = -0$ and $\cos(-0) = +1$

Required properties

- the ULP bound applies independently to both results
- results need not match separate FSINA and FCOSA executions

For nontrapping execution, the sine and cosine destinations are written atomically.

A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

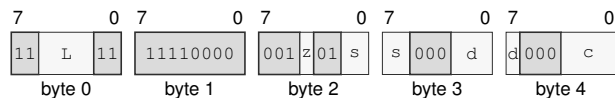
If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

`FSINCOSA.X(z:S/D) Fn(s), Fn(d), Fn(c)`

extralong · 5 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FSINCOSA.X(z:S/D) Fn(s), Fn(d), Fn(c)`

Instruction Fields:

Length field `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.

Size field `z`—Selects S/D.

Register field `s`—Selects the operand 1.

Register field `d`—Selects the operand 2.

Register field `c`—Selects the operand 3.

Destination overlap—When `sin_dst = cos_dst` designate the same architectural register, the instruction raises `ILLEGAL_INSTRUCTION.INVALID_OPERAND_RELATION` before architectural effects.

FSINHA**Approximate Floating-Point Hyperbolic Sine****FSINHA**

Operation: Computes the $\sinh(x)$ reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FSINHA.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0021. If PRESENT is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\sinh(x)$ on all finite values and signed infinity after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite or zero, the result is the source.

Exact anchors

- $\sinh(+0) = +0$
- $\sinh(-0) = -0$
- $\sinh(\text{infinity}) = \text{infinity}$ with the input sign

Required properties

- odd
- monotonically increasing

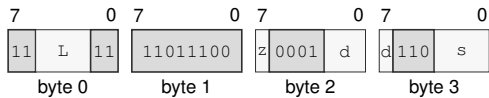
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

`FSINHA.X(z:S/D) Fn(s), Fn(d)`
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for `FSINHA.X(z:S/D) Fn(s), Fn(d)`

Instruction Fields:

- Length field** `L`—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** `z`—Selects S/D.
- Register field** `d`—Selects the destination operand.
- Register field** `s`—Selects the source operand.

FTANA**Approximate Floating-Point Tangent****FTANA**

Operation: Computes the $\tan(x)$ in radians reference over finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FTANA.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0003. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\tan(x)$ in radians on finite x with $\text{abs}(x) \leq \pi / 4$ after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite or $\text{abs}(\text{the source}) > \pi / 4$, NV is accrued and the result is the architectural default quiet NaN.
- When the source is zero, the result is the source.

Exact anchors

- $\tan(+0) = +0$
- $\tan(-0) = -0$

Required properties

- odd
- monotonically increasing on the contract domain
- result is in $[-1, 1]$

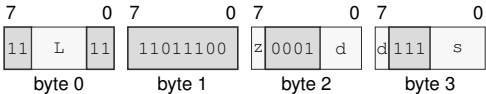
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

FTANA.X(z:S/D) Fn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FTANA.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Register field** s—Selects the source operand.

FTANHA**Approximate Floating-Point Hyperbolic Tangent****FTANHA**

Operation: Computes the $\tanh(x)$ reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: `FTANHA.X(z:S/D) Fn(s), Fn(d)`

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	preserved
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0023. If PRESENT is clear, the instruction raises `ILLEGAL_INSTRUCTION` before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is $\tanh(x)$ on all finite values and signed infinity after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is infinite, the result is one with the sign of the source.
- When the source is zero, the result is the source.

Exact anchors

- $\tanh(+0) = +0$
- $\tanh(-0) = -0$
- $\tanh(\text{signed infinity}) = \text{signed } 1$

Required properties

- odd
- monotonically increasing
- result is in $[-1, 1]$

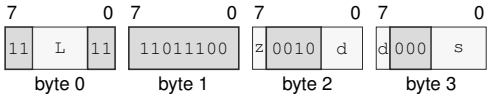
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

FTANHA.X(z:S/D) Fn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FTANHA.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Register field** s—Selects the source operand.

FTENTOXa**Approximate Floating-Point 10 To x****FTENTOXa**

Operation: Computes the 10^{**x} reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: FTENTOXa.X(z:S/D) Fn(s), Fn(d)

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0034. If PRESENT is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is 10^{**x} on all finite values and signed infinity after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is +infinity, the result is +infinity.
- When the source is -infinity, the result is positive zero.
- When the source is zero, the result is positive one.

Exact anchors

- 10^{**} signed 0 = +1
- 10^{**} +infinity = +infinity
- 10^{**} -infinity = +0

Required properties

- strictly increasing for finite inputs
- result is nonnegative

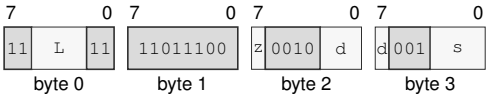
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

FTENTOXAX(z:S/D) Fn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FTENTOXAX(z:S/D) Fn(s), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Register field** s—Selects the source operand.

FTWOTOXA**Approximate Floating-Point 2 To x****FTWOTOXA**

Operation: Computes the 2^{**x} reference over all finite values and signed infinity after DAZ and writes an implementation-bounded S- or D-format approximation under the selected FPTRANSA accuracy contract.

Assembler Syntax: FTWOTOXA.X(z:S/D) Fn(s), Fn(d)

Attributes:
 Class = fpu
 Family = fpu transcendental approx
 Privilege = unprivileged
 Length = 4 bytes
 Feature = FPTRANSA
 Repeat = REP, REPG

Detailed Semantics**FFLAGS Effects**

Flag	Effect
NV	may accrue
DZ	preserved
OF	may accrue
UF	may accrue
NX	preserved

The required FPTRANSA_ACCURACY contract is 0x0033. If PRESENT is clear, the instruction raises ILLEGAL_INSTRUCTION before reading operands or floating-point state.

After FSTATUS.DAZ processing, the reference function is 2^{**x} on all finite values and signed infinity after DAZ. The result error is at most 4 ULP for S and 4 ULP for D.

Special values

- When the source is +infinity, the result is +infinity.
- When the source is -infinity, the result is positive zero.
- When the source is zero, the result is positive one.

Exact anchors

- 2^{**} signed 0 = +1
- 2^{**} +infinity = +infinity
- 2^{**} -infinity = +0

Required properties

- strictly increasing for finite inputs
- result is nonnegative

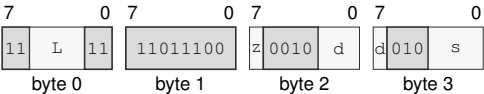
A signaling NaN accrues NV. NaN results are quieted and then processed by FSTATUS.DN. FSTATUS.FTZ replaces a subnormal result with the required signed zero and accrues UF. OF and UF are accrued as specified by the common FPTRANSA model, while NX is unchanged.

If any accrued exception is enabled as a trap, the instruction raises `FLOATING_POINT_FAULT` before writing a destination; otherwise it commits the result and accrued flags together.

Instruction Forms:

FTWOTOXA.X(z:S/D) Fn(s), Fn(d)
long · 4 bytes · unprivileged

Instruction Format:



Format — Instruction format for FTWOTOXA.X(z:S/D) Fn(s), Fn(d)

Instruction Fields:

- Length field** L—Encodes the total instruction length as $3 + L$ bytes. The selected length must cover all required bytes; trailing bytes are uninterpreted padding.
- Size field** z—Selects S/D.
- Register field** d—Selects the destination operand.
- Register field** s—Selects the source operand.

A REFERENCE INDEXES AND REVISION HISTORY

These indexes are generated from the instruction definitions, encoding store, register and extension manifests, architectural tables, and navigation metadata. Page references point to the normative definition or instruction entry.

A.1 Canonical Field Names

The following spellings are canonical within the named owner. A one-letter field is interpreted in its owner or instruction-form context.

Table A-1. Canonical Field Names

Owner	Canonical fields	Normative definition
FLAGS	Z, N, C, V	definition (p. 10)
STATUS	IE, PM, RF, TF, NI, EA	definition (p. 10)
PTCR	root_page, PABITS_SEL, LA57, PE	definition (p. 13)
ASCR	ASID, AE	definition (p. 13)
ECR	MAX_EDEPTH, NMI_P, V	definition (p. 13)
URCTL	V, STATUS, FLAGS	definition (p. 13)
PMC	EN	definition (p. 13)
PTE	P, W, X, U, G, A, D, AT, CP, SWO, T, PFN	definition (p. 47)
instruction_header	L	definition (p. 20)

A.2 Architectural State Index

Reader and writer columns identify the architectural interfaces or execution classes that access each state group. Reset values are taken from the generated architectural reset-state source.

Table A-2. Architectural State Index

State	Fields	Readers	Writers	Reset
R0--R15 (p. 10)	—	Rn source operands and effective-address base or index terms	writable Rn destinations and effective-address auto-update	zero
SP (p. 10)	—	stack operations and SP-relative effective addresses	stack operations and writable SP destinations	zero
PC (p. 10)	—	instruction fetch and PC-relative effective addresses	control transfers, event return, and RESET	bootpc
CS, DS, SS, GS0, GS1, GS2, GS3, GS4, GS5 (p. 12)	base, e, m, B	segment pre-translation, RDSEG, fixed CS reads, and SAVE	WRSEG, segment-changing control transfers, event return, and RESTORE	zero
FLAGS, STATUS (p. 10)	Z, N, C, V, IE, PM, RF, TF, NI, EA	condition evaluation, RDFLAGS, RDSTATUS, event entry, and SAVE	instruction flag results, WRFLAGS, WRSTATUS, event transitions, and RESTORE	FLAGS=zero; STATUS=supervisor_ only
F0--F15 (p. 12)	—	floating-point source operands and SAVE	floating-point destinations and RESTORE	zero
FFLAGS, FSTATUS (p. 10)	NV, DZ, OF, UF, NX, NV_EN, DZ_EN, OF_EN, UF_EN, NX_EN, RM, FTZ, DAZ, DN	floating-point execution, dedicated reads, and SAVE	floating-point results, dedicated writes, RESET, and RESTORE	zero
CYCLE, INSTRET, PTWALK (p. 87)	—	RDPMC	architecturally defined counter increment events and RESET	zero
PTCR (p. 13)	root_page, PABITS_SEL, LA57, PE	RDCR	WRCR and named architectural transitions	zero
ASCR (p. 13)	ASID, AE	RDCR	WRCR and named architectural transitions	zero

Table A-2 (continued)

State	Fields	Readers	Writers	Reset
ECR (p. 13)	MAX_EDEPTH, NMI_P, V	RDCR	WRCR and named architectural transitions	zero
SPC (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
SCS (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
SDS (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
URPC (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
URSP (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
URCS (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
URDS (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
URSS (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
URCTL (p. 13)	V, STATUS, FLAGS	RDCR	WRCR and named architectural transitions	zero
EPC (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
ECS (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
EDS (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
SSS (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
SSP (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
ISS (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
ISP (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
FSS (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
FSP (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
DSS (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
DSP (p. 13)	—	RDCR	WRCR and named architectural transitions	zero
BOOTPC (p. 13)	—	RDCR	WRCR and named architectural transitions	platform_ supplied_ cold_ preserved_ warm
BOOTCFG (p. 13)	—	RDCR	WRCR and named architectural transitions	platform_ supplied_ cold_ preserved_ warm
PMC (p. 13)	EN	RDCR	WRCR and named architectural transitions	zero

A.3 Architectural Event Index

Table A-3. Architectural Event Index

Code	Event	Producer	Priority	Frame	Definition
EXC:00	DEBUG_TRACE	completion of a TF-selected trace unit	5	BASIC	definition (p. 65)
EXC:02	BREAKPOINT	BKPT; BKPT	4	BASIC	definition (p. 65)
EXC:03	ILLEGAL_INSTRUCTION	decode or instruction-validity check; DIVMODS, DIVMODU, FSINCOSA, ILLEGAL	3	ERROR	definition (p. 65)
EXC:08	PRIVILEGE_FAULT	privilege check	3	BASIC	definition (p. 65)
EXC:09	PAGE_FAULT	segment, translation, access, or atomic-alignment check	3	PAGE_FAULT	definition (p. 65)
EXC:0a	DIVIDE_ERROR	integer divide operation; DIVMODS, DIVMODU, DIVS, DIVU, MODS, MODU	3	ERROR	definition (p. 65)
EXC:0d	INVALID_CONTROL_STATE	control-state validation; ERET, SYSCALL, SYSRET, WRCR, WRSTATUS	3	ERROR	definition (p. 65)
EXC:0e	FLOATING_POINT_FAULT	enabled floating-point exception; FSINCOSA	3	ERROR	definition (p. 65)
EXC:18	DOUBLE_FAULT	architectural event-delivery failure	1	AUXILIARY	definition (p. 65)
EXC:19	MACHINE_CHECK	processor or memory-system machine-check source	2	AUXILIARY	definition (p. 65)
EXC:1a	BUS_ERROR	synchronous acknowledged bus failure	3	AUXILIARY	definition (p. 65)
NMI:source	NMI	admitted non-maskable interrupt source	6	BASIC	definition (p. 65)
INTERRUPT:platform	INTERRUPT	admitted maskable interrupt source	7	BASIC	definition (p. 65)

A.4 CPUID Feature Index

Table A-4. CPUID Feature Index

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
FP (p. 83)	00000001:0000:1/bit0	FABS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FADD	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FBNDII	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FBNDIX	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FBNDXI	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FBNDXX	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FCEIL	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FCLASS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FCLR	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FCMP	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FCOPYSIGN	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FCVT	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FCVTU	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FDIV	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FFLOOR	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FGETEXP	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FGETMAN	F0–F15, FFLAGS, FSTATUS, SAVE component FP

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
FP (p. 83)	00000001:0000:1/bit0	FINT	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FINTRZ	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FMADD	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FMAX	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FMIN	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FMOD	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FMOV	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FMOVCR	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FMOVcc	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FMSUB	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FMUL	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FNEG	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FNMADD	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FNMSUB	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FPOPP	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FPUSHP	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FREM	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FROUND	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FSCALE	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FSQRT	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FSUB	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FTEST	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FTRUNC	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	FXCHG	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	RDFFLAGS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	RDFSTATUS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	WRFFLAGS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FP (p. 83)	00000001:0000:1/bit0	WRFSTATUS	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FACOSA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FASINA	F0–F15, FFLAGS, FSTATUS, SAVE component FP

Table A-4 (continued)

Predicate	Class:leaf:index/bit	Gated instruction	Related state or component
FPTRANSA (p. 83)	00000001:0000:1/bit1	FATANA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FATANHA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FCOSA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FCOSHA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FETOXA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FETOXM1A	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FLOG10A	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FLOG2A	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FLOGNA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FLOGNP1A	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FSINA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FSINCOSA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FSINHA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FTANA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FTANHA	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FTENTOX A	F0–F15, FFLAGS, FSTATUS, SAVE component FP
FPTRANSA (p. 83)	00000001:0000:1/bit1	FTWOTOXA	F0–F15, FFLAGS, FSTATUS, SAVE component FP

A.5 Architecture Revision History

The current working specification remains in the Unreleased row until a release assigns an ARCHITECTURE_REVISION. Released rows use the revision value returned by CPUID.

Table A-5. Architecture Revision History

Status	Architecture revision	Changes
Unreleased	–	Closed the CPUID, SAVE/RESTORE, floating-point, cache-maintenance, and effective-address contracts.; Defined reset, memory policy, translation, event, repeat, and instruction-encoding boundaries.; Defined atomic ordering, paging boundaries, compatibility identity, stack, WAIT/HALT, and conformance contracts.; Unified terminology, canonical field names, reference indexes, and revision-history navigation.